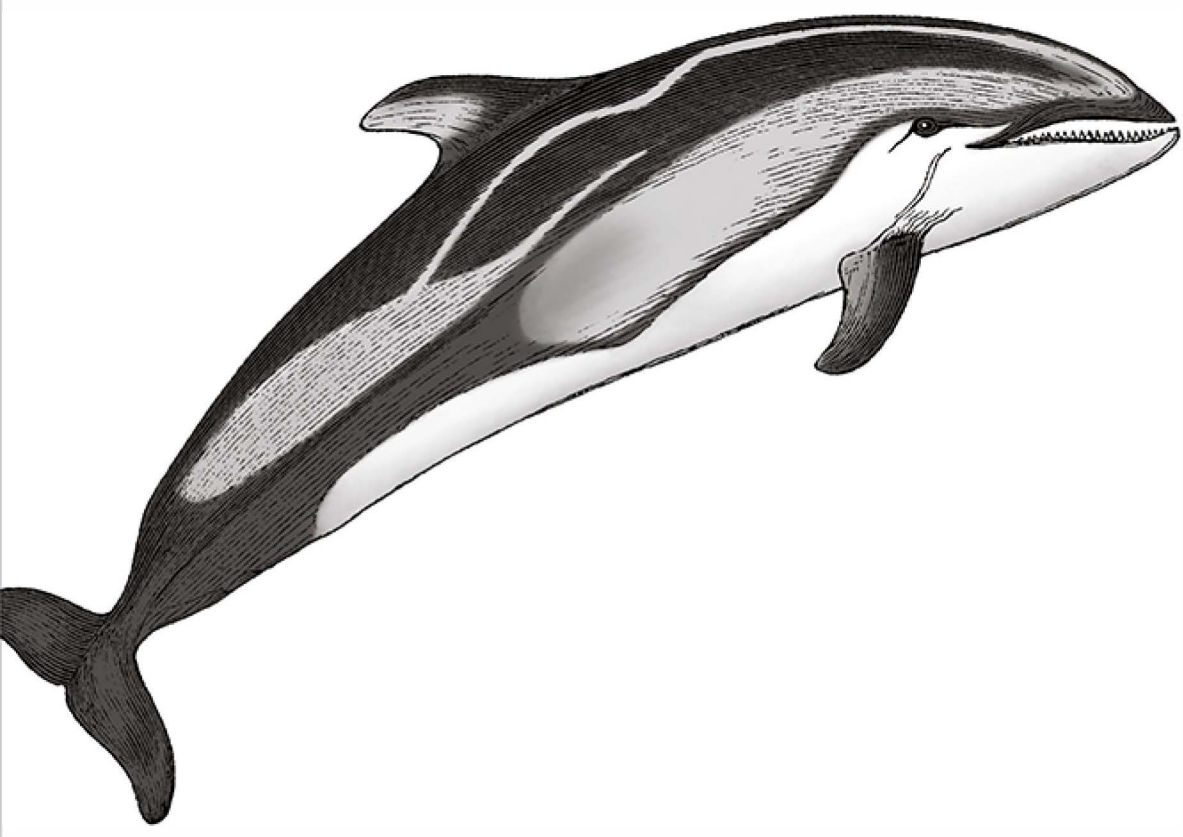
O'REILLY

Wydanie III

Kubernetes

Tworzenie niezawodnych systemów rozproszonych



Brendan Burns, Joe Beda
Kelsey Hightower, Lachlan Evenson

Helion

Tytuł oryginału: Kubernetes: Up and Running: Dive into the Future of Infrastructure, 3rd Edition

Tłumaczenie: Łukasz Piwko z wykorzystaniem fragmentów poprzedniego wydania w przekładzie Lecha Lachowskiego

ISBN: 978-83-8322-337-7

© 2023 Helion S.A.

Authorized Polish translation of the English edition of *Kubernetes: Up and Running, 3E*
ISBN 9781098110208 © 2022 Brendan Burns, Joe Beda, Kelsey Hightower, and Lachlan Evenson.

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from the Publisher.

Wszelkie prawa zastrzeżone. Nieautoryzowane rozpowszechnianie całości lub fragmentu niniejszej publikacji w jakiejkolwiek postaci jest zabronione. Wykonywanie kopii metodą kserograficzną, fotograficzną, a także kopiowanie książki na nośniku filmowym, magnetycznym lub innym powoduje naruszenie praw autorskich niniejszej publikacji.

Wszystkie znaki występujące w tekście są zastrzeżonymi znakami firmowymi bądź towarowymi ich właścicieli.

Autor oraz wydawca dołożyli wszelkich starań, by zawarte w tej książce informacje były kompletne i rzetelne. Nie biorą jednak żadnej odpowiedzialności ani za ich wykorzystanie, ani za związane z tym ewentualne naruszenie praw patentowych lub autorskich. Autor oraz wydawca nie ponoszą również żadnej odpowiedzialności za ewentualne szkody wynikłe z wykorzystania informacji zawartych w książce.

Drogi Czytelniku!
Jeżeli chcesz ocenić tę książkę, zajrzyj pod adres
https://helion.pl/user/opinie/kuber3_ebook
Możesz tam wpisać swoje uwagi, spostrzeżenia, recenzję.

Pliki z przykładami omawianymi w książce można znaleźć pod adresem:
https://ftp.helion.pl/przyklady/kuber3.zip

Helion S.A.
ul. Kościuszki 1c, 44-100 Gliwice
tel. 32 230 98 63
e-mail: *helion@helion.pl*
WWW: *https://helion.pl* (księgarnia internetowa, katalog książek)

- Poleć książkę na Facebook.com
- Kup w wersji papierowej
- Oceń książkę

- Księgarnia internetowa
- Lubię to! » Nasza społeczność

Spis treści

Przedmowa **13**

1. Wprowadzenie **19**
Prędkość 20
Wartość niemutowalności 21
Deklaratywna konfiguracja 22
Systemy samonaprawiające się 23
Skalowanie usługi i zespołów programistycznych 23
Rozłączność 24
Łatwe skalowanie aplikacji i klastrów 24
Skalowanie zespołów programistycznych za pomocą mikrousług 25
Separacja zagadnień dla zapewnienia spójności i skalowania 26
Zapewnianie abstrakcji infrastruktury 27
Wydajność 28
Ekosystem o pochodzeniu chmurowym 29
Podsumowanie 30

2. Tworzenie i uruchamianie kontenerów **31**
Obrazy kontenerów 32
Budowanie obrazów aplikacji za pomocą Dockera 34
Pliki Dockerfile 34
Optymalizacja rozmiarów obrazu 36
Bezpieczeństwo obrazu 37
Wieloetapowe budowanie obrazów 37
Przechowywanie obrazów w zdalnym rejestrze 39
Środowisko wykonawcze kontenera Dockera 40
Uruchamianie kontenerów za pomocą Dockera 41
Odkrywanie aplikacji kuard 41
Ograniczanie wykorzystania zasobów 41
Czyszczenie 42
Podsumowanie 43

3. Wdrażanie klastra Kubernetes 44
Instalowanie Kubernetes w usłudze dostawcy publicznej chmury 45
Google Kubernetes Engine 45
Instalowanie Kubernetes w Azure Kubernetes Service 45
Instalowanie Kubernetes w Amazon Web Services 46
Lokalna instalacja Kubernetes za pomocą minikube 46
Uruchamianie Kubernetes w Dockerze 47
Klient Kubernetes 47
Sprawdzanie statusu klastra 48
Wyświetlanie węzłów roboczych klastra Kubernetes 48
Komponenty klastra 50
Serwer proxy Kubernetes 50
Serwer DNS Kubernetes 51
Interfejs użytkownika Kubernetes 51
Podsumowanie 51

4. Typowe polecenia kubectl 52
Przestrzenie nazw 52
Konteksty 52
Przeglądanie obiektów interfejsu API Kubernetes 53
Tworzenie, aktualizacja i niszczenie obiektów Kubernetes 54
Dodawanie etykiet i adnotacji do obiektów 55
Polecenia debugowania 55
Zarządzanie klastrem 57
Uzupełnianie poleceń 57
Inne sposoby pracy z klastrami 58
Podsumowanie 58

5. Kapsuły 59
Kapsuły w Kubernetes 60
Myślenie w kategoriach kapsuł 60
Manifest kapsuły 61
Tworzenie kapsuły 62
Tworzenie manifestu kapsuły 62
Uruchamianie kapsuł 63
Wyświetlanie listy kapsuł 63
Szczegółowe informacje o kapsule 64
Usuwanie kapsuły 65
Uzyskiwanie dostępu do kapsuły 65
Uzyskiwanie większej ilości informacji za pomocą dzienników 65
Uruchamianie poleceń w kontenerze przy użyciu exec 66
Kopiowanie plików do i z kontenerów 66

Kontrole działania 66
Sonda żywotności 67
Sonda gotowości 68
Sonda rozruchu 68
Zaawansowana konfiguracja sondy 68
Rodzaje kontroli działania 69
Zarządzanie zasobami 69
Żądania zasobów: minimalne wymagane zasoby 70
Ograniczanie wykorzystania zasobów za pomocą limitów 71
Utrwalanie danych za pomocą woluminów 72
Używanie woluminów z kapsułami 72
Różne sposoby używania woluminów z kapsułami 72
Wszystko razem 74
Podsumowanie 75

6. Etykiety i adnotacje 76
Etykiety 76
Stosowanie etykiet 77
Modyfikowanie etykiet 78
Selektory etykiet 79
Selektory etykiet w obiektach API 81
Etykiety w architekturze Kubernetes 81
Adnotacje 82
Czyszczenie 83
Podsumowanie 83

7. Wykrywanie usług 84
Co to jest wykrywanie usług? 84
Obiekt Service 85
DNS usługi 86
Kontrole gotowości 87
Udostępnianie usługi poza klastrem 88
Integracja z usługą równoważenia obciążenia 89
Szczegóły dla zaawansowanych 91
Punkty końcowe 91
Ręczne wykrywanie usług 92
kube-proxy i adresy IP klastra 93
Zmienne środowiskowe adresu IP klastra 94
Łączenie z innymi środowiskami 94
Łączenie z zasobami poza klastrem 95
Łączenie zasobów zewnętrznych z usługami w klastrze 95
Czyszczenie 96
Podsumowanie 96

8. Równoważenie obciążenia HTTP przy użyciu Ingress 97
Specyfikacja Ingress i kontrolery Ingress 98
Instalacja Contour 99
Konfiguracja DNS 99
Konfiguracja pliku lokalnych hostów 100
Praca z Ingress 100
Najprostszy sposób użycia 101
Używanie nazw hosta 102
Ścieżki 103
Czyszczenie 104
Techniki zaawansowane i pułapki 104
Uruchamianie kilku kontrolerów Ingress 104
Wiele obiektów Ingress 105
Ingress i przestrzenie nazw 105
Przepisywanie ścieżek 105
Serwowanie przez TLS 106
Inne implementacje Ingress 107
Przyszłość Ingress 107
Podsumowanie 108

9. Obiekt ReplicaSet 109
Pętle uzgadniania 110
Relacje między kapsułami i obiektami ReplicaSet 110
Adaptowanie istniejących kontenerów 110
Poddawanie kontenerów kwarantannie 111
Projektowanie z wykorzystaniem ReplicaSet 111
Specyfikacja ReplicaSet 111
Szablony kapsuł 112
Etykiety 113
Tworzenie obiektu ReplicaSet 113
Inspekcja obiektu ReplicaSet 113
Znajdowanie ReplicaSet z poziomu kapsuły 114
Znajdowanie zestawu kapsuł dla ReplicaSet 114
Skalowanie kontrolerów ReplicaSet 114
Skalowanie imperatywne za pomocą polecenia kubectl scale 114
Skalowanie deklaratywne za pomocą kubectl apply 115
Automatyczne skalowanie kontrolera ReplicaSet 116
Usuwanie obiektów ReplicaSet 117
Podsumowanie 117

10. Obiekt Deployment 118
Twoje pierwsze wdrożenie 119
Tworzenie obiektów Deployment 120
Zarządzanie obiektami Deployment 122
Aktualizowanie obiektów Deployment 123
Skalowanie obiektu Deployment 123
Aktualizowanie obrazu kontenera 123
Historia wersji 124
Strategie wdrażania 127
Strategia Recreate 127
Strategia RollingUpdate 127
Spowalnianie wdrażania w celu zapewnienia poprawnego działania usługi 130
Usuwanie wdrożenia 131
Monitorowanie wdrożenia 132
Podsumowanie 132

11. Obiekt DaemonSet 133
Planista DaemonSet 134
Tworzenie obiektów DaemonSet 134
Ograniczanie użycia kontrolerów DaemonSet do określonych węzłów 136
Dodawanie etykiet do węzłów 137
Selektory węzłów 137
Aktualizowanie obiektu DaemonSet 138
Usuwanie obiektu DaemonSet 139
Podsumowanie 139

12. Obiekt Job 141
Obiekt Job 141
Wzorce obiektu Job 141
Zadania jednorazowe 142
Równoległość 146
Kolejki robocze 147
Obiekt CronJob 151
Podsumowanie 151

13. Obiekty ConfigMap i tajne dane 153
Obiekty ConfigMap 153
Tworzenie obiektów ConfigMap 153
Używanie obiektów ConfigMap 154
Tajne dane 157
Tworzenie tajnych danych 158
Korzystanie z tajnych danych 159
Prywatne rejestry kontenera 160

Ograniczenia dotyczące nazewnictwa 160
Zarządzanie obiektami ConfigMap i tajnymi danymi 161
Wyświetlanie obiektów 161
Tworzenie obiektów 162
Aktualizowanie obiektów 162
Podsumowanie 164

14. Model kontroli dostępu oparty na rolach w Kubernetes ... 165
Kontrola dostępu oparta na rolach 166
Tożsamość w Kubernetes 166
Role i powiązania ról 167
Role i powiązania ról w Kubernetes 167
Techniki zarządzania funkcją RBAC 169
Testowanie autoryzacji za pomocą narzędzia can-i 169
Zarządzanie funkcją RBAC w kontroli źródła 170
Tematy zaawansowane 170
Agregowanie ról klastrowych 170
Wykorzystywanie grup do wiązań 171
Podsumowanie 172

15. Siatki usług ... 173
Szyfrowanie i uwierzytelnianie przy użyciu Mutual TLS 174
Kształtowanie ruchu 174
Introspekcja 175
Czy naprawdę potrzebujesz siatki usług 175
Introspekcja implementacji siatki usług 176
Krajobraz siatek usług 176
Podsumowanie 177

16. Integracja rozwiązań do przechowywania danych i Kubernetes ... 178
Importowanie usług zewnętrznych 179
Usługi bez selektorów 180
Ograniczenia usług zewnętrznych: sprawdzanie poprawności działania 181
Uruchamianie niezawodnych singletonów 181
Uruchamianie singletona MySQL 182
Dynamiczne przydzielanie woluminów 185
Natywne magazyny danych Kubernetes z wykorzystaniem obiektów StatefulSet 186
Właściwości obiektów StatefulSet 187
Ręcznie zreplikowany klaster MongoDB z wykorzystaniem obiektów StatefulSet 187
Automatyzacja tworzenia klastra MongoDB 189
Trwałe woluminy i obiekty StatefulSet 192
Ostatnia rzecz: sondy gotowości 193
Podsumowanie 193

17. Rozszerzanie Kubernetes 194
Co znaczy rozszerzanie Kubernetes 194
Punkty rozszerzalności 195
Wzorce tworzenia zasobów 202
Tylko dane 202
Kompilatory 202
Operatory 203
Jak zacząć 203
Podsumowanie 203

18. Dostęp do Kubernetes z poziomu popularnych języków programowania 204
API Kubernetes — perspektywa klienta 204
OpenAPI i generowane biblioteki klientów 205
Kwestia kubectl x 205
Programowanie API Kubernetes 206
Instalacja bibliotek klienckich 206
Uwierzytelnianie w API Kubernetes 206
Dostęp do API Kubernetes 208
Generowanie list i tworzenie kapsuł w Pythonie, Javie i .NET 208
Tworzenie i łatanie nowych obiektów 210
Obserwowanie zmian w API Kubernetes 211
Interakcja z kapsułami 213
Podsumowanie 215

19. Zabezpieczanie aplikacji w Kubernetes 216
Kontekst zabezpieczeń 216
Wyzwania związane z kontekstem zabezpieczeń 221
Pod Security 222
Czym jest Pod Security 222
Stosowanie standardów Pod Security 223
Zarządzanie kontami usług 225
Kontrola dostępu oparta na rolach 226
RuntimeClass 226
Network Policy 227
Siatka usług 230
Narzędzia do weryfikacji zabezpieczeń 230
Bezpieczeństwo obrazów 232
Podsumowanie 232

20. Polityki i zarządzanie klastrami Kubernetes 233
Dlaczego polityka i zarządzanie są ważne 233
Przepływ wstępu 234
Polityka i zarządzanie z narzędziem Gatekeeper 235
Czym jest Open Policy Agent 235
Instalacja Gatekeepera 235
Konfigurowanie polityk 237
Szablony ograniczeń 240
Tworzenie ograniczeń 240
Audyt 241
Modyfikacja 242
Replikacja danych 244
Metryki 245
Biblioteka polityk 245
Podsumowanie 246

21. Wdrożenia aplikacji w wielu klastrach 247
Zanim zaczniesz 248
Podejście od góry z równoważeniem obciążenia 249
Budowa aplikacji dla wielu klastrów 251
Replikowane silosy — najprostszy model międzyregionalny 252
Fragmentowanie — dane regionalne 253
Większa elastyczność — routing mikrousług 254
Podsumowanie 255

22. Organizacja aplikacji 256
Podstawowe zasady 256
Systemy plików jako źródło prawdy 256
Rola recenzji kodu 257
Bramy i flagi funkcji 257
Zarządzanie aplikacją w systemie kontroli źródła 258
Układ systemu plików 258
Wersje okresowe 259
Konstruowanie aplikacji w sposób umożliwiający jej rozwój, testowanie i wdrażanie 261
Cele 261
Progresja wydania 262
Parametryzacja aplikacji za pomocą szablonów 264
Parametryzacja przy użyciu narzędzia Helm i szablonów 264
Parametryzacja systemu plików 265

Wdrażanie aplikacji na całym świecie 265
Architektura umożliwiająca wdrażanie aplikacji na całym świecie 265
Implementacja wdrożenia światowego 267
Pulpity i monitorowanie wdrożeń światowych 268
Podsumowanie 268

A Budowanie własnego klastra Kubernetes 269

Przedmowa

Zespół Kubernetes chciałby podziękować każdemu administratorowi systemów, który zrywał się o trzeciej nad ranem, by zrestartować jakiś proces. Każdemu programiście, który przesyłał kod do środowiska produkcyjnego tylko po to, aby przekonać się, że nie działa tak, jak działał na jego laptopie. Każdemu architektowi systemów, który omyłkowo skierował obciążenie testowe do serwera produkcyjnego ze względu na pozostawioną niezaktualizowaną nazwę hosta. To właśnie ten fizyczny ból, dziwne godziny pracy i dziwaczne błędy zainspirowały nas do stworzenia systemu Kubernetes. W jednym zdaniu: Kubernetes ma radykalnie uprościć zadanie budowania, wdrażania i utrzymywania systemów rozproszonych. Do jego powstania przyczyniły się dekady rzeczywistych doświadczeń w budowaniu niezawodnych systemów; został zaprojektowany od podstaw w taki sposób, aby uczynić to zadanie jeśli nie euforycznym, to przynajmniej przyjemnym. Mamy nadzieję, że spodoba Ci się ta książka!

Kto powinien przeczytać tę książkę?

Bez względu na to, czy dopiero zaczynasz pracę z systemami rozproszonymi, czy też od lat wdrażasz chmurowo natywne systemy, kontenery i Kubernetes mogą pomóc Ci w osiągnięciu nowych poziomów prędkości, zwinności, niezawodności i wydajności. Ta książka opisuje orkiestrator klastrów Kubernetes oraz sposoby wykorzystania jego narzędzi i interfejsów API do usprawnienia procesów rozwoju, dostarczania, zabezpieczania i utrzymywania rozproszonych aplikacji. W założeniu nie musisz mieć żadnych wcześniejszych doświadczeń z systemem Kubernetes, ale abyś mógł maksymalnie skorzystać na lekturze tej książki, powinieneś znać zagadnienia związane z tworzeniem i wdrażaniem aplikacji serwerowych. Przyda Ci się znajomość takich pojęć jak mechanizmy równoważenia obciążenia i sieciowe magazyny danych, choć nie jest ona wymagana. Podobnie, doświadczenie pracy z systemami Linux, kontenerami linuksowymi i Dockerem, choć nie jest niezbędne, pomoże Ci w pełni wykorzystać wiedzę zawartą w tej książce.

Dlaczego napisaliśmy tę książkę?

Jesteśmy zaangażowani w Kubernetes od samych jego początków. To było naprawdę niezwykłe doświadczenie, gdy obserwowaliśmy, jak ciekawość, w dużej mierze wykorzystywana w eksperymentach, przeradza się w istotną infrastrukturę produkcyjną, która napędza duże aplikacje

produkcyjne w różnych dziedzinach, od uczenia maszynowego po usługi internetowe. Gdy ta transformacja się dokonała, zaczęło stawać się jasne, że książka, która omawiałaby zarówno sposób korzystania z podstawowych koncepcji w Kubernetes, jak i motywacje do rozwijania tych koncepcji, będzie ważnym wkładem w proces tworzenia natywnych aplikacji chmurowych. Mamy nadzieję, że podczas lektury tej książki nie tylko dowiesz się, jak na bazie Kubernetes budować niezawodne, skalowalne aplikacje, ale także zapoznasz się z podstawowymi wyzwaniami związanymi z systemami rozproszonymi, które doprowadziły do powstania Kubernetes.

Dlaczego zaktualizowaliśmy tę książkę?

Ekosystem Kubernetes rozwija się i przekształca od ukazania się pierwszego i drugiego wydania tej książki. Pojawiło się wiele wersji samej platformy Kubernetes, a sporo narzędzi i metod pracy stało się de facto standardami. W trzecim wydaniu skupiliśmy się na tematach, które zyskały na znaczeniu w ekosystemie Kubernetes, a więc na bezpieczeństwie, dostępie do Kubernetes z poziomu języków programowania oraz na tworzeniu aplikacji wieloklastrowych. Ponadto zaktualizowaliśmy informacje zawarte we wcześniej napisanych rozdziałach, uwzględniając zmiany, jakie zaszły w Kubernetes od czasu pojawienia się pierwszego wydania tej książki. Za kilka lat z pewnością ponownie ją zaktualizujemy (co zrobimy z przyjemnością), ponieważ Kubernetes nieustannie ewoluuje.

Kilka słów na temat aktualnego stanu aplikacji natywnych w chmurze

Od pierwszych języków programowania, poprzez programowanie obiektowe, aż po rozwój wirtualizacji i infrastruktury chmury historia informatyki jest historią rozwijania abstrakcji, które ukrywają złożoność i umożliwiają tworzenie coraz bardziej zaawansowanych aplikacji. Mimo to rozwój niezawodnych, skalowalnych aplikacji jest wciąż znacznie trudniejszy, niż powinien być. W ostatnich latach kontenery i interfejsy API orkiestracji kontenerów, takie jak Kubernetes, stały się ważną abstrakcją, która radykalnie upraszcza rozwijanie niezawodnych, skalowalnych systemów rozproszonych. Kontenery i orkiestratory umożliwiają programistom budowanie i wdrażanie aplikacji z szybkością, zwinnością i niezawodnością, które zaledwie kilka lat temu mogły się wydawać fantastyką naukową.

Poruszanie się po tej książce

Ta książka jest zorganizowana w zarysowany poniżej sposób. W rozdziale 1. przedstawiamy ogólne korzyści płynące z używania Kubernetes bez wdawania się w szczegóły. Jeśli jesteś początkującym użytkownikiem Kubernetes, to świetny wstęp, aby zrozumieć, dlaczego warto przeczytać resztę książki.

Rozdział 2. zawiera szczegółowe wprowadzenie do kontenerów i rozwoju aplikacji kontenerowych. Jeśli nigdy przedtem nie bawiłeś się Dockerem, ten rozdział będzie przydatnym przewodnikiem. Jeżeli jesteś już ekspertem od Dockera, najprawdopodobniej będzie to dla Ciebie ogólny przegląd zagadnień.

Rozdział 3. opisuje, jak wdrożyć Kubernetes. Chociaż większa część tej książki koncentruje się na sposobach *korzystania* z Kubernetes, zanim zaczniesz to robić, musisz uruchomić klaster. Uruchomienie klastra do celów produkcyjnych wykracza poza zakres tej książki, ale ten rozdział przedstawia kilka prostych sposobów tworzenia klastra, żebyś mógł zrozumieć, jak używać systemu Kubernetes. W rozdziale 4. opisujemy często używane polecenia do obsługi klastrów Kubernetes.

Rozpoczynając od rozdziału 5., zaczynamy omawiać szczegóły wdrażania aplikacji przy użyciu Kubernetes. Omawiamy kapsuły (rozdział 5.), etykiety i adnotacje (rozdział 6.), usługi (rozdział 7.), Ingress (rozdział 8.) i obiekty `ReplicaSet` (rozdział 9.). Tworzą one zbiór podstawowych elementów, których potrzebujesz, aby wdrożyć swoją usługę w Kubernetes. Następnie omawiamy wdrożenia (rozdział 10.), które spajają cykl życia kompletnej aplikacji.

Po tych zagadnieniach zajmujemy się bardziej wyspecjalizowanymi obiektami w Kubernetes, takimi jak `DaemonSet` (rozdział 11.), `Job` (rozdział 12.) oraz `ConfigMap` i tajne dane (rozdział 13.). Chociaż informacje zawarte w tych rozdziałach są niezbędne do obsługi wielu aplikacji produkcyjnych, jeśli dopiero uczysz się systemu Kubernetes, możesz je pominąć i wrócić do nich później, po zdobyciu większego doświadczenia i wiedzy technicznej.

W dalszej kolejności wprowadzamy pojęcie kontroli dostępu opartej na rolach (rozdział 14.), opisujemy siatki usług (rozdział 15.) oraz przedstawiamy zagadnienia dotyczące integracji pamięci masowej (rozdział 16.) z Kubernetes. Opisujemy też rozszerzanie Kubernetes (rozdział 17.), dostęp z poziomu języków programowania (rozdział 18.), a także techniki zabezpieczania kapsuł (rozdział 19.) oraz nadzorowanie i zasady Kubernetes (rozdział 20.).

Na zakończenie podsumowujemy zawartą w książce wiedzę kilkoma przykładami tego, jak opracowywać i wdrażać aplikacje wieloklastrowe w Kubernetes (rozdział 21.), oraz omawiamy metody organizowania aplikacji w systemie kontroli źródła (rozdział 22.).

Zasoby online

Powinieneś zainstalować Dockera (*https://docker.com*). Warto, byś zapoznał się też z dokumentacją Dockera, jeśli jeszcze tego nie zrobiłeś.

Powinieneś również zainstalować narzędzie wiersza poleceń kubectl (*https://kubernetes.io*). Możesz także dołączyć do kanału slackowego Kubernetes (*http://slack.kubernetes.io*), gdzie znajdziesz dużą społeczność użytkowników, którzy są chętni do rozmowy i odpowiadania na pytania niemal o każdej porze dnia.

Wreszcie, gdy będziesz się rozwijać i stawać coraz bardziej zaawansowanym użytkownikiem, możesz chcieć zaangażować się w tworzenie opartego na licencji open source repozytorium Kubernetes na GitHubie (*https://github.com/kubernetes/kubernetes*).

Konwencje stosowane w tej książce

W tej książce wykorzystywane są następujące konwencje typograficzne:

Czcionka pogrubiona

Stosowana do podkreślenia ważnych pojęć i przy wprowadzaniu nowych terminów.

Czcionka pochylona

Stosowana do zapisywania nazw folderów i plików, rozszerzeń plików, nazw ścieżek, adresów URL, nazw opcji, nazw okien programów oraz nazw przycisków.

`Czcionka o stałej szerokości znaków`

Stosowana do zapisywania poleceń i fragmentów kodu występujących w tekście.

`Pogrubiona czcionka o stałej szerokości znaków`

Stosowana do zaznaczania fragmentów poleceń, które należy samodzielnie wpisać.

`Pochylona czcionka o stałej szerokości znaków`

Stosowana w poleceniach do zaznaczania argumentów, pod które należy samodzielnie podstawić określone wartości.

Ta ikona oznacza wskazówki, podpowiedzi lub ogólne uwagi.

Ta ikona oznacza ostrzeżenie lub przestrogę.

Korzystanie z przykładów kodu

Umieszczone w książce przykłady kodu możesz pobrać z serwera FTP wydawnictwa Helion, pod adresem *https://ftp.helion.pl/przyklady/kuber3.zip*. Po ściągnięciu archiwum rozpakuj je na swoim dysku.

Książka ta ma pomóc Ci w pracy. Ogólnie rzecz biorąc, kodu znajdującego się w tej książce można używać we własnych programach i dokumentacjach bez proszenia kogokolwiek o zgodę, chyba że wykorzystasz duże fragmenty. Jeśli na przykład w pisanym programie użyjesz kilku fragmentów kodu z tej książki, nie musisz pytać o pozwolenie. Aby sprzedawać i rozprowadzać płyty CD-ROM z przykładami, trzeba mieć zezwolenie. Aby odpowiedzieć komuś na pytanie, cytując fragment tej książki wraz z kodem źródłowym, nie trzeba mieć zezwolenia. Aby wykorzystać dużą ilość kodu źródłowego z tej książki w dokumentacji własnego produktu, trzeba mieć pozwolenie.

Informacje o źródle użytych fragmentów są mile widziane, ale niewymagane. Notka powinna zawierać nazwisko autora, tytuł publikacji, nazwę wydawcy oraz datę i miejsce publikacji, na przykład Brendan Burns, Joe Beda, Kelsey Hightower, Lachlan Evenson, *Kubernetes. Tworzenie niezawodnych systemów rozproszonych. Wydanie III*, Helion, Gliwice 2023.

Podziękowania

Pragniemy podziękować wszystkim osobom, które pomogły nam w pracy nad tą książką. Do tego grona zaliczają się nasze redaktorki Virginia Wilson i Sarah Grey oraz cały wspaniały zespół z wydawnictwa O'Reilly, a także redaktorzy merytoryczni, których trafne uwagi w znacznym stopniu przyczyniły się do podniesienia jakości tej książki. Chcemy podziękować również wszystkim czytelnikom pierwszego i drugiego wydania, którzy zgłosili znalezione błędy, dzięki czemu mogliśmy je poprawić w trzecim wydaniu. Dziękujemy Wam wszystkim! Jesteśmy wdzięczni

ROZDZIAŁ 1.
Wprowadzenie

Kubernetes jest open source'owym orkiestratorem do wdrażania kontenerowych aplikacji. Został opracowany przez Google pod wpływem inspiracji płynących z dziesięcioletnich doświadczeń we wdrażaniu skalowalnych, niezawodnych systemów w kontenerach za pośrednictwem ukierunkowanych użytkowo interfejsów API[1].

Kubernetes, który powstał w 2014 r., dziś jest jednym z największych i najpopularniejszych projektów open source na świecie. Stał się standardowym API do budowy aplikacji chmurowych i jest wykorzystywany w prawie każdej chmurze publicznej. Kubernetes to sprawdzona infrastruktura do tworzenia systemów rozproszonych odpowiednia dla twórców natywnych aplikacji chmurowych o różnej skali, od klastra komputerów Raspberry Pi po centra danych pełne najnowszych maszyn. Kubernetes zapewnia oprogramowanie niezbędne do efektywnego budowania i wdrażania niezawodnych, skalowalnych systemów rozproszonych.

Być może zastanawiasz się, co rozumiemy przez „niezawodne, skalowalne systemy rozproszone". Coraz więcej usług jest oferowanych przez sieć za pośrednictwem interfejsów API. Te interfejsy API są często dostarczane przez **system rozproszony** — różne elementy implementujące API działają na różnych maszynach połączonych siecią i koordynujących swoje działania poprzez komunikację sieciową. Ponieważ coraz częściej polegamy na tych API we wszystkich aspektach naszego codziennego życia (na przykład przy znajdowaniu drogi dojazdu do najbliższego szpitala), te systemy muszą być wysoce *niezawodne*. Nie mogą zawieść, nawet jeśli część systemu ulegnie awarii lub stanie się niedostępna z jakiegoś powodu. Muszą także być *dostępne* nawet podczas wdrażania nowych wersji oprogramowania lub w trakcie innych działań konserwacyjnych. Wreszcie, ponieważ coraz większa część świata jest obecna w internecie i korzysta z takich usług, muszą być one wysoce *skalowalne*, aby można było zwiększać ich możliwości, nadążając za coraz większym zapotrzebowaniem, bez radykalnego przeprojektowywania systemu rozproszonego, który implementuje te usługi. W wielu przypadkach oznacza to także automatyczne zwiększanie (i zmniejszanie) możliwości, aby aplikacja była maksymalnie wydajna.

W zależności od tego, kiedy i dlaczego ta książka trafiła w Twoje ręce, możesz być w różnym stopniu zaznajomiony z kontenerami, systemami rozproszonymi i platformą Kubernetes. Może planujesz

[1] Brendan Burns i in., *Borg, Omega, and Kubernetes: Lessons Learned from Three Container-Management Systems over a Decade*, „ACM Queue", 2016, 14, s. 70 – 93; dostępne pod adresem *http://bit.ly/2vIrL4S*.

budowę aplikacji na bazie chmury publicznej, w prywatnych centrach danych lub w jakimś środowisku hybrydowym. Bez względu na to, jakie masz doświadczenie, wierzymy, że ta książka pozwoli Ci w pełni wykorzystać możliwości Kubernetes.

Istnieje wiele powodów, dla których używa się kontenerów i kontenerowych interfejsów API, takich jak Kubernetes, ale uważamy, że wszystkie można tak naprawdę sprowadzić do jednej z poniższych korzyści:

- prędkość tworzenia,
- skalowanie (zarówno oprogramowania, jak i zespołów programistycznych),
- zapewnianie abstrakcji infrastruktury,
- wydajność,
- ekosystem chmurowy.

W kolejnych podrozdziałach opiszemy, w jaki sposób Kubernetes może pomóc w uzyskaniu wymienionych korzyści.

Prędkość

Prędkość jest obecnie prawie zawsze kluczowym czynnikiem w tworzeniu oprogramowania. Zmienna natura oprogramowania, od wersji pudełkowych dostarczanych na płytach CD lub DVD po oprogramowanie udostępniane przez internet i usługi sieciowe zmieniające się co kilka godzin, powoduje, że różnica między Tobą a Twoją konkurencją często sprowadza się do szybkości, z jaką możesz rozwijać i wdrażać nowe komponenty i funkcje, lub szybkości, z jaką reagujesz na innowacje wprowadzane przez innych.

Należy jednak zauważyć, że prędkość, o którą nam chodzi, nie jest zdefiniowana po prostu jako szybkość. Użytkownicy zawsze oczekują systematycznego ulepszania produktu, ale tak naprawdę bardziej interesuje ich niezawodna usługa. Dawno, dawno temu nie stanowiło to problemu, gdy usługa była codziennie wyłączana o północy dla celów konserwacyjnych. Ale dzisiaj nasi użytkownicy oczekują nieprzerwanej dostępności, nawet jeśli oprogramowanie, z którego korzystają, ciągle się zmienia.

W związku z tym prędkość mierzy się nie pod względem samej liczby funkcjonalności, które można dostarczyć co godzinę lub dziennie, ale raczej pod względem liczby rzeczy, które można dostarczyć przy jednoczesnym utrzymaniu dużej dostępności usługi.

Kontenery i Kubernetes mogą oferować narzędzia potrzebne, aby działać szybko, zachowując dostępność. Do podstawowych koncepcji, które to umożliwiają, należą:

- niemutowalność,
- deklaratywna konfiguracja,
- samonaprawiające się systemy,
- współdzielone biblioteki i narzędzia nadające się do wielokrotnego użytku.

Wszystkie te pomysły są ze sobą powiązane w taki sposób, aby radykalnie poprawić szybkość, z jaką można niezawodnie wdrożyć nowe oprogramowanie.

Wartość niemutowalności

Kontenery i Kubernetes zachęcają programistów do budowania systemów rozproszonych, które są zgodne z zasadami niemutowalnej infrastruktury. Dzięki *niemutowalnej* infrastrukturze żaden artefakt, który powstanie w systemie, nie zmienia się w wyniku modyfikacji dokonywanych przez użytkownika.

Tradycyjnie komputery i systemy oprogramowania były traktowane jako infrastruktura **mutowalna**. W przypadku infrastruktury mutowalnej zmiany są wprowadzane jako przyrostowe aktualizacje istniejącego systemu. Mogą one zostać wykonane wszystkie naraz lub ich wykonanie może być rozłożone na dłuższy okres. Dobrym przykładem aktualizacji mutowalnego systemu jest upgrade za pomocą narzędzia `apt-get update`. Uruchomienie polecenia `apt` powoduje pobranie kolejno wszelkich zaktualizowanych plików binarnych, skopiowanie ich w miejsce starszych plików binarnych i wykonanie przyrostowych aktualizacji plików konfiguracyjnych. W przypadku systemu mutowalnego aktualny stan infrastruktury nie jest reprezentowany jako pojedynczy artefakt, ale raczej jako nagromadzenie przyrostowych aktualizacji i zmian w czasie. W wielu systemach te przyrostowe aktualizacje pochodzą nie tylko z aktualizacji systemu, ale także z modyfikacji wprowadzonych przez operatora. Ponadto istnieje duże prawdopodobieństwo, że w systemach obsługiwanych przez duże zespoły zmiany będą wprowadzane przez wiele osób i w licznych przypadkach nigdzie nie zostaną zarejestrowane.

W systemie niemutowalnym natomiast zamiast serii przyrostowych aktualizacji i zmian budowany jest całkowicie nowy kompletny obraz, a aktualizacja po prostu powoduje zastąpienie całego obrazu nowszym obrazem w jednej operacji. Nie ma żadnych przyrostowych zmian. Jak można sobie wyobrazić, jest to rewolucja w stosunku do tradycyjnego modelu zarządzania konfiguracją.

Aby odnieść to konkretnie do świata kontenerów, rozważmy dwa różne sposoby aktualizowania oprogramowania:

- Można zalogować się do kontenera, uruchomić polecenie pobrania nowego oprogramowania, wyłączyć stary serwer i uruchomić nowy.
- Można zbudować nowy obraz kontenera, przesłać go do rejestru kontenerów, wyłączyć istniejący kontener i uruchomić nowy.

Na pierwszy rzut oka te dwa podejścia mogą wydawać się w dużej mierze nie do odróżnienia. Co jest więc takiego w procesie budowania nowego kontenera, co powoduje zwiększenie niezawodności?

Kluczową różnicą jest tworzony artefakt oraz zapis sposobu jego utworzenia. Właśnie zapisy ułatwiają dokładne zrozumienie różnic między nową a starą wersją, a jeśli coś pójdzie nie tak, pomagają ustalić, co się zmieniło i jak to naprawić.

Ponadto tworzenie nowego obrazu zamiast modyfikowania istniejącego oznacza, że stary obraz nadal istnieje i można go szybko wykorzystać do przywrócenia poprzedniej wersji, jeśli wystąpi błąd. Po skopiowaniu nowego pliku binarnego na istniejący plik binarny takie wycofanie zmiany jest prawie niemożliwe.

Niemutowalne obrazy kontenerów stanowią rdzeń wszystkiego, co zbudujesz za pomocą Kubernetes. Istnieje możliwość imperatywnej zmiany działających kontenerów, ale jest to antywzorzec,

który należy stosować tylko w skrajnych przypadkach, gdy nie ma żadnych innych opcji (na przykład jest to jedyny sposób na tymczasową naprawę kluczowego systemu produkcyjnego). I nawet wtedy zmiany muszą być rejestrowane poprzez późniejszą aktualizację deklaratywnej konfiguracji po ugaszeniu pożaru.

Deklaratywna konfiguracja

Niemutowalność rozciąga się poza kontenery działające w klastrze w takim zakresie, w jakim poinstruujesz Kubernetes poprzez opis swojej aplikacji. W Kubernetes wszystko jest **obiektem deklaratywnej konfiguracji** reprezentującym pożądany stan systemu. Zadaniem Kubernetes jest zapewnienie, żeby faktyczny stan świata odpowiadał pożądanemu stanowi.

Podobnie jak w przypadku infrastruktury mutowalnej i niemutowalnej, deklaratywna konfiguracja to alternatywa dla konfiguracji **imperatywnej**, gdzie stan świata jest definiowany przez wykonanie serii instrukcji, a nie poprzez zadeklarowanie żądanego stanu świata. Podczas gdy polecenia imperatywne definiują akcje, konfiguracje deklaratywne definiują stan.

Abyś zrozumiał te dwa podejścia, rozważmy zadanie utworzenia trzech replik jakiegoś oprogramowania. Przy podejściu imperatywnym konfiguracja wyglądałaby następująco: „uruchom A, uruchom B i uruchom C". Odpowiadająca jej konfiguracja deklaratywna to: „liczba replik równa się trzy".

Ponieważ deklaratywna konfiguracja opisuje stan świata, nie musi zostać wykonana, aby można było ją zrozumieć. Jej rezultat jest konkretnie zadeklarowany. Ponieważ skutki konfiguracji deklaratywnej można zrozumieć przed jej wykonaniem, jest znacznie mniej narażona na błędy. Ponadto tradycyjne narzędzia i metody używane przy tworzeniu oprogramowania, takie jak system kontroli wersji, przeglądanie kodu i testy jednostkowe, mogą być stosowane w konfiguracji deklaratywnej w sposób, który jest niemożliwy do wykorzystania dla instrukcji imperatywnych. Idea przechowywania konfiguracji deklaratywnej w systemie kontroli źródła nazywa się „infrastruktura jako kod".

Ostatnio koncepcja infrastruktury jako kodu z kontrolą źródła jako źródłem prawdy zaczęła się formalizować w formie GitOps. Po wprowadzeniu metodyki GitOps zmiany w produkcji wykonuje się w całości przez wysyłanie do repozytorium Git, a następnie są one automatycznie odzwierciedlane w klastrze. W istocie klaster produkcyjny Kubernetes jest traktowany jako środowisko tylko do odczytu. Ponadto GitOps coraz chętniej wdraża się w usługach chmurowych Kubernetes jako najprostsze rozwiązanie umożliwiające deklaratywne zarządzanie infrastrukturą chmurową.

Połączenie deklaratywnego stanu przechowywanego w systemie kontroli wersji i zdolności Kubernetes do dopasowywania rzeczywistości do tego stanu powoduje, że wycofywanie zmian staje się banalnie łatwe. Wymaga po prostu ponownego zadeklarowania poprzedniego deklaratywnego stanu systemu. W przypadku systemów imperatywnych zwykle jest to niemożliwe, ponieważ imperatywne instrukcje opisują jedynie, jak dostać się z punktu A do punktu B, ale rzadko zawierają instrukcje odwrotne, które pozwoliłyby wrócić do poprzedniego stanu.

Systemy samonaprawiające się

Kubernetes to internetowy system samonaprawiający się. Gdy otrzyma konfigurację żądanego stanu, nie podejmuje jedynie działań prowadzących do jednorazowego zapewnienia zgodności bieżącego stanu ze stanem żądanym. Podejmuje *ciągłe* działania w celu zapewnienia stałej zgodności bieżącego stanu z żądanym. Oznacza to, że Kubernetes nie tylko zainicjuje Twój system, ale i będzie go chronić przed wszelkimi awariami lub zakłóceniami, które mogą go zdestabilizować i negatywnie wpłynąć na jego niezawodność.

Bardziej tradycyjny system naprawiania obejmuje serię ręcznych czynności lub interwencji człowieka wykonywanych w odpowiedzi na pewnego rodzaju alarm. Taka imperatywna naprawa jest droższa (ponieważ zazwyczaj wymaga utrzymywania w stanie gotowości dyspozycyjnego operatora, który w razie konieczności zostanie wezwany do naprawy systemu). Jest również zasadniczo wolniejsza, gdyż pozostająca w gotowości osoba często musi się obudzić w środku nocy i zalogować do systemu, aby zareagować na awarię. Co więcej, jest to mniej pewne rozwiązanie, ponieważ z imperatywną serią działań naprawczych wiążą się wszystkie problemy dotyczące zarządzania imperatywnego opisane w poprzednim punkcie. Samonaprawiające się systemy, takie jak Kubernetes, zmniejszają obciążenie operatorów i poprawiają ogólną niezawodność systemu poprzez szybsze wykonywanie rzetelnych napraw.

Możemy przywołać konkretny przykład tego samonaprawiania: jeśli wskażesz systemowi Kubernetes żądany stan trzech replik, to nie tylko utworzy on te trzy repliki, ale i będzie stale gwarantował istnienie dokładnie trzech replik. Jeżeli ręcznie utworzysz czwartą replikę, Kubernetes zniszczy jedną, aby przywrócić założoną liczbę trzech replik. Jeżeli ręcznie zniszczysz którąś replikę, Kubernetes utworzy jedną, żeby przywrócić żądany stan.

Internetowe systemy samonaprawiające się zwiększają prędkość programisty, ponieważ czas i energię poświęcane zwykle na obsługę i konserwację może on przeznaczyć na opracowywanie i testowanie nowych funkcjonalności.

W ostatnim czasie dużo dzieje się w temacie bardziej zaawansowanej formy samonaprawy w paradygmacie *operatorów* Kubernetes. W tym przypadku koduje się zaawansowaną logikę potrzebną do utrzymania, skalowania i naprawy określonego programu (na przykład MySQL) w aplikacji operującej, która działa jako kontener w klastrze. Zadaniem kodu w operatorze jest bardziej precyzyjne i zaawansowane sprawdzanie stanu oraz przeprowadzanie takich samych napraw, jakie można wykonać za pomocą ogólnych mechanizmów samonaprawy Kubernetes. Taki kod często jest reprezentowany jako „operatory". Do tematu operatorów wrócimy w rozdziale 17.

Skalowanie usługi i zespołów programistycznych

Wraz z rozwojem produktu konieczne staje się skalowanie zarówno oprogramowania, jak i rozwijających je zespołów. Na szczęście Kubernetes może być pomocny w obu tych kwestiach. Kubernetes osiąga skalowalność poprzez faworyzowanie architektur **rozłącznych**.

Rozłączność

W architekturze rozłącznej poszczególne komponenty są od siebie oddzielone poprzez zdefiniowane interfejsy API i mechanizmy równoważenia obciążenia usług. Interfejsy API i mechanizmy równoważenia obciążenia izolują każdy element systemu od pozostałych. Interfejsy API zapewniają bufor między implementatorem i konsumentem, a mechanizmy równoważenia obciążenia zapewniają bufor między działającymi instancjami każdej usługi.

Oddzielenie komponentów poprzez mechanizmy równoważenia obciążenia ułatwia skalowanie programów, które składają się na usługę, ponieważ zwiększenie rozmiaru (a co za tym idzie, możliwości) programu odbywa się bez zmiany lub rekonfiguracji którejkolwiek z pozostałych warstw usługi.

Oddzielenie serwerów za pomocą interfejsów API ułatwia skalowanie zespołów programistycznych, ponieważ każdy zespół może skupić się na pojedynczej, mniejszej *mikrousłudze* o zrozumiałym zakresie. Zwięzłe i rzeczowe interfejsy API pomiędzy mikrousługami ograniczają komunikację między zespołami wymaganą do budowania i wdrażania oprogramowania. Narzut komunikacyjny jest często głównym czynnikiem ograniczającym skalowanie zespołów.

Łatwe skalowanie aplikacji i klastrów

Jeśli zachodzi konieczność skalowania usługi, niemutowalny, deklaratywny charakter Kubernetes sprawia, że takie skalowanie staje się trywialne do zaimplementowania. Ponieważ kontenery są niemutowalne, a liczba replik to po prostu liczba w konfiguracji deklaratywnej, skalowanie usługi w górę jest jedynie kwestią zmiany liczby w pliku konfiguracyjnym i wskazania w ten sposób nowego deklaratywnego stanu — Kubernetes zajmie się resztą. Alternatywnie możesz ustawić automatyczne skalowanie i pozwolić, żeby Kubernetes zrobił to za Ciebie.

Oczywiście w przypadku tego rodzaju skalowania zakłada się, że w klastrze są dostępne zasoby do wykorzystania. Czasami konieczne jest skalowanie w górę samego klastra. Kubernetes ułatwia również to zadanie. Ponieważ wszystkie maszyny w klastrze są identyczne, a same aplikacje są oddzielone od szczegółowych mechanizmów działania maszyn przez kontenery, dodanie kolejnych zasobów do klastra sprowadza się do utworzenia obrazu nowej maszyny i dołączenia go do klastra. Można to wykonać za pomocą kilku prostych poleceń lub przy użyciu wstępnie skonfigurowanego obrazu maszyny.

Jednym z wyzwań związanych ze skalowaniem zasobów maszyn jest przewidywanie ich wykorzystania. Jeśli pracujesz na infrastrukturze fizycznej, czas pozyskania nowej maszyny mierzony jest w dniach lub tygodniach. Zarówno w przypadku infrastruktury fizycznej, jak i chmurowej przewidywanie przyszłych kosztów jest trudne, ponieważ trudno jest ocenić z góry rozwój i skalowanie potrzeb konkretnych aplikacji.

Kubernetes może uprościć prognozowanie przyszłych kosztów obliczeniowych. Abyś mógł łatwo zrozumieć, dlaczego tak jest, rozważmy skalowanie trzech zespołów: A, B i C. Z doświadczenia wiadomo, że rozwój każdego zespołu jest bardzo zmienny, a przez to trudny do przewidzenia. Jeśli zapewniasz poszczególne maszyny dla każdej usługi, musisz prognozować na podstawie maksymalnego oczekiwanego rozwoju każdej usługi, ponieważ maszyny przeznaczone dla jednego

zespołu nie mogą zostać użyte przez inny zespół. Jeśli zamiast tego skorzystasz z Kubernetes do oddzielenia zespołów od konkretnych używanych przez nie maszyn, możesz prognozować rozwój na podstawie łącznego rozwoju wszystkich trzech usług. Połączenie trzech zmiennych wskaźników wzrostu w pojedynczy wskaźnik wzrostu zmniejsza szum statystyczny i zapewnia bardziej wiarygodną prognozę oczekiwanego rozwoju. Ponadto oddzielenie zespołów od konkretnych maszyn oznacza, że zespoły mogą współdzielić ułamkowe części swoich maszyn, przez co zmniejszy się jeszcze bardziej narzut związany z prognozowaniem wzrostu zasobów obliczeniowych.

W końcu Kubernetes umożliwia osiągnięcie automatycznego skalowania (w górę i w dół) zasobów. W szczególności w środowisku chmurowym, w którym nowe maszyny można tworzyć za pośrednictwem interfejsów API, połączenie Kubernetes z automatycznym skalowaniem zarówno dla aplikacji, jak i samych klastrów sprawia, że zawsze można dostosować koszty do aktualnego obciążenia.

Skalowanie zespołów programistycznych za pomocą mikrousług

Jak zauważono w różnych badaniach, idealnym rozmiarem zespołu jest „zespół na dwie pizze" (ang. *two-pizza team*), czyli mniej więcej od sześciu do ośmiu osób, ponieważ ten rozmiar grupy często zapewnia dobrą wymianę wiedzy, szybkie podejmowanie decyzji i zachowanie zdrowego rozsądku w dążeniu do celu. Większe zespoły z reguły cierpią z powodu hierarchiczności, słabej widoczności i konfliktów wewnętrznych, które zmniejszają zwinność działania i prawdopodobieństwo odniesienia sukcesu. Jednak wiele projektów wymaga znacznie większej ilości zasobów, aby odnieść sukces i osiągnąć cele. Dlatego istnieje pewna rozbieżność pomiędzy idealnym rozmiarem zespołu pod kątem zachowania zwinności i niezbędnym rozmiarem zespołu dla osiągnięcia założonych celów projektu.

Typowym rozwiązaniem tego problemu było tworzenie rozłącznych, skoncentrowanych na usługach zespołów, z których każdy buduje jedną mikrousługę. Każdy mały zespół jest odpowiedzialny za zaprojektowanie i dostarczenie usługi, która jest wykorzystywana przez inne małe zespoły. Agregacja wszystkich tych usług ostatecznie zapewnia zaimplementowanie całkowitego zakresu produktu.

Kubernetes udostępnia liczne abstrakcje i interfejsy API, które ułatwiają tworzenie takich rozłącznych architektur mikrousług.

- **Kapsuły** (ang. *pods*), czyli grupy kontenerów, mogą łączyć opracowane przez różne zespoły obrazy kontenerów w pojedynczą nadającą się do wdrożenia jednostkę.
- **Usługi** Kubernetes zapewniają równoważenie obciążenia, nazywanie i wykrywanie w celu izolowania poszczególnych mikrousług.
- **Przestrzenie nazw** zapewniają izolację i kontrolę dostępu, dzięki czemu dla każdej mikrousługi można kontrolować stopień interakcji z innymi usługami.
- **Obiekty `Ingress`** zapewniają łatwy w użyciu frontend, który może łączyć wiele mikrousług w jedną zewnętrzną powierzchnię interfejsu API.

Wreszcie, oddzielenie obrazu kontenera aplikacji i maszyny oznacza, że różne usługi mogą współdziałać na tej samej maszynie bez wzajemnego zakłócania swojej pracy, co zmniejsza narzut i koszty architektur mikrousługowych. Funkcje Kubernetes, takie jak sprawdzanie poprawności działania i wdrażanie nowych wersji oprogramowania, gwarantują spójne podejście do wdrażania aplikacji i niezawodność, która zapewnia, że mnożenie się zespołów mikrousługowych nie spowoduje szybkiego wzrostu liczby różnych podejść do produkcyjnego cyklu życia i działania usług.

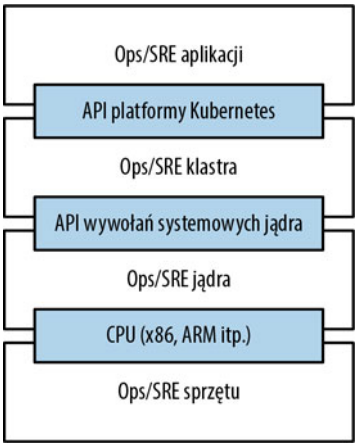
Separacja zagadnień dla zapewnienia spójności i skalowania

Kubernetes wprowadza spójność do działania, a rozłączność i separacja zagadnień oferowana przez stos Kubernetes doprowadziły do znacznego zwiększenia spójności także na niższych poziomach infrastruktury. Dzięki temu można skalować funkcje operacyjne do zarządzania wieloma maszynami za pomocą jednego małego zespołu skoncentrowanego na konkretnych zadaniach. Mówiliśmy już o oddzielaniu kontenera aplikacji i maszyny (lub systemu operacyjnego), ale ważnym aspektem tej rozłączności jest to, że API orkiestracji kontenerów staje się rzeczowym kontraktem oddzielającym odpowiedzialność operatora aplikacji od odpowiedzialności operatora orkiestracji klastra. Taki podział obowiązków określa się często powiedzeniem: „nie moja małpa, nie mój cyrk". Programista aplikacji opiera się na umowie **SLA** (ang. *service-level agreement*) dostarczanej przez interfejs API orkiestracji kontenera i nie martwi się o szczegóły dotyczące tego, jak powstaje ta umowa. Podobnie inżynier do spraw niezawodności API orkiestracji kontenera skupia się na dostarczaniu umowy SLA interfejsów API orkiestracji bez przejmowania się korzystającymi z niej aplikacjami.

Ta separacja zagadnień oznacza, że niewielki zespół pracujący z wykorzystaniem klastra Kubernetes może być odpowiedzialny za obsługę setek, a nawet tysięcy zespołów obsługujących aplikacje w ramach tego klastra (zobacz rysunek 1.1). Podobnie niewielki zespół może być odpowiedzialny za kilkadziesiąt (lub więcej) klastrów działających na całym świecie. Należy zauważyć, że to samo oddzielenie kontenerów i systemu operacyjnego umożliwia inżynierom do spraw niezawodności systemu operacyjnego skoncentrowanie się na umowach SLA systemu operacyjnego poszczególnych komputerów. Staje się kolejną linią rozdzielania odpowiedzialności, przy czym operatorzy Kubernetes polegają na SLA systemu operacyjnego, a operatorzy systemu operacyjnego martwią się wyłącznie o dostarczanie tej umowy SLA. Ponownie pozwala to na przypisanie niewielkiego zespołu ekspertów do spraw systemu operacyjnego do floty tysięcy maszyn.

Oczywiście tylko nieliczne organizacje mogą sobie pozwolić na oddelegowanie choćby małego zespołu do zarządzania systemem operacyjnym. W tych środowiskach doskonałym wyborem jest Kubernetes jako usługa (ang. *Kubernetes-as-a-Service* — KaaS) w chmurze publicznej. Usługa KaaS stawała się bardziej dostępna wraz ze wzrostem popularności platformy Kubernetes i dziś jest już oferowana w prawie każdej chmurze publicznej. Oczywiście KaaS ma pewne ograniczenia wynikające z faktu, że operator za nas podejmuje decyzje dotyczące budowy i konfiguracji klastrów Kubernetes. Na wielu platformach na przykład wyłączone są funkcje alfa, które mogą destabilizować zarządzany klaster.

Oprócz w pełni zarządzanej usługi Kubernetes istnieje także kwitnący ekosystem firm i projektów pomagających w instalacji Kubernetes i zarządzaniu nim. Między „rzuceniem się na głęboką wodę" a skorzystaniem z w pełni zarządzanej usługi znajduje się szeroki wachlarz rozwiązań pośrednich.



Rysunek 1.1. Sposób oddzielania różnych zespołów operacyjnych za pomocą interfejsów API

Decyzję, czy używać KaaS, czy też zarządzać klastrem Kubernetes samodzielnie (lub skorzystać z rozwiązania pośredniego), każdy użytkownik musi podjąć na podstawie własnych umiejętności i wymagań w danej sytuacji. Dla małych organizacji KaaS często zapewnia łatwe w obsłudze rozwiązanie, które pozwala im poświęcić czas i energię na budowanie oprogramowania wspierającego ich działalność, a nie na zarządzanie klastrem. W przypadku większej organizacji, która może sobie pozwolić na dedykowany zespół do zarządzania własnym klastrem Kubernetes, samodzielne zarządzanie nim może mieć sens, ponieważ zapewnia większą elastyczność pod względem możliwości i działania klastra.

Zapewnianie abstrakcji infrastruktury

Celem chmury publicznej jest oferowanie programistom łatwej w użyciu, samoobsługowej infrastruktury. Interfejsy API chmur koncentrują się jednak zbyt często na odzwierciedlaniu infrastruktury oczekiwanej przez dział IT, a nie na koncepcjach (na przykład „maszyny wirtualne" zamiast „aplikacji"), z których chcą korzystać programiści. Ponadto w wielu przypadkach chmura jest dostarczana z wieloma szczegółami implementacji lub usług, które są charakterystyczne dla dostawcy chmury. Bezpośrednie korzystanie z tych interfejsów API utrudnia uruchamianie aplikacji w wielu środowiskach lub rozpraszanie jej na chmurę i środowiska fizyczne.

Przejście na interfejsy API kontenerów ukierunkowanych na aplikacje, takich jak Kubernetes, zapewnia dwie konkretne korzyści. Po pierwsze, jak już wcześniej pisaliśmy, oddziela programistów od konkretnych maszyn. Nie tylko ułatwia to realizowanie zadań IT dotyczących maszyn, ponieważ w celu skalowania klastra można po prostu dodawać i agregować maszyny, ale także, w kontekście chmury, zapewnia wysoki stopień przenośności, ponieważ programiści używają interfejsu API wyższego poziomu, który jest implementowany pod kątem określonych interfejsów API infrastruktury chmury.

Gdy programiści budują aplikacje pod kątem obrazów kontenerów i wdrażają je z uwzględnieniem przenośnych interfejsów API Kubernetes, przenoszenie aplikacji między środowiskami lub nawet uruchamianie jej w środowiskach hybrydowych jest po prostu kwestią przesłania deklaratywnej konfiguracji do nowego klastra. Kubernetes ma wiele wtyczek, które pozwalają abstrahować od konkretnej chmury. Usługi Kubernetes umożliwiają na przykład tworzenie mechanizmów równoważenia obciążenia na wszystkich głównych chmurach publicznych, a także w kilku różnych infrastrukturach prywatnych i fizycznych. W podobny sposób można używać klas `PersistentVolumes` i `PersistentVolumeClaims` Kubernetes do oddzielania aplikacji od konkretnych implementacji pamięci masowej. Oczywiście aby osiągnąć tę przenośność, należy unikać usług zarządzanych w chmurze (na przykład Amazon DynamoDB, Azure CosmosDB lub Google Cloud Spanner), co oznacza, że trzeba wdrożyć rozwiązanie pamięci masowej typu open source, takie jak Cassandra, MySQL lub MongoDB, i zarządzać nim.

Można podsumować, że bazowanie na ukierunkowanych na aplikacje abstrakcjach Kubernetes gwarantuje, iż wysiłek wkładany w budowanie, wdrażanie aplikacji i zarządzanie nimi zapewni naprawdę przenośne (między wieloma różnymi środowiskami) rozwiązanie.

Wydajność

Oprócz korzyści dla programistów i obszaru zarządzania IT, jakie oferują kontenery i Kubernetes, z tą abstrakcją wiąże się również konkretna korzyść ekonomiczna. Ponieważ programiści będą mogli przestać myśleć w kategoriach maszyn, ich aplikacje będą mogły współużytkować te same maszyny bez negatywnego wpływu na same aplikacje. Oznacza to, że zadania zlecane przez wielu użytkowników mogą być ciasno upakowane na mniejszej liczbie komputerów.

Wydajność może być mierzona jako stosunek użytecznej pracy wykonywanej przez maszynę lub proces do całkowitej ilości zużytej energii. Jeśli chodzi o wdrażanie aplikacji oraz zarządzanie nimi, wiele dostępnych narzędzi i procesów (na przykład skrypty *bash*, aktualizacje `apt` lub imperatywne zarządzanie konfiguracją) jest w jakiś sposób nieefektywnych. Podczas analizowania efektywności warto pomyśleć zarówno o kosztach działania serwera, jak i kosztach ludzkich wymaganych do zarządzania nim.

Uruchomienie serwera wiąże się z kosztem opartym na zużyciu energii, wymaganiach dotyczących chłodzenia, przestrzeni centrów danych i surowej mocy obliczeniowej. Gdy serwer zostanie zamontowany w szafie rackowej i włączony, dosłownie zaczyna bić licznik. Każdy czas bezczynności procesora to zmarnowane pieniądze. Dlatego częścią obowiązków administratora systemu staje się utrzymywanie wykorzystania serwera na akceptowalnym poziomie, co wymaga ciągłego zarządzania. Tutaj do gry wchodzą kontenery i przepływ pracy Kubernetes. Kubernetes zapewnia narzędzia, które automatyzują dystrybucję aplikacji w klastrze maszyn, pozwalając utrzymać wyższy poziom wykorzystania, niż jest to możliwe w przypadku tradycyjnych narzędzi.

Dalsze zwiększenie wydajności wynika z faktu, że środowisko testowe programisty może być szybko i tanio utworzone jako zestaw kontenerów działających w osobistym widoku udostępnionego klastra Kubernetes przy użyciu funkcjonalności **przestrzeni nazw**. W przeszłości utworzenie klastra testowego dla programisty mogło oznaczać uruchomienie trzech maszyn. W przypadku Kubernetes

prościej jest, aby wszyscy programiści współużytkowali pojedynczy klaster testowy, co pozwala agregować wykorzystanie mocy obliczeniowej na mniejszym zestawie maszyn. Zmniejszenie ogólnej liczby używanych maszyn zwiększa z kolei wydajność każdego systemu: ponieważ wykorzystywanych jest więcej zasobów (CPU, RAM itp.) na każdym komputerze, zmniejsza się całkowity koszt każdego kontenera.

Obniżenie kosztów instancji deweloperskich w stosie umożliwia prace rozwojowe, które wcześniej mogły być zbyt kosztowne. W przypadku aplikacji wdrożonej za pomocą Kubernetes możliwe staje się na przykład zaimplementowanie i przetestowanie wszystkich poprawek przekazanych przez każdego programistę w całym stosie.

Gdy koszt każdego wdrożenia jest mierzony w kategoriach niewielkiej liczby kontenerów zamiast wielu kompletnych maszyn wirtualnych, koszty testów są znacznie niższe. Wracając do pierwotnej wartości Kubernetes, te zwiększone możliwości testowania powodują również zwiększenie prędkości, ponieważ mamy zarówno silne sygnały co do niezawodności kodu, jak i dużą liczbę szczegółowych informacji potrzebnych do szybkiej identyfikacji miejsca, w którym mógł zostać wprowadzony błąd.

W końcu, jak zostało napisane wcześniej, automatyczne skalowanie, aby dodawać i usuwać zasoby zgodnie z zapotrzebowaniem, można wykorzystać w celu zapewnienia ogólnej wydajności aplikacji przy jednoczesnym zachowaniu ich wymaganych parametrów.

Ekosystem o pochodzeniu chmurowym

Kubernetes z definicji jest rozszerzalnym środowiskiem wspieranym przez szeroką i życzliwą społeczność. Te jego charakterystyczne cechy i powszechność w tak wielu środowiskach obliczeniowych doprowadziły do powstania dużego i energicznego ekosystemu narzędzi i usług, które wyrosły na gruncie Kubernetes. Wzorem Kubernetes (a także Dockera i Linuksa wcześniej) większość tych projektów również jest typu open source. To znaczy, że programista rozpoczynający pracę nie musi wszystkiego robić od początku. Od czasu pojawienia się Kubernetes dla platformy tej powstały narzędzia pozwalające wykonywać niemal wszystkie możliwe zadania z różnych dziedzin, od uczenia maszynowego, przez ciągły rozwój, po modele programowania bezserwerowego. W większości przypadków problemem jest nie tyle znalezienie potencjalnego rozwiązania, ile wybranie odpowiedniego spośród dużej liczby dostępnych. Bogactwo narzędzi w ekosystemie chmurowym samo w sobie stało się dla wielu osób wystarczającym powodem, aby zacząć pracować z Kubernetes. Podczas korzystania z ekosystemu chmurowego można używać stworzonych i wspieranych przez społeczność projektów w prawie każdej części swojego systemu, dlatego decydujące się na to osoby mogą skupić się na swojej podstawowej działalności i usługach, które stanowią ich unikalną ofertę.

Jak w przypadku każdego ekosystemu open source, największe wyzwanie stanowi tu obfitość możliwych rozwiązań oraz to, że często brakuje integracji od a do z. Jednym ze sposobów, by poradzić sobie z tymi problemami, jest korzystanie z wytycznych technicznych Cloud Native Computing Foundation (CNCF). CNCF jest niczym neutralna centralna platforma dla kodu i własności intelektualnej wszystkich projektów chmurowych. Określa trzy poziomy dojrzałości projektów, co

ułatwia ich adaptację. Większość projektów w CNCF znajduje się w fazie **piaskownicy** (ang. *sandbox*), a to oznacza, że wciąż są na wczesnym etapie rozwoju i ich adaptacja jest niezalecana, chyba że jest się wczesnym użytkownikiem lub osobą zainteresowaną rozwijaniem danego projektu. Następny etap na drodze do dojrzałości to **inkubacja** (ang. *incubating*). Projekty znajdujące się w tej fazie wykazały już swoją przydatność i stabilność w środowisku produkcyjnym. Nadal jednak trwają nad nimi prace i rosną skupione wokół nich społeczności. Podczas gdy projektów w fazie piaskownicy są setki, projektów w fazie inkubacji jest zaledwie nieco ponad 20. Ostatni poziom to **absolwent** (ang. *graduated*). Projekty znajdujące się w tej fazie są dojrzałe i szeroko rozpowszechnione. Jest ich tylko kilka i należy do nich na przykład sam Kubernetes.

Innym sposobem, by poruszać się po ekosystemie chmurowym, jest integracja z Kubernetes jako usługą (ang. *Kubernetes as a Service*). Obecnie większość ofert KaaS ma także dodatkowe usługi w postaci projektów open source z ekosystemu chmurowego. Jako że te usługi są zintegrowane z produktami obsługującymi chmurę, można mieć pewność, że są dojrzałe i gotowe do użycia w produkcji.

Podsumowanie

Kubernetes został stworzony w celu radykalnej zmiany sposobu budowania i wdrażania aplikacji w chmurze. Zasadniczo został zaprojektowany tak, aby zaoferować programistom większą prędkość, wydajność i zwinność. Obecnie bazuje na nim wiele internetowych usług i aplikacji używanych na co dzień. Prawdopodobnie też korzystasz z Kubernetes, tylko o tym nie wiesz! Mamy nadzieję, że ten rozdział dał Ci pewne wyobrażenie, dlaczego warto wdrażać aplikacje za pomocą Kubernetes. Teraz, gdy jesteś już o tym przekonany, w kolejnych rozdziałach dowiesz się, *jak* wdrożyć aplikację.

ROZDZIAŁ 2.
Tworzenie i uruchamianie kontenerów

Kubernetes to platforma do tworzenia i wdrażania rozproszonych aplikacji oraz zarządzania nimi. Te aplikacje mają wiele różnych kształtów i rozmiarów, ale ostatecznie wszystkie składają się z jednego programu lub kilku **programów** działających na poszczególnych maszynach. Akceptują dane wejściowe, manipulują danymi, a następnie zwracają wyniki. Zanim w ogóle będziemy mogli zacząć myśleć nad budową rozproszonego systemu, musimy najpierw zastanowić się, jak budować **obrazy kontenerów aplikacji**, które zawierają te programy i są elementami naszego rozproszonego systemu.

Aplikacje zazwyczaj składają się ze środowiska wykonawczego dla języka programowania, bibliotek i kodu źródłowego. W wielu przypadkach aplikacja opiera się na zewnętrznych bibliotekach, takich jak `libc` i `libssl`. Te zewnętrzne biblioteki są zwykle dostarczane jako współdzielone komponenty w systemie operacyjnym zainstalowanym na konkretnym komputerze.

Problemy pojawiają się, gdy aplikacja opracowana na laptopie programisty jest zależna od współdzielonej biblioteki, która nie jest dostępna po wdrożeniu programu w produkcyjnym systemie operacyjnym. Nawet wtedy, gdy środowiska programistyczne i produkcyjne współdzielą dokładnie taką samą wersję systemu operacyjnego, mogą wystąpić problemy, gdy programiści zapomną załączyć zależne pliki zasobów we wdrażanym do produkcji pakiecie.

Tradycyjne metody wykonywania wielu programów na jednym komputerze wymagają, aby wszystkie te programy wykorzystywały te same wersje bibliotek współdzielonych dostępnych w systemie. Jeśli programy te zostały utworzone przez różne zespoły lub organizacje, takie wspólne zależności tylko niepotrzebnie zwiększają poziom złożoności i zacieśniają powiązania między tymi zespołami.

Program może zostać pomyślnie wykonany tylko wtedy, gdy można go niezawodnie wdrożyć na maszynie, na której powinien działać. Zbyt często nowoczesne techniki wdrażania polegają na uruchamianiu skryptów imperatywnych, które ze swojej natury charakteryzują się zawiłymi przypadkami awarii.

To sprawia, że wdrożenie nowej wersji całości lub części systemu rozproszonego staje się pracochłonnym i trudnym zadaniem.

W rozdziale 1. stanowczo opowiedzieliśmy się za wartością niemutowalnych obrazów i niemutowalnej infrastruktury. Okazuje się, że dokładnie to zapewnia obraz kontenera. Jak zobaczymy, łatwo rozwiązuje on wszystkie opisane powyżej problemy związane z zarządzaniem zależnościami i hermetyzacją.

Podczas pracy z aplikacjami często pomocne jest spakowanie ich w sposób ułatwiający dzielenie się nimi z innymi. Docker, czyli dla większości domyślne narzędzie kontenerowe, ułatwia pakowanie aplikacji i przesyłanie ich do zdalnego rejestru, skąd mogą być później pobierane przez innych użytkowników. W czasie pisania tej książki wszystkie najważniejsze chmury publiczne umożliwiały korzystanie z rejestrów kontenerów, a wiele z nich pozwalało także na budowanie obrazów w chmurze. Użytkownik mógł też prowadzić własny rejestr przy użyciu systemów open source lub komercyjnych. Rejestry takie ułatwiają zarządzanie prywatnymi obrazami i ich wdrażanie, podczas gdy usługi budowania obrazów upraszczają integrację z systemami ciągłej dostawy.

W tym rozdziale i w pozostałej części książki będziemy pracować z prostą przykładową aplikacją, którą zbudowaliśmy dla celów tej książki, aby pokazać przepływ pracy w działaniu. Możesz pobrać tę aplikację w formie archiwum z serwera FTP wydawnictwa Helion, pod adresem *https://ftp.helion.pl/przyklady/kuber3.zip*.

Obrazy kontenerów upakowują program i jego zależności w głównym systemie plików jako pojedynczy artefakt. Najpopularniejszym formatem obrazów kontenerów jest format obrazu Dockera, który został ustandaryzowany przez organizację Open Container Initiative jako format obrazu OCI. Kubernetes obsługuje zarówno obrazy Dockera, jak i OCI przez środowisko wykonawcze Dockera i inne. Obrazy Dockera zawierają również dodatkowe metadane używane przez środowisko wykonawcze kontenera do uruchamiania instancji działającej aplikacji na podstawie zawartości obrazu kontenera.

Ten rozdział obejmuje następujące tematy:

- pakowanie aplikacji za pomocą formatu obrazu Dockera,
- uruchamianie aplikacji przy użyciu środowiska wykonawczego kontenera Dockera.

Obrazy kontenerów

Prawie wszyscy po raz pierwszy stykamy się z technologią kontenerową poprzez obraz kontenera. **Obraz kontenera** jest pakietem binarnym, który hermetyzuje wszystkie pliki niezbędne do uruchomienia aplikacji w kontenerze systemu operacyjnego. W zależności od tego, w jaki sposób zaczynamy eksperymentować z kontenerami, budujemy obraz kontenera z lokalnego systemu plików lub pobieramy istniejący obraz z **rejestru kontenerów**. W obu przypadkach, gdy obraz kontenera będzie już obecny na komputerze, można go uruchomić w celu utworzenia działającej aplikacji w kontenerze systemu operacyjnego.

Najpopularniejszym i najbardziej rozpowszechnionym formatem obrazów kontenerów jest format obrazu Dockera, który został opracowany przez open source'owy projekt Docker do pakowania, dystrybucji i uruchamiania kontenerów za pomocą polecenia `docker`. Później firma Docker, Inc. i inne organizacje rozpoczęły prace mające na celu standaryzację formatu obrazu kontenera za pośrednictwem projektu Open Container Initiative (OCI). Chociaż zestaw standardów OCI w 2017 r. został opublikowany jako standard 1.0, przyjmowanie tych standardów nadal postępuje bardzo powoli. Format obrazu Dockera nadal jest de facto standardem i składa się z wielu warstw systemu plików. Każda warstwa dodaje, usuwa lub modyfikuje pliki z poprzedniej warstwy

w systemie plików. Jest to przykład **nakładkowego** systemu plików. System taki jest wykorzystywany zarówno przy pakowaniu obrazu, jak i podczas jego wykorzystywania. W czasie działania istnieje wiele różnych konkretnych implementacji takich systemów plików, w tym: `aufs`, `overlay` i `overlay2`.

Warstwy kontenera

Określenia „format obrazów Dockera" i „obrazy kontenerów" mogą być mylące. Obraz nie jest pojedynczym plikiem, lecz specyfikacją pliku manifestu określającą inne pliki. Użytkownicy często traktują manifest i związane z nim pliki jako moduł. To pozwala na efektywniejsze przechowywanie i przesyłanie. Z formatem tym powiązany jest interfejs API do wysyłania i pobierania obrazów do rejestru.

Obrazy kontenerów są zbudowane z wielu warstw systemu plików, gdzie każda warstwa dziedziczy i modyfikuje warstwy ją poprzedzające. Aby lepiej to wyjaśnić, zbudujmy kilka kontenerów. Zwróć uwagę, że poprawna kolejność warstw to od dołu do góry, ale dla ułatwienia zastosujemy odwrotne podejście:

```
.
└── kontener A: tylko bazowy system operacyjny, taki jak Debian
    └── kontener B: zbudowany na #A poprzez dodanie Ruby v2.1.10
    └── kontener C: zbudowany na #A poprzez dodanie Golang v1.6
```

W tym momencie mamy trzy kontenery: A, B i C. B i C są rozgałęzieniem z A i nie współdzielą niczego poza plikami bazowego kontenera. Idąc dalej, możemy budować na bazie B, dodając Ruby on Rails (wersję 4.2.6). Możemy również zapewnić obsługę starszej aplikacji, która wymaga starszej wersji Ruby on Rails (na przykład wersji 3.2.x). Możemy zbudować obraz obsługujący tę aplikację również na bazie B, planując migrację aplikacji do wersji 4 pewnego dnia:

```
. (kontynuacja poprzedniego listingu)
└── kontener B: zbudowany na #A poprzez dodanie Ruby v2.1.10
    └── kontener D: zbudowany na #B poprzez dodanie Rails v4.2.6
    └── kontener E: zbudowany na #B poprzez dodanie Rails v3.2.x
```

Koncepcyjnie każda warstwa obrazu kontenera opiera się na poprzedniej. Każda referencja do warstwy nadrzędnej jest wskaźnikiem. Ten przykład obrazuje prosty zestaw kontenerów, inne kontenery w prawdziwym świecie mogą stanowić część większego i rozległego skierowanego grafu acyklicznego.

Obrazy kontenerów są zwykle połączone z plikiem konfiguracyjnym kontenera, który zawiera instrukcje konfiguracji środowiska kontenera i wykonania punktu wejścia aplikacji. Konfiguracja kontenera często obejmuje informacje na temat sposobu konfiguracji sieci, izolacji przestrzeni nazw, ograniczania zasobów (cgroups) oraz ograniczeń `syscall`, które należy nałożyć na uruchomioną instancję kontenera. Główny system plików kontenera i plik konfiguracyjny są zazwyczaj spakowane za pomocą formatu obrazu Dockera.

Kontenery dzielą się na dwie główne kategorie:

- kontenery systemowe,
- kontenery aplikacji.

Kontenery systemowe próbują naśladować maszyny wirtualne i często uruchamiają proces pełnego rozruchu. Zazwyczaj zawierają zestaw usług systemowych zwykle znajdujących się w maszynie wirtualnej, takich jak `ssh`, `cron` i `syslog`. Ten typ kontenerów był znacznie popularniejszy w czasach, gdy Docker był jeszcze nowością. Z czasem korzystanie z takich kontenerów zaczęto postrzegać jako złą praktykę i przewagę zyskały kontenery aplikacji.

Kontenery aplikacji różnią się od kontenerów systemowych pod tym względem, że zazwyczaj uruchamiają pojedynczą aplikację. Chociaż uruchamianie pojedynczej aplikacji na kontener może się wydawać niepotrzebnym ograniczeniem, takie podejście zapewnia doskonały poziom szczegółowości do komponowania skalowalnych aplikacji i jest rozwiązaniem szeroko wykorzystywanym przez kapsuły. Szczegółowy opis kapsuł zamieściliśmy w rozdziale 5.

Budowanie obrazów aplikacji za pomocą Dockera

Ogólnie rzecz biorąc, systemy orkiestracji kontenerów, takie jak Kubernetes, koncentrują się na budowaniu oraz wdrażaniu systemów rozproszonych składających się z kontenerów aplikacji. Dlatego w pozostałej części rozdziału skupimy się na kontenerach aplikacji.

Pliki Dockerfile

Plik *Dockerfile* może być używany do zautomatyzowania tworzenia obrazu kontenera Dockera.

Zaczniemy od budowy obrazu aplikacji dla prostego programu Node.js. Przykład ten wyglądałby bardzo podobnie także w wielu innych dynamicznych językach programowania, takich jak Python czy Ruby.

Prosta aplikacja npm/Node/Express składa się z dwóch plików: *package.json* (listing 2.1) i *server.js* (listing 2.2). Umieść je w dowolnym katalogu, po czym wykonaj polecenie `npm install express --save`, aby ustanowić zależność od aplikacji Express i ją zainstalować.

Listing 2.1. Plik package.json

```
{
  "name": "simple-node",
  "version": "1.0.0",
  "description": "Przykładowa aplikacja do książki Kubernetes. Tworzenie
  ↪niezawodnych systemów rozproszonych",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "author": ""
}
```

Listing 2.2. Plik server.js

```
var express = require('express');

var app = express();
app.get('/', function (req, res) {
  res.send('Witaj, świecie!');
});
app.listen(3000, function () {
  console.log('Listening on port 3000!');
  console.log(' http://localhost:3000');
});
```

Aby spakować ten program jako obraz Dockera, musimy utworzyć dwa dodatkowe pliki: *.dockerignore* (listing 2.3) i *Dockerfile* (listing 2.4). Plik *Dockerfile* zawiera instrukcje na temat budowy obrazu kontenera, natomiast *.dockerignore* określa, które pliki należy zignorować podczas kopiowania plików do obrazu. Kompletny opis składni pliku *Dockerfile* znajduje się na stronach Dockera (*https://dockr.ly/2XUanvl*).

Listing 2.3. Plik .dockerignore

```
node_modules
```

Listing 2.4. Plik Dockerfile

```
# Zacznij od obrazu Node.js 16 (LTS) ❶
FROM node:16

# Określenie katalogu w obrazie, w którym będą wykonywane wszystkie polecenia ❷
WORKDIR /usr/src/app

# Skopiowanie plików pakietu i instalacja zależności ❸
COPY package*.json ./
RUN npm install
RUN npm install express

# Skopiowanie wszystkich plików aplikacji do obrazu ❹
COPY . .

# Domyślne polecenie uruchomienia kontenera ❺
CMD [ "npm", "start" ]
```

❶ Każdy plik *Dockerfile* bazuje na innych obrazach kontenerów. W tym wierszu informujemy, że zaczynamy od obrazu `node:16` z Docker Hub. Jest to gotowa konfiguracja obrazu z systemem Node.js 16.

❷ W tym wierszu ustawiamy katalog roboczy w obrazie kontenera dla wszystkich następnych poleceń.

❸ Te trzy wiersze inicjalizują zależności dla Node.js. Najpierw kopiujemy pliki pakietu, między innymi *package.json* i *package-lock.json*, do obrazu. Następnie za pomocą polecenia `RUN` wykonujemy w kontenerze instalację potrzebnych zależności.

❹ Teraz kopiujemy pozostałe pliki programu do obrazu. Pomijamy tylko plik *node_modules*, który wykluczyliśmy w pliku *.dockerignore*.

❺ Na koniec określamy polecenie, które ma zostać wykonane po uruchomieniu kontenera.

Wykonaj poniższe polecenie, aby utworzyć obraz *simple-node* Dockera:

```
$ docker build -t simple-node .
```

Obraz ten możesz uruchomić za pomocą poniższego polecenia. Program działający w tym kontenerze jest dostępny pod adresem *http://localhost:3000*:

```
$ docker run --rm -p 3000:3000 simple-node
```

Teraz obraz *simple-node* znajduje się w lokalnym rejestrze Dockera, w którym został zbudowany, i jest dostępny tylko na jednym komputerze. Jednak największą zaletą Dockera jest możliwość udostępniania obrazów na tysiącach komputerów szerokiej społeczności skupionej wokół tej technologii.

Optymalizacja rozmiarów obrazu

Gdy zaczynamy eksperymentować z obrazami kontenera, pojawia się kilka pułapek, które prowadzą do powstawania zbyt dużych obrazów. Pierwszą rzeczą, którą należy zapamiętać, jest to, że pliki usuwane przez kolejne warstwy w systemie w rzeczywistości są nadal obecne w obrazach; są po prostu niedostępne. Rozważmy następującą sytuację:

```
.
└── warstwa A: zawiera duży plik o nazwie 'BigFile'
    └── warstwa B: usuwa 'BigFile'
        └── warstwa C: opiera się na B, dodając statyczny plik binarny
```

Możesz uznać, że *BigFile* nie jest już obecny w tym obrazie. W końcu kiedy uruchomisz obraz, nie będzie już dostępny. Jednak w rzeczywistości wciąż istnieje w warstwie A, co oznacza, że za każdym razem, gdy wysyłasz lub pobierasz obraz, *BigFile* nadal jest transmitowany przez sieć, nawet jeśli nie masz już do niego dostępu.

Kolejna pułapka dotyczy buforowania i budowania obrazu. Pamiętaj, że każda warstwa jest niezależnym rozgałęzieniem dla warstw znajdujących się poniżej. Za każdym razem, kiedy zmienisz jakąś warstwę, zmieni ona każdą warstwę następującą po niej. Zmiana kolejnych warstw oznacza, że należy je od nowa zbudować, ponownie wysłać do rejestru i pobrać, aby wdrożyć obraz do środowiska programistycznego.

Abyś lepiej to zrozumiał, rozważmy dwa obrazy:

```
.
└── warstwa A: zawiera bazowy system operacyjny
    └── warstwa B: dodaje kod źródłowy server.js
        └── warstwa C: instaluje pakiet 'node'
```

oraz:

```
.
└── warstwa A: zawiera bazowy system operacyjny
    └── warstwa B: instaluje pakiet 'node'
        └── warstwa C: dodaje kod źródłowy server.js
```

Wydaje się oczywiste, że oba te obrazy będą zachowywać się identycznie, i rzeczywiście po pierwszym ich pobraniu tak będzie. Zastanów się jednak, co się stanie, gdy zmieni się plik *server.js*. W jednym przypadku wysłania lub pobrania wymaga tylko sama zmiana, ale w drugim trzeba przesłać i pobrać zarówno *server.js*, jak i warstwę dostarczającą pakiet `node`, ponieważ warstwa `node` jest zależna od warstwy *server.js*. Na ogół najlepiej jest porządkować warstwy od tych z najmniejszym do tych z największym prawdopodobieństwem zmiany, aby zoptymalizować rozmiar obrazu do przesyłania i pobierania. Dlatego w przykładzie na listingu 2.4 kopiujemy pliki *package*.json* i instalujemy zależności przed skopiowaniem pozostałych plików programu. Programista będzie aktualizował i zmieniał pliki programu znacznie częściej niż zależności.

Bezpieczeństwo obrazu

Jeśli chodzi o bezpieczeństwo, nie ma dróg na skróty. Podczas budowania obrazów, które zostaną ostatecznie uruchomione w produkcyjnym klastrze Kubernetes, należy przestrzegać najlepszych praktyk dotyczących pakowania i dystrybucji aplikacji. Nie powinno się na przykład budować kontenerów z wbudowanym hasłem — i dotyczy to nie tylko ostatniej warstwy, ale każdej warstwy obrazu. Jednym z nielogicznych problemów wprowadzanych przez warstwy kontenerów jest to, że usunięcie pliku w jednej warstwie nie powoduje usunięcia danego pliku z poprzednich warstw. Nadal zajmuje on miejsce i dostęp do niego może uzyskać każdy, kto ma odpowiednie narzędzia — przedsiębiorczy haker może po prostu utworzyć obraz składający się wyłącznie z warstw, które zawierają hasło.

Haseł i obrazów *nigdy* nie należy łączyć. Jeśli to zrobisz, zostaniesz zhakowany i Ty przyniesiesz wstyd całej firmie lub wydziałowi. Wszyscy chcemy kiedyś wystąpić w telewizji, ale są na to lepsze sposoby.

Ponadto, jako że obrazy kontenerów mają uruchamiać indywidualne aplikacje, najlepszą praktyką jest minimalizowanie liczby plików w kontenerze. Każda dodatkowa biblioteka w obrazie stanowi potencjalne źródło luk bezpieczeństwa w aplikacji. Niektóre języki pozwalają uzyskać bardzo małe obrazy z mocno ograniczonym zbiorem zależności. Taki obraz nie jest podatny na ataki, na które są narażone biblioteki, których on nie używa.

Wieloetapowe budowanie obrazów

Przypadkowa budowa dużych obrazów najczęściej zdarza się, gdy kompilacja programu jest wykonywana w ramach procesu budowy obrazu kontenera aplikacji. Kompilacja kodu podczas budowania obrazu wydaje się czymś naturalnym i jest najprostszym sposobem na zbudowanie obrazu kontenera z programu. Problem polega na tym, że po niej pozostają wszystkie niepotrzebne narzędzia programistyczne, które w większości są bardzo duże i zalegając w kontenerze, niepotrzebnie spowalniają procesy wdrażania.

W celu rozwiązania tego problemu w Dockerze wprowadzono **wieloetapowe procesy budowy**. Polegają one na tym, że zamiast pojedynczego obrazu plik Dockera może utworzyć kilka obrazów. Utworzenie każdego z nich stanowi pewien etap, a elementy z poprzednich etapów mogą być kopiowane do bieżącego etapu.

W ramach konkretnego przykładu przeanalizujemy budowę aplikacji *kuard*. Jest to dość skomplikowany program wykorzystujący bibliotekę React.js do budowy interfejsu użytkownika (z własnym procesem budowy), który jest włączony w program Go. Program Go jest wykonywany przez serwer API, z którym współpracuje interfejs napisany przy użyciu React.js.

Zawartość prostego pliku *Dockerfile* może wyglądać tak:

```
FROM golang:1.17-alpine

# Instalacja Node i NPM
RUN apk update && apk upgrade && apk add --no-cache git nodejs bash npm

# Pobranie zależności dla składnika Go
RUN go get -u github.com/jteeuwen/go-bindata/...
RUN go get github.com/tools/godep
RUN go get github.com/kubernetes-up-and-running/kuard

WORKDIR /go/src/github.com/kubernetes-up-and-running/kuard

# Skopiowanie wszystkich źródeł
COPY . .

# Zestaw zmiennych wymaganych przez skrypt budujący
ENV VERBOSE=0
ENV PKG=github.com/kubernetes-up-and-running/kuard
ENV ARCH=amd64
ENV VERSION=test

# Budowa — ten skrypt należy do źródeł przychodzących.
RUN build/build.sh

CMD [ "/go/bin/kuard" ]
```

Ten plik *Dockerfile* tworzy obraz kontenera zawierający statyczny plik wykonywalny, a dodatkowo wszystkie narzędzia programistyczne języka Go oraz narzędzia potrzebne do budowy interfejsu w React.js i kod źródłowy aplikacji. To wszystko w ostatecznej wersji aplikacji jest niepotrzebne. Cały obraz w sumie zajmuje aż 500 MB.

Poniżej natomiast znajduje się treść wieloetapowego pliku *Dockerfile*:

```
# ETAP 1: budowa
FROM golang:1.17-alpine AS build

# Instalacja Node i NPM
RUN apk update && apk upgrade && apk add --no-cache git nodejs bash npm

# Pobranie zależności dla części w Go
RUN go get -u github.com/jteeuwen/go-bindata/...
RUN go get github.com/tools/godep

WORKDIR /go/src/github.com/kubernetes-up-and-running/kuard

# Skopiowanie wszystkich źródeł
COPY . .
```

```
# Zestaw zmiennych wymaganych przez skrypt budujący
ENV VERBOSE=0
ENV PKG=github.com/kubernetes-up-and-running/kuard
ENV ARCH=amd64
ENV VERSION=test

# Budowa — ten skrypt należy do źródeł przychodzących.
RUN build/build.sh

# ETAP 2: wdrożenie
FROM alpine

USER nobody:nobody
COPY --from=build /go/bin/kuard /kuard

CMD [ "/kuard" ]
```

Ten plik *Dockerfile* tworzy dwa obrazy. Pierwszy to obraz roboczy zawierający kompilator Go, narzędzia potrzebne React.js i kod źródłowy programu. Drugi to obraz wdrożeniowy, który zawiera tylko skompilowany plik binarny. Wieloetapowa budowa kontenera może spowodować zmniejszenie rozmiaru obrazu kontenera nawet o kilkaset megabajtów, a tym samym radykalne zwiększenie prędkości wdrażania, która w znacznej mierze zależy od przepustowości sieci. Obraz uzyskany dzięki temu plikowi *Dockerfile* zajmuje tylko około 20 MB.

Te skrypty są obecne w repozytorium `kuard` na GitHub (*https://oreil.ly/6c9MX*). Poniżej znajdują się polecenia umożliwiające budowę i uruchomienie tego obrazu:

```
# Uwaga: w systemie Windows może być konieczne poprawienie zakończeń wierszy za pomocą:
# --config core.autocrlf=input
$ git clone https://github.com/kubernetes-up-and-running/kuard
$ cd kuard
$ docker build -t kuard .
$ docker run --rm -p 8080:8080 kuard
```

Przechowywanie obrazów w zdalnym rejestrze

Co komu po obrazie kontenera, jeśli jest on dostępny tylko na jednym komputerze?

Kubernetes opiera się na fakcie, że obrazy opisane w manifeście kapsuły są dostępne na każdym komputerze w klastrze. Jedną z opcji udostępnienia obrazu na wszystkich komputerach w klastrze będzie wyeksportowanie obrazu `kuard` i zaimportowanie go na nich wszystkich. Nie można wyobrazić sobie niczego bardziej nużącego niż taki sposób zarządzania obrazami Dockera. W proces ręcznego importowania i eksportowania obrazów Dockera wpisany jest błąd ludzki. Po prostu powiedz nie!

Standardem w społeczności Dockera jest przechowywanie obrazów Dockera w zdalnym rejestrze. Istnieje mnóstwo opcji, jeśli chodzi o rejestry Dockera, i to, co wybierzesz, zależy w dużej mierze od Twoich potrzeb dotyczących bezpieczeństwa i zasad współpracy.

Ogólnie rzecz biorąc, pierwszą decyzją, jaką należy podjąć w odniesieniu do rejestru, jest użycie rejestru prywatnego lub publicznego. Rejestry publiczne umożliwiają pobieranie przechowywanych w nich obrazów każdemu, rejestry prywatne natomiast wymagają uwierzytelnienia w celu

pobrania obrazu. Wybierając między publicznym i prywatnym rejestrem, warto wziąć pod uwagę konkretny przypadek użycia.

Publiczne rejestry doskonale nadają się do dzielenia się obrazami ze światem, ponieważ umożliwiają łatwe korzystanie z obrazów kontenerów bez konieczności uwierzytelniania się. Możesz bez problemu dystrybuować swoje oprogramowanie jako obraz kontenera i mieć pewność, że wszyscy użytkownicy w dowolnym miejscu na świecie będą mieli dokładnie takie samo doświadczenie.

Prywatne repozytorium z kolei najlepiej nadaje się do przechowywania aplikacji, które są prywatne dla Twojej usługi i nie chcesz, aby ktoś inny ich używał. Ponadto prywatne repozytoria często zapewniają lepszą dostępność i dają gwarancję bezpieczeństwa, ponieważ są specjalnie dostosowane do potrzeb użytkownika i do jego obrazów, a nie przeznaczone dla wszystkich.

Bez względu na wspomniane powyżej kwestie, aby przesłać obraz, musisz uwierzytelnić się w rejestrze. Zasadniczo możesz to zrobić za pomocą polecenia `docker login`, choć istnieją pewne różnice przy niektórych rejestrach. W przykładach przedstawionych w tej książce przesyłamy obrazy do rejestru Google Cloud Platform, zwanego Google Container Registry (GCR), choć istnieją też inne usługi w chmurze, takie jak na przykład Azure i Amazon Web Services (AWS), które także zawierają rejestry kontenerów. Dla nowych użytkowników hostujących publicznie dostępne obrazy doskonałym miejscem na początek jest Docker Hub (*https://hub.docker.com*).

Po zalogowaniu się możesz oznakować obraz `kuard`, dodając prefiks do docelowego rejestru Dockera. Możesz też dodać kolejny identyfikator, który najczęściej wykorzystuje się do określenia wersji lub wariantu obrazu i wstawia się po dwukropku (`:`):

```
$ docker tag kuard gcr.io/kuar-demo/kuard-amd64:blue
```

Następnie możesz przesłać obraz `kuard`:

```
$ docker push gcr.io/kuar-demo/kuard-amd64:blue
```

Teraz gdy obraz `kuard` jest dostępny w zdalnym rejestrze, czas go wdrożyć za pomocą Dockera. Kiedy wysłaliśmy obraz do GCR, został on oznaczony jako publiczny, dzięki czemu będzie dostępny wszędzie bez konieczności uwierzytelniania się.

Środowisko wykonawcze kontenera Dockera

Kubernetes udostępnia interfejs API do opisywania wdrożenia aplikacji, ale w kwestii konfigurowania kontenera aplikacji przy użyciu charakterystycznych dla kontenera interfejsów API natywnych dla docelowego systemu operacyjnego opiera się na środowisku wykonawczym kontenera. W systemie Linux oznacza to konfigurowanie cgroups i przestrzeni nazw. Interfejs do tego środowiska wykonawczego kontenerów jest zdefiniowany w standardzie Container Runtime Interface (CRI). API CRI implementuje kilka różnych programów, w tym `containerd-cri` zbudowany przez Dockera i implementacja `cri-o` autorstwa Red Hat. Wraz z narzędziami Dockera instalowane jest także środowisko `containerd`, które wykorzystuje demon Dockera.

Od Kubernetes 1.25 można używać tylko środowisk kontenerowych obsługujących CRI. Na szczęście dostawcy zarządzanych usług Kubernetes praktycznie zautomatyzowali proces przechodzenia na nowe rozwiązania.

Uruchamianie kontenerów za pomocą Dockera

Choć generalnie w Kubernetes kontenery wykonuje się za pomocą demona o nazwie `kubelet`, który działa na każdym węźle, na początek łatwiej jest posługiwać się wierszem poleceń Dockera. Do wdrażania kontenerów można używać narzędzia CLI Dockera. Aby wdrożyć kontener z obrazu `gcr.io/kuar-demo/kuard-amd64:blue`, wykorzystaj następujące polecenie:

```
$ docker run -d --name kuard \
  --publish 8080:8080 \
  gcr.io/kuar-demo/kuard-amd64:blue
```

To polecenie uruchamia kontener `kuard` i mapuje port 8080 na lokalnym komputerze na 8080 w kontenerze. Opcję `--publish` można skrócić do postaci `-p`. To przekierowanie jest konieczne, ponieważ każdy kontener otrzymuje swój własny adres IP, więc nasłuchiwanie na *lokalnym hoście* wewnątrz kontenera nie powoduje nasłuchiwania na danej maszynie. Bez przekierowania portu połączenia będą niedostępne dla Twojej maszyny. Opcja `-d` oznacza, że kontener ma działać w tle (demon), a `--name kuard` nadaje kontenerowi łatwą do zapamiętania nazwę.

Odkrywanie aplikacji kuard

Kontener `kuard` udostępnia prosty interfejs sieciowy, który można załadować, wpisując w przeglądarce adres *http://localhost:8080*, lub za pomocą wiersza poleceń:

```
$ curl http://localhost:3000
```

`kuard` udostępnia również wiele interesujących funkcji, które omówimy w dalszej części książki.

Ograniczanie wykorzystania zasobów

Docker umożliwia aplikacjom korzystanie z mniejszej ilości zasobów używanych przez aplikacje, udostępniając bazową technologię cgroups dostarczaną przez jądro Linuksa. Z funkcji tych korzysta także Kubernetes w celu ograniczania ilości zasobów używanych przez każdą kapsułę.

Ograniczanie zasobów pamięci

Jedną z kluczowych zalet uruchamiania aplikacji w kontenerze jest możliwość ograniczenia wykorzystania zasobów. Pozwala to na współistnienie wielu aplikacji na tym samym sprzęcie i zapewnia uczciwy podział zasobów.

Aby ograniczyć `kuard` do 200 MB pamięci i 1 GB partycji wymiany, z poleceniem `docker run` użyj flag `--memory` i `--memory-swap`.

Zatrzymaj i usuń bieżący kontener `kuard`:

```
$ docker stop kuard
$ docker rm kuard
```

Następnie uruchom kolejny kontener `kuard`, używając odpowiednich flag, aby ograniczyć wykorzystanie pamięci:

```
$ docker run -d --name kuard \
  --publish 8080:8080 \
```

```
    --memory 200m \
    --memory-swap 1G \
    gcr.io/kuar-demo/kuard-amd64:blue
```

Jeśli program znajdujący się w kontenerze zacznie wykorzystywać zbyt dużo pamięci, to zostanie zamknięty.

Ograniczanie zasobów procesora

Kolejnym kluczowym zasobem na komputerze jest procesor. Ogranicz użycie procesora za pomocą flagi `-cpu-shares` z poleceniem `docker run`:

```
$ docker run -d --name kuard \
  --publish 8080:8080 \
  --memory 200m \
  --memory-swap 1G \
  --cpu-shares 1024 \
  gcr.io/kuar-demo/kuard-amd64:blue
```

Czyszczenie

Po zakończeniu budowania obrazu możesz go usunąć za pomocą polecenia `docker rmi`:

```
docker rmi <nazwa_znacznika>
```

Lub w ten sposób:

```
docker rmi <id_obrazu>
```

Obrazy można usuwać za pomocą nazwy znacznika (na przykład `gcr.io/kuar--demo/kuard-amd64:blue`) lub poprzez identyfikator obrazu. Podobnie jak w przypadku wszystkich wartości identyfikatorów w narzędziu `docker`, identyfikator obrazu można skrócić, o ile pozostanie unikatowy. Zwykle niezbędne są tylko trzy lub cztery znaki identyfikatora.

Należy pamiętać, że jeśli nie usuniesz obrazu bezpośrednio, pozostanie on w Twoim systemie na zawsze, *nawet* jeżeli zbudujesz nowy obraz o identycznej nazwie. Zbudowanie takiego nowego obrazu po prostu powoduje przeniesienie znacznika do nowego obrazu; stary obraz nie zostaje usunięty ani zastąpiony.

Dlatego w wyniku powtarzania czynności podczas tworzenia nowego obrazu często uzyskujesz bardzo wiele różnych obrazów, które ostatecznie zajmują niepotrzebnie miejsce na komputerze. Aby zobaczyć obrazy aktualnie znajdujące się na Twoim komputerze, możesz użyć polecenia `docker images`. Następnie możesz usunąć znaczniki, których już nie używasz.

Docker zawiera ogólne narzędzie czyszczące o nazwie `docker system prune`. Usuwa ono wszystkie zatrzymane kontenery, nieoznakowane obrazy oraz nieużywane warstwy obrazów nagromadzone w buforze w ramach procesu budowy. Korzystaj z niego ostrożnie.

Nieco bardziej wyrafinowanym podejściem jest ustawienie zadania `cron` do uruchamiania mechanizmu czyszczenia niepotrzebnych obrazów. Na przykład polecenie `docker system prune` możesz uruchomić jako cykliczne zadanie `cron` raz dziennie lub raz na godzinę, w zależności od tego, ile obrazów tworzysz.

Podsumowanie

Kontenery aplikacji zapewniają czystą abstrakcję dla aplikacji, a aplikacje spakowane w formacie obrazu Dockera stają się łatwe do kompilowania, wdrażania i dystrybuowania. Kontenery gwarantują również izolację między aplikacjami działającymi na tym samym komputerze, co pomaga uniknąć konfliktów zależności.

W następnych rozdziałach zobaczysz, jak możliwość montowania zewnętrznych katalogów pozwala na uruchamianie w kontenerze nie tylko aplikacji bezstanowych, ale także takich jak `mysql` i inne, które generują dużą ilość danych.

ROZDZIAŁ 3.

Wdrażanie klastra Kubernetes

Ponieważ udało Ci się już zbudować kontener aplikacji, Twoim kolejnym krokiem będzie nauczenie się, jak go zamienić w kompletny, niezawodny i skalowalny system rozproszony. Oczywiście w tym celu potrzebujesz działającego klastra Kubernetes. W tym momencie w większości chmur publicznych dostępne są oparte na chmurze usługi Kubernetes, które ułatwiają tworzenie klastra za pomocą kilku instrukcji wiersza poleceń. Gorąco polecamy takie podejście, jeśli dopiero zaczynasz swoją przygodę z Kubernetes. Nawet jeżeli ostatecznie planujesz uruchomić Kubernetes na „gołym metalu", dobrze jest zacząć pracować z tym systemem, poznać jego działanie, a dopiero potem uczyć się, jak zainstalować go na fizycznych maszynach. Ponadto zarządzanie klastrem Kubernetes samo w sobie jest skomplikowane i większość ludzi woli zdać się w tej kwestii na chmurę — zwłaszcza wziąwszy pod uwagę, że w większości chmur usługa zarządzania jest bezpłatna.

Oczywiście korzystanie z rozwiązania chmurowego wymaga uiszczenia opłaty za zasoby w chmurze oraz posiadania aktywnego połączenia sieciowego z chmurą. Z tych powodów postawienie na działanie lokalne może być bardziej atrakcyjne, a w takim przypadku narzędzie `minikube` zapewnia łatwy sposób uruchomienia lokalnego klastra Kubernetes na maszynie wirtualnej na lokalnym laptopie lub komputerze stacjonarnym. Chociaż jest to kuszące, `minikube` tworzy tylko jednowęzłowy klaster, który nie pozwala wykorzystać wszystkich możliwości kompletnego klastra Kubernetes. Dlatego zalecamy, byś zaczął od rozwiązania chmurowego, chyba że naprawdę nie jest ono odpowiednie w Twojej sytuacji. Niedawno pojawiło się rozwiązanie Docker-in-Docker, które pozwala na uruchomienie klastra wielu węzłów na jednej maszynie. Ale ten projekt jest jeszcze w fazie beta, więc jeśli będziesz z niego korzystać, możesz spodziewać się nieoczekiwanych problemów.

Jeżeli chcesz zacząć od gołego metalu, dodatek A znajdujący się na końcu tej książki zawiera instrukcje dotyczące budowania klastra na bazie zestawu jednopłytowych komputerów Raspberry Pi. W tych instrukcjach wykorzystano narzędzie `kubeadm` i mogą one być zastosowane również do innych maszyn niż Raspberry Pi.

Instalowanie Kubernetes w usłudze dostawcy publicznej chmury

Ten rozdział opisuje instalację Kubernetes dla trzech głównych dostawców usług w chmurze: Amazon Web Services, Microsoft Azure oraz Google Cloud Platform.

Jeśli do zarządzania klastrem Kubernetes wybierzesz dostawcę chmury, będziesz musiał zainstalować tylko jedną z tych opcji. Gdy skonfigurujesz już klaster i będziesz mieć wszystko gotowe, możesz pominąć podrozdział „Klient Kubernetes" w dalszej części rozdziału, chyba że chcesz zainstalować Kubernetes gdzieś indziej.

Google Kubernetes Engine

Google Cloud Platform oferuje hostowaną usługę **KaaS** (ang. *Kubernetes-as-a-Service*) o nazwie Google Kubernetes Engine (GKE). Aby zacząć korzystać z GKE, potrzebujesz konta Google Cloud Platform z włączoną opcją naliczania płatności oraz zainstalowanego narzędzia `gcloud` (*https://oreil.ly/uuUQD*).

Po zainstalowaniu `gcloud` najpierw ustaw domyślną strefę:

```
$ gcloud config set compute/zone us-west1-a
```

Następnie możesz utworzyć klaster:

```
$ gcloud container clusters create kuar-cluster --num-nodes=3
```

To zajmie kilka minut. Gdy klaster będzie gotowy, możesz uzyskać poświadczenia dla klastra za pomocą następującego polecenia:

```
$ gcloud container clusters get-credentials kuar-cluster
```

W razie problemów kompletne instrukcje dotyczące tworzenia klastra GKE możesz znaleźć w dokumentacji Google Cloud Platform (*https://oreil.ly/HMwnD*).

Instalowanie Kubernetes w Azure Kubernetes Service

Microsoft Azure oferuje hostowaną usługę KaaS w ramach Azure Container Service. Najprostszym sposobem na rozpoczęcie korzystania z usługi Azure Container Service jest użycie wbudowanej powłoki Azure Cloud Shell w portalu Azure. Powłokę możesz aktywować poprzez kliknięcie jej ikony w pasku narzędzi w prawym górnym rogu:

Powłoka ma automatycznie zainstalowane i skonfigurowane narzędzie `az` do pracy ze środowiskiem Azure.

Alternatywnie możesz zainstalować interfejs wiersza poleceń (CLI) narzędzia `az` na lokalnym komputerze (*https://oreil.ly/xpLCa*).

Po uruchomieniu powłoki możesz wpisać następujące polecenie:

```
$ az group create --name=kuar --location=westus
```

Po utworzeniu grupy zasobów możesz utworzyć klaster:

```
$ az aks create --resource-group=kuar --name=kuar-cluster
```

Zajmie to kilka minut. Po utworzeniu klastra możesz uzyskać dla niego poświadczenia za pomocą poniższego polecenia:

```
$ az aks get-credentials --resource-group=kuar --name=kuar-cluster
```

Jeżeli nie masz jeszcze zainstalowanego narzędzia `kubectl`, możesz zainstalować je w ten sposób:

```
$ az aks install-cli
```

Kompletne instrukcje instalowania Kubernetes na platformie Azure można znaleźć w dokumentacji Azure *(https://oreil.ly/hsLWA*).

Instalowanie Kubernetes w Amazon Web Services

Amazon oferuje zarządzaną usługę Kubernetes o nazwie Elastic Kubernetes Service (EKS). Najprostszym sposobem na utworzenie klastra EKS jest użycie narzędzia wiersza poleceń open source `eksctl` (*https://eksctl.io*).

Gdy je zainstalujesz i dodasz do ścieżki, możesz utworzyć klaster za pomocą następującego polecenia:

```
$ eksctl create cluster
```

Szersze informacje na temat opcji instalacji (na przykład rozmiaru węzła itd.) znajdziesz w pomocy, którą możesz wyświetlić przy użyciu polecenia `help`:

```
$ eksctl create cluster --help
```

Instalacja klastra zawiera odpowiednią konfigurację narzędzia wiersza poleceń `kubectl`. Jeśli jeszcze go nie zainstalowałeś, możesz to zrobić zgodnie z instrukcjami zawartymi w dokumentacji (*https://oreil.ly/rorrD*).

Lokalna instalacja Kubernetes za pomocą minikube

Jeśli wystarczy Ci programowanie lokalne lub nie chcesz płacić za zasoby w chmurze, możesz zainstalować prosty jednowęzłowy klaster za pomocą narzędzia `minikube`. Ewentualnie, jeśli zainstalowałeś już Docker Desktop, jest on dołączony do instalacji Kubernetes dla jednego komputera.

`minikube` (lub Docker Desktop) jest dobrą symulacją klastra Kubernetes, ale tak naprawdę jest przeznaczony do lokalnego programowania, uczenia się i eksperymentowania. Ponieważ działa tylko na maszynie wirtualnej na jednym węźle, nie zapewnia niezawodności rozproszonego klastra Kubernetes. Ponadto niektóre funkcje opisane w tej książce wymagają integracji z dostawcą chmury. W `minikube` te funkcje są niedostępne lub działają w ograniczonym zakresie.

Aby korzystać z narzędzia `minikube`, musisz mieć zainstalowanego na swoim komputerze hipernadzorcę. Dla systemów Linux i macOS jest to na ogół `virtualbox` (*https://virtualbox.org*). W systemie Windows domyślną opcją jest hipernadzorca `Hyper-V`. Przed użyciem `minikube` upewnij się, że zainstalowałeś hipernadzorcę.

Narzędzie `minikube` możesz znaleźć na GitHubie (*https://oreil.ly/iHcuV*). Dostępne są do pobrania pliki binarne dla systemów Linux, macOS i Windows. Po zainstalowaniu narzędzia `minikube` możesz utworzyć lokalny klaster za pomocą następującego polecenia:

```
$ minikube start
```

Spowoduje to utworzenie lokalnej maszyny wirtualnej, dostarczenie Kubernetes i utworzenie lokalnej konfiguracji `kubectl`, wskazującej na ten klaster. Jak zostało wspomniane wcześniej, klaster ten ma tylko jeden węzeł, więc choć jest przydatny, różni się od większości produkcyjnych wdrożeń Kubernetes.

Po zakończeniu pracy z klastrem można zatrzymać maszynę wirtualną w ten sposób:

```
$ minikube stop
```

Jeśli chcesz usunąć klaster, możesz uruchomić poniższe polecenie:

```
$ minikube delete
```

Uruchamianie Kubernetes w Dockerze

Niedawno pojawił się nowy sposób na uruchomienie klastra Kubernetes, który polega na wykorzystaniu kontenerów Docker do symulacji węzłów Kubernetes zamiast uruchamiania wszystkiego w maszynie wirtualnej. Projekt kind (*https://kind.sigs.k8s.io/*) bardzo ułatwia uruchamianie klastrów testowych w Dockerze i zarządzanie nimi (*kind* to skrót od słów *Kubernetes IN Docker*). Projekt ten nie jest jeszcze ukończony (nie ma jeszcze wersji 1.0), ale powszechnie używają go wszyscy, którzy potrzebują szybkiej metody budowania klastrów Kubernetes do celów testowych.

Instrukcje instalacji dla swojej platformy znajdziesz w witrynie projektu kind (*https://oreil.ly/EOgJn*). Po zainstalowaniu narzędzia klaster utworzysz za pomocą następujących poleceń:

```
$ kind create cluster --wait 5m \
$ export KUBECONFIG="$(kind get kubeconfig-path)"
$ kubectl cluster-info
$ kind delete cluster
```

Klient Kubernetes

Oficjalnym klientem Kubernetes jest `kubectl`: narzędzie wiersza poleceń służące do interakcji z interfejsem API Kubernetes. Narzędzie `kubectl` może być używane do zarządzania większością obiektów Kubernetes, takich jak kapsuły, kontrolery replik `ReplicaSet` i usługi. Można się nim również posługiwać do eksploracji i weryfikacji ogólnego stanu zdrowia klastra.

Użyjemy narzędzia `kubectl` do eksploracji właśnie utworzonego klastra.

Sprawdzanie statusu klastra

Pierwszą rzeczą, którą możesz zrobić, jest sprawdzenie uruchomionej wersji klastra:

```
$ kubectl version
```

Zostaną wyświetlone dwie różne wersje: wersja lokalnego narzędzia `kubectl` oraz wersja serwera API Kubernetes.

Nie przejmuj się, jeśli te wersje są różne. Narzędzia Kubernetes są kompatybilne wstecz i w przód z różnymi wersjami interfejsu API Kubernetes, o ile pozostaniesz przy dwóch numeracjach dodatkowych (ang. *minor versions*) wersji narzędzi oraz klastra i nie będziesz próbować używać nowszych funkcji na starszym klastrze. Kubernetes stosuje się do ogólnie przyjętego systemu numeracji oprogramowania i tę wersję dodatkową oznacza środkowa liczba (na przykład 18 w wersji 1.18.2). Staraj się tylko nie wychodzić poza obsługiwany zakres, który obejmuje trzy wersje. Jeśli tego nie dopilnujesz, możesz mieć kłopoty.

Skoro ustaliliśmy już, że możesz komunikować się z klastrem Kubernetes, zbadamy ten klaster bardziej szczegółowo.

Po pierwsze, możemy przeprowadzić prostą diagnostykę dla klastra. Jest to dobry sposób, aby sprawdzić, czy klaster jest generalnie zdrowy:

```
$ kubectl get componentstatuses
```

Dane wyjściowe powinny wyglądać tak:

```
NAME                 STATUS   MESSAGE              ERROR
scheduler            Healthy  ok
controller-manager   Healthy  ok
etcd-0               Healthy  {"health": "true"}
```

Kubernetes jest cały czas ulepszany i zmieniany, w związku z czym wyniki użycia polecenia `kubectl` też mogą się zmieniać. Dlatego nie przejmuj się, jeśli to, co zobaczysz, nie będzie wyglądało dokładnie tak samo jak w przykładzie.

Możesz zobaczyć tutaj komponenty, które składają się na klaster Kubernetes. Komponent `controller-manager` jest odpowiedzialny za uruchamianie różnych kontrolerów, które regulują zachowanie w klastrze: na przykład zapewniają, że wszystkie repliki usługi są dostępne i zdrowe. Komponent `scheduler` jest odpowiedzialny za umieszczanie różnych kapsuł w różnych węzłach w klastrze. Serwer `etcd` jest natomiast magazynem dla klastra, gdzie przechowywane są wszystkie obiekty API.

Wyświetlanie węzłów roboczych klastra Kubernetes

Następnie możemy wyświetlić listę wszystkich węzłów w naszym klastrze:

```
$ kubectl get nodes
NAME        STATUS        ROLES                          AGE       VERSIONS
kube0       Ready         control-plane,master           45d       v1.22.4
kube1       Ready         <none>                         45d       v1.22.4
kube2       Ready         <none>                         45d       v1.22.4
kube3       Ready         <none>                         45d       v1.22.4
```

Widać, że jest to klaster z czterema węzłami, który działa od 45 dni. W Kubernetes węzły są podzielone na węzły główne `control-plane` zawierające kontenery, takie jak serwer API, planista itp., które zarządzają klastrem, i węzły robocze (`worker`), w których będą działać kontenery. Kubernetes zazwyczaj nie planuje pracy na węzłach `control-plane`, aby zapewnić, że obciążenia robocze generowane przez użytkownika nie będą wpływać negatywnie na ogólne działanie klastra.

Aby uzyskać więcej informacji o konkretnym węźle, takim jak `kube1`, możesz użyć polecenia `kubectl describe`:

```
$ kubectl describe nodes kube1
```

Najpierw otrzymujemy podstawowe informacje o węźle:

```
Name:               kube1
Role:
Labels:             beta.kubernetes.io/arch=arm
                    beta.kubernetes.io/os=linux
                    kubernetes.io/hostname=node-1
```

Widać, że ten węzeł działa pod kontrolą systemu operacyjnego Linux i na procesorze ARM.

Następnie widzimy informacje o działaniu samego węzła `kube1` (daty zostały usunięte dla uproszczenia):

```
Conditions:
Type                 Status   ...   Reason                        Message
-----                ------         ------                        -------
NetworkUnavailable   False     ...   FlannelIsUp                   Flannel...
MemoryPressure       False     ...   KubeletHasSufficientMemory    kubelet...
DiskPressure         False     ...   KubeletHasNoDiskPressure      kubelet...
PIDPressure          False     ...   KubeletHasSufficientPID       kubelet...
Ready                True      ...   KubeletReady                  kubelet...
```

Te statusy pokazują, że węzeł ma wystarczającą ilość miejsca przestrzeni dyskowej i w pamięci i zgłasza węzłowi głównemu Kubernetes, że jest zdrowy. Następnie wyświetlane są informacje o parametrach maszyny:

```
Capacity:
 alpha.kubernetes.io/nvidia-gpu:       0
 cpu:                                  4
 memory:                               882636Ki
 pods:                                 110
Allocatable:
 alpha.kubernetes.io/nvidia-gpu:       0
 cpu:                                  4
 memory:                               882636Ki
 pods:                                 110
```

Dalej znajdują się informacje o oprogramowaniu w węźle, w tym między innymi o uruchomionej wersji Dockera, wersji Kubernetes i jądra Linuksa:

```
System Info:
 Machine ID:                 44d8f5dd42304af6acde62d233194cc6
 System UUID:                c8ab697e-fc7e-28a2-7621-94c691120fb9
 Boot ID:                    e78d015d-81c2-4876-ba96-106a82da263e
 Kernel Version:             4.19.0-18-amd64
 OS Image:                   Debian GNU/Linux 10 (buster)
 Operating System:           linux
```

```
  Architecture:                  amd64
  Container Runtime Version:     containerd://1.4.12
  Kubelet Version:               v1.22.4
  Kube-Proxy Version:            v1.22.4
PodCIDR:                         10.244.1.0/24
PodCIDRs:                        10.244.1.0/24
```

Na koniec mamy informacje o kapsułach, które aktualnie działają na tym węźle:

```
Non-terminated Pods:              (3 in total)
  Namespace   Name      CPU Requests CPU Limits Memory Requests Memory Limits
  ---------   ----      ------------ ---------- --------------- -------------
  kube-system kube-dns… 260m (6%)    0 (0%)     140Mi (16%)     220Mi (25%)
  kube-system kube-fla… 0 (0%)       0 (0%)     0 (0%)          0 (0%)
  kube-system kube-pro… 0 (0%)       0 (0%)     0 (0%)          0 (0%)
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.
  CPU Requests  CPU Limits    Memory Requests Memory Limits
  ------------  ----------    --------------- -------------
  260m (6%)     0 (0%)        140Mi (16%)     220Mi (25%)
No events.
```

Z tych danych wyjściowych możemy odczytać informacje o kapsułach w węźle (na przykład kapsule `kube-dns`, która dostarcza usługi DNS dla klastra), procesorze i pamięci, których żąda z węzła każda kapsuła, a także o całkowitych żądanych zasobach. Warto zauważyć, że Kubernetes śledzi zarówno *żądanie*, jak i górny *limit* zasobów dla każdej kapsuły działającej na danej maszynie. Różnica pomiędzy żądaniami i limitami została opisana szczegółowo w rozdziale 5., ale mówiąc w skrócie, obecność zasobów *żądanych* przez kapsułę jest gwarantowana w węźle, limit kapsuły jest natomiast maksymalną ilością danego zasobu, którą kapsuła może konsumować. Limit kapsuły może być większy niż jego żądanie, a w takim przypadku dodatkowe zasoby są dostarczane z zachowaniem należytej staranności. Ich obecność w węźle nie jest gwarantowana.

Komponenty klastra

Jednym z interesujących aspektów Kubernetes jest to, że wiele komponentów, które tworzą klaster Kubernetes, jest w rzeczywistości wdrażanych za pomocą samego Kubernetes. Przyjrzymy się kilku z nich. Te komponenty korzystają z wielu koncepcji, które przedstawimy w kolejnych rozdziałach. Wszystkie te komponenty działają w przestrzeni nazw `kube-system`[1].

Serwer proxy Kubernetes

Serwer proxy Kubernetes jest odpowiedzialny za routing ruchu sieciowego do usług w klastrze Kubernetes, dla których włączony jest mechanizm równoważenia obciążenia. Aby wykonać swoją pracę, serwer proxy musi być obecny na każdym węźle w klastrze. Do tego celu Kubernetes ma obiekt API o nazwie `DaemonSet` (poznasz go w rozdziale 11.), który jest używany w wielu klastrach. Jeżeli Twój klaster uruchamia serwer proxy Kubernetes za pomocą obiektu `DaemonSet`, możesz wyświetlić serwery proxy przy użyciu następującego polecenia:

[1] Jak dowiesz się w kolejnym rozdziale, przestrzeń nazw w Kubernetes jest jednostką do organizowania zasobów w tym systemie. Możesz ją potraktować jak folder w systemie plików.

```
$ kubectl get daemonSets --namespace=kube-system kube-proxy
NAME         DESIRED    CURRENT    READY    UP-TO-DATE   AVAILABLE   NODE   SELECTOR
kube-proxy   5          5          5        5            5           ...    45d
```

W zależności od konfiguracji klastra obiekt `DaemonSet` kontenera `kube-proxy` może mieć inną nazwę. Istnieje też możliwość, że obiekt ten w ogóle nie będzie używany. Niezależnie od tego kontener `kube-proxy` powinien działać na wszystkich węzłach klastra.

Serwer DNS Kubernetes

Kubernetes uruchamia również serwer DNS, który zapewnia nazywanie i wykrywanie dla usług zdefiniowanych w klastrze. Ten serwer DNS również działa w klastrze jako zreplikowana usługa. W zależności od wielkości klastra może w nim działać jeden serwer DNS lub kilka. Usługa DNS jest uruchamiana jako wdrożenie Kubernetes, które zarządza tymi replikami (może mieć nazwę `coredns` lub inną):

```
$ kubectl get deployments --namespace=kube-system core-dns
NAME        DESIRED    CURRENT    UP-TO-DATE    AVAILABLE    AGE
core-dns    1          1          1             1            45d
```

Istnieje również usługa Kubernetes, która wykonuje zadania równoważenia obciążenia dla serwera DNS:

```
$ kubectl get services --namespace=kube-system core-dns
NAME        CLUSTER-IP    EXTERNAL-IP    PORT(S)        AGE
core-dns    10.96.0.10    <none>         53/UDP,53/TCP  45d
```

To pokazuje, że usługa DNS dla klastra ma adres 10.96.0.10. Jeżeli zalogujesz się do kontenera w klastrze, zobaczysz, że został on umieszczony w pliku */etc/resolv.conf* dla tego kontenera.

Interfejs użytkownika Kubernetes

Jeśli chcesz przedstawić wizualnie swój klaster w graficznym interfejsie użytkownika, możesz wykorzystać narzędzia, które większość dostawców chmury udostępnia w swoim GUI. Jeśli Twój dostawca tego nie robi lub wolisz GUI w klastrze, istnieje wspierany przez społeczność GUI, który możesz zainstalować. Sprawdź w dokumentacji (*https://oreil.ly/wKfEx*), jak zainstalować pulpit dla tych klastrów. Ponadto są dostępne rozszerzenia do środowisk programistycznych, takich jak Visual Studio Code, pozwalające sprawdzić stan klastra.

Podsumowanie

Mamy nadzieję, że na tym etapie masz już uruchomiony klaster Kubernetes (lub trzy klastry) i użyłeś kilku poleceń do eksploracji utworzonego klastra. W następnej kolejności poświęcimy trochę więcej czasu na zbadanie interfejsu wiersza poleceń dla tego klastra Kubernetes i nauczysz się posługiwać narzędziem `kubectl`. W pozostałej części książki będziesz używać `kubectl` i swojego klastra testowego do eksploracji różnych obiektów z interfejsu API Kubernetes.

ROZDZIAŁ 4.

Typowe polecenia kubectl

`kubectl` jest potężnym narzędziem wiersza poleceń, a w kolejnych rozdziałach będziesz go używać do tworzenia obiektów interfejsu API Kubernetes i interakcji z nimi. Zanim jednak to nastąpi, warto, żebyś poznał podstawowe polecenia `kubectl`, które mają zastosowanie do wszystkich obiektów Kubernetes.

Przestrzenie nazw

Kubernetes używa **przestrzeni nazw** do porządkowania obiektów w klastrze. Możesz potraktować poszczególne przestrzenie nazw jako foldery zawierające zestawy obiektów. Domyślnie narzędzie wiersza poleceń `kubectl` współdziała z przestrzenią nazw `default`. Jeśli chcesz użyć innej przestrzeni nazw, możesz przekazać narzędziu `kubectl` flagę `--namespace`, na przykład polecenie `kubectl --namespace=mystuff` odwołuje się do obiektów w przestrzeni nazw `mystuff`. Jeżeli wolisz bardziej zwięzły zapis, możesz użyć skróconej flagi `-n`. Jeśli chcesz korzystać ze wszystkich przestrzeni nazw — aby na przykład utworzyć listę wszystkich kapsuł w swoim klastrze — możesz przekazać flagę `--all-namespaces`.

Konteksty

Jeśli chcesz zmienić domyślną przestrzeń nazw w sposób permanentny, możesz użyć **kontekstu**. Zmiana zostanie zapisana w pliku konfiguracyjnym `kubectl`, zwykle zlokalizowanym w *$HOME/.kube/config*. Ten plik konfiguracyjny przechowuje również informacje o tym, jak znaleźć klaster i uwierzytelnić się w nim. Możesz na przykład utworzyć kontekst z inną domyślną przestrzenią nazw dla poleceń `kubectl` w następujący sposób:

```
$ kubectl config set-context my-context --namespace=mystuff
```

Tworzysz tak nowy kontekst, ale tak naprawdę nie zaczynasz go jeszcze używać. Aby skorzystać z tego nowo utworzonego kontekstu, możesz uruchomić następujące polecenie:

```
$ kubectl config use-context my-context
```

Kontekstów można również używać do zarządzania różnymi klastrami lub różnymi użytkownikami w celu uwierzytelniania w tych klastrach, korzystając z flagi `--users` lub `--clusters` z poleceniem `set-context`.

Przeglądanie obiektów interfejsu API Kubernetes

Wszystko, co jest zawarte w Kubernetes, jest reprezentowane przez zasób RESTful. W całej książce odnosimy się do tych zasobów jako **obiektów Kubernetes**. Każdy obiekt Kubernetes istnieje w unikatowej ścieżce HTTP; na przykład *https://your-k8s.com/api/v1/namespaces/default/pods/my-pod* prowadzi do reprezentacji kapsuły w domyślnej przestrzeni nazw o nazwie `my-pod`. Polecenie `kubectl` powoduje wysłanie żądania HTTP do tych adresów URL, aby uzyskać dostęp do obiektów Kubernetes znajdujących się na tych ścieżkach.

Najbardziej podstawowym poleceniem do przeglądania obiektów Kubernetes za pomocą narzędzia `kubectl` jest polecenie `get`. Jeśli uruchomisz `kubectl get <nazwa_zasobu>`, otrzymasz listę wszystkich zasobów w bieżącej przestrzeni nazw. Jeżeli chcesz uzyskać konkretny zasób, możesz użyć `kubectl get <nazwa_zasobu> <nazwa_obiektu>`.

Domyślnie do przeglądania odpowiedzi z serwera API `kubectl` używa czytelnego dla człowieka formatu drukowania, ale w tym formacie, aby zmieścić każdy obiekt w jednej linii terminala, usuniętych zostaje wiele szczegółowych informacji o obiektach. Jednym ze sposobów uzyskania nieco większej ilości informacji jest dodanie flagi `-o wide`, która pozwala otrzymać więcej szczegółów w dłuższej linii. Jeśli chcesz przeglądać całe obiekty, możesz je również wyświetlać jako surowy JSON lub YAML, używając odpowiednio flag `-o json` lub `-o yaml`.

Typową opcją służącą do manipulowania wyjściem `kubectl` jest usuwanie nagłówków, co jest często użyteczne przy łączeniu `kubectl` z uniksowymi potokami (na przykład `kubectl ... | awk ...`). Jeżeli określisz flagę `--no-headers`, `kubectl` pominie nagłówki u góry czytelnej dla człowieka tabeli.

Innym częstym zadaniem jest wyodrębnianie określonych pól z obiektu. Do wybierania pól w zwracanym obiekcie narzędzie `kubectl` używa języka zapytań JSONPath. Szczegółowy opis JSONPath wykracza poza zakres tego rozdziału, ale jako przykład poniższe polecenie spowoduje wyodrębnienie i wydrukowanie adresu IP kapsuły:

```
$ kubectl get pods my-pod -o jsonpath --template={.status.podIP}
```

Przy użyciu listy typów rozdzielanych przecinkami można także wyświetlić wiele obiektów różnego typu, na przykład:

```
$ kubectl get pods,services
```

To polecenie spowoduje wyświetlenie wszystkich kapsuł i usług z danej przestrzeni nazw.

Jeżeli chcesz uzyskać bardziej szczegółowe informacje na temat konkretnego obiektu, skorzystaj z polecenia `describe`:

```
$ kubectl describe <nazwa_zasobu> <nazwa_obiektu>
```

Zapewni ono bogaty, wielowierszowy, czytelny dla człowieka opis obiektu oraz wszelkich innych istotnych powiązanych obiektów i zdarzeń w klastrze Kubernetes.

Jeśli chcesz wyświetlić listę dostępnych pól dla każdego obsługiwanego typu obiektu Kubernetes, możesz użyć polecenia `explain`:

```
$ kubectl explain pods
```

Czasami warto ciągle obserwować stan wybranego zasobu Kubernetes, aby zobaczyć ewentualne zmiany, gdy się pojawią. Możesz na przykład czekać na ponowne uruchomienie aplikacji. Stałą obserwację umożliwia flaga `--watch`. Można ją dodać do każdego polecenia `kubectl get`, aby w sposób ciągły monitorować stan wybranego zasobu.

Tworzenie, aktualizacja i niszczenie obiektów Kubernetes

Obiekty w interfejsie API Kubernetes są reprezentowane jako pliki JSON lub YAML. Te pliki są zwracane przez serwer w odpowiedzi na zapytanie lub przesyłane do serwera jako część żądania API. Tych plików YAML lub JSON możesz używać do tworzenia, aktualizowania lub usuwania obiektów na serwerze Kubernetes.

Załóżmy, że masz prosty obiekt przechowywany w *obj.yaml*. Możesz użyć `kubectl` do utworzenia tego obiektu w Kubernetes, uruchamiając następujące polecenie:

```
$ kubectl apply -f obj.yaml
```

Zwróć uwagę, że nie musisz określać typu zasobu obiektu; jest on uzyskiwany z samego pliku obiektu.

Podobnie po wprowadzeniu zmian w obiekcie można ponownie użyć polecenia `apply`, aby zaktualizować obiekt:

```
$ kubectl apply -f obj.yaml
```

Narzędzie `apply` modyfikuje tylko te obiekty, które różnią się od aktualnych obiektów w klastrze. Jeśli tworzone przez użytkownika obiekty już znajdują się w klastrze, po prostu zakończy działanie bez wprowadzania żadnych zmian. Ta cecha sprawia, że narzędzie to jest przydatne w pętlach zapewniających zgodność stanu klastra ze stanem systemu plików. Stan można uzgadniać za jego pomocą wielokrotnie.

Jeśli chcesz się dowiedzieć, co polecenie `apply` zrobi, nie wprowadzając żadnych zmian, dodaj flagę `--dry-run`, aby wydrukować obiekty w terminalu bez wysyłania ich do serwera.

Jeśli zamiast edytować lokalny plik, chcesz dokonać interaktywnych zmian, możesz użyć polecenia `edit`, które spowoduje pobranie najnowszego stanu obiektu, a następnie uruchomienie edytora zawierającego daną definicję:

```
$ kubectl edit <nazwa_zasobu> <nazwa_obiektu>
```

Po zapisaniu plik zostanie automatycznie przesłany z powrotem do klastra Kubernetes.

Ponadto polecenie `apply` powoduje zapisanie historii poprzednich konfiguracji w adnotacji w obiekcie. Rekordy te można modyfikować i przeglądać za pomocą poleceń `edit-last-applied`, `set-last-applied` i `view-last-applied`. Na przykład:

```
$ kubectl apply -f myobj.yaml view-last-applied
```

To polecenie zwróci informację o ostatnim stanie zastosowanym do obiektu.

Kiedy chcesz usunąć obiekt, możesz po prostu uruchomić następujące polecenie:

```
$ kubectl delete -f obj.yaml
```

Należy jednak pamiętać, że `kubectl` nie poprosi o potwierdzenie usunięcia. Natychmiast po wydaniu polecenia obiekt *zostanie* usunięty.

Podobnie możesz usunąć obiekt przy użyciu typu zasobu i nazwy:

```
$ kubectl delete <nazwa_zasobu> <nazwa_obiektu>
```

Dodawanie etykiet i adnotacji do obiektów

Etykiety i adnotacje to znaczniki dla Twoich obiektów. Różnice między nimi omówimy w rozdziale 6., na razie możesz aktualizować etykiety i adnotacje dowolnego obiektu Kubernetes za pomocą poleceń `annotate` i `label`. Aby dodać na przykład etykietę `color=red` do kapsuły o nazwie `bar`, możesz uruchomić takie polecenie:

```
$ kubectl label pods bar color=red
```

Składnia adnotacji jest identyczna.

Domyślnie polecenia `label` i `annotate` nie pozwalają na nadpisanie istniejącej etykiety. W tym celu musisz dodać flagę `--overwrite`.

Jeśli chcesz usunąć etykietę, możesz użyć składni `<nazwa_etykiety>-`:

```
$ kubectl label pods bar color-
```

Spowoduje to usunięcie etykiety `color` z kapsuły o nazwie `bar`.

Polecenia debugowania

Narzędzie `kubectl` udostępnia także kilka poleceń do debugowania kontenerów. Aby wyświetlić dzienniki uruchomionego kontenera, możesz użyć takiego polecenia:

```
$ kubectl logs <nazwa_kapsuły>
```

Jeśli masz wiele kontenerów w kapsule, możesz wybrać kontener do przeglądania za pomocą flagi `-c`.

Domyślnie polecenie `kubectl logs` wyświetla bieżące dzienniki i kończy działanie. Jeśli zamiast tego chcesz w sposób ciągły przesyłać dzienniki do terminala bez kończenia działania polecenia, możesz dodać flagę `-f` (ang. *follow*) wiersza poleceń.

Możesz także użyć polecenia `exec`, aby wykonać polecenie w uruchomionym kontenerze:

```
$ kubectl exec -it <nazwa_kapsuły> -- bash
```

Zapewni to interaktywną powłokę wewnątrz działającego kontenera, dzięki czemu będziesz mógł wykonać więcej poleceń debugowania.

Jeśli nie masz dostępu do powłoki bash ani żadnego innego terminala w swoim kontenerze, zawsze możesz podpiąć się do działającego procesu:

```
$ kubectl attach -it <nazwa-kapsuły>
```

Polecenie `attach` jest podobne do `kubectl logs`, ale umożliwia wysyłanie danych do działającego procesu przy założeniu, że proces ten może wczytywać dane z wejścia standardowego.

Możesz też kopiować pliki do kontenera i z kontenera za pomocą polecenia `cp`:

```
$ kubectl cp <nazwa_kapsuły>:/ścieżka/do/zdalnego/pliku /ścieżka/do/lokalnego/pliku
```

Spowoduje to skopiowanie pliku z uruchomionego kontenera na lokalny komputer. Możesz również określić katalogi lub odwrócić składnię, aby skopiować plik z lokalnego komputera z powrotem do kontenera.

Jeśli chcesz mieć dostęp do kapsuły przez sieć, możesz przekazywać do niej ruch sieciowy z lokalnego komputera za pomocą polecenia `port-forward`. Umożliwia ono bezpieczne tunelowanie ruchu sieciowego do kontenerów, które mogą być niedostępne publicznie. Na przykład poniższe polecenie:

```
$ kubectl port-forward <nazwa-kapsuły> 8080:80
```

otwiera połączenie przekazujące ruch z komputera lokalnego na porcie 8080 na port 80 kontenera zdalnego.

Polecenia `port-forward` można też używać z usługami i wówczas zamiast nazwy kapsuły należy wpisać parametr `services/<nazwa-usługi>`. Trzeba tylko pamiętać, że w przypadku przekazywania portu do usługi żądania będą przekazywane tylko do jednej kapsuły w tej usłudze. Nie będą przechodzić przez mechanizmy równoważenia obciążenia usługi.

Jeśli chcesz wyświetlić zdarzenia Kubernetes, możesz użyć polecenia `kubectl get events`, które pozwala wyświetlić listę 10 ostatnich zdarzeń na wszystkich obiektach w danej przestrzeni nazw:

```
$ kubectl get events
```

Ponadto po dodaniu `--watch` do polecenia `kubectl get` można strumieniować zdarzenia w miarę ich występowania. Dodatek parametru `-A` sprawi, że będą pokazywane zdarzenia ze wszystkich przestrzeni nazw.

Jeśli natomiast chcesz sprawdzić zużycie zasobów przez klaster, za pomocą polecenia `top` możesz wyświetlić listę zasobów wykorzystywanych przez węzły bądź kapsuły. To polecenie:

```
$ kubectl top nodes
```

wyświetli informację o całkowitym poziomie wykorzystania CPU i pamięci przez węzły zarówno w jednostkach bezwzględnych (na przykład liczbie rdzeni), jak i procentowanych w odniesieniu do dostępnych zasobów (na przykład liczby wszystkich rdzeni). Z kolei to polecenie:

```
$ kubectl top pods
```

pokaże wszystkie kapsuły i ich zużycie zasobów. Domyślnie wyświetla tylko kapsuły należące do bieżącej przestrzeni nazw, ale po dodaniu flagi `--all-namespaces` można uzyskać informacje na temat wszystkich kapsuł w klastrze.

Te polecenia `top` zwracają wyniki tylko wtedy, gdy w klastrze działa serwer metryk. Takie serwery są obecne w prawie każdym zarządzanym środowisku Kubernetes, a także w wielu niezarządzanych. Jeśli wykonanie któregoś z poleceń `top` się nie uda, może to oznaczać, że trzeba zainstalować serwer metryk.

Zarządzanie klastrem

Za pomocą narzędzia `kubectl` można zarządzać także samym klastrem. Czynnościami wykonywanymi najczęściej w ramach zarządzania klastrem są: kordonowanie i drenowanie wybranego węzła. **Kordonowanie** węzła zapobiega planowaniu przyszłych kapsuł na danym komputerze. Kiedy **drenujemy** węzeł, usuwamy wszelkie kapsuły, które aktualnie działają na tym komputerze. Te polecenia przydają się na przykład w sytuacji, gdy konieczne jest usunięcie fizycznej maszyny w celu jej naprawy lub modernizacji. W takim przypadku można użyć poleceń `kubectl cordon` i `kubectl drain`, aby bezpiecznie usunąć maszynę z klastra. Kiedy zostanie naprawiona, można wykonać polecenie `kubectl uncordon`, by ponownie włączyć na niej planowanie kapsuł. Polecenie `undrain` nie istnieje. Tworzone kapsuły będą naturalnie planowane na pustym węźle. W przypadku szybkich czynności mających wpływ na węzeł (na przykład przy ponownym uruchomieniu) zwykle nie ma potrzeby wykonywać poleceń `cordon` ani `drain`. Jest to konieczne tylko wtedy, gdy komputer będzie wyłączony z użytku na tyle długo, że chcemy, aby kapsuły zostały przeniesione na inną maszynę.

Uzupełnianie poleceń

Narzędzie `kubectl` integruje się z powłoką, dzięki czemu włącza funkcję uzupełniania poleceń i zasobów za pomocą klawisza *Tab*. W niektórych środowiskach przed aktywacją tej funkcji może być konieczne zainstalowanie pakietu uzupełniania powłoki bash. Można to zrobić przy użyciu odpowiedniego menedżera pakietów:

```
# macOS
$ brew install bash-completion

# CentOS/Red Hat
$ yum install bash-completion

# Debian/Ubuntu
$ apt-get install bash-completion
```

Podczas instalacji w systemie macOS postępuj według instrukcji dotyczących aktywacji funkcji uzupełniania narzędzia brew przy użyciu `${HOME}/.bash_profile`.

Po zainstalowaniu modułu uzupełniania powłoki bash możesz tymczasowo aktywować go dla swojego terminala za pomocą następującego polecenia:

```
$ source <(kubectl completion bash)
```

Aby funkcja ta włączała się automatycznie dla każdego terminala, dodaj to polecenie do pliku `${HOME}/.bashrc`:

```
$ echo "source <(kubectl completion bash)" >> ${HOME}/.bashrc
```

Użytkownicy powłoki zsh znajdą podobne instrukcje w internecie (*https://oreil.ly/aYujA*).

Inne sposoby pracy z klastrami

Narzędzie `kubectl` nie jest jedynym narzędziem do pracy z klastrami Kubernetes.

Niektóre edytory mają na przykład dostępne wtyczki integrujące Kubernetes:

- Visual Studio Code (*http://bit.ly/32ijGV1*)
- IntelliJ (*http://bit.ly/2Gen1eG*)
- Eclipse (*http://bit.ly/2XHi6gP*)

Większość zarządzanych usług Kubernetes ma graficzny interfejs do Kubernetes zintegrowany z ich interfejsem sieciowym. Zarządzany Kubernetes w chmurze publicznej także integruje się z zaawansowanymi narzędziami do monitoringu, które pozwalają sprawdzić szczegółowe informacje na temat działania aplikacji.

Istnieje również kilka graficznych interfejsów open source do Kubernetes, na przykład Rancher Dashboard (*https://oreil.ly/mliob*) i Headlamp (*https://oreil.ly/lDvbs*).

Podsumowanie

`kubectl` to potężne narzędzie do zarządzania aplikacjami w klastrze Kubernetes. Ten rozdział ilustruje wiele typowych zastosowań tego narzędzia, ale `kubectl` ma też dostępną sporą ilość wbudowanej pomocy. Możesz zacząć przeglądać tę pomoc w ten sposób:

```
$ kubectl help
```

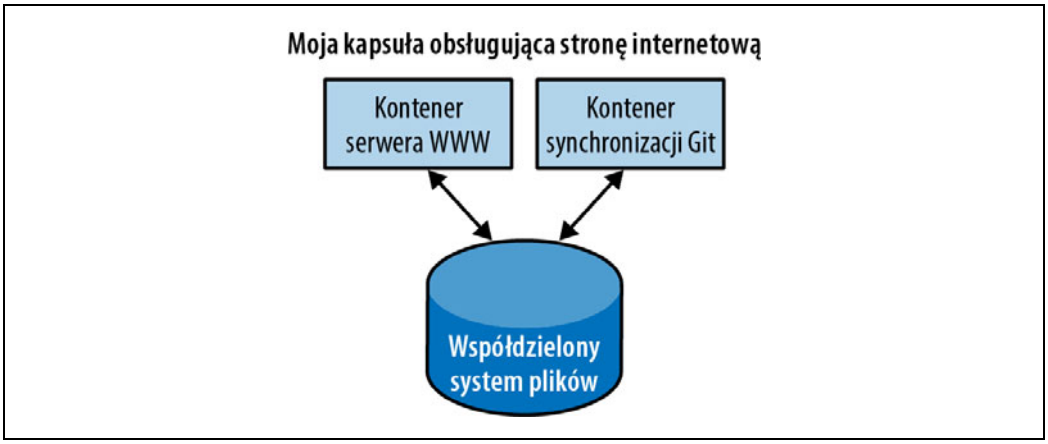
lub:

```
$ kubectl help <nazwa polecenia>
```

ROZDZIAŁ 5.
Kapsuły

W poprzednich rozdziałach omawialiśmy, jak można zabrać się za konteneryzowanie aplikacji, ale w rzeczywistych wdrożeniach kontenerowych aplikacji często trzeba kolokować wiele aplikacji w jednej niepodzielnej jednostce rozplanowanej na pojedynczej maszynie.

Na rysunku 5.1 przedstawiono kanoniczny przykład takiego wdrożenia; widać na nim kontener obsługujący żądania WWW i kontener synchronizujący system plików ze zdalnym repozytorium Git.



Rysunek 5.1. Przykładowa kapsuła z dwoma kontenerami i współdzielonym systemem plików

Na początku może wydawać się kuszące opakowanie zarówno serwera WWW, jak i synchronizatora Git w jeden kontener. Jednak po bliższej analizie jasne stają się powody ich rozdzielenia. Po pierwsze, te dwa różne kontenery mają całkowicie odmienne wymagania dotyczące wykorzystania zasobów. Weźmy na przykład pamięć. Ponieważ serwer WWW obsługuje żądania użytkowników, chcemy zagwarantować, żeby zawsze był dostępny i responsywny. Z kolei synchronizator Git nie jest tak naprawdę skierowany do użytkownika i charakteryzuje się jakością usług typu *best effort*.

Załóżmy, że nasz synchronizator Git ma wyciek pamięci. Musimy zagwarantować, żeby synchronizator Git nie wykorzystywał pamięci, której chcemy używać dla serwera WWW, ponieważ mogłoby to wpłynąć na wydajność tego serwera, a nawet spowodować jego awarię.

Ten rodzaj izolacji zasobów jest dokładnie tym, w czym najlepiej sprawdzają się kontenery. Rozdzielając dwie aplikacje na dwa osobne kontenery, możemy zapewnić niezawodne działanie serwera WWW.

Oczywiście te dwa kontenery są symbiotyczne; nie ma sensu rozplanowywać serwera WWW na jednej maszynie, a synchronizatora Git na drugiej. Dlatego Kubernetes grupuje wiele kontenerów w pojedynczą niepodzielną jednostkę zwaną **kapsułą** (ang. *pod*).

Choć początkowo koncepcja grupowania wielu kontenerów w jednej kapsule wydawała się kontrowersyjna i niejasna, z czasem została przyjęta w różnych aplikacjach jako metoda wdrażania infrastruktury. Na przykład kilka implementacji siatki usług za pomocą drugiego kontenera **przyczepki** wprowadza mechanizmy zarządzania siecią do kapsuł aplikacji.

Kapsuły w Kubernetes

Kapsuła to zbiór kontenerów aplikacji i woluminów działających w tym samym środowisku wykonawczym. Najmniejszymi możliwymi do wdrożenia w klastrze Kubernetes artefaktami są właśnie kapsuły, a nie kontenery. Oznacza to, że wszystkie kontenery w kapsule zawsze lądują na tej samej maszynie.

Poszczególne kontenery w ramach kapsuły działają we własnych grupach kontrolnych (cgroups), ale współdzielą wiele przestrzeni nazw Linuksa.

Aplikacje działające w tej samej kapsule współdzielą ten sam adres IP i przestrzeń portów (sieciową przestrzeń nazw), mają tę samą nazwę hosta (przestrzeń nazw UTS) i mogą komunikować się za pomocą natywnych kanałów komunikacji międzyprocesowej poprzez kolejki komunikatów System V IPC lub POSIX (przestrzeń nazw IPC). Aplikacje w różnych kapsułach są od siebie odizolowane; mają różne adresy IP, różne nazwy hostów itd. Kontenery w różnych kapsułach działających na tym samym węźle mogą równie dobrze znajdować się na różnych serwerach.

Myślenie w kategoriach kapsuł

Jednym z najczęstszych pytań, które pojawiają się przy adaptowaniu Kubernetes, jest: „Co należy umieścić w kapsule?”.

Czasami ludzie widzą kapsuły i myślą: „Aha! Kontener WordPress i kontener bazy danych MySQL łączą się, tworząc instancję Wordpressa”. Jednak taki rodzaj kapsuły jest w rzeczywistości przykładem antywzorca dla budowy kapsuł. Istnieją ku temu dwa powody. Po pierwsze, WordPress i jego baza danych nie są tak naprawdę symbiotyczne. Jeżeli kontener WordPress i kontener bazy danych trafią na różne maszyny, nadal mogą ze sobą całkiem skutecznie współpracować, ponieważ komunikują się przez połączenie sieciowe. Po drugie, niekoniecznie powinieneś skalować WordPressa i bazę danych jako jednostkę. Sam WordPress jest głównie bezstanowy, w odpowiedzi na obciążenie frontendu możesz więc chcieć skalować frontendy WordPressa, tworząc kolejne kapsuły WordPressa. Skalowanie bazy danych MySQL jest znacznie trudniejsze i prawdopodobnie należałoby raczej zwiększyć ilość zasobów przypisanych do pojedynczej kapsuły MySQL. Jeśli

zgrupujesz kontenery WordPress i MySQL w jednej kapsule, będziesz musiał użyć tej samej strategii skalowania dla obu kontenerów, co nie jest zbyt dobrym rozwiązaniem.

Ogólnie rzecz biorąc, pytanie, które należy zadać sobie przy projektowaniu kapsuł, jest następujące: „Czy te kontenery będą działać poprawnie, jeżeli wylądują na różnych maszynach?”. Jeśli odpowiedź brzmi „nie”, wtedy właściwym sposobem pogrupowania tych kontenerów jest kapsuła. Jeżeli odpowiedź brzmi „tak”, to prawdopodobnie prawidłowym rozwiązaniem jest wiele kapsuł. W przykładzie przywołanym na początku tego rozdziału dwa kontenery współdziałają poprzez lokalny system plików. Nie mogłyby działać poprawnie, gdyby zostały rozplanowane na różnych maszynach.

W pozostałej części tego rozdziału opiszemy, jak tworzyć, analizować i usuwać kapsuły w Kubernetes oraz zarządzać nimi.

Manifest kapsuły

Kapsuły są opisane w *manifeście* kapsuły. Manifest kapsuły jest po prostu plikiem tekstowym reprezentującym obiekt API Kubernetes. Twórcy Kubernetes mocno wierzą w **konfigurację deklaratywną**. Deklaratywna konfiguracja oznacza zapisywanie żądanego stanu świata w pliku konfiguracyjnym, a następnie przesyłanie tej konfiguracji do usługi, która podejmuje działania mające zapewnić, że pożądany stan stanie się stanem rzeczywistym.

Konfiguracja deklaratywna różni się od **konfiguracji imperatywnej**, w której po prostu wykonuje się serię czynności (na przykład `apt-get install foo`), aby zmodyfikować stan systemu. Lata doświadczeń w środowisku produkcyjnym nauczyły nas, że utrzymywanie pisemnego zapisu żądanego stanu systemu prowadzi do uzyskania łatwiejszego w zarządzaniu, niezawodnego systemu. Konfiguracja deklaratywna zapewnia wiele korzyści, w tym możliwość przeglądania kodu pod kątem konfiguracji i dokumentowania aktualnego stanu świata dla rozproszonych zespołów. Dodatkowo jest podstawą wszystkich zachowań samonaprawiania się w Kubernetes, które pozwalają aplikacji działać bez podejmowania działań przez użytkownika.

Serwer interfejsu API Kubernetes akceptuje i przetwarza manifesty kapsuł przed ich zapisaniem w pamięci trwałej (`etcd`). Planista również wykorzystuje API Kubernetes do znajdowania kapsuł, które nie zostały rozplanowane dla jakiegoś węzła. Następnie planista umieszcza te kapsuły na węzłach w zależności od zasobów i innych ograniczeń wyrażonych w manifestach kapsuł. Na tej samej maszynie można umieścić wiele kapsuł, o ile dostępne są wystarczające zasoby. Jednak rozplanowywanie wielu replik tej samej aplikacji na tę samą maszynę jest gorsze pod kątem niezawodności, ponieważ taka maszyna jest pojedynczą domeną awarii. W związku z tym planista Kubernetes próbuje sprawić, żeby kapsuły z tej samej aplikacji były rozmieszczane na różnych maszynach w celu zapewnienia niezawodności w przypadku wystąpienia awarii. Po rozplanowaniu na węzeł kapsuły nie przemieszczają się i aby je przenieść, trzeba je bezpośrednio zniszczyć i ponownie rozplanować.

Wiele instancji kapsuły można wdrożyć, powtarzając opisany tutaj przepływ pracy. Jednak do uruchamiania wielu instancji kapsuły lepiej nadają się obiekty `ReplicaSet` (zobacz rozdział 8.).

(Okazuje się, że są one również lepsze przy uruchamianiu pojedynczej kapsuły, ale do tego dojdziemy później).

Tworzenie kapsuły

Najprostszym sposobem na utworzenie kapsuły jest wykonanie imperatywnego polecenia `kubectl run`. Aby uruchomić na przykład nasz serwer `kuard`, użyj następującego polecenia:

```
$ kubectl run kuard --generator=run-pod/v1 \
  --image=gcr.io/kuar-demo/kuard-amd64:blue
```

W ten sposób możesz sprawdzić status tej kapsuły:

```
$ kubectl get pods
```

Początkowo możesz zobaczyć kontener jako oczekujący (`Pending`), ale ostatecznie przekonasz się, że jego status zmieni się na działający (`Running`), co oznacza, że kapsuła i jej kontenery zostały pomyślnie utworzone.

Na razie możesz usunąć tę kapsułę, uruchamiając następujące polecenie:

```
$ kubectl delete pods/kuard
```

Przejdziemy teraz do ręcznego napisania pełnego manifestu kapsuły.

Tworzenie manifestu kapsuły

Manifesty kapsuł można pisać przy użyciu formatów YAML lub JSON, ale zasadniczo preferowany jest YAML, ponieważ daje nieco większe możliwości edycji i oferuje opcję dodawanie komentarzy. Manifesty kapsuł (i inne obiekty API Kubernetes) należy tak naprawdę traktować w ten sam sposób, w jaki traktuje się kod źródłowy, a elementy takie jak komentarze pomagają objaśnić kapsułę nowym członkom zespołu.

Manifesty kapsuł zawierają kilka kluczowych pól i atrybutów, między innymi: sekcję `metadata` do opisania kapsuły i jej etykiet, sekcję `spec` opisującą woluminy oraz listę kontenerów, które będą działały w kapsule.

W rozdziale 2. wdrożyliśmy `kuard` za pomocą następującego polecenia Dockera:

```
$ docker run -d --name kuard \
  --publish 8080:8080 \
  gcr.io/kuar-demo/kuard-amd64:blue
```

Podobny rezultat można osiągnąć, wpisując kod z listingu 5.1 do pliku o nazwie *kuard-pod.yaml*, a następnie używając poleceń `kubectl` w celu załadowania tego manifestu do Kubernetes.

Listing 5.1. kuard-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard
spec:
  containers:
    - image: gcr.io/kuar-demo/kuard-amd64:blue
```

```
    name: kuard
    ports:
      - containerPort: 8080
        name: http
        protocol: TCP
```

Choć początkowo zarządzanie aplikacją w taki sposób może się wydawać kłopotliwe, zapisywanie rekordów pożądanego stanu jest najlepszą praktyką w długiej perspektywie, w szczególności w dużych zespołach zajmujących się wieloma aplikacjami.

Uruchamianie kapsuł

W poprzednim podrozdziale utworzyliśmy manifest kapsuły, który może być użyty do zainicjowania kapsuły uruchamiającej `kuard`. Aby uruchomić pojedynczą instancję `kuard`, skorzystaj z polecenia `kubectl apply`:

```
$ kubectl apply -f kuard-pod.yaml
```

Manifest kapsuły zostanie przesłany na serwer API Kubernetes. Następnie system Kubernetes rozplanuje tę kapsułę, aby została uruchomiona na zdrowym węźle w klastrze, gdzie będzie monitorowana przez proces demona `kubelet`. Nie przejmuj się, jeśli na razie nie rozumiesz tych wszystkich ruchomych części Kubernetes; w miarę lektury tej książki będziesz poznawał coraz więcej szczegółów.

Wyświetlanie listy kapsuł

Skoro mamy już uruchomioną kapsułę, dowiedzmy się o niej czegoś więcej. Używając narzędzia wiersza poleceń `kubectl`, możemy wyświetlić wszystkie kapsuły działające w klastrze. Na razie powinna w nim być tylko jedna kapsuła, którą utworzyliśmy w poprzednim kroku:

```
$ kubectl get pods
NAME        READY        STATUS        RESTARTS     AGE
kuard       1/1          Running       0            44s
```

Możesz zobaczyć nazwę kapsuły (`kuard`), którą nadaliśmy jej w poprzednim pliku YAML. Oprócz liczby gotowych kontenerów (`1/1`) dane wyjściowe pokazują również status, liczbę ponownych uruchomień kapsuły oraz wiek kapsuły.

Jeśli uruchomiłeś to polecenie natychmiast po utworzeniu kapsuły, możesz zobaczyć następujące dane wyjściowe:

```
NAME        READY        STATUS        RESTARTS    AGE
kuard       0/1          Pending       0           1s
```

Stan oczekujący (`Pending`) wskazuje, że kapsuła została przesłana, ale nie została jeszcze rozplanowana. Jeżeli wystąpi jakiś bardziej znaczący błąd (na przykład próba utworzenia kapsuły z obrazem kontenera, który nie istnieje), również zostanie wyświetlony w polu statusu.

Domyślnie narzędzie wiersza poleceń `kubectl` zwraca zwięzłe informacje, ale możesz uzyskać ich więcej, stosując odpowiednią flagę wiersza poleceń. Dodanie flagi `-o wide` do jakiegokolwiek polecenia `kubectl` spowoduje wydrukowanie nieco większej ilości danych wyjściowych (nadal mieszczących się w pojedynczej linii). Dodanie flagi `-o json` lub `-o yaml` spowoduje wydrukowanie kompletnych obiektów odpowiednio w formacie JSON lub YAML. Jeśli chcesz zobaczyć wyczerpujące informacje na temat tego, co robi `kubectl`, możesz dodać flagę `--v=10`, aby zwiększyć liczbę szczegółów kosztem czytelności.

Szczegółowe informacje o kapsule

Czasami jednoliniowy wydruk jest niewystarczający, ponieważ jest zbyt lapidarny. Kubernetes przechowuje dodatkowo różne zdarzenia związane z kapsułami, które są obecne w strumieniu zdarzeń, ale nie są dołączone do obiektu kapsuły.

W celu uzyskania większej ilości informacji na temat kapsuły (lub dowolnego obiektu Kubernetes) możesz użyć polecenia `kubectl describe`. Aby opisać na przykład kapsułę, którą wcześniej utworzyliśmy, możemy uruchomić następujące polecenie:

```
$ kubectl describe pods kuard
```

Te dane wyjściowe dostarczają wiele informacji o kapsule w różnych sekcjach. Na początku znajdują się informacje podstawowe:

```
Name:           kuard
Namespace:      default
Node:           node1/10.0.15.185
Start Time:     Sun, 02 Jul 2017 15:00:38 -0700
Labels:         <none>
Annotations:    <none>
Status:         Running
IP:             192.168.199.238
Controllers:    <none>
```

Następnie mamy informacje o kontenerach działających w kapsule:

```
Containers:
  kuard:
    Container ID:    docker://055095…
    Image:           gcr.io/kuar-demo/kuard-amd64:blue
    Image ID:        docker-pullable://gcr.io/kuar-demo/kuard-amd64@sha256:a580…
    Port:            8080/TCP
    State:           Running
      Started:       Sun, 02 Jul 2017 15:00:41 -0700
   Ready:            True
   Restart Count:    0
   Environment:     <none>
   Mounts:
     /var/run/secrets/kubernetes.io/serviceaccount from default-token-cg5f5 (ro)
```

Na koniec podawane są zdarzenia związane z kapsułą, takie jak czas jej rozplanowania, czas pobrania obrazu oraz informacje, czy i kiedy kapsuła musiała zostać ponownie uruchomiona z powodu niepowodzenia kontroli poprawności działania:

```
Events:
  Seen  From                SubObjectPath              Type      Reason       Message
  ----  ----                -------------              --------  ------       -------
  50s  default-scheduler                           Normal   Scheduled   Success…
  49s  kubelet, node1     spec.containers{kuard}   Normal   Pulling     pulling…
  47s  kubelet, node1     spec.containers{kuard}   Normal   Pulled      Success…
  47s  kubelet, node1     spec.containers{kuard}   Normal   Created     Created…
  47s  kubelet, node1     spec.containers{kuard}   Normal   Started     Started…
```

Usuwanie kapsuły

Gdy nadejdzie czas usunięcia kapsuły, możesz ją usunąć na podstawie nazwy:

```
$ kubectl delete pods/kuard
```

Możesz również użyć tego samego pliku, który został wykorzystany do jej utworzenia:

```
$ kubectl delete -f kuard-pod.yaml
```

Po usunięciu kapsuła *nie* jest natychmiast niszczona. Jeżeli uruchomisz w tym momencie polecenie `kubectl get pods`, zobaczysz, że kapsuła jest w stanie kończenia działania (`Terminating`). Wszystkie kapsuły mają **okres karencji** związany z kończeniem działania. Domyślnie jest to 30 sekund. Po wejściu kapsuły w stan `Terminating` nie otrzymuje już ona żadnych nowych żądań. W scenariuszu serwowania okres karencji jest ważny dla niezawodności, ponieważ pozwala kapsule przed usunięciem zakończyć wszelkie aktywne żądania, które mogą być w trakcie przetwarzania.

Po usunięciu kapsuły wszelkie dane przechowywane w kontenerach związanych z tą kapsułą również zostaną usunięte. Jeśli chcesz zachować dane na wielu instancjach kapsuły, musisz użyć obiektu `PersistentVolumes` opisanego na końcu tego rozdziału.

Uzyskiwanie dostępu do kapsuły

Gdy kapsuła jest już uruchomiona, możesz chcieć z różnych powodów uzyskać do niej dostęp. Na przykład by załadować usługę internetową, która działa w kapsule. Możesz chcieć przejrzeć jej dzienniki w celu debugowania napotkanego problemu lub nawet wykonać inne polecenia w kontekście kapsuły, które pomogą w debugowaniu. Poniższe punkty rozdziału opisują szczegółowo różne sposoby interakcji z kodem i danymi działającymi wewnątrz kapsuły.

Uzyskiwanie większej ilości informacji za pomocą dzienników

Gdy aplikacja wymaga debugowania, pomocne jest pozyskanie większej ilości informacji, niż zapewnia `describe`, aby zrozumieć, co robi aplikacja. Kubernetes oferuje dwa polecenia do debugowania uruchomionych kontenerów. Polecenie `kubectl logs` pozwala pobrać bieżące dzienniki z działającej instancji:

```
$ kubectl logs kuard
```

Dodanie flagi `-f` pozwoli na ciągłe przesyłanie dzienników.

Użycie polecenia `kubectl logs` zawsze powoduje próbę uzyskania dzienników z aktualnie uruchomionego kontenera. Po dodaniu flagi `--previous` pobrane zostaną dzienniki z poprzedniej instancji kontenera. Jest to użyteczne na przykład wtedy, gdy kontenery są ciągle restartowane z powodu problemu z rozruchem kontenera.

Chociaż polecenie `kubectl logs` jest użyteczne w przypadku jednorazowego debugowania kontenerów w środowiskach produkcyjnych, zasadniczo bardziej przydaje się usługa agregacji dzienników. Istnieje kilka open source'owych narzędzi agregacji dzienników, takich jak `fluentd` i `elasticsearch`, a także wielu chmurowych dostawców rejestrowania dzienników. Usługi agregacji dzienników zapewniają więcej pojemności do przechowywania dzienników z dłuższego przedziału czasu oraz wszechstronne funkcje wyszukiwania i filtrowania dzienników. Ponadto oferują one możliwość agregowania dzienników z wielu kapsuł w jeden widok.

Uruchamianie poleceń w kontenerze przy użyciu exec

Czasami dzienniki są niewystarczające i żeby naprawdę określić, co się dzieje, musisz wykonywać polecenia w kontekście samego kontenera. W tym celu możesz użyć poniższego polecenia:

```
$ kubectl exec kuard date
```

Możesz również uzyskać interaktywną sesję, dodając flagi `-it`:

```
$ kubectl exec -it kuard ash
```

Kopiowanie plików do i z kontenerów

W poprzednim rozdziale pokazaliśmy, jak uzyskać dostęp do plików w kapsule za pomocą polecenia `kubectl cp`. Ogólnie rzecz biorąc, kopiowanie plików do kontenera jest antywzorcem. Naprawdę powinieneś traktować zawartość kontenera jako niemutowalną. Jednak czasami jest to sposób na natychmiastowe powstrzymanie krwawienia i przywrócenie usługi do zdrowia, ponieważ jest szybszy niż zbudowanie, przesłanie do rejestru i wdrożenie nowego obrazu. Gdy krwawienie zostanie już zatrzymane, zawsze bardzo ważne jest, abyś od razu przystąpił do tworzenia i wdrażania obrazu, ponieważ w przeciwnym razie na pewno zapomnisz o lokalnej zmianie, którą wprowadziłeś do swojego kontenera, i nadpiszesz ją podczas późniejszego regularnego wprowadzania kolejnej wersji.

Kontrole działania

Po uruchomieniu aplikacji jako kontenera w Kubernetes aplikacja ta jest automatycznie utrzymywana przy życiu za pomocą **kontroli działania procesu** (ang. *process health check*). Ta kontrola po prostu zapewnia, że główny proces aplikacji będzie zawsze uruchomiony. Jeśli proces zostanie wyłączony, Kubernetes uruchomi go ponownie.

Jednak w większości przypadków prosta kontrola działania procesu jest niewystarczająca. Jeżeli proces ulegnie na przykład zakleszczeniu i nie będzie mógł obsługiwać żądań, system sprawdzania działania nadal będzie traktował aplikację jako zdrową, ponieważ jej proces będzie wciąż uruchomiony.

Aby rozwiązać ten problem, Kubernetes wprowadził kontrole działania pod kątem **żywotności** aplikacji. Kontrole żywotności uruchamiają logikę charakterystyczną dla danej aplikacji (na przykład ładowanie strony internetowej), by zweryfikować, czy aplikacja nie tylko nadal działa, ale także czy działa poprawnie. Ponieważ kontrole żywotności są charakterystyczne dla danej aplikacji, należy je zdefiniować w manifeście kapsuły.

Sonda żywotności

Po uruchomieniu procesu `kuard` potrzebujemy sposobu weryfikacji, czy proces ten naprawdę działa poprawnie i nie powinien zostać zrestartowany. Sondy żywotności są definiowane dla poszczególnych kontenerów, co oznacza, że każdy kontener wewnątrz kapsuły jest sprawdzany osobno. W kodzie z listingu 5.2 dodajemy do naszego kontenera `kuard` sondę żywotności, która uruchamia zażądanie HTTP dla ścieżki `/healthy` w kontenerze.

Listing 5.2. kuard-pod-health.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard
spec:
  containers:
    - image: gcr.io/kuar-demo/kuard-amd64:blue
      name: kuard
      livenessProbe:
        httpGet:
          path: /healthy
          port: 8080
        initialDelaySeconds: 5
        timeoutSeconds: 1
        periodSeconds: 10
        failureThreshold: 3
      ports:
        - containerPort: 8080
          name: http
          protocol: TCP
```

Powyższy manifest kapsuły używa sondy `httpGet` do wykonania żądania `GET` HTTP dla punktu końcowego `/healthy` na porcie `8080` kontenera `kuard`. Sonda ustawia początkowe opóźnienie `initialDelaySeconds` na `5`, a zatem wywoływanie zostanie wykonane po upływie pięciu sekund od momentu utworzenia wszystkich kontenerów w kapsule. Sonda musi odpowiedzieć w ciągu jednosekundowego limitu czasu, a kod statusu HTTP musi być równy lub większy niż 200 i mniejszy niż 400, aby wykonanie żądania zostało uznane za udane. Kubernetes będzie wywoływał tę sondę co 10 sekund. Jeżeli zawiodą więcej niż trzy sondy z rzędu, kontener ulegnie awarii i zostanie uruchomiony ponownie.

Możesz zobaczyć to w akcji, otwierając stronę statusu kontenera `kuard`. Utwórz kapsułę przy użyciu tego manifestu, a następnie przekieruj port do tej kapsuły:

```
$ kubectl apply -f kuard-pod-health.yaml
$ kubectl port-forward kuard 8080:8080
```

Wpisz w przeglądarce adres *http://localhost:8080*. Kliknij zakładkę *Liveness Probe* (sonda żywotności). Powinna wyświetlić się tabela zawierającą listę wszystkich sond, które otrzymała ta instancja `kuard`. Jeżeli klikniesz link *fail* na tej stronie, `kuard` zacznie „oblewać" kontrole poprawności działania. Poczekaj, aż Kubernetes zrestartuje kontener. W tym momencie wyświetlacz zostanie zresetowany i zainicjowany od nowa. Szczegółowe informacje na temat restartu można uzyskać za pomocą polecenia `kubectl describe pods kuard`. Sekcja *Events* (zdarzenia) będzie zawierała dane wyjściowe podobne do tych:

```
Killing container with id docker://2ac946...:pod "kuard_default(9ee84...)"
container "kuard" is unhealthy, it will be killed and re-created.
```

Choć domyślnie niepomyślny wynik testu żywotności powoduje ponowne uruchomienie kapsuły, o tym, co tak naprawdę się stanie, decyduje parametr `restartPolicy` kapsuły. Są trzy możliwości: `Always` (zawsze uruchamiaj ponownie; domyślne ustawienie), `OnFailure` (ponowne uruchomienie tylko w przypadku niepomyślnego wyniku testu żywotności lub kodu wyjścia różnego od zera) oraz `Never` (nigdy nie uruchamiaj ponownie).

Sonda gotowości

Oczywiście sonda żywotności nie jest jedynym rodzajem kontroli działania, który chcemy wykorzystać. Kubernetes dokonuje rozróżnienia pomiędzy **żywotnością** a **gotowością**. Żywotność określa, czy aplikacja działa poprawnie. Kontenery, które nie przejdą kontroli żywotności, są restartowane. Gotowość opisuje, kiedy kontener jest gotowy do obsługi żądań użytkownika. Kontenery, które nie przejdą kontroli gotowości, są usuwane z mechanizmów równoważenia obciążenia usług. Sondy gotowości są konfigurowane podobnie do sond żywotności. Szczegółowo zbadamy usługi Kubernetes w rozdziale 7.

Połączenie sond gotowości i żywotności pomaga zagwarantować, że w klastrze będą działały tylko zdrowe kontenery.

Sonda rozruchu

Sondy rozruchu ostatnio wprowadzone do Kubernetes umożliwiają zarządzanie w alternatywny sposób powoli uruchamiającymi się kontenerami. Kiedy jest uruchamiana kapsuła, sonda rozruchu zostaje uruchomiona przed wszystkimi innymi jej sondami. Sonda ta działa, aż przekroczy limit czasu (w którym to przypadku kapsuła zostaje zrestartowana) lub wykona swoje zadanie pomyślnie (i wówczas kontrolę przejmuje sonda żywotności). Sondy rozruchu pozwalają na powolne sondowanie w przypadku powoli uruchamiających się kontenerów, a jednocześnie umożliwiają responsywne sprawdzanie żywotności po zainicjalizowaniu takiego kontenera.

Zaawansowana konfiguracja sondy

Sondy w Kubernetes mają kilka zaawansowanych opcji, w tym opcję pozwalającą określić, ile czasu po uruchomieniu kapsuły rozpocząć sondowanie, ile nieudanych prób oznacza prawdziwą awarię oraz po ilu udanych próbach można zresetować licznik awarii. Wszystkie te konfiguracje

mają wartości domyślne, ale ich zmiana może być konieczna w niektórych trudniejszych przypadkach, na przykład dla aplikacji, które z natury są niepewne albo długo się uruchamiają.

Rodzaje kontroli działania

Oprócz kontroli HTTP Kubernetes obsługuje także kontrole działania `tcpSocket`, które otwierają gniazdo TCP; jeśli połączenie się powiedzie, kontrola się powiedzie. Ten styl sondowania jest przydatny dla aplikacji innych niż HTTP, na przykład baz danych lub interfejsów API innych niż oparte na HTTP.

Ponadto Kubernetes umożliwia przeprowadzanie sond `exec`. Wykonują one jakiś skrypt lub program w kontekście kontenera. Zgodnie z powszechnie przyjętą konwencją, jeżeli skrypt zwróci zerowy kod wyjścia, kontrola została wykonana z powodzeniem. W przeciwnym razie wykonanie sondy nie powiedzie się. Skrypty `exec` są często przydatne dla niestandardowych logik walidacji aplikacji, których nie można zweryfikować za pomocą wywołania HTTP.

Zarządzanie zasobami

Większość programistów przechodzi na kontenery i orkiestratory takie jak Kubernetes z powodu zapewnianych przez nie znacznych usprawnień w zakresie pakowania obrazów i niezawodnego wdrażania. Oprócz podstawowych elementów ukierunkowanych na aplikacje, które upraszczają tworzenie systemów rozproszonych, równie ważna jest możliwość zwiększenia ogólnego wykorzystania węzłów obliczeniowych, które tworzą klaster. Podstawowe koszty eksploatacji maszyny, wirtualnej lub fizycznej, są zasadniczo stałe niezależnie od tego, czy jest ona bezczynna, czy w pełni obciążona. Dlatego zapewnienie maksymalnej aktywności maszyn zwiększa efektywność każdego dolara wydanego na infrastrukturę.

Ogólnie rzecz biorąc, tę efektywność mierzy się za pomocą wskaźnika **wykorzystania**. Wykorzystanie jest definiowane jako ilość aktywnie używanych zasobów podzielona przez ilość zakupionych zasobów. Jeśli zakupisz na przykład maszynę jednordzeniową, a Twoja aplikacja będzie używać jedną dziesiątą rdzenia, wskaźnik wykorzystania wyniesie 10%. Dzięki systemom planowania, takim jak Kubernetes, które zarządzają pakowaniem zasobów, można zwiększyć poziom wykorzystania do ponad 50%. W tym celu musisz dostarczyć systemowi Kubernetes informacje o zasobach wymaganych przez aplikację, aby mógł ustalić optymalny sposób spakowania kontenerów na zakupionych maszynach.

Kubernetes umożliwia użytkownikom określenie dwóch różnych metryk zasobów. **Żądania** zasobów określają minimalną ilość zasobu wymaganą do uruchomienia aplikacji. **Limity** zasobów określają maksymalną ilość zasobów, które aplikacja może wykorzystywać. Przyjrzymy się im bliżej w kolejnych podrozdziałach.

Kubernetes rozpoznaje wiele różnych notacji do określania zasobów, od literałów (`12345`) po milirdzenie (`100m`). Bardzo ważne jest, aby rozróżniać jednostki MB/GB/PB i MiB/GiB/PiB. Pierwsze reprezentują znane wszystkim potęgi dwójki (na przykład 1 MB == 1024 KB), a drugie — to potęgi dziesiątki (1 MiB == 1000 KiB).

Błędy często wynikają z zapisywania jednostek „mili-” za pomocą małej litery *m*, a jednostek „mega-” za pomocą wielkiej litery *M*. Mówiąc inaczej, 400 m to 0,4 MB, a nie 400 MB — to duża różnica!

Żądania zasobów: minimalne wymagane zasoby

W Kubernetes kapsuła żąda zasobów wymaganych do uruchomienia swoich kontenerów. Kubernetes gwarantuje, że te zasoby będą dostępne dla kapsuły. Najczęściej żądanymi zasobami są CPU i pamięć, ale Kubernetes obsługuje również inne typy zasobów, między innymi GPU. Aby na przykład zażądać, żeby kontener `kuard` wylądował na maszynie z wolną połową zasobów procesora i otrzymał przydział 128 MB pamięci, definiujemy kapsułę tak, jak pokazano na listingu 5.3.

Listing 5.3. kuard-pod-resreq.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard
spec:
  containers:
    - image: gcr.io/kuar-demo/kuard-amd64:blue
      name: kuard
      resources:
        requests:
          cpu: "500m"
          memory: "128Mi"
      ports:
        - containerPort: 8080
          name: http
          protocol: TCP
```

Zasoby są żądane dla poszczególnych kontenerów, a nie dla kapsuł. Całkowita ilość zasobów zażądanych przez kapsułę jest sumą wszystkich zasobów żądanych przez wszystkie kontenery w kapsule. Jest tak dlatego, że w wielu przypadkach różne kontenery mają bardzo różne wymagania odnośnie do zasobów procesora. Przykładowo w kapsule serwera WWW i synchronizatora danych serwer WWW jest skierowany do użytkownika i prawdopodobnie wymaga dużej mocy obliczeniowej, a synchronizator danych może obejść się mniejszą ilością zasobów procesora.

Żądania są używane podczas rozplanowywania kapsuł na węzły. Planista Kubernetes gwarantuje, że suma wszystkich żądań wszystkich kapsuł na węźle nie przekroczy pojemności węzła. Dlatego kapsuła działająca na węźle może mieć zagwarantowane posiadanie przynajmniej wymaganych zasobów. Co ważne, „żądanie” określa minimum. Nie określa górnego limitu zasobów, z których może korzystać kapsuła. Aby dowiedzieć się, co to oznacza, spójrz na przykład.

Wyobraź sobie, że mamy kontener, którego kod próbuje wykorzystać wszystkie dostępne rdzenie procesora. Załóżmy, że tworzymy kapsułę z tym kontenerem, która żąda połowy procesora. Kubernetes rozplanowuje tę kapsułę na maszynę z dwoma rdzeniami procesora. Dopóki jest to jedyna kapsuła na maszynie, będzie wykorzystywać wszystkie dwa dostępne rdzenie, mimo że zażądała tylko połowy procesora.

Jeżeli na tę maszynę trafi druga kapsuła z tym samym kontenerem i tym samym żądaniem 0,5 CPU, wtedy każda kapsuła otrzyma jeden rdzeń. Jeśli rozplanowana zostanie trzecia identyczna kapsuła, każda kapsuła otrzyma 0,66 rdzenia. Wreszcie, jeśli rozplanowana zostanie czwarta identyczna kapsuła, każda kapsuła otrzyma pół rdzenia, którego żąda, a węzeł będzie miał komplet.

Żądania CPU są realizowane za pomocą funkcjonalności `cpu-share` jądra Linuksa.

Żądania pamięci są obsługiwane podobnie do żądań procesora, ale istnieje pewna ważna różnica. Jeśli kontener przekracza swoją żądaną pamięć, system operacyjny nie może po prostu usunąć pamięci z procesu, ponieważ została ona przydzielona. Dlatego gdy w systemie zabraknie pamięci, `kubelet` kończy działanie kontenerów, których użycie pamięci jest większe niż żądana pamięć. Te kontenery są automatycznie restartowane, ale z mniejszą dostępną dla nich pamięcią na komputerze.

Ponieważ żądania zasobów gwarantują dostępność zasobów dla kapsuły, są one niezbędne w celu zapewnienia, by kontenery miały wystarczające zasoby w sytuacjach dużego obciążenia.

Ograniczanie wykorzystania zasobów za pomocą limitów

Oprócz ustawiania zasobów wymaganych przez kapsułę, co ustanawia minimalną ilość zasobów dostępnych dla kapsuły, możesz także definiować maksimum wykorzystania zasobów przez kapsułę, określając *limity* zasobów.

W naszym poprzednim przykładzie utworzyliśmy kapsułę `kuard`, która zażądała co najmniej pół rdzenia i 128 MB pamięci. W manifeście kapsuły na listingu 5.4 rozszerzamy tę konfigurację, aby dodać limit 1 CPU i 256 MB pamięci.

Listing 5.4. kuard-pod-reslim.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard
spec:
  containers:
    - image: gcr.io/kuar-demo/kuard-amd64:blue
      name: kuard
      resources:
        requests:
          cpu: "500m"
          memory: "128Mi"
        limits:
          cpu: "1000m"
          memory: "256Mi"
      ports:
        - containerPort: 8080
          name: http
          protocol: TCP
```

Po ustanowieniu ograniczeń dla kontenera jądro jest skonfigurowane w taki sposób, aby uniemożliwić wykorzystanie przekraczające te limity. Kontener z limitem procesora wynoszącym pół rdzenia będzie otrzymywał tylko pół rdzenia, nawet jeśli procesor będzie bezczynny. Kontener z limitem

pamięci 256 MB nie będzie mógł wykorzystywać dodatkowej pamięci (nie powiedzie się na przykład `malloc`), jeśli jego wykorzystanie pamięci przekroczy 256 MB.

Utrwalanie danych za pomocą woluminów

Po usunięciu kapsuły lub ponownym uruchomieniu kontenera usuwane są również wszystkie dane z systemu plików kontenera. Często jest to dobre rozwiązanie, ponieważ raczej nie należy pozostawiać śmieciowego kodu, który akurat zapisała jakaś bezstanowa aplikacja internetowa. W innych przypadkach posiadanie dostępu do trwałego magazynu danych jest ważną cechą zdrowej aplikacji. Kubernetes modeluje takie trwałe magazyny przechowywania danych.

Używanie woluminów z kapsułami

Aby dodać wolumin do manifestu kapsuły, musimy dodać do naszej konfiguracji dwa nowe fragmenty. Pierwszy to nowa sekcja `spec.volumes`. Ta tablica definiuje w manifeście kapsuły wszystkie woluminy, do których kontenery mogą uzyskiwać dostęp. Należy zwrócić uwagę, że nie wszystkie kontenery muszą montować wszystkie woluminy zdefiniowane w kapsule. Drugim dodatkiem jest tablica `volumeMounts` w definicji kontenera. Ta tablica określa woluminy, które są montowane w konkretnym kontenerze, oraz ścieżki montowania poszczególnych woluminów. Zauważ, że dwa różne kontenery w kapsule mogą montować ten sam wolumin na różnych ścieżkach.

Manifest na listingu 5.5 definiuje pojedynczy nowy wolumin o nazwie `kuard-data`, który kontener `kuard` montuje w ścieżce `/data`.

Listing 5.5. kuard-pod-vol.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard
spec:
  volumes:
    - name: "kuard-data"
      hostPath:
        path: "/var/lib/kuard"
  containers:
    - image: gcr.io/kuar-demo/kuard-amd64:blue
      name: kuard
      volumeMounts:
        - mountPath: "/data"
          name: "kuard-data"
      ports:
        - containerPort: 8080
          name: http
          protocol: TCP
```

Różne sposoby używania woluminów z kapsułami

Istnieje wiele sposobów wykorzystania danych w aplikacji. Poniżej opisanych zostało kilka zalecanych wzorców dla Kubernetes.

Komunikacja/synchronizacja

W pierwszym przykładzie kapsuły zobaczyłeś, w jaki sposób dwa kontenery korzystały ze współdzielonego woluminu, aby serwować witrynę, zachowując przy tym synchronizację ze zdalną lokalizacją Git (rysunek 5.1). W tym celu kapsuła używa woluminu `emptyDir`. Zakresem takiego woluminu jest czas życia kapsuły, ale może on być współdzielony między dwoma kontenerami, tworząc podstawę komunikacji między kontenerami synchronizacji Git i serwera WWW.

Pamięć podręczna

Aplikacja może używać woluminu, który jest istotny pod kątem wydajności, ale nie jest wymagany do poprawnego działania aplikacji. Aplikacja może na przykład przechowywać wstępnie zrenderowane miniatury większych obrazów. Oczywiście można je rekonstruować z oryginalnych obrazów, ale wówczas obsługa miniatur jest bardziej kosztowna. Taka pamięć podręczna powinna przetrwać ponowne uruchomienie kontenera spowodowane niepowodzeniami kontroli poprawności działania, dlatego również w przypadku użycia pamięci podręcznej dobrze sprawdza się `emptyDir`.

Trwałe dane

Czasami będziesz używać woluminu dla prawdziwie trwałych danych, czyli takich, które są niezależne od czasu życia konkretnej kapsuły i powinny być przesuwane między węzłami w klastrze, jeśli jakiś węzeł ulegnie awarii lub kapsuła z jakiegoś powodu zostanie przeniesiona na inną maszynę. W tym celu Kubernetes obsługuje wiele różnych zdalnych woluminów sieciowej pamięci masowej, w tym szeroko obsługiwane protokoły, takie jak NFS lub iSCSI, a także sieciowe pamięci masowe dostawców chmury, takie jak Elastic Block Store, Azure Files, Azure Disk i Persistent Disk firmy Google.

Montowanie systemu plików hosta

Inne aplikacje w rzeczywistości nie potrzebują trwałego woluminu, ale niektóre z nich wymagają dostępu do bazowego systemu plików hosta. Mogą na przykład potrzebować dostępu do systemu plików */dev* w celu uzyskiwania nieprzetworzonego dostępu na poziomie bloków do urządzenia w systemie. Dla takich przypadków Kubernetes obsługuje wolumin `hostPath`, który może montować w kontenerze dowolne lokalizacje w węźle roboczym. Na listingu 5.5 użyliśmy typu woluminu `hostPath`. Utworzony wolumin to */var/lib/kuard* na hoście.

Oto przykład użycia serwera NFS:

```
...
# Powyżej reszta definicji kapsuły
volumes:
    - name: "kuard-data"
      nfs:
        server: my.nfs.server.local
        path: "/exports"
```

Trwałe woluminy to szeroki temat. W rozdziale 16. został omówiony bardziej szczegółowo.

Wszystko razem

Wiele aplikacji ma charakter stanowy i jako takie muszą zachowywać wszelkie dane i zapewniać dostęp do bazowej pamięci masowej niezależnie od tego, na jakiej maszynie działa aplikacja. Jak widziałeś wcześniej, można to osiągnąć za pomocą trwałego woluminu obsługiwanego przez sieciową pamięć masową. Chcemy również zapewnić, żeby przez cały czas działała zdrowa instancja aplikacji, co oznacza, iż chcemy mieć pewność, że kontener uruchamiający `kuard` będzie gotowy, zanim udostępnimy go klientom.

Poprzez połączenie trwałych woluminów, sond gotowości i żywotności oraz ograniczenia zasobów Kubernetes zapewnia wszystko, co jest potrzebne do niezawodnego działania stanowej aplikacji. Listing 5.6 zbiera to wszystko w jeden manifest.

Listing 5.6. kuard-pod-full.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard
spec:
  volumes:
    - name: "kuard-data"
      nfs:
        server: my.nfs.server.local
        path: "/exports"
  containers:
      - image: gcr.io/kuar-demo/kuard-amd64:blue
        name: kuard
        ports:
          - containerPort: 8080
            name: http
            protocol: TCP
        resources:
          requests:
            cpu: "500m"
            memory: "128Mi"
          limits:
            cpu: "1000m"
             memory: "256Mi"
        volumeMounts:
          - mountPath: "/data"
            name: "kuard-data"
        livenessProbe:
          httpGet:
            path: /healthy
            port: 8080
          initialDelaySeconds: 5
          timeoutSeconds: 1
          periodSeconds: 10
          failureThreshold: 3
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 30
```

```
        timeoutSeconds: 1
        periodSeconds: 10
        failureThreshold: 3
```

W tym rozdziale definicja kapsuły została znacznie rozszerzona. Każda nowa funkcja dodana do aplikacji powoduje dołączenie kolejnej sekcji do jej definicji.

Podsumowanie

Kapsuły reprezentują niepodzielną jednostkę pracy w klastrze Kubernetes. Składają się z jednego kontenera lub większej liczby kontenerów współpracujących symbiotycznie. Aby utworzyć kapsułę, należy napisać manifest kapsuły i przesłać go do serwera API Kubernetes za pomocą jakiegoś narzędzia wiersza poleceń lub (rzadziej) poprzez wywołania HTTP i JSON wysyłane bezpośrednio do serwera.

Po przesłaniu manifestu do serwera interfejsu API planista Kubernetes znajduje maszynę, na której może zmieścić się kapsuła, i rozplanowuje kapsułę na tę maszynę. Po rozplanowaniu kapsuły demon `kubelet` na tej maszynie jest odpowiedzialny za utworzenie odpowiadających tej kapsule kontenerów oraz przeprowadzanie kontroli działania określonych w manifeście kapsuły.

Gdy kapsuła zostanie rozplanowana na węzeł, nie jest przeprowadzane ponowne rozplanowanie, jeśli ten węzeł ulegnie awarii. W celu uzyskania wielu replik tej samej kapsuły trzeba je utworzyć i nazwać ręcznie. W rozdziale 9. przedstawimy obiekt `ReplicaSet` i pokażemy, jak może on zautomatyzować tworzenie wielu identycznych kapsuł i pomóc upewnić się, że będą one odtwarzane w przypadku awarii maszyny węzła.

ROZDZIAŁ 6.
Etykiety i adnotacje

Kubernetes z założenia ma rozwijać się razem z aplikacją, zarówno pod względem wielkości, jak i złożoności. Etykiety i adnotacje to fundamentalne koncepcje w Kubernetes. Umożliwiają pracę w zbiorach elementów, które odwzorowują *Twój* sposób myślenia o aplikacji. Możesz organizować, oznaczać i indeksować wszystkie swoje zasoby, aby reprezentowały takie grupy, które mają największy sens dla aplikacji.

Etykiety to dowolne pary klucz-wartość, które można dołączać do obiektów Kubernetes, takich jak kapsuły i `ReplicaSet`. Umożliwiają dodanie do tych obiektów informacji identyfikujących. Etykiety stanowią podstawę grupowania obiektów.

Adnotacje natomiast zapewniają mechanizm przechowywania przypominający etykiety: są parami klucz-wartość zaprojektowanymi do przechowywania nieidentyfikujących informacji, które mogą być wykorzystywane przez narzędzia i biblioteki. Adnotacje, w odróżnieniu od etykiet, nie służą do tworzenia zapytań, filtrowania ani rozróżniania kapsuł w jakikolwiek inny sposób.

Etykiety

Etykiety zapewniają metadane identyfikujące dla obiektów. Są to podstawowe cechy obiektów, które będą używane do ich grupowania, przeglądania i obsługi. Decyzja o dodaniu do Kubernetes etykiet była oparta na doświadczeniach firmy Google w zakresie uruchamiania dużych i złożonych aplikacji. Z tych doświadczeń wyciągnięto dwa wnioski:

- Środowisko produkcyjne nie znosi singletonów. Gdy wdrażane jest oprogramowanie, użytkownicy często zaczynają od pojedynczej instancji. Jednak w miarę dojrzewania aplikacji te singletony mnożą się i stają się zbiorami obiektów. Z tego względu Kubernetes używa etykiet do obsługi zbiorów obiektów zamiast pojedynczych instancji.
- Każda hierarchia narzucona przez system będzie dla wielu użytkowników niewystarczająca. Ponadto grupowanie użytkowników i hierarchia mogą z czasem ulegać zmianie. Użytkownik może na przykład na początku mieć wyobrażenie, że wszystkie aplikacje składają się z wielu usług. Jednak z czasem usługa może zostać udostępniona w wielu aplikacjach. Etykiety Kubernetes są wystarczająco elastyczne, aby można je było dostosować do tych i innych sytuacji.

Więcej informacji o tym, w jaki sposób Google podchodzi do systemów produkcyjnych, znajdziesz w książce *Site Reliability Engineering. Jak Google zarządza systemami produkcyjnymi* autorstwa Betsy Beyer i in. (Helion).

Etykiety mają prostą składnię. Są to pary klucz-wartość, w których oba elementy są reprezentowane przez łańcuchy znaków. Klucze etykiet można podzielić na dwie oddzielone ukośnikiem części: opcjonalny przedrostek i nazwę. Jeśli przedrostek został określony, musi być poddomeną DNS z limitem 253 znaków. Nazwa klucza jest wymagana i musi być krótsza niż 63 znaki. Nazwy muszą ponadto zaczynać się i kończyć znakiem alfanumerycznym i dopuszczalne jest używanie kresek (`-`), podkreśleń (`_`) oraz kropek (`.`) między znakami.

Wartościami etykiet są łańcuchy znaków o maksymalnej długości 63 znaków. Do wartości etykiet stosuje się te same reguły co do kluczy etykiet. Tabela 6.1 przedstawia prawidłowe klucze i wartości etykiet.

Tabela 6.1. Przykłady etykiet

Klucz	Wartość
`acme.com/app-version`	`1.0.0`
`appVersion`	`1.0.0`
`app.version`	`1.0.0`
`kubernetes.io/cluster-service`	`true`

Domeny wykorzystywane w etykietach i adnotacjach powinny w jakiś sposób zgadzać się z daną jednostką. Na przykład projekt może definiować kanoniczny zestaw etykiet służących do identyfikacji różnych etapów wdrażania aplikacji (*staging*, *canary*, *production* itp.). Alternatywnie dostawca chmury może zdefiniować własne adnotacje rozszerzające obiekty Kubernetes do aktywacji funkcji specyficznych dla swojej usługi.

Stosowanie etykiet

Utworzymy tutaj kilka wdrożeń (w taki sposób, w jaki tworzy się tablicę kapsuł) z paroma ciekawymi etykietami. Będziemy mieli dwie aplikacje (zwane `alpaca` i `bandicoot`) i dwa środowiska dla każdej z nich. Będziemy mieli także dwie różne wersje.

Najpierw utwórz wdrożenie `alpaca-prod` i ustaw etykiety `ver`, `app` i `env`:

```
$ kubectl run alpaca-prod \
  --image=gcr.io/kuar-demo/kuard-amd64:blue \
  --replicas=2 \
  --labels="ver=1,app=alpaca,env=prod"
```

Następnie utwórz wdrożenie `alpaca-test` i ustaw etykiety `ver`, `app` i `env` z odpowiednimi wartościami:

```
$ kubectl run alpaca-test \
  --image=gcr.io/kuar-demo/kuard-amd64:green \
  --replicas=1 \
  --labels="ver=2,app=alpaca,env=test"
```

Na koniec utwórz dwa wdrożenia dla aplikacji `bandicoot`. Tutaj nazywamy środowiska `prod` i `staging`:

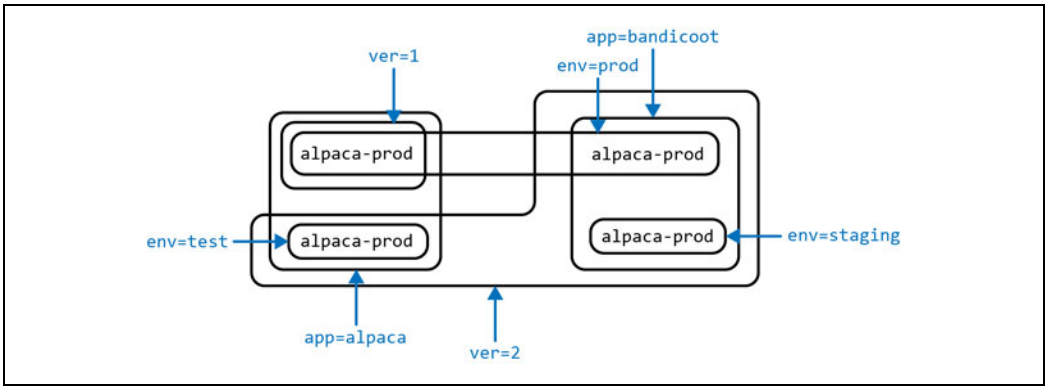
```
$ kubectl run bandicoot-prod \
  --image=gcr.io/kuar-demo/kuard-amd64:green \
  --replicas=2 \
  --labels="ver=2,app=bandicoot,env=prod"
$ kubectl run bandicoot-staging \
  --image=gcr.io/kuar-demo/kuard-amd64:green \
  --replicas=1 \
  --labels="ver=2,app=bandicoot,env=staging"
```

W tym momencie powinieneś mieć cztery wdrożenia: `alpaca-prod`, `alpaca-test`, `bandicoot-prod` i `bandicoot-staging`:

```
$ kubectl get deployments --show-labels

NAME                ... LABELS
alpaca-prod         ... app=alpaca,env=prod,ver=1
alpaca-test         ... app=alpaca,env=test,ver=2
bandicoot-prod      ... app=bandicoot,env=prod,ver=2
bandicoot-staging   ... app=bandicoot,env=staging,ver=2
```

Możemy to przedstawić jako diagram Venna oparty na etykietach (zobacz rysunek 6.1).



Rysunek 6.1. Wizualizacja etykiet zastosowanych w naszych wdrożeniach

Modyfikowanie etykiet

Etykiety można również stosować (lub aktualizować) do obiektów po ich utworzeniu.

```
$ kubectl label deployments alpaca-test "canary=true"
```

Należy pamiętać o jednym zastrzeżeniu. W tym przykładzie polecenie `kubectl label` zmieni tylko etykietę w samym wdrożeniu; nie wpłynie to na obiekty (`ReplicaSet` i `Pod`) tworzone przez to wdrożenie. Aby je zmienić, musisz zmienić szablon osadzony we wdrożeniu (zobacz rozdział 10.).

Możesz także użyć opcji `-L` z poleceniem `kubectl get`, aby wyświetlić wartości etykiet w kolumnie:

```
$ kubectl get deployments -L canary
NAME                DESIRED    CURRENT    ... CANARY
alpaca-prod         2          2          ... <none>
alpaca-test         1          1          ... true
bandicoot-prod      2          2          ... <none>
bandicoot-staging   1          1          ... <none>
```

Etykietę możesz usunąć, stosując przyrostek w postaci kreski:

```
$ kubectl label deployments alpaca-test "canary-"
```

Selektory etykiet

Selektory etykiet są wykorzystywane do filtrowania obiektów Kubernetes na podstawie zestawu etykiet. Selektory używają prostych wyrażeń logicznych. Są one wykorzystywane zarówno przez użytkowników końcowych (za pośrednictwem takich narzędzi jak `kubectl`), jak i przez różne typy obiektów (na przykład do określenia relacji obiektu `ReplicaSet` do jego obiektów `Pod`). Każde wdrożenie (poprzez `ReplicaSet`) powoduje utworzenie zestawu obiektów `Pod` (kapsuł) za pomocą etykiet określonych w szablonie osadzonym we wdrożeniu. Konfiguruje się to poleceniem `kubectl run`.

Uruchomienie polecenia `kubectl get pods` powinno skutkować zwróceniem wszystkich aktualnych uruchomionych w klastrze kapsuł. Powinniśmy mieć w sumie sześć kapsuł `kuard` w trzech środowiskach:

```
$ kubectl get pods --show-labels

NAME                              ... LABELS
alpaca-prod-3408831585-4nzfb      ... app=alpaca,env=prod,ver=1,...
alpaca-prod-3408831585-kga0a      ... app=alpaca,env=prod,ver=1,...
alpaca-test-1004512375-3r1m5      ... app=alpaca,env=test,ver=2,...
bandicoot-prod-373860099-0t1gp    ... app=bandicoot,env=prod,ver=2,...
bandicoot-prod-373860099-k2wcf    ... app=bandicoot,env=prod,ver=2,...
bandicoot-staging-1839769971-3ndv ... app=bandicoot,env=staging,ver=2,...
```

Możesz zobaczyć nową etykietę, której nie widziałeś wcześniej: `pod-template-hash`. Ta etykieta jest stosowana przez wdrożenie, dzięki czemu może ono śledzić, które kapsuły zostały wygenerowane z określonych wersji szablonu. Pozwala to zarządzać aktualizacjami w czysty sposób, co zostanie omówione szczegółowo w rozdziale 10.

Gdybyśmy chcieli wyświetlić jedynie listę kapsuł z etykietą `ver` ustawioną na `2`, moglibyśmy użyć flagi `--selector`:

```
$ kubectl get pods --selector="ver=2"

NAME                                READY    STATUS    RESTARTS    AGE
alpaca-test-1004512375-3r1m5        1/1      Running   0           3m
bandicoot-prod-373860099-0t1gp      1/1      Running   0           3m
bandicoot-prod-373860099-k2wcf      1/1      Running   0           3m
bandicoot-staging-1839769971-3ndv5  1/1      Running   0           3m
```

Jeśli określimy dwa selektory oddzielone przecinkiem, zwrócone zostaną tylko te obiekty, które spełniają oba warunki. Jest to operacja logiczna AND:

```
$ kubectl get pods --selector="app=bandicoot,ver=2"

NAME                                READY    STATUS    RESTARTS    AGE
bandicoot-prod-373860099-0t1gp      1/1      Running   0           4m
bandicoot-prod-373860099-k2wcf      1/1      Running   0           4m
bandicoot-staging-1839769971-3ndv5  1/1      Running   0           4m
```

Możemy również zapytać, czy dana etykieta zawiera którąś z wartości ze zbioru wartości. Tutaj zapytamy o wszystkie kapsuły, w których etykieta `app` jest ustawiona na `alpaca` lub `bandicoot` (czyli o wszystkie sześć kapsuł):

```
$ kubectl get pods --selector="app in (alpaca,bandicoot)"

NAME                                READY    STATUS    RESTARTS    AGE
alpaca-prod-3408831585-4nzfb        1/1      Running   0           6m
alpaca-prod-3408831585-kga0a        1/1      Running   0           6m
alpaca-test-1004512375-3r1m5        1/1      Running   0           6m
bandicoot-prod-373860099-0t1gp      1/1      Running   0           6m
bandicoot-prod-373860099-k2wcf      1/1      Running   0           6m
bandicoot-staging-1839769971-3ndv5  1/1      Running   0           6m
```

Na koniec możemy zapytać, czy dana etykieta jest w ogóle ustawiona. Tutaj prosimy o wszystkie wdrożenia z etykietą `canary` ustawioną na dowolną wartość:

```
$ kubectl get deployments --selector="canary"

NAME          DESIRED  CURRENT  UP-TO-DATE  AVAILABLE  AGE
alpaca-test   1        1        1           1          7m
```

Istnieją również „odwrotne” wersje każdego operatora, tak jak pokazano w tabeli 6.2.

Tabela 6.2. Operatory selektorów

Operator	Opis
`key=value`	klucz `key` jest ustawiony na wartość `value`
`key!=value`	`key` nie jest ustawiony na `value`
`key in (value1, value2)`	`key` jest jedną z wartości `value1` lub `value2`
`key notin (value1, value2)`	`key` nie jest jedną z wartości `value1` lub `value2`
`key`	`key` jest ustawiony
`!key`	`key` nie jest ustawiony

Na przykład zapytanie o to, czy klucz, w tym przypadku `canary`, nie jest ustawiony, może wyglądać następująco:

```
$ kubectl get deployments --selector='!canary'
```

Pozytywne i negatywne selektory można grupować, na przykład:

```
$ kubectl get pods -l 'ver=2,!canary'
```

Selektory etykiet w obiektach API

Obiekt Kubernetes odwołuje się do zestawu innych obiektów Kubernetes za pomocą selektora etykiet. Zamiast prostego łańcucha znaków, tak jak opisaliśmy w poprzednim punkcie, używana jest parsowana struktura.

Ze względów historycznych (Kubernetes nie narusza kompatybilności API!) istnieją dwie formy stosowania selektorów. Większość obiektów obsługuje nowszy, bardziej wszechstronny zestaw operatorów selektorów. Selektor `app=alpaca,ver in (1, 2)` zostanie przekonwertowany na następujący:

```
selector:
  matchLabels:
    app: alpaca
  matchExpressions:
    - {key: ver, operator: In, values: [1, 2]}
```

W tym przykładzie użyto kompaktowej składni YAML. Jest to pozycja na liście (`matchExpressions`), która jest mapą z trzema wpisami. Ostatni wpis (`values`) ma wartość, która jest listą z dwoma elementami. Wszystkie warunki są ewaluowane jako logiczne AND. Jedynym sposobem na przedstawienie operatora `!=` jest przekonwertowanie go na wyrażenie `NotIn` z jedną wartością.

Starsza forma określania selektorów (używana w obiektach `ReplicationController` i usługach) obsługuje tylko operator `=`. Jest to prosty zestaw par klucz-wartość, z których wszystkie muszą pasować, aby obiekt docelowy został wybrany. Selektor `app=alpaca,ver=1` byłby reprezentowany w następujący sposób:

```
selector:
  app: alpaca
  ver: 1
```

Etykiety w architekturze Kubernetes

Oprócz tego, że umożliwiają użytkownikom organizację infrastruktury, etykiety odgrywają także ważną rolę w łączeniu ze sobą różnych spokrewnionych obiektów Kubernetes. System Kubernetes jest celowo pozbawiony powiązań, to znaczy nie ma hierarchii i wszystkie składniki działają niezależnie. Mimo to w wielu przypadkach obiekty muszą się do siebie odnosić i relacje te wyrażają właśnie etykiety oraz selektory etykiet.

Na przykład obiekty `ReplicaSet`, służące do tworzenia i utrzymywania wielu replik kapsuły, znajdują kapsuły, którymi zarządzają, za pomocą selektora. Analogicznie obiekt równoważenia obciążenia usługi znajduje kapsuły, do których powinien kierować ruch, za pomocą zapytania z selektorem. Podczas tworzenia kapsuły przy użyciu jej selektora węzłów można określić zbiór węzłów, na których może zostać rozplanowana. Jeśli trzeba ograniczyć natężenie ruchu w klastrze, można użyć `NetworkPolicy` z etykietami określającymi kapsuły, które powinny lub nie powinny komunikować się ze sobą.

Etykiety to rodzaj uniwersalnego kleju, który spaja całą aplikację Kubernetes. Początkowo Twoja aplikacja będzie miała skromny zestaw etykiet i zapytań, ale z czasem na pewno będzie się on rozrastał i stawał coraz bardziej zaawansowany.

Adnotacje

Adnotacje zapewniają miejsce do przechowywania dodatkowych metadanych dla obiektów Kubernetes, a jedynym celem ich wykorzystania jest wspomaganie narzędzi i bibliotek. Umożliwiają innym programom kierującym Kubernetes poprzez API przechowywanie w obiektach nieprzezroczystych danych. Adnotacje mogą być używane dla samego narzędzia lub do przekazywania informacji konfiguracyjnych między zewnętrznymi systemami.

Podczas gdy etykiety służą do identyfikowania i grupowania obiektów, adnotacje są stosowane do zapewniania dodatkowych informacji o pochodzeniu obiektu, sposobie jego użycia lub związanych z nim regułach. W pewnym zakresie etykiety i adnotacje nakładają się na siebie i jedynie od nas zależy, kiedy będziemy stosować jedne lub drugie. W razie wątpliwości dodawaj informacje do obiektu jako adnotacje i zamieniaj je na etykiety, jeżeli zechcesz ich użyć w selektorze.

Adnotacje mają następujące zastosowania:

- Śledzenie „przyczyny" ostatniej aktualizacji obiektu.
- Przekazywanie wyspecjalizowanej polityki planowania do specjalistycznego planisty.
- Poszerzanie danych o ostatnim narzędziu aktualizującym zasób i sposobie jego zaktualizowania (używane do wykrywania zmian za pomocą innych narzędzi i wykonywania inteligentnego scalania).
- Przechowywanie informacji dotyczących kompilacji, wydania lub obrazu, które nie są odpowiednie dla etykiet (mogą obejmować wartość skrótu Git, znacznik czasu, liczbę pobrań itd.).
- Umożliwienie obiektowi `Deployment` (zobacz rozdział 10.) śledzenie obiektów `ReplicaSet`, którymi zarządza pod kątem wprowadzania nowych wersji.
- Zapewnianie dodatkowych danych, aby poprawić jakość wizualną lub użyteczność interfejsu użytkownika. Obiekty mogą na przykład zawierać łącze dla ikony (lub wersję ikony zakodowaną w base64).
- Realizowanie prototypowej funkcjonalności alfa w Kubernetes (zamiast tworzenia pierwszoklasowych pól API parametry dla tej funkcjonalności są kodowane w adnotacji).

Adnotacje są wykorzystywane w różnych miejscach w Kubernetes, a podstawowym przypadkiem użycia jest wprowadzanie wdrożeń. Podczas wprowadzania wdrożeń adnotacje są używane do śledzenia statusu wprowadzania i zapewniania niezbędnych informacji wymaganych do wycofania wdrożenia w celu powrotu do poprzedniego stanu.

Użytkownicy powinni unikać wykorzystywania serwera API Kubernetes jako bazy danych ogólnego zastosowania. Adnotacje są dobre dla małych porcji danych, które są ściśle związane z konkretnymi zasobami. Jeśli chcesz przechowywać dane w Kubernetes, ale nie masz żadnego oczywistego obiektu, z którym możesz je powiązać, rozważ przechowywanie tych danych w innej, bardziej odpowiedniej bazie danych.

Klucze adnotacji mają ten sam format co klucze etykiet. Ponieważ jednak są one często używane do przekazywania informacji między narzędziami, ważniejsza jest ta część klucza, która stanowi jego „przestrzeń nazw". Przykładowe klucze to: `deployment.kubernetes.io/revision` lub `kubernetes.io/change-cause`.

Wartość adnotacji jest polem łańcucha znaków o dowolnej postaci. Zapewnia to maksymalną elastyczność, ponieważ użytkownicy mogą przechowywać dowolne dane; nie ma walidacji żadnego formatu, gdyż jest to dowolny tekst. Często na przykład dokumenty JSON są kodowane jako łańcuchy znaków i przechowywane w adnotacji. Należy zwrócić uwagę, że serwer Kubernetes nie ma wiedzy o wymaganym formacie wartości adnotacji. Jeżeli adnotacje są używane do przekazywania lub przechowywania danych, nie ma gwarancji, że dane będą prawidłowe. Może to utrudnić wyszukiwanie błędów.

Adnotacje są definiowane w sekcji `metadata` w każdym obiekcie Kubernetes:

```
...
metadata:
  annotations:
    example.com/icon-url: "https://example.com/icon.png"
...
```

Adnotacje są bardzo przydatne i umożliwiają postępowanie zgodnie z zasadą stosowania luźnych powiązań, ale używaj ich rozważnie, aby uniknąć bałaganu z danymi bez określonego typu.

Czyszczenie

Łatwo można wyczyścić wszystkie wdrożenia, które rozpoczęliśmy w tym rozdziale:

```
$ kubectl delete deployments --all
```

Jeśli chcesz wyczyścić tylko niektóre, możesz użyć flagi `--selector`, aby wybrać, które wdrożenia usunąć.

Podsumowanie

Etykiety służą do identyfikacji i, opcjonalnie, grupowania obiektów w klastrze Kubernetes. Etykiety są również używane w zapytaniach selektora w celu zapewnienia elastycznego grupowania w środowisku wykonawczym obiektów takich jak kapsuły.

Adnotacje zapewniają obiektowe magazyny danych typu klucz-wartość do przechowywania metadanych, które mogą być używane przez narzędzia automatyzacji i biblioteki klienckie. Adnotacje mogą również służyć do przechowywania danych konfiguracyjnych dla zewnętrznych narzędzi, takich jak niezależne programy planujące i narzędzia do monitorowania.

Etykiety i adnotacje są kluczem do zrozumienia, w jaki sposób współpracują ze sobą najważniejsze elementy w klastrze Kubernetes, aby zapewnić pożądany stan klastra. Właściwe używanie etykiet i adnotacji pozwala w pełni wykorzystać elastyczność Kubernetes i zapewnia punkt wyjścia do budowania narzędzi automatyzacji i przepływów pracy wdrożeń.

ROZDZIAŁ 7.

Wykrywanie usług

Kubernetes jest bardzo dynamicznym systemem. Ten system jest zaangażowany w umieszczanie kapsuł na węzłach, gwarantowanie ich poprawnego działania i w razie potrzeby ponowne ich rozplanowywanie. Istnieją sposoby automatycznej zmiany liczby kapsuł na podstawie obciążenia, na przykład poziome autoskalowanie kapsuł (zobacz rozdział 9., podrozdział „Skalowanie kontrolerów ReplicaSet"). Ponieważ system ten jest oparty na interfejsie API, zachęca to innych programistów do tworzenia coraz wyższych poziomów automatyzacji.

Chociaż dynamiczna natura Kubernetes ułatwia uruchamianie wielu procesów, powoduje problemy, jeśli chodzi o *znajdowanie* tych elementów. Większość tradycyjnej infrastruktury sieci nie została zbudowana z uwzględnieniem takiego poziomu dynamizmu, jaki prezentuje Kubernetes.

Co to jest wykrywanie usług?

Ogólna nazwa dla opisanej w tym podrozdziale klasy problemów i rozwiązań to **wykrywanie usług** (ang. *service discovery*). Narzędzia wykrywania usług pomagają rozwiązać problem określania, które procesy nasłuchują na konkretnych adresach dla poszczególnych usług. Dobry system wykrywania usług pozwala użytkownikom w szybki i niezawodny sposób pozyskiwać te informacje. Dobry system charakteryzuje się również małymi opóźnieniami, klienty są aktualizowane wkrótce po zmianie informacji związanych z usługą. Wreszcie, dobry system wykrywania usług może przechowywać bogatszą definicję tego, czym jest usługa. Przykładowo z usługą może być powiązanych wiele portów.

System DNS (ang. *Domain Name System*) to tradycyjny system wykrywania usług w internecie. DNS jest przeznaczony do względnie stabilnego rozwiązywania nazw dzięki obszernemu i wydajnemu buforowaniu. Świetnie sprawdza się w internecie, ale jest niewystarczający w dynamicznym świecie Kubernetes.

Niestety wiele systemów (na przykład domyślnie Java) szuka nazw bezpośrednio w DNS i nigdy ponownie ich nie rozwiązuje. Może to doprowadzić do tego, że klienty będą buforować nieaktualne mapowania i komunikować się z niewłaściwymi adresami IP. Nawet przy krótkich czasach TTL i właściwie działających klientach istnieje naturalne opóźnienie między zmianą rozwiązywania nazw a momentem odnotowania tego przez klienta. Istnieją również naturalne ograniczenia ilości i rodzaju informacji, które mogą być zwracane dla typowego zapytania DNS. Wszystko zaczyna się psuć po 20 – 30 rekordach A dla jednej nazwy. Niektóre problemy mogą być rozwiązane

przez rekordy SRV, ale często są one bardzo trudne w użyciu. Wreszcie, sposób obsługi wielu adresów IP w rekordzie DNS przez klienty polega na tym, że zwykle pobierają one pierwszy adres IP i polegają na serwerze DNS, aby ustalił kolejność rekordów losowo lub na podstawie algorytmu round-robin. Nie ma żadnego substytutu dla lepiej zaprojektowanego mechanizmu równoważenia obciążenia.

Obiekt Service

Prawdziwe wykrywanie usług w Kubernetes rozpoczyna się od obiektu `Service`. Obiekt `Service` służy do tworzenia nazwanego selektora etykiet. Jak zobaczysz, ten obiekt robi dla nas również inne ciekawe rzeczy.

Tak jak wykorzystanie polecenia `kubectl run` jest łatwym sposobem tworzenia wdrożenia Kubernetes, tak możemy używać polecenia `kubectl expose` do tworzenia usług. Szczegółowy opis wdrożeń znajdziesz w rozdziale 10., więc na razie możesz je traktować jako egzemplarze mikrousług. Utwórzmy kilka wdrożeń i usług, żebyś mógł zobaczyć, jak działają:

```
$ kubectl create deployment alpaca-prod \
    --image=gcr.io/kuar-demo/kuard-amd64:blue \
    --port=8080
$ kubectl scale deployment alpaca-prod --replicas 3
$ kubectl expose deployment alpaca-prod
$ kubectl create deployment bandicoot-prod \
    --image=gcr.io/kuar-demo/kuard-amd64:green \
    --port=8080
$ kubectl scale deployment bandicoot-prod --replicas 2
kubectl expose deployment bandicoot-prod
$ kubectl get services -o wide

NAME            CLUSTER-IP    ... PORT(S)  ... SELECTOR
alpaca-prod     10.115.245.13 ... 8080/TCP ... app=alpaca
bandicoot-prod  10.115.242.3  ... 8080/TCP ... app=bandicoot
kubernetes      10.115.240.1  ... 443/TCP  ... <none>
```

Po uruchomieniu tych poleceń mamy trzy usługi. Te, które właśnie utworzyliśmy, to: `alpaca-prod` i `bandicoot-prod`. Usługa `kubernetes` jest tworzona automatycznie, abyś mógł znaleźć interfejs API Kubernetes z poziomu aplikacji i komunikować się z nim.

Gdy spojrzymy na kolumnę `SELECTOR`, zobaczymy, że usługa `alpaca-prod` nadaje po prostu nazwę selektorowi i określa, na których portach należy komunikować się z tą usługą. Wykorzystanie polecenia `kubectl expose` to wygodny sposób, by pobrać zarówno selektor etykiet, jak i odpowiednie porty (w tym przypadku 8080) z definicji wdrożenia.

Ponadto do tej usługi przypisywany jest nowy typ wirtualnego adresu IP, zwany **adresem IP klastra**. Jest to specjalny adres IP, którego system będzie używał do równoważenia obciążenia na wszystkich kapsułach, które są identyfikowane przez selektor.

Aby wejść w interakcję z usługami, skonfigurujemy przekierowanie portów do jednej z kapsuł `alpaca`. Uruchom to polecenie i pozostaw je uruchomione w oknie terminala. Uzyskując dostęp do kapsuły `alpaca` pod adresem *http://localhost:48858*, możesz sprawdzić, czy przekierowanie portów działa:

```
$ ALPACA_POD=$(kubectl get pods -l app=alpaca \
    -o jsonpath='{.items[0].metadata.name}')
$ kubectl port-forward $ALPACA_POD 48858:8080
```

DNS usługi

Ponieważ adres IP klastra jest wirtualny, jest stabilny i należy nadać mu adres DNS. Nie musimy się już przejmować żadnymi problemami związanymi z buforowaniem przez klienty wyników DNS. W ramach przestrzeni nazw wystarczy po prostu użyć nazwy usługi, aby połączyć się z jedną z kapsuł identyfikowanych przez usługę.

Kubernetes zapewnia usługę DNS udostępnianą kapsułom działającym w klastrze. Ta usługa DNS Kubernetes została zainstalowana jako składnik systemu, gdy został utworzony klaster. Sama usługa DNS jest zarządzana przez Kubernetes i jest świetnym przykładem tego, jak ten buduje usługi, opierając się na samym sobie. Usługa DNS Kubernetes zapewnia nazwy DNS dla adresów IP klastra.

Możesz to sprawdzić, rozwijając sekcję *DNS Query* na stronie statusu serwera `kuard`. Wykonaj kwerendę rekordu A dla `alpaca-prod`. Dane wyjściowe powinny wyglądać mniej więcej tak:

```
;; opcode: QUERY, status: NOERROR, id: 12071
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;alpaca-prod.default.svc.cluster.local. IN       A

;; ANSWER SECTION:
alpaca-prod.default.svc.cluster.local.  30      IN     A        10.115.245.13
```

Pełna nazwa DNS to `alpaca-prod.default.svc.cluster.local.`; przeanalizujmy ją:

`alpaca-prod`
: to nazwa danej usługi.

`default`
: jest przestrzenią nazw, w której znajduje się ta usługa.

`svc`
: wskazuje, że jest to usługa. Dzięki temu Kubernetes może później udostępniać inne typy rzeczy jako DNS.

`cluster.local.`
: podstawowa nazwa domeny dla klastra. Jest to ustawienie domyślne i zobaczysz je w większości klastrów. Administratorzy mogą je zmienić, aby umożliwić stosowanie unikatowych nazwy DNS w wielu klastrach.

Gdy odwołujesz się do usługi we własnej przestrzeni nazw, możesz po prostu użyć nazwy usługi (`alpaca-prod`). Możesz także odwoływać się do usługi w innej przestrzeni nazw za pomocą `alpaca-prod.default`. Oczywiście można również użyć w pełni kwalifikowanej nazwy usługi (`alpaca-prod.default.svc.cluster.local.`). Wypróbuj każdą z nich w sekcji *DNS Query* serwera `kuard`.

Kontrole gotowości

Często gdy aplikacja jest uruchamiana po raz pierwszy, nie jest gotowa do obsługi żądań. Zwykle wymagane jest jakieś inicjowanie, które może zająć od kilkudziesięciu milisekund do kilku minut. Jedną z przyjemnych czynności wykonywanych przez obiekt `Service` jest śledzenie poprzez kontrole gotowości, które z kapsuł są gotowe. Zmodyfikujmy nasze wdrożenie, aby dodać kontrolę gotowości dołączoną do kapsuły, zgodnie z opisem w rozdziale 5.:

```
$ kubectl edit deployment/alpaca-prod
```

To polecenie spowoduje pobranie aktualnej wersji wdrożenia `alpaca-prod` i wyświetlenie jej w edytorze. Po zapisaniu i zamknięciu edytora obiekt zostanie ponownie zapisany w Kubernetes. Jest to szybki sposób edycji obiektu bez zapisywania go w pliku YAML.

Dodaj następującą sekcję kodu:

```
spec:
  ...
  template:
    ...
    spec:
      containers:
        ...
        name: alpaca-prod
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          periodSeconds: 2
          initialDelaySeconds: 0
          failureThreshold: 3
          successThreshold: 1
```

Ustawiamy w ten sposób, że kapsuły tworzone przez to wdrożenie będą sprawdzane pod kątem gotowości poprzez żądanie HTTP `GET` dla `/ready` na porcie `8080`. Ta kontrola jest przeprowadzana co 2 sekundy od momentu uruchomienia kapsuły. Jeżeli nie powiodą się trzy kolejne kontrole, kapsuła zostanie uznana za niegotową. Jeśli jednak powiedzie się jedna kontrola, kapsuła ponownie zostanie uznana za gotową.

Ruch jest wysyłany tylko do gotowych kapsuł.

Taka aktualizacja definicji wdrożenia spowoduje usunięcie i odtworzenie kapsuł `alpaca`. W związku z tym musimy ponownie uruchomić zastosowane wcześniej polecenie przekierowania portów:

```
$ ALPACA_POD=$(kubectl get pods -l app=alpaca-prod \
    -o jsonpath='{.items[0].metadata.name}')
$ kubectl port-forward $ALPACA_POD 48858:8080
```

Po wpisaniu w przeglądarce adresu *http://localhost:48858* powinieneś zobaczyć stronę debugowania dla tej instancji `kuard`. Rozwiń sekcję *Readiness Probe* (kontrola gotowości). Strona powinna zacząć aktualizować się przy każdej nowej kontroli gotowości z systemu, co powinno odbywać się co 2 sekundy.

W innym oknie terminala uruchom polecenie `watch` na punktach końcowych dla usługi `alpaca-prod`. Punkty końcowe są sposobem niższego poziomu pozwalającym na wykrywanie, do czego usługa wysyła ruch, i zostaną omówione w dalszej części tego rozdziału. Zastosowana tutaj opcja `--watch` powoduje, że polecenie `kubectl` pozostaje uruchomione i wyświetla wszelkie aktualizacje. To prosty sposób, aby zobaczyć, jak z biegiem czasu zmienia się obiekt Kubernetes:

```
$ kubectl get endpoints alpaca-prod --watch
```

Teraz wróć do przeglądarki i kliknij link *Fail* (niepowodzenie kontroli) dla kontroli gotowości. Powinieneś zobaczyć, że serwer nie zwraca kodów błędu 500. Po trzech nieudanych kontrolach serwer zostanie usunięty z listy punktów końcowych dla usługi. Kliknij link *Succeed* (powodzenie kontroli), a zauważysz, że po pojedynczym sprawdzeniu gotowości punkt końcowy zostanie dodany z powrotem.

Taka kontrola gotowości jest sposobem, aby przeciążony lub chory serwer zasygnalizował systemowi, że nie chce już otrzymywać ruchu. To świetna metoda na zaimplementowanie łagodnego zamknięcia systemu. Serwer może zasygnalizować, że nie chce już otrzymywać ruchu, poczekać, aż zostaną zamknięte istniejące połączenia, a następnie czysto zakończyć działanie.

Naciśnij w obu oknach terminala *Ctrl+C*, aby wyjść z poleceń `port-forward` i `watch`.

Udostępnianie usługi poza klastrem

Jak dotąd wszystko, co omówiliśmy w tym rozdziale, dotyczyło udostępniania usług wewnątrz klastra. Często adresy IP dla kapsuł są dostępne tylko z wewnątrz klastra. Na pewnym etapie będziemy musieli dopuścić nowy ruch!

Najbardziej uniwersalnym sposobem na to jest użycie funkcjonalności o nazwie `NodePort`, która jeszcze bardziej usprawnia usługę. Oprócz adresu IP klastra system wybiera port (lub określa go użytkownik), na który każdy węzeł w klastrze przekazuje ruch skierowany do danej usługi.

Dzięki tej funkcjonalności możesz skontaktować się z usługą, jeśli jesteś w stanie dotrzeć do dowolnego węzła w klastrze. Korzystasz z `NodePort`, nie wiedząc, gdzie uruchomione są jakiekolwiek kapsuły dla tej usługi. Takie rozwiązanie można zintegrować ze sprzętowym lub programowym mechanizmem równoważenia obciążenia, aby jeszcze bardziej udostępnić usługę.

Wypróbuj ten sposób, modyfikując usługę `alpaca-prod`:

```
$ kubectl edit service alpaca-prod
```

Zmień pole `spec.type` na `NodePort`. Możesz to również zrobić podczas tworzenia usługi za pomocą polecenia `kubectl expose`, określając `--type=NodePort`. System przydzieli nowy `NodePort`:

```
$ kubectl describe service alpaca-prod

Name:                   alpaca-prod
Namespace:              default
Labels:                 app=alpaca
Annotations:            <none>
Selector:               app=alpaca
Type:                   NodePort
IP:                     10.115.245.13
```

```
Port:                     <unset> 8080/TCP
NodePort:                 <unset> 32711/TCP
Endpoints:                10.112.1.66:8080,10.112.2.104:8080,10.112.2.105:8080
Session Affinity:         None
No events.
```

Widzimy tutaj, że system przypisał do tej usługi port 32711. Teraz w celu uzyskania dostępu do usługi możemy skomunikować się z dowolnym węzłem klastra na tym porcie. Jeżeli znajdujesz się w tej samej sieci, możesz uzyskać do niego bezpośredni dostęp. Jeśli Twój klaster znajduje się w chmurze, możesz użyć tunelowania SSH w taki sposób:

```
$ ssh <węzeł> -L 8080:localhost:32711
```

Gdy otworzysz w przeglądarce stronę *http://localhost:8080*, zostaniesz połączony z tą usługą. Każde żądanie wysyłane do niej będzie losowo kierowane do jednej z kapsuł, które implementują tę usługę. Odśwież stronę kilka razy, a zobaczysz, że będziesz losowo przydzielany do różnych kapsuł.

Po zakończeniu zamknij sesję SSH.

Integracja z usługą równoważenia obciążenia

Jeśli klaster integruje się z zewnętrznymi usługami równoważenia obciążenia, możesz użyć typu `LoadBalancer`, o ile obsługuje go chmura, na której pracujesz (i Twój klaster jest skonfigurowany tak, aby można było z niego skorzystać). Rozszerza to funkcjonalność `NodePort` poprzez dodatkowe skonfigurowanie chmury w taki sposób, by utworzyła nowy moduł równoważenia obciążenia i skierowała go na węzły w klastrze. Większość chmurowych klastrów Kubernetes umożliwia integrację z usługami równoważenia obciążenia, istnieje też kilka projektów implementujących taką integrację dla popularnych usług tego typu, choć skorzystanie z nich wymaga trochę więcej ręcznej pracy.

Wyedytuj ponownie usługę `alpaca-prod` (`kubectl edit service alpaca-prod`) i zmień `spec.type` na `LoadBalancer`.

Usługa typu `LoadBalancer` po utworzeniu jest eksponowana w publicznym internecie. Zanim utworzysz taką usługę, upewnij się, że można ją bezpiecznie ujawnić całemu światu. W dalszej części tego rozdziału opisujemy bardziej szczegółowo kwestie bezpieczeństwa. Ponadto w rozdziałach 9. i 20. znajdują się wytyczne na temat zabezpieczania aplikacji.

Jeżeli od razu wykonasz polecenie `kubectl get services`, zobaczysz, że w kolumnie `EXTERNAL-IP` dla `alpaca-prod` znajduje się teraz informacja `<pending>` (oczekuje). Po chwili przekonasz się, że chmura przypisze usłudze publiczny adres. Możesz zajrzeć do konsoli na swoim koncie dla chmury i sprawdzić, jaką pracę konfiguracyjną wykonał dla Ciebie Kubernetes:

```
$ kubectl describe service alpaca-prod

Name:                     alpaca-prod
Namespace:                default
Labels:                   app=alpaca
Selector:                 app=alpaca
Type:                     LoadBalancer
```

```
IP:                     10.115.245.13
LoadBalancer Ingress:   104.196.248.204
Port:                   <unset> 8080/TCP
NodePort:               <unset> 32711/TCP
Endpoints:              10.112.1.66:8080,10.112.2.104:8080,10.112.2.105:8080
Session Affinity:       None
Events:
  FirstSeen ... Reason                  Message
  --------- ... ------                  -------
  3m        ... Type                    NodePort -> LoadBalancer
  3m        ... CreatingLoadBalancer    Creating load balancer
  2m        ... CreatedLoadBalancer     Created load balancer
```

Ten przykład pochodzi z klastra uruchomionego i zarządzanego na platformie Google Cloud Platform poprzez GKE. Sposób skonfigurowania `LoadBalancer` jest specyficzny dla każdej chmury. Ponadto niektóre chmury mają mechanizmy równoważenia obciążenia oparte na systemie DNS (na przykład AWS Elastic Load Balancing [ELB]). W takim przypadku zamiast adresu IP zobaczysz nazwę hosta. Dodatkowo, w zależności od dostawcy chmury, może chwilę potrwać, zanim system równoważenia obciążenia będzie w pełni sprawny.

Tutaj widzimy, że do usługi `alpaca-prod` jest teraz przypisany adres `104.196.248.204`. Otwórz przeglądarkę i wypróbuj go!

Tworzenie chmurowego mechanizmu równoważenia może zająć trochę czasu. W większości usług chmurowych należy spodziewać się, że potrwa to kilka minut.

W prezentowanych dotychczas przykładach pojawiały się *zewnętrzne* usługi równoważenia obciążenia, a więc usługi połączone z publicznym internetem. Są świetne w przypadku udostępniania światu usług, ale aplikacje często chcemy udostępniać tylko w obrębie własnej sieci prywatnej. W tym celu trzeba skorzystać z *wewnętrznej* usługi równoważenia obciążenia. Niestety obsługa takich usług została dodana do Kubernetes później, przez co zaimplementowano ją przy użyciu adnotacji do obiektów. Aby na przykład utworzyć wewnętrzną usługę równoważenia obciążenia w klastrze Azure Kubernetes Service, należy dodać adnotację `service.beta.kubernetes.io/azure-load-balancer-internal: "true"` do zasobu `Service`. Poniżej znajdują się ustawienia dla niektórych popularnych chmur:

Microsoft Azure

```
service.beta.kubernetes.io/azure-load-balancer-internal: "true"
```

Amazon Web Services

```
service.beta.kubernetes.io/aws-load-balancer-internal: "true"
```

Alibaba Cloud

```
service.beta.kubernetes.io/alibaba-cloud-loadbalancer-address-type: "intranet"
```

Google Cloud Platform

```
cloud.google.com/load-balancer-type: "Internal"
```

Po dodaniu tej adnotacji do zasobu `Service` powinno to wyglądać tak:

```
...
metadata:
  ...
  name: some-service
  annotations:
    service.beta.kubernetes.io/azure-load-balancer-internal: "true"
...
```

Kiedy utworzymy usługę z jedną z tych adnotacji, usługa ta będzie dostępna wewnętrznie, a nie w publicznym internecie.

Istnieje kilka innych adnotacji rozszerzających zachowanie `LoadBalancer`, w tym umożliwiających użycie istniejącego adresu IP. Rozszerzenia danego dostawcy powinny być opisane na jego stronie internetowej.

Szczegóły dla zaawansowanych

Kubernetes ma być z założenia systemem rozszerzalnym. Jako taki ma warstwy, które umożliwiają bardziej skomplikowane integracje. Zrozumienie szczegółów implementacji wyrafinowanych koncepcji, takich jak usługi, może pomóc w rozwiązywaniu problemów lub tworzeniu bardziej zaawansowanych połączeń. W tym podrozdziale podejrzymy, co kryje się pod powierzchnią.

Punkty końcowe

Niektóre aplikacje (i sam system) chcą móc korzystać z usług bez używania adresu IP klastra. Odbywa się to za pomocą kolejnego typu obiektu zwanego `Endpoints`. Dla każdego obiektu `Service` Kubernetes tworzy obiekt `Endpoints`, który zawiera adresy IP dla tej usługi:

```
$ kubectl describe endpoints alpaca-prod

Name:           alpaca-prod
Namespace:      default
Labels:         app=alpaca
Subsets:
  Addresses:              10.112.1.54,10.112.2.84,10.112.2.85
  NotReadyAddresses:      <none>
  Ports:
    Name        Port      Protocol
    ----        ----      --------
    <unset>     8080      TCP

No events.
```

Aby skorzystać z usługi, zaawansowana aplikacja może komunikować się bezpośrednio z API Kubernetes w celu wyszukania punktów końcowych i wywołania ich. Interfejs API Kubernetes ma nawet możliwość „obserwowania” obiektów i otrzymywania powiadomień od razu po ich zmianie. Dzięki temu klient może zareagować natychmiast, gdy tylko zmienią się adresy IP powiązane z usługą.

Sprawdźmy to. W oknie terminala wpisz następujące polecenie i pozostaw je uruchomione:

```
$ kubectl get endpoints alpaca-prod --watch
```

Wyświetlony zostanie bieżący stan punktu końcowego, po czym polecenie „zawiśnie”:

```
NAME          ENDPOINTS                                                  AGE
alpaca-prod   10.112.1.54:8080,10.112.2.84:8080,10.112.2.85:8080         1m
```

Teraz otwórz *kolejne* okno terminala i usuń oraz ponownie utwórz wdrożenie obsługujące `alpaca-prod`:

```
$ kubectl delete deployment alpaca-prod
$ kubectl create deployment alpaca-prod \
  --image=gcr.io/kuar-demo/kuard-amd64:blue \
  --port=8080 \
$ kubectl scale deployment alpaca-prod --replicas=3
```

Jeśli wrócisz do danych wyjściowych dla obserwowanego punktu końcowego, zobaczysz, że gdy usunąłeś i odtworzyłeś te kapsuły, dane wyjściowe polecenia odzwierciedliły najbardziej aktualny zestaw adresów IP związanych z usługą. Twoje dane wyjściowe będą wyglądać mniej więcej tak:

```
NAME          ENDPOINTS                                                  AGE
alpaca-prod   10.112.1.54:8080,10.112.2.84:8080,10.112.2.85:8080         1m
alpaca-prod   10.112.1.54:8080,10.112.2.84:8080                          1m
alpaca-prod   <none>                                                     1m
alpaca-prod   10.112.2.90:8080                                           1m
alpaca-prod   10.112.1.57:8080,10.112.2.90:8080                          1m
alpaca-prod   10.112.0.28:8080,10.112.1.57:8080,10.112.2.90:8080         1m
```

Obiekt `Endpoints` sprawdza się świetnie, jeśli piszesz nowy kod, który od początku jest budowany do działania na Kubernetes. Jednak większość projektów nie oferuje tej możliwości! Przeważająca część istniejących systemów jest zbudowana do pracy ze zwykłymi starymi adresami IP, które nie zmieniają się tak często.

Ręczne wykrywanie usług

Usługi Kubernetes opierają się na selektorach etykiet, które znajdują się nad kapsułami. Oznacza to, że do podstawowego wykrywania usług możesz używać interfejsu API Kubernetes, nie korzystając w ogóle z obiektu `Service`! Przyjrzyjmy się temu bliżej.

Dzięki poleceniu `kubectl` (i za pośrednictwem API) możemy łatwo zobaczyć, jakie adresy IP są przypisane do każdej kapsuły w naszych przykładowych wdrożeniach:

```
$ kubectl get pods -o wide --show-labels

NAME                      ... IP          ... LABELS
alpaca-prod-12334-87f8h   ... 10.112.1.54 ... app=alpaca,env=prod,ver=1
alpaca-prod-12334-jssmh   ... 10.112.2.84 ... app=alpaca
alpaca-prod-12334-tjp56   ... 10.112.2.85 ... app=alpaca
bandicoot-prod-5678-sbxzl ... 10.112.1.55 ... app=bandicoot
bandicoot-prod-5678-x0dh8 ... 10.112.2.86 ... app=bandicoot
```

Wszystko pięknie, ale co by się stało, gdybyśmy mieli kilkaset kapsuł? Prawdopodobnie należałoby przefiltrować je na podstawie etykiet zastosowanych w ramach wdrożenia. Zróbmy to dla samej aplikacji `alpaca`:

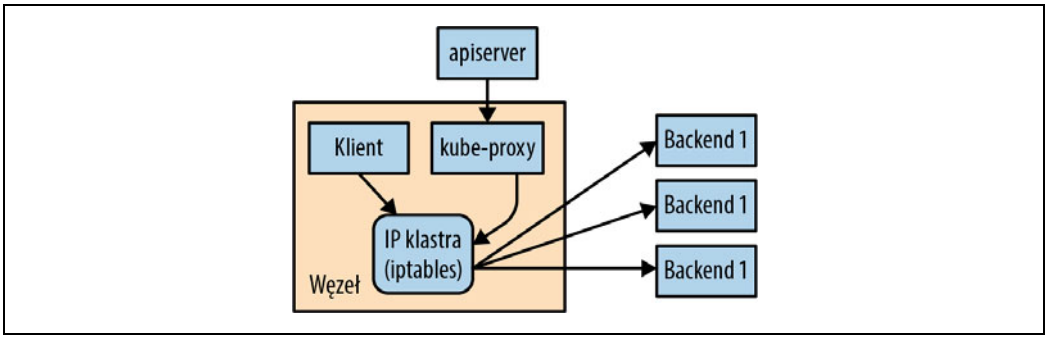
```
$ kubectl get pods -o wide --selector=app=alpaca

NAME                          ... IP          ...
alpaca-prod-3408831585-bpzdz ... 10.112.1.54 ...
alpaca-prod-3408831585-kncwt ... 10.112.2.84 ...
alpaca-prod-3408831585-l9fsq ... 10.112.2.85 ...
```

W tym momencie mamy podstawowe wykrywanie usług! Zawsze możemy użyć etykiet do zidentyfikowania interesującego nas zestawu kapsuł, pobrać wszystkie kapsuły dla tych etykiet i wygrzebać adresy IP. Jednak utrzymywanie w synchronizacji właściwego zestawu etykiet może być trudnym zadaniem. Właśnie dlatego utworzono obiekt `Service`.

kube-proxy i adresy IP klastra

Adresy IP klastra to stabilne wirtualne adresy IP, które równoważą obciążenie ruchu na wszystkich punktach końcowych usługi. Ta magia jest wykonywana przez komponent działający na każdym węźle w klastrze i noszący nazwę `kube-proxy` (zobacz rysunek 7.1).



Rysunek 7.1. Konfigurowanie i używanie adresu IP klastra

Na rysunku 7.1 `kube-proxy` obserwuje nowe usługi w klastrze za pośrednictwem serwera API. Następnie programuje zestaw reguł `iptables` w jądrze tego hosta w celu przepisania miejsc docelowych pakietów, aby pakiety te były kierowane do jednego z punktów końcowych dla tej usługi. Jeśli zmieni się zestaw punktów końcowych dla usługi (ze względu na przychodzące i odchodzące kapsuły lub z powodu nieudanych kontroli gotowości), zestaw reguł `iptables` jest przepisywany.

Sam adres IP klastra jest zwykle przypisywany przez serwer API podczas tworzenia usługi. Jednak w czasie jej tworzenia użytkownik może określić konkretny adres IP klastra. Po ustawieniu adres IP klastra nie może być modyfikowany bez usuwania i ponownego tworzenia obiektu `Service`.

Zakres adresów usług Kubernetes jest konfigurowany za pomocą flagi `--service-cluster-ip-range` w pliku binarnym *kube-apiserver*. Zakres adresów usług nie powinien pokrywać się z podsieciami i zakresami IP przypisanymi do poszczególnych mostków Dockera lub węzłów Kubernetes. Ponadto każdy bezpośrednio wybierany adres IP klastra musi pochodzić z tego zakresu i nie może być jeszcze w użyciu.

Zmienne środowiskowe adresu IP klastra

Większość użytkowników powinna korzystać z usług DNS w celu odnajdowania adresów IP klastra, ale istnieje jeszcze kilka starszych mechanizmów, które mogą nadal być w użyciu. Jednym z nich jest wstrzykiwanie do kapsuł zestawu zmiennych środowiskowych podczas ich uruchamiania.

Aby przyjrzeć się temu mechanizmowi, spójrzmy na konsolę dla instancji `bandicoot` serwera `kuard`. Wprowadź w terminalu następujące polecenia:

```
$ BANDICOOT_POD=$(kubectl get pods -l app=bandicoot \
    -o jsonpath='{.items[0].metadata.name}')
$ kubectl port-forward $BANDICOOT_POD 48858:8080
```

Teraz wpisz w przeglądarce adres *http://localhost:48858*, aby wyświetlić stronę statusu dla tego serwera. Rozwiń sekcję *Server env* (środowisko serwera) i zwróć uwagę na zestaw zmiennych środowiskowych dla usługi `alpaca`. Strona statusu powinna pokazywać tabelę podobną do tabeli 7.1.

Tabela 7.1. Zmienne środowiskowe usługi

Nazwa	Wartość
ALPACA_PROD_PORT	tcp://10.115.245.13:8080
ALPACA_PROD_PORT_8080_TCP	tcp://10.115.245.13:8080
ALPACA_PROD_PORT_8080_TCP_ADDR	10.115.245.13
ALPACA_PROD_PORT_8080_TCP_PORT	8080
ALPACA_PROD_PORT_8080_TCP_PROTO	tcp
ALPACA_PROD_SERVICE_HOST	10.115.245.13
ALPACA_PROD_SERVICE_PORT	8080

Dwie główne zmienne środowiskowe, z których można korzystać, to `ALPACA_PROD_SERVICE_HOST` i `ALPACA_PROD_SERVICE_PORT`. Pozostałe zmienne środowiskowe są tworzone dla kompatybilności z (przestarzałymi już) zmiennymi łączy Dockera.

Problem z podejściem opartym na zmiennych środowiskowych polega na tym, że wymaga ono określonej kolejności tworzenia zasobów. Usługi muszą być tworzone przed kapsułami, które się do nich odwołują. Może to wprowadzić sporo komplikacji podczas wdrażania zestawu usług, które składają się na większą aplikację. Ponadto wielu użytkownikom korzystanie z *samych* zmiennych środowiskowych wydaje się dziwne. Z tego powodu prawdopodobnie lepszym rozwiązaniem jest DNS.

Łączenie z innymi środowiskami

Choć dobrze jest mieć w klastrze mechanizm wykrywania usług, wiele rzeczywistych aplikacji wymaga zintegrowania aplikacji chmurowych wdrożonych w Kubernetes z aplikacjami wdrożonymi w starszych środowiskach. Ponadto użytkownik może zintegrować klaster Kubernetes w chmurze z infrastrukturą wdrożoną lokalnie. W tym obszarze zastosowań Kubernetes wciąż trwają intensywne prace badawcze i rozwojowe.

Łączenie z zasobami poza klastrem

Podczas łączenia klastra Kubernetes ze starymi zasobami zlokalizowanymi poza klastrem można zadeklarować usługę Kubernetes za pomocą usług niewykorzystujących selektorów i ręcznie przypisać jej adres IP spoza klastra. Dzięki temu funkcja wykrywania usług Kubernetes przez DNS działa prawidłowo, ale ruch sieciowy jest kierowany do zasobu zewnętrznego. Aby utworzyć usługę bez selektora, należy usunąć pole `spec.selector` z zasobu, pozostawiając sekcje `metadata` i `ports` bez zmian. Ponieważ taka usługa nie ma selektora, nie są do niej automatycznie dodawane żadne punkty końcowe. To znaczy, że należy je dodać ręcznie. Zazwyczaj dodawany punkt końcowy ma stały adres IP (na przykład adres IP serwera baz danych), więc wystarczy go dodać tylko raz. Jeśli jednak ten adres IP kiedykolwiek się zmieni, będzie konieczne zaktualizowanie odpowiedniego punktu końcowego. Aby utworzyć lub zaktualizować zasób punktu końcowego, należy użyć poniższego punktu:

```
apiVersion: v1
kind: Endpoints
metadata:
    # Ta nazwa musi odpowiadać nazwie Twojej usługi
    name: my-database-server
subsets:
    - addresses:
        # Zamień adres IP na prawdziwy adres IP swojego serwera
        - ip: 1.2.3.4
    ports:
        # Zamień ten port na porty, które chcesz udostępnić
        - port: 1433
```

Łączenie zasobów zewnętrznych z usługami w klastrze

Łączenie zewnętrznych zasobów z usługami Kubernetes jest odrobinę trudniejsze. Jeśli jest taka możliwość w środowisku chmurowym, z którego korzystasz, to najprostszym sposobem jest utworzenie „wewnętrznego” mechanizmu równoważącego obciążenie, który będzie działał w Twojej prywatnej sieci wirtualnej i będzie mógł kierować ruch ze stałego adresu IP do klastra. Wówczas przy użyciu zwykłych technik DNS możesz udostępnić ten adres zasobowi zewnętrznemu. Jeśli wewnętrzna usługa równoważenia obciążenia jest niedostępna, to można użyć usługi `NodePort`, aby udostępnić usługę na adresie IP węzłów w klastrze. Użytkownik może zaprogramować fizyczną usługę równoważenia obciążenia do obsługi ruchu na tych węzłach lub może użyć usługi równoważenia obciążenia opartej na DNS, by rozproszyć ruch między węzłami.

Jeśli żadne z tych rozwiązań nie wchodzi w grę w Twoim przypadku, możesz skorzystać z bardziej skomplikowanych rozwiązań, takich jak uruchomienie komponentu `kube-proxy` na zewnętrznym zasobie i zaprogramowanie tego komputera tak, aby korzystał z serwera DNS w klastrze Kubernetes. Prawidłowa konfiguracja takiego rozwiązania jest znacznie trudniejsza i dlatego z tej opcji powinno się korzystać tylko w środowiskach lokalnych. Istnieją też różne projekty open source (na przykład HashiCorp’s Consul) służące do łączenia zasobów w klastrze z zasobami poza klastrem. Aby prawidłowo skorzystać z takich opcji, trzeba dysponować dużą wiedzą zarówno w dziedzinie sieci, jak i Kubernetes, więc należy je traktować jako ostatnią deskę ratunku.

Czyszczenie

Aby wyczyścić wszystkie obiekty utworzone w tym rozdziale, uruchom następujące polecenie:

```
$ kubectl delete services,deployments -l app
```

Podsumowanie

Kubernetes to dynamiczny system, który rzuca wyzwanie tradycyjnym metodom nazywania i łączenia usług za pośrednictwem sieci. Obiekt `Service` zapewnia elastyczność i wszechstronny sposób udostępniania usług zarówno w klastrze, jak i poza nim. Dzięki omówionym w tym rozdziale technikom możesz łączyć ze sobą usługi i udostępniać je poza klastrem.

Korzystanie z dynamicznych mechanizmów wykrywania usług w Kubernetes wymaga znajomości kilku nowych koncepcji i na pierwszy rzut oka może wydawać się skomplikowane, ale zrozumienie i zaadaptowanie tych technik jest kluczem do uwolnienia mocy systemu Kubernetes. Gdy Twoja aplikacja będzie mogła dynamicznie znajdować usługi i reagować na dynamiczne rozmieszczanie aplikacji, będziesz mógł przestać przejmować się tym, gdzie uruchamiane są różne procesy i kiedy są przenoszone. Kluczowym elementem układanki jest logiczne myślenie o usługach i pozwolenie, żeby Kubernetes zajął się szczegółami rozmieszczania kontenerów.

Oczywiście odkrywanie usług to tylko wierzchołek góry lodowej, jeśli chodzi o obsługę aplikacji sieciowych w Kubernetes. W rozdziale 8. opisujemy obiekt `Ingress`, który dotyczy warstwy 7 (HTTP), a więc równoważenia obciążenia i routingu, a w rozdziale 15. prezentujemy siatki usług, które są nowszym rozwiązaniem w zakresie pracy z siecią w chmurze, oferującym wiele nowych możliwości oprócz odkrywania usług i równoważenia obciążenia.

ROZDZIAŁ 8.

Równoważenie obciążenia HTTP przy użyciu Ingress

Bardzo ważnym elementem każdej aplikacji jest przyjmowanie i wyprowadzanie ruchu sieciowego. Jak wyjaśniliśmy w rozdziale 7., w Kubernetes istnieje cały zestaw funkcji umożliwiających udostępnianie usług na zewnątrz klastra. W wielu przypadkach funkcje te są w zupełności wystarczające.

Obiekt `Service` operuje jednak na poziomie warstwy 4. (według modelu OSI[1]). Oznacza to, że tylko przekazuje połączenia TCP i UDP, w ogóle do nich nie zaglądając. Z tego powodu hosting wielu aplikacji w klastrze wykorzystuje dużo różnych udostępnionych usług. Jeśli usługi te są typu `NodePort`, klienty w każdej usłudze muszą łączyć się z unikalnym portem. Gdy natomiast usługi te są typu `LoadBalancer`, użytkownik przydziela każdej usłudze zasoby chmurowe (często kosztowne lub skąpe). Z kolei w przypadku usług HTTP (warstwa 7.) nasze możliwości są znacznie większe.

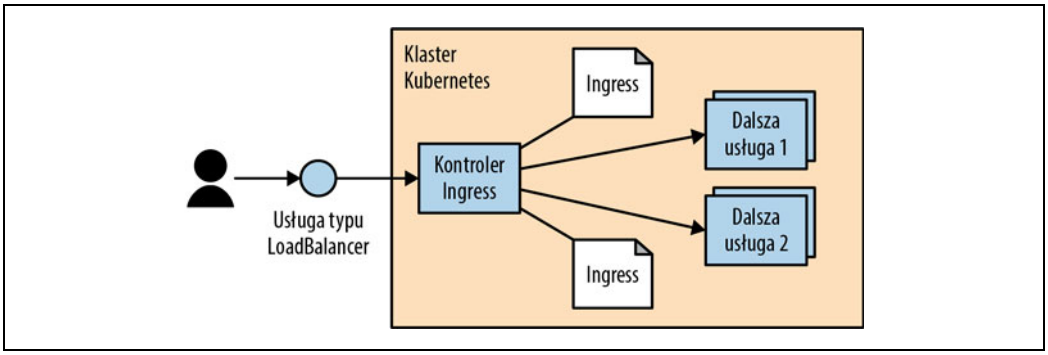
Kiedy tego typu problem trzeba rozwiązać w innym środowisku niż Kubernetes, użytkownicy często korzystają z „hostingu wirtualnego", czyli techniki pozwalającej utrzymywać kilka witryn HTTP pod jednym adresem IP. Połączenia z portami HTTP (80) i HTTPS (443) są zazwyczaj przyjmowane przez mechanizm równoważenia obciążenia lub odwrotny proxy. Później program przetwarza dane żądanie HTTP i na podstawie zawartości nagłówka `Host` i adresu URL żądania przekazuje wywołanie HTTP do innego programu. W ten sposób mechanizm równoważący obciążenie lub odwrotny proxy pełni funkcję „policjanta pilnującego ruchu", który dekoduje i przekierowuje połączenia przychodzące do odpowiedniego serwera.

W Kubernetes system równoważący obciążenie HTTP nazywa się Ingress. Służy on do implementacji omówionego właśnie wzorca „hostingu wirtualnego".

Jedną z największych trudności w posługiwaniu się tym wzorcem jest konieczność samodzielnego zarządzania plikiem konfiguracyjnym mechanizmu równoważącego obciążenie. W dynamicznym środowisku oraz gdy liczba wirtualnych hostów stanie się duża, zadanie to robi się bardzo skomplikowane. System Kubernetes Ingress upraszcza je poprzez: a) standaryzację konfiguracji, b) przeniesienie jej do standardowego obiektu Kubernetes oraz c) połączenie wielu obiektów Ingress w jedną konfigurację dla mechanizmu równoważącego obciążenie.

[1] Model Open Systems Interconnection, OSI (*https://en.wikipedia.org/wiki/OSI_model*), to standardowy sposób opisu budowy poszczególnych warstw sieci na bazie innych warstw. TCP i UDP znajdują się na warstwie 4., a HTTP — na 7.

Typowa implementacja tego rozwiązania wygląda mniej więcej tak jak na rysunku 8.1. Kontroler Ingress to system składający się z dwóch części. Pierwsza z nich to serwer proxy Ingress, który jest dostępny poza klastrem i wykorzystuje usługę typu `LoadBalancer`. Wysyła on żądania do dalszych serwerów. Drugi składnik to operator Ingress. Jego zadaniem jest odczytywanie i monitorowanie obiektów Ingress w API Kubernetes oraz konfigurowanie serwera proxy Ingress, aby kierował ruchem w sposób określony w zasobie Ingress. Istnieje wiele różnych implementacji kontrolera Ingress. W niektórych dwa wymienione komponenty są połączone w jeden kontener, a w innych są one rozdzielone i rozwijane osobno w klastrze Kubernetes. Na rysunku 8.1 jest pokazany przykładowy kontroler Ingress.



Rysunek 8.1. Typowa konfiguracja kontrolera Ingress

Specyfikacja Ingress i kontrolery Ingress

Choć koncepcja Ingress jest prosta, na poziomie implementacyjnym system ten bardzo różni się od praktycznie każdego innego obiektu w Kubernetes. Dzieli się ona na wspólną specyfikację zasobów i implementację kontrolera. W Kubernetes nie ma żadnego „standardowego" kontrolera Ingress, przez co użytkownik musi zainstalować jedną z wielu opcjonalnych implementacji.

Obiekty Ingress tworzy i modyfikuje się dokładnie tak samo jak wszystkie inne, tylko że domyślnie żaden kod ich nie wykorzystuje. Użytkownik (lub wybrana przez niego dystrybucja) sam musi zainstalować kontroler zewnętrzny i nim zarządzać. W tym sensie kontroler jest jak wtyczka.

Taki sposób implementacji systemu Ingress wynika z wielu przyczyn. Najważniejsza z nich jest taka, że nie istnieje jeden uniwersalny mechanizm równoważący obciążenie HTTP. Oprócz licznych rozwiązań programowych tego typu (zarówno open source, jak i zastrzeżonych) i funkcji dostępnych w środowiskach chmurowych (na przykład ELB w AWS) istnieją też sprzętowe mechanizmy równoważące obciążenie. Drugi powód jest taki, że obiekt Ingress został dodany do Kubernetes wcześniej niż jakiekolwiek funkcje rozszerzania (rozdział 17.). Zapewne z czasem Ingress zacznie wykorzystywać te mechanizmy.

Instalacja Contour

Istnieje wiele kontrolerów Ingress, ale w przykładach w tym rozdziale używamy Contour. Służy on do konfiguracji systemu równoważącego obciążenie open source (będącego projektem CNCF) o nazwie Envoy. System ten można dynamicznie konfigurować za pośrednictwem interfejsu API. Zadanie kontrolera Ingress Contour polega na tłumaczeniu obiektów Ingress na postać zrozumiałą dla Envoy.

Projekt Contour znajduje się pod adresem *https://oreil.ly/5IHmq*. Został stworzony przez Heptio we współpracy z prawdziwymi klientami i jest wykorzystywany w środowiskach produkcyjnych, ale obecnie stanowi niezależny projekt open source.

Do instalacji Contour wystarczy proste polecenie:

```
$ kubectl apply -f https://projectcontour.io/quickstart/contour.yaml
```

Należy tylko pamiętać, że musi je wykonać użytkownik posiadający uprawnienia administratora klastra.

Ten jeden wiersz działa w większości konfiguracji. Tworzy przestrzeń nazw `projectcontour`, a w niej wdrożenie (z dwiema replikami) i skierowaną na zewnątrz usługę typu `LoadBalancer`. Dodatkowo nadaje odpowiednie uprawnienia poprzez konto usługi i instaluje definicję `CustomResourceDefinition` (rozdział 17.) umożliwiającą korzystanie z pewnych zaawansowanych funkcji opisanych w dalszej części rozdziału, w podrozdziale „Przyszłość Ingress”.

Jako że instalacja ma charakter globalny, użytkownik, który ją przeprowadza, musi mieć szerokie uprawnienia w klastrze.

Po zakończeniu instalacji można pobrać zewnętrzny adres Contour:

```
$ kubectl get -n projectcontour service envoy -o wide
NAME      CLUSTER-IP     EXTERNAL-IP            PORT(S) ...
contour   10.106.53.14   a477...amazonaws.com   80:30274/TCP ...
```

Spójrz na kolumnę `EXTERNAL-IP`. Może zawierać adres IP (w przypadku GCP i Azure) lub nazwę hosta (w przypadku AWS). W innych chmurach i środowiskach może to być coś jeszcze innego. Jeśli klaster Kubernetes nie obsługuje usług typu `LoadBalancer`, należy zmienić typ w kodzie YAML instalacji Contour na `NodePort` i przekazywać ruch do komputerów w klastrze przez mechanizm działający w danej konfiguracji.

W przypadku minikube kolumna `EXTERNAL-IP` prawdopodobnie będzie pusta. Aby to zmienić, należy otworzyć nowe okno terminala i uruchomić tunel `minikube`. Spowoduje to skonfigurowanie tras sieciowych, dzięki czemu każdej usłudze typu `LoadBalancer` zostanie przypisany unikalny adres IP.

Konfiguracja DNS

Aby Ingress dobrze działał, należy ustawić rekordy DNS na zewnętrzny adres mechanizmu równoważącego obciążenie. Można zmapować wiele nazw hosta na jeden zewnętrzny punkt końcowy,

a kontroler Ingress jak policjant na skrzyżowaniu będzie kierował żądania przychodzące do odpowiednich usług na podstawie nazwy hosta.

Na potrzeby tego rozdziału posługujemy się przykładową domeną *example.com*. Musimy skonfigurować dwa rekordy DNS: *alpaca.example.com* i *bandicoot.example.com*. Jeśli masz adres IP zewnętrznego mechanizmu równoważącego obciążenie, utwórz rekordy A. Jeśli masz nazwę hosta, skonfiguruj rekordy CNAME.

Projekt ExternalDNS (*https://oreil.ly/ILdEj*) to dodatek do klastra umożliwiający zarządzanie rekordami DNS. Monitoruje klaster Kubernetes i synchronizuje adresy IP zasobów Kubernetes Service z zewnętrznym dostawcą DNS. Obsługuje wielu dostawców DNS, w tym zarówno tradycyjnych rejestratorów domen, jak i dostawców chmury prywatnej.

Konfiguracja pliku lokalnych hostów

Jeśli nie masz domeny lub korzystasz z lokalnego rozwiązania takiego jak minikube, możesz utworzyć lokalną konfigurację, dodając adres IP do pliku */etc/hosts*. Aby to zrobić, musisz jednak mieć uprawnienia administratora. Położenie tego pliku może się różnić w zależności od platformy, a sprawienie, żeby konfiguracja zadziałała, może wymagać wykonania paru dodatkowych czynności. Na przykład w systemie Windows plik ten najczęściej znajduje się na ścieżce *C:\Windows\System32\drivers\etc\hosts*, natomiast w nowych wersjach systemu macOS po wprowadzeniu zmian do tego pliku należy wykonać polecenie `sudo killall -HUP mDNSResponder`.

Otwórz ten plik i dodaj do niego następujący wiersz:

```
<adres-ip> alpaca.example.com bandicoot.example.com
```

W miejsce `<adres-ip>` wpisz zewnętrzny adres IP kontrolera Contour. Jeśli masz tylko nazwę hosta (na przykład od AWS), adres IP (który może się zmienić) uzyskasz za pomocą polecenia `host -t a <adres>`.

Nie zapomnij usunąć tych zmian, gdy skończysz pracę!

Praca z Ingress

Mamy skonfigurowany kontroler Ingress, więc możemy zacząć z niego korzystać. Zaczniemy od utworzenia kilku przykładowych usług, z którymi będziemy mogli pracować, za pomocą następujących poleceń:

```
$ kubectl create deployment be-default \
  --image=gcr.io/kuar-demo/kuard-amd64:blue \
  --replicas=3 \
  --port=8080
$ kubectl expose deployment be-default
$ kubectl create deployment alpaca \
  --image=gcr.io/kuar-demo/kuard-amd64:green \
  --replicas=3 \
  --port=8080
$ kubectl expose deployment alpaca
$ kubectl create deployment bandicoot \
```

```
  --image=gcr.io/kuar-demo/kuard-amd64:purple \
  --replicas=3 \
  --port=8080
$ kubectl expose deployment bandicoot
$ kubectl get services -o wide

NAME            CLUSTER-IP    ... PORT(S)  ... SELECTOR
alpaca          10.115.245.13 ... 8080/TCP ... run=alpaca
bandicoot       10.115.242.3  ... 8080/TCP ... run=bandicoot
be-default      10.115.246.6  ... 8080/TCP ... run=be-default
kubernetes     10.115.240.1  ... 443/TCP  ... <none>
```

Najprostszy sposób użycia

Najprostszym sposobem użycia Ingress jest przekazywanie wszystkiego, co się trafi, do dalszych usług. Obsługa poleceń rozkazujących do Ingress w `kubectl` jest ograniczona, więc zaczniemy od utworzenia pliku YAML (listing 8.1).

Listing 8.1. simple-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: simple-ingress
spec:
  defaultBackend:
    service:
      name: alpaca
      port:
         number: 8080
```

Plik ten przekazujemy do polecenia `kubectl apply`:

```
$ kubectl apply -f simple-ingress.yaml
ingress.extensions/simple-ingress created
```

Poprawność konfiguracji można sprawdzić za pomocą poleceń `kubectl get` i `kubectl describe`:

```
$ kubectl get ingress
NAME            HOSTS ADDRESS PORTS AGE
simple-ingress  *             80    13m

$ kubectl describe ingress simple-ingress
Name:             simple-ingress
Namespace:        default
Address:
Default backend:  alpaca:8080
(172.17.0.6:8080,172.17.0.7:8080,172.17.0.8:8080)
Rules:
  Host Path Backends
  ---- ---- --------
  *    *    alpaca:8080 (172.17.0.6:8080,172.17.0.7:8080,172.17.0.8:8080)
Annotations:
  ...
Events: <none>
```

Dzięki tej konfiguracji *wszystkie* żądania HTTP trafiające do kontrolera Ingress będą przekazywane do usługi `alpaca`. Teraz mamy dostęp do instancji usługi `alpaca` obrazu `kuard` na każdym surowym IP/CNAME usługi. W tym przypadku jest to *alpaca.example.com* lub *bandicoot.example.com*. Na razie zyskaliśmy niewiele ponad prostą usługę typu `LoadBalancer`, ale teraz poeksperymentujemy z bardziej skomplikowanymi konfiguracjami.

Używanie nazw hosta

Ciekawie robi się, gdy zaczynamy kierować ruchem na podstawie właściwości żądań. Najbardziej typowym przykładem takiego działania jest sprawdzanie przez Ingress nagłówka HTTP `host` (który jest ustawiony na domenę DNS w oryginalnym adresie URL) i kierowanie ruchem na podstawie jego zawartości. Dodamy nowy obiekt Ingress, który będzie przekazywał do usługi `alpaca` ruch skierowany do *alpaca.example.com* (listing 8.2).

Listing 8.2. host-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: host-ingress
spec:
  defaultBackend:
    service:
      name: be-default
      port:
        number: 8080
rules:
- host: alpaca.example.com
  http:
    paths:
    - pathType: Prefix
      path: /
      backend:
        service:
          name: alpaca
          port:
            number: 8080
```

Tworzymy ten obiekt Ingress za pomocą polecenia `kubectl apply`:

```
$ kubectl apply -f host-ingress.yaml
ingress.extensions/host-ingress created
```

Sprawdzamy, czy wszystko jest w porządku:

```
$ kubectl get ingress
NAME            HOSTS               ADDRESS PORTS AGE
host-ingress    alpaca.example.com          80    54s
simple-ingress  *                           80    13m

$ kubectl describe ingress host-ingress
Name:             host-ingress
Namespace:        default
Address:
Default backend:  be-default:8080 (<none>)
```

```
Rules:
  Host                Path   Backends
  ----                ----   --------
  alpaca.example.com
                      /    alpaca:8080 (<none>)
Annotations:
  ...
Events: <none>
```

W tym przykładzie kilka elementów może być niejasnych. Pierwszy z nich to odwołanie do `default-http-backend`. Jest to stosowana tylko przez niektóre kontrolery Ingress konwencja obsługi żądań, które nie są obsługiwane w żaden inny sposób. Kontrolery te wysyłają takie żądania do usługi o nazwie `default-http-backend` w przestrzeni nazw `kube-system`. Jest to konwencja stosowana po stronie klienta w `kubectl`. Kolejny — nie ma punktów końcowych dla usługi `alpaca`. To błąd `kubectl`, który został naprawiony w wersji Kubernetes 1.14.

Niezależnie od tego teraz powinno się dać uzyskać dostęp do usługi `alpaca` za pomocą adresu *http://alpaca.example.com/*. Jeśli użytkownik spróbuje uzyskać dostęp do punktu końcowego usługi inną metodą, powinien zostać skierowany do usługi domyślnej.

Ścieżki

Następny interesujący scenariusz to kierowanie ruchu nie tylko na podstawie nazwy hosta, ale również na podstawie ścieżki zawartej w żądaniu HTTP. W tym przypadku wystarczy zdefiniować ścieżkę w rekordzie `paths` (listing 8.3). W prezentowanym przykładzie kierujemy wszystko, co przychodzi do *http://bandicoot.example.com/*, do usługi `bandicoot`, ale dodatkowo wysyłamy *http://bandicoot.example.com/a* do usługi `alpaca`. Tego typu scenariusz można wykorzystać do hostowania kilku usług na różnych ścieżkach w jednej domenie.

Listing 8.3. path-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: path-ingress
spec:
  rules:
  - host: bandicoot.example.com
    http:
      paths:
      - pathType: Prefix
      - path: "/"
        backend:
          service:
            name: bandicoot
            port:
              number: 8080
      - pathType: Prefix
        path: "/a/"
        backend:
          service:
            name: alpaca
            port:
              number: 8080
```

Jeśli w systemie Ingress jest kilka ścieżek na tym samym hoście, zostaje wybrana ta z najdłuższym prefiksem. W związku z tym ruch skierowany do */a/* jest przekazywany do usługi `alpaca`, a cały pozostały — zaczynający się od / — do usługi `bandicoot`.

Podczas przekazywania żądań do dalszej usługi ścieżka pozostaje niezmieniona. To oznacza, że do odpowiednio skonfigurowanego serwera trafia żądanie *bandicoot.example.com/a/*. Usługa musi być gotowa na obsługę ruchu na tej podścieżce. Na taką ewentualność `kuard` ma specjalny kod testujący, pozwalający odpowiadać na żądania do ścieżki głównej, oraz zestaw gotowych podścieżek (*/a/*, */b/* i */c/*).

Czyszczenie

Aby wszystko posprzątać, należy wykonać następujące polecenia:

```
$ kubectl delete ingress host-ingress path-ingress simple-ingress
$ kubectl delete service alpaca bandicoot be-default
$ kubectl delete deployment alpaca bandicoot be-default
```

Techniki zaawansowane i pułapki

Ingress ma jeszcze kilka innych ciekawych funkcji, choć poziom ich obsługi zależy od implementacji kontrolera. Poza tym dwa kontrolery mogą implementować daną funkcję w nieco odmienny sposób.

Wiele funkcji rozszerzeń jest udostępnionych w postaci adnotacji do obiektu Ingress. Uważaj, ponieważ podczas korzystania z nich łatwo się pomylić, a na dodatek są trudne do weryfikacji. Sporo z nich ma zastosowanie do całego obiektu Ingress, przez co mogą być zbyt ogólne. Aby ograniczyć zakres adnotacji, zawsze można podzielić obiekt Ingress na kilka obiektów. Kontroler powinien je wczytywać i łączyć.

Uruchamianie kilku kontrolerów Ingress

Istnieje wiele implementacji kontrolera Ingress i czasami w jednym klastrze trzeba uruchomić ich kilka. Dlatego powstał zasób `IngressClass`, aby Ingress mógł użyć jego konkretnej implementacji. Kiedy tworzymy zasób Ingress, to określamy odpowiedni zasób w polu `spec.ingressClassName`.

W wersjach Kubernetes starszych od 1.18 pole `IngressClassName` nie istnieje. Zamiast niego była używana adnotacja `kubernetes.io/ingress.class`. Choć wiele kontrolerów nadal pozwala na posługiwanie się tą techniką, zaleca się odchodzenie od niej, ponieważ w przyszłości ta adnotacja prawdopodobnie zostanie wycofana.

Jeśli nie ma adnotacji `spec.ingressClassName`, zostaje użyty domyślny kontroler Ingress. Określa się go przez dodanie adnotacji `ingressclass.kubernetes.io/is-default-class` do odpowiedniego zasobu `IngressClass`.

Wiele obiektów Ingress

Jeśli użytkownik określi kilka obiektów Ingress, kontrolery Ingress powinny odczytać je wszystkie i spróbować połączyć je w spójną konfigurację. Jeśli jednak konfiguracje będą zduplikowane i sprzeczne, to wynik będzie niezdefiniowany. Najprawdopodobniej każdy kontroler Ingress zareaguje inaczej. Nawet jedna implementacja może różnie reagować w zależności od czynników, które mogą być niejasne.

Ingress i przestrzenie nazw

Interakcje Ingress z przestrzeniami nazw nie zawsze są oczywiste. Z jednej strony ze względu na przyjęte środki ostrożności obiekt Ingress może odnosić się tylko do usługi w tej samej przestrzeni nazw. To znaczy, że obiekt taki nie może wskazywać podścieżki do usługi należącej do innej przestrzeni nazw niż on sam.

Z drugiej strony kilka obiektów Ingress należących do różnych przestrzeni nazw może określać podścieżki dla tego samego hosta. Obiekty te są wówczas łączone w jedną konfigurację dla kontrolera Ingress.

Taka międzyprzestrzenna funkcjonalność oznacza, że Ingress musi być koordynowany globalnie w skali klastra. Jeśli ta koordynacja zawiedzie, obiekt Ingress z jednej przestrzeni nazw może spowodować problemy (niezdefiniowane zachowanie) w innej przestrzeni.

Kontrolery Ingress zazwyczaj nie mają wbudowanych żadnych ograniczeń dotyczących tego, jakie przestrzenie nazw mogą określać wybrane nazwy hostów i ścieżki. Zaawansowani użytkownicy mogą próbować wymusić politykę używania własnego kontrolera wstępu. Podejmowane są też próby rozwiązania tego problemu w Ingress. Zostały one opisane w podrozdziale „Przyszłość Ingress”.

Przepisywanie ścieżek

Niektóre implementacje kontrolerów Ingress umożliwiają przepisywanie ścieżek. Można to wykorzystać w celu modyfikacji ścieżki w żądaniu HTTP podczas jej przekazywania. Zazwyczaj używa się do tego adnotacji do obiektu Ingress i stosuje się to do wszystkich żądań określonych przez ten obiekt. Jeśli na przykład ktoś korzysta z kontrolera Ingress NGINX, to może użyć adnotacji `nginx.ingress.kubernetes.io/rewrite-target: /`. Czasami może to powodować, że pewne usługi będą działać na ścieżce, do której nie zostały przystosowane.

Istnieje wiele implementacji, które nie tylko obsługują przepisywanie ścieżek, ale dodatkowo pozwalają na ich określanie przy użyciu wyrażeń regularnych. Na przykład kontroler NGINX umożliwia definiowanie części ścieżek za pomocą wyrażeń regularnych i wykorzystywanie tych części potem do przepisywania. Szczegóły tej operacji (i wariant używanych wyrażeń regularnych) zależą od implementacji.

Przepisywanie ścieżek to jednak nie jest cudowny środek i niestety często prowadzi do błędów. W przypadku wielu aplikacji sieciowych zakłada się, że mogą one odwoływać się do swoich własnych części za pomocą ścieżek bezwzględnych. Dana aplikacja może być hostowana na */podścieżce*, ale odbierać żądania na ścieżce /. Następnie może odesłać użytkownika do */ścieżki-aplikacji*. Wówczas

powstaje pytanie, czy jest to „wewnętrzny” odnośnik aplikacji (wtedy ścieżka powinna mieć postać */podścieżka/ścieżkaaplikacji*), czy odnośnik do innej aplikacji. Dlatego jeśli jest taka możliwość, w bardziej skomplikowanych aplikacjach lepiej jest unikać używania podścieżek.

Serwowanie przez TLS

Strony internetowe coraz częściej muszą być bezpiecznie serwowane przy użyciu TLS i HTTPS. Ingress to umożliwia (podobnie jak większość kontrolerów Ingress).

Przede wszystkim użytkownicy muszą określić sekret wraz z certyfikatem i kluczami TLS — jak pokazano na listingu 8.4. Sekret można też utworzyć za pomocą polecenia `kubectl create secret tls <nazwa-sekretu> --cert <plik-pem-certyfikatu> --key <plik-pem-klucza-prywatnego>`.

Listing 8.4. tls-secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  creationTimestamp: null
  name: tls-secret-name
type: kubernetes.io/tls
data:
tls.crt: <base64 encoded certificate>
tls.key: <base64 encoded private key>
```

Po wysłaniu certyfikatu można się do niego odnieść w obiekcie Ingress. W przykładzie na listingu 8.5 zdefiniowaliśmy listę certyfikatów z nazwami hostów, dla których te certyfikaty mają być używane. Jeśli kilka obiektów Ingress określa certyfikaty dla jednego hosta, efekt jest niezdefiniowany.

Listing 8.5. tls-ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-ingress
spec:
  tls:
  - hosts:
    - alpaca.example.com
    secretName: tls-secret-name
  rules:
  - host: alpaca.example.com
    http:
      paths:
      - backend:
        serviceName: alpaca
        servicePort: 8080
```

Wysyłanie sekretów TLS i zarządzanie nimi może być trudne. Ponadto certyfikaty często są kosztowne. W celu rozwiązania tego problemu powstała usługa non profit o nazwie Let’s Encrypt (*https://letsencrypt.org/*) oferująca bezpłatną certyfikację na bazie API. Dzięki temu, że usługa ta udostępnia API, klaster Kubernetes można skonfigurować tak, aby automatycznie pobierał i instalował certyfikaty. Konfiguracja może być trudna, ale obsługa jest już bardzo łatwa. Brakuje jeszcze tylko projektu open source o nazwie *cert-manager* (*https://cert-manager.io*) założonego

i wspieranego przez brytyjski start-up Jetstack, który został wciągnięty do CNCF. Szczegółowe informacje na temat instalacji cert-manager i tego, jak zacząć pracę z tym projektem, znajdują się na stronie *cert-manager.io* i w repozytorium GitHub (*https://oreil.ly/S0PU4*).

Inne implementacje Ingress

Istnieje wiele implementacji kontrolerów Ingress. Wszystkie bazują na podstawowym obiekcie Ingress i mają pewne wyjątkowe cechy. Całe to środowisko jest bardzo aktywne.

Każdy dostawca chmury ma implementację Ingress udostępniającą specyficzny chmurowy system równoważący obciążenie na poziomie warstwy 7. Kontrolery te, zamiast konfigurować programowy system równoważący obciążenie działający w kapsule, wykorzystują obiekty Ingress do konfiguracji chmurowych systemów równoważących obciążenie poprzez API. To pozwala zmniejszyć obciążenie klastra i upraszcza zarządzanie operatorom, ale często pociąga za sobą koszty.

Najpopularniejszym ogólnym kontrolerem Ingress jest prawdopodobnie kontroler open source NGINX (*https://oreil.ly/EstHX*). Pamiętaj, że istnieje również komercyjny kontroler bazujący na zastrzeżonym NGINX Plus. Kontroler open source wczytuje obiekty Ingress i scala je, tworząc plik konfiguracyjny NGINX. Następnie sygnalizuje procesowi NGINX potrzebę ponownego uruchomienia z nową konfiguracją (podczas serwowania istniejącego napływu połączeń). Kontroler NGINX ma bardzo dużo funkcji i opcji dostępnych poprzez adnotacje (*https://oreil.ly/V8nM7*).

Emissary (*https://oreil.ly/5HDun*) i Gloo (*https://oreil.ly/rZDlX*) to z kolei kontrolery Ingress oparte na Envoy, które stanowią przede wszystkim bramy API.

Traefik (*https://traefik.io/*) to odwrotny serwer proxy zaimplementowany w Go, który może także pełnić funkcję kontrolera Ingress. Ma zestaw bardzo przyjaznych dla programisty funkcji i pulpitów.

To tylko kilka przykładów. Ekosystem Ingress jest bardzo aktywny i można w nim znaleźć wiele nowych projektów, jak również ofert komercyjnych, w oryginalny sposób wykorzystujących skromny obiekt Ingress.

Przyszłość Ingress

Jak widać, obiekt Ingress stanowi bardzo praktyczną abstrakcję do konfiguracji systemów równoważących obciążenie na warstwie 7., ale nie zapewnia wszystkich funkcji potrzebnych użytkownikom i różnym implementacjom. Wiele funkcji w Ingress jest niedookreślonych, przez co poszczególne implementacje mogą je wykorzystywać na różne sposoby. To utrudnia przenoszenie konfiguracji między implementacjami.

Inny problem stanowi fakt, że Ingress łatwo jest źle skonfigurować. Sposób, w jaki obiekty są scalane, otwiera drogę do konfliktów, które mogą być różnie rozwiązywane przez poszczególne implementacje. Ponadto łączenie obiektów pochodzących z różnych przestrzeni nazw podaje w wątpliwość sens używania przestrzeni nazw.

Do tego Ingress powstał, zanim jeszcze idea siatek (których przykładami są Istio i Linkerd) zdobyła popularność. Dlatego wciąż trwają prace nad pogodzeniem Ingress z siatkami usług. Siatki usług zostały opisane bardziej szczegółowo w rozdziale 15.

Przyszłością równoważenia obciążenia HTTP dla Kubernetes wydaje się interfejs API **Gateway**, który jest tworzony przez specjalną grupę zainteresowań ds. sieci (SIG) Kubernetes. Ma to być nowocześniejszy API do routingu w Kubernetes. Choć głównym celem API Gateway jest równoważenie obciążenia HTTP, zawiera on także zasoby do kontroli równoważenia warstwy 4 (TCP). Interfejs API Gateway wciąż jest w fazie rozwoju, więc na razie zalecane jest pozostanie przy istniejących zasobach Ingress i Service. Aktualny stan API Gateway można sprawdzić w internecie (*https://oreil.ly/zhlil*).

Podsumowanie

Ingress to wyjątkowy system w Kubernetes. Jest po prostu schematem, a implementacje kontrolerów tego schematu muszą być instalowane i zarządzane osobno. Jest to też jednak krytyczny system umożliwiający udostępnianie usług użytkownikom w praktyczny i ekonomiczny sposób. Należy się spodziewać, że w miarę dojrzewania klastra Kubernetes Ingress będzie zyskiwać na znaczeniu.

ROZDZIAŁ 9.
Obiekt ReplicaSet

Wcześniej opisaliśmy, jak uruchamiać indywidualne kontenery jako kapsuły. Ale takie kapsuły są zasadniczo jednorazowymi singletonami. Najczęściej potrzeba wielu replik kontenera działających w określonym czasie. Istnieje wiele powodów tego typu replikacji:

Redundancja
: Wiele uruchomionych instancji oznacza, że awaria może być tolerowana.

Skala
: Wiele uruchomionych instancji oznacza, że można obsłużyć więcej żądań.

Fragmentowanie
: Różne repliki mogą równolegle obsługiwać różne części obliczeń.

Oczywiście można ręcznie utworzyć wiele kopii kapsuły, korzystając z wielu różnych (choć w dużej mierze podobnych) manifestów kapsuły, ale takie podejście jest żmudne i narażone na błędy. W ujęciu logicznym użytkownik zarządzający zestawem zreplikowanych kapsuł traktuje je pod względem definiowania i zarządzania jako pojedynczy podmiot. Tym właśnie jest obiekt `ReplicaSet`. `ReplicaSet` działa jako menedżer kapsuł w całym klastrze, zapewniając przez cały czas działanie odpowiednich typów i właściwej liczby kapsuł.

Ponieważ obiekty `ReplicaSet` ułatwiają tworzenie zestawów zreplikowanych kapsuł i zarządzanie nimi, są elementami konstrukcyjnymi używanymi do opisywania typowych wzorców wdrażania aplikacji i zapewniają podstawy samonaprawiania się dla aplikacji na poziomie infrastruktury. Kapsuły zarządzane przez `ReplicaSet` są automatycznie ponownie rozplanowywane w przypadku wystąpienia określonych stanów awaryjnych, takich jak awarie węzłów i podział sieci na partycje.

Najprościej można to ująć w taki sposób, że `ReplicaSet` łączy w pojedynczym obiekcie API zarówno foremkę do ciastek, jak i żądaną liczbę ciastek. Kiedy definiujemy `ReplicaSet`, definiujemy specyfikację kapsuł, które chcemy utworzyć (foremkę do ciastek), oraz żądaną liczbę replik (ciastek). Dodatkowo musimy zdefiniować sposób wykrywania kapsuł, które obiekt `ReplicaSet` powinien kontrolować. Rzeczywistym przykładem zarządzania zreplikowanymi kapsułami jest **pętla uzgadniania** (ang. *reconciliation loop*). Takie pętle mają fundamentalne znaczenie dla większości projektów i implementacji Kubernetes.

Pętle uzgadniania

Główne pojęcia wykorzystywane w ramach koncepcji pętli uzgadniania to pojęcia stanu **żądanego** i stanu **obserwowanego** lub **bieżącego**. Stan żądany to ten, do którego dążymy. W przypadku `ReplicaSet` jest to żądana liczba replik i definicja kapsuły, która ma być replikowania. Żądanym stanem są na przykład trzy repliki kapsuły, w której działa serwer `kuard`. Stan bieżący natomiast to aktualnie obserwowany stan systemu. Przykładowo obecnie działają tylko dwie kapsuły serwera `kuard`.

Pętla uzgadniania działa w trybie ciągłym, obserwując bieżący stan świata i podejmując działania, aby dopasować stan obserwowany do stanu żądanego. Biorąc pod uwagę poprzedni przykład, pętla uzgadniania tworzy nową kapsułę `kuard` i stara się, aby obserwowany stan był zgodny z żądanym stanem trzech replik.

Podejście wykorzystujące pętlę uzgadniania ma wiele zalet, jeśli chodzi o zarządzanie stanem. Jest to oparty na celach, samonaprawiający się system, który często można łatwo wyrazić w kilku liniach kodu. Należy zwrócić uwagę, że pętla uzgadniania dla obiektów `ReplicaSet` jest pojedynczą pętlą, a mimo to obsługuje działania użytkownika polegające na skalowaniu `ReplicaSet` w górę lub w dół, a także awarie węzłów lub węzły ponownie dołączające do klastra po czasowej nieobecności.

W dalszej części książki zobaczysz liczne przykłady pętli uzgadniania.

Relacje między kapsułami i obiektami ReplicaSet

Jednym z kluczowych motywów systemu Kubernetes jest rozłączność. W szczególności ważne jest to, że wszystkie podstawowe koncepcje Kubernetes są w odniesieniu do siebie nawzajem modułowe i można je wymieniać i zastępować innymi komponentami. W tym duchu relacja między obiektami `ReplicaSet` i kapsułami jest luźno powiązana. Chociaż `ReplicaSet` tworzą kapsuły i zarządzają nimi, nie są właścicielami tworzonych przez siebie kapsuł. Obiekty `ReplicaSet` do identyfikacji zestawu kapsuł, którymi powinny zarządzać, używają zapytań etykietowych. Następnie w celu utworzenia kapsuł, którymi zarządzają, korzystają z dokładnie tego samego API kapsuły, którego użyliśmy bezpośrednio w rozdziale 5. Pojęcie „wchodzenia przez frontowe drzwi" to kolejna centralna koncepcja projektowa w Kubernetes. Obiekty `ReplicaSet`, które tworzą wiele kapsuł, oraz usługi równoważące obciążenie dla tych kapsuł są również całkowicie oddzielnymi, rozłącznymi obiektami API. Oprócz wspierania modularności, oddzielenie kapsuł i obiektów `ReplicaSet` umożliwia kilka ważnych zachowań omówionych w kolejnych punktach.

Adaptowanie istniejących kontenerów

Pomimo że wartość deklaratywnej konfiguracji oprogramowania jest oczywista, czasami łatwiej jest budować coś imperatywnie. Szczególnie na początku można po prostu wdrożyć pojedynczą kapsułę z obrazem kontenera bez zarządzającego nią `ReplicaSet`. Można nawet zdefiniować system równoważenia obciążenia, który obsługuje ruch skierowany do tej pojedynczej kapsuły.

Gdyby obiekty `ReplicaSet` były właścicielami tworzonych przez siebie kapsuł, jedynym sposobem na rozpoczęcie replikacji kapsuły byłoby jej usunięcie, a następnie ponowne uruchomienie za pomocą `ReplicaSet`. To mogłoby być destrukcyjne, ponieważ w pewnym momencie nie byłoby uruchomionej żadnej kopii kontenera. Ponieważ jednak `ReplicaSet` są oddzielone od zarządzanych przez nie kapsuł, można po prostu utworzyć obiekt `ReplicaSet`, który „zaadaptuje" istniejącą kapsułę i doda kolejne kopie kontenerów. W ten sposób można płynnie przejść od pojedynczej imperatywnej kapsuły do zestawu zreplikowanych kapsuł zarządzanych przez `ReplicaSet`.

Poddawanie kontenerów kwarantannie

Gdy serwer zachowuje się niewłaściwie, często kontrole działania na poziomie kapsuł automatycznie restartują daną kapsułę. Jeśli jednak kontrole poprawności działania są niekompletne, kapsuła może zachowywać się niewłaściwie, a mimo to nadal być częścią zreplikowanego zestawu. W takich sytuacjach najprościej byłoby po prostu wyłączyć kapsułę, programistom do debugowania problemu pozostałyby jednak tylko dzienniki. Zamiast tego można zmodyfikować zestaw etykiet w chorej kapsule. Spowoduje to odłączenie tej kapsuły od `ReplicaSet` (i usługi), aby można było ją debugować. Kontroler `ReplicaSet` zauważy, że brakuje kapsuły, i utworzy nową kopię, ponieważ jednak kapsuła będzie nadal działać, będzie dostępna dla programistów do interaktywnego debugowania, co jest znacznie bardziej wartościowe niż debugowanie na podstawie dzienników.

Projektowanie z wykorzystaniem ReplicaSet

Kontrolery `ReplicaSet` są zaprojektowane w taki sposób, aby reprezentowały pojedynczą, skalowalną mikrousługę w ramach architektury. Kluczową cechą `ReplicaSet` jest to, że każda kapsuła tworzona przez kontroler `ReplicaSet` jest całkowicie jednorodna. Zazwyczaj przed takimi kapsułami jest umieszczany mechanizm równoważenia obciążenia usług systemu Kubernetes, który rozprowadza ruch między kapsuły składające się na usługę. Ogólnie mówiąc, obiekty `ReplicaSet` są przeznaczone dla usług bezstanowych lub prawie bezstanowych. Elementy utworzone przez `ReplicaSet` są wymienne. Przy skalowaniu `ReplicaSet` w dół do usunięcia zostaje wybrana dowolna kapsuła. Taka operacja skalowania w dół nie powinna zmienić zachowania aplikacji.

Aplikacje zazwyczaj używają obiektu `Deployment`, ponieważ umożliwia on zarządzanie wydaniami nowych wersji. Kontrolery `ReplicaSet` są wykorzystywane przez obiekty `Deployment` i należy zrozumieć sposób ich działania, aby być w stanie usunąć potencjalne błędy, jeśli się pojawią.

Specyfikacja ReplicaSet

Podobnie jak wszystkie obiekty w Kubernetes, `ReplicaSet` są definiowane przy użyciu specyfikacji. Wszystkie obiekty `ReplicaSet` muszą mieć unikatowe nazwy (zdefiniowane za pomocą pola `metadata.name`), sekcję `spec` opisująca liczbę kapsuł (replik), które powinny działać w danym momencie w całym klastrze, oraz szablon kapsuły opisujący kapsułę, która powinna zostać utworzona, gdy nie osiągnięto jeszcze zdefiniowanej liczby replik. Listing 9.1 pokazuje minimalną definicję

kontrolera `ReplicaSet`. Zwróć uwagę na sekcje `replicas`, `selector` i `template`, ponieważ dają wgląd w sposób działania kontrolera `ReplicaSet`.

Listing 9.1. kuard-rs.yaml

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  labels:
    app: kuard
    version: "2"
  name: kuard
spec:
  replicas: 1
  selector:
    matchLabels:
      app: kuard
      version: "2"
  template:
    metadata:
      labels:
        app: kuard
        version: "2"
    spec:
      containers:
        - name: kuard
          image: "gcr.io/kuar-demo/kuard-amd64:green"
```

Szablony kapsuł

Jak wspomnieliśmy wcześniej, gdy liczba kapsuł w bieżącym stanie jest mniejsza niż liczba kapsuł w żądanym stanie, kontroler `ReplicaSet` tworzy nowe kapsuły. Te kapsuły są konstruowane przy użyciu szablonu kapsuły, który jest zawarty w specyfikacji kontrolera `ReplicaSet`. Są one tworzone w dokładnie taki sam sposób, w jaki tworzyliśmy kapsuły z pliku YAML w poprzednich rozdziałach. Ale zamiast używać pliku, kontroler `ReplicaSet` systemu Kubernetes tworzy manifest kapsuły na podstawie szablonu kapsuły i wysyła go bezpośrednio na serwer API. Poniżej przedstawiono przykład szablonu kapsuły w `ReplicaSet`:

```
template:
  metadata:
    labels:
      app: helloworld
      version: v1
  spec:
    containers:
      - name: helloworld
        image: kelseyhightower/helloworld:v1
        ports:
          - containerPort: 80
```

Etykiety

W każdym klastrze o rozsądnej wielkości przez cały czas działa wiele różnych kapsuł — w jaki więc sposób pętla uzgadniania kontrolera `ReplicaSet` wykrywa zestaw kapsuł dla konkretnego `ReplicaSet`? Kontrolery `ReplicaSet` monitorują stan klastra za pomocą zestawu etykiet kapsuły. Etykiety służą do filtrowania listy kapsuł i śledzenia kapsuł działających w klastrze. Gdy obiekt `ReplicaSet` jest tworzony, pobiera listę kapsuł z API Kubernetes i filtruje wyniki według etykiet. Na podstawie liczby kapsuł zwróconej przez zapytanie `ReplicaSet` usuwa lub tworzy kapsuły, aby zagwarantować żądaną liczbę replik. Etykiety używane do filtrowania są definiowane w sekcji `spec` obiektu `ReplicaSet` i są kluczem do zrozumienia jego działania.

Selektor w specyfikacji `ReplicaSet` powinien być właściwym podzbiorem etykiet z szablonu kapsuły.

Tworzenie obiektu ReplicaSet

Kontrolery `ReplicaSet` są tworzone przez przesłanie obiektu `ReplicaSet` do interfejsu API Kubernetes. W tym podrozdziale utworzymy `ReplicaSet` za pomocą pliku konfiguracyjnego i polecenia `kubectl apply`.

Plik konfiguracyjny `ReplicaSet` z listingu 9.1 zapewni, że w danym momencie będzie działać jedna kopia kontenera `gcr.io/kuar-demo/kuard-amd64:green`. Użyj polecenia `kubectl apply`, aby przesłać `ReplicaSet` serwera `kuard` do API Kubernetes:

```
$ kubectl apply -f kuard-rs.yaml
replicaset "kuard" created
```

Po zaakceptowaniu `ReplicaSet` serwera `kuard` kontroler `ReplicaSet` wykryje, że nie ma uruchomionych kapsuł `kuard` odpowiadających żądanemu stanowi, i utworzona zostanie nowa kapsuła `kuard` na podstawie zawartości szablonu kapsuły:

```
$ kubectl get pods
NAME          READY    STATUS    RESTARTS    AGE
kuard-yvzgd   1/1      Running   0           11s
```

Inspekcja obiektu ReplicaSet

Podobnie jak w przypadku kapsuł i innych obiektów API Kubernetes, jeśli jesteś zainteresowany dalszymi szczegółami dotyczącymi obiektu `ReplicaSet`, polecenie `describe` pozwoli Ci uzyskać znacznie więcej informacji o jego stanie. Oto przykład użycia polecenia `describe` w celu zdobycia szczegółowych informacji na temat wcześniej utworzonego kontrolera `ReplicaSet`:

```
$ kubectl describe rs kuard
Name:         kuard
Namespace:    default
Selector:     app=kuard,version=2
Labels:       app=kuard
              version=2
```

```
Replicas:     1 current / 1 desired
Pods Status:  1 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
```

Możesz zobaczyć selektor etykiet dla `ReplicaSet` oraz stan wszystkich replik zarządzanych przez ten kontroler `ReplicaSet`.

Znajdowanie ReplicaSet z poziomu kapsuły

Czasami możesz się zastanawiać, czy kapsuła jest zarządzana przez jakiś kontroler `ReplicaSet`, a jeśli tak, to przez który. Aby umożliwić tego rodzaju wykrywanie, kontroler `ReplicaSet` dodaje sekcję `ownerReferences` do każdej kapsuły, którą tworzy. Gdy uruchomisz poniższe polecenie, poszukaj sekcji `ownerReferences`:

```
$ kubectl get pods <nazwa_kapsuły> -o=jsonpath='{.metadata.ownerReferences[0].name}'
```

Jeśli ma to zastosowanie, wyświetlona zostanie nazwa kontrolera `ReplicaSet` zarządzającego tą kapsułą.

Znajdowanie zestawu kapsuł dla ReplicaSet

Możesz także określić zestaw kapsuł zarządzanych przez kontroler `ReplicaSet`. Po pierwsze, możesz uzyskać zestaw etykiet za pomocą polecenia `kubectl describe`. W poprzednim przykładzie selektorem etykiet było `app=kuard,version=2`. Aby znaleźć kapsuły pasujące do tego selektora, użyj flagi `--selector` lub skrótu `-l`:

```
$ kubectl get pods -l app=kuard,version=2
```

Jest to dokładnie to samo zapytanie, które wykonuje `ReplicaSet` w celu ustalenia bieżącej liczby kapsuł.

Skalowanie kontrolerów ReplicaSet

Kontrolery `ReplicaSet` można skalować w górę lub w dół poprzez aktualizację klucza `spec.replicas` w obiekcie `ReplicaSet` przechowywanym w Kubernetes. Po przeprowadzeniu skalowania `ReplicaSet` w górę do interfejsu API Kubernetes przesyłane są nowe kapsuły przy użyciu szablonu kapsuły zdefiniowanego w `ReplicaSet`.

Skalowanie imperatywne za pomocą polecenia kubectl scale

Najprostszym sposobem skalowania jest użycie polecenia `kubectl scale`. Aby wykonać na przykład skalowanie w górę do czterech replik, można uruchomić następujące polecenie:

```
$ kubectl scale replicasets kuard --replicas=4
```

Chociaż takie imperatywne polecenia są przydatne dla celów demonstracyjnych i do szybkiego reagowania w sytuacjach awaryjnych (na przykład w odpowiedzi na nagły wzrost obciążenia), ważne jest również zaktualizowanie wszelkich konfiguracyjnych plików tekstowych, aby zawierały odpowiednią liczbę replik, która została ustawiona za pomocą imperatywnego polecenia `scale`. Przyczyna tego stanie się oczywista, kiedy rozważysz następujący scenariusz:

Alicja jest na dyżurze, gdy nagle pojawia się duży wzrost obciążenia usługi, którą zarządza. Alicja używa polecenia `scale` w celu zwiększenia do 10 liczby serwerów obsługujących żądania i sytuacja zostaje opanowana. Zapomina jednak zaktualizować konfiguracje `ReplicaSet` przesłane do systemu kontroli wersji.

Kilka dni później Bob przygotowuje cotygodniowe wdrożenie. Edytuje konfiguracje kontrolerów `ReplicaSet` przechowywane w systemie kontroli wersji, aby użyć nowego obrazu kontenera, ale nie zauważa, że liczba replik w pliku to obecnie 5, a nie 10, co Alicja ustawiła w odpowiedzi na zwiększone obciążenie. Bob kontynuuje wdrażanie, w wyniku którego nastąpi aktualizacja obrazu kontenera i liczba replik zmniejszy się o połowę, co spowoduje natychmiastowe przeciążenie lub przestój.

Mamy nadzieję, że ten przykład ilustruje konieczność upewnienia się, że po imperatywnych zmianach natychmiast nastąpi deklaratywna zmiana w systemie kontroli wersji. Jeśli taka potrzeba nie wydaje Ci się wystarczająco oczywista, zalecamy wprowadzanie jedynie deklaratywnych zmian zgodnie z tym, co opisaliśmy w kolejnym punkcie.

Skalowanie deklaratywne za pomocą kubectl apply

W świecie deklaratywnym zmian dokonuje się poprzez edycję pliku konfiguracyjnego w systemie kontroli wersji, a następnie stosuje się te zmiany w klastrze. Aby wykonać skalowanie kontrolera `ReplicaSet` serwera `kuard`, edytuj plik konfiguracyjny *kuard-rs.yaml* i ustaw wartość `replicas` na 3:

```
...
spec:
  replicas: 3
...
```

W trybie wieloużytkownikowym powinieneś mieć udokumentowaną inspekcję kodu dla tych zmian i ostatecznie przesłać zmiany do systemu kontroli wersji. Tak czy inaczej, możesz następnie użyć polecenia `kubectl apply`, aby przesłać zaktualizowany kontroler `ReplicaSet` serwera `kuard` do serwera API:

```
$ kubectl apply -f kuard-rs.yaml
replicaset "kuard" configured
```

Teraz gdy mamy już zaktualizowany `ReplicaSet` serwera `kuard`, kontroler `ReplicaSet` wykryje, że zmieniła się liczba żądanych kapsuł i trzeba podjąć działania, aby osiągnąć pożądany stan. Jeśli użyłeś imperatywnego polecenia `scale` w poprzednim punkcie, kontroler `ReplicaSet` zniszczy jedną kapsułę, by uzyskać liczbę trzech kapsuł. W przeciwnym razie prześle do interfejsu API Kubernetes dwie nowe kapsuły, korzystając z szablonu kapsuły zdefiniowanego w obiekcie `ReplicaSet` serwera `kuard`. Tak czy inaczej, użyj polecenia `kubectl get pods`, aby wyświetlić listę działających kapsuł `kuard`. Powinieneś zobaczyć następujące dane wyjściowe — trzy działające kapsuły, z których dwie są młodsze, ponieważ zostały później uruchomione:

```
$ kubectl get pods
NAME          READY    STATUS    RESTARTS    AGE
kuard-3a2sb   1/1      Running   0           26s
kuard-wuq9v   1/1      Running   0           26s
kuard-yvzgd   1/1      Running   0           2m
```

Automatyczne skalowanie kontrolera ReplicaSet

Chociaż czasami będziesz chciał mieć bezpośrednią kontrolę nad liczbą replik w `ReplicaSet`, zwykle będziesz chciał mieć po prostu „wystarczającą” liczbę replik. Definicja „wystarczającej liczby replik” różni się w zależności od potrzeb kontenerów w `ReplicaSet`. W przypadku serwera WWW, takiego jak na przykład `nginx`, możesz chcieć skalować według użycia procesora. Z kolei pamięć podręczna może wymagać skalowania zgodnie z wykorzystaniem pamięci. W niektórych przypadkach potrzebne może być skalowanie w reakcji na niestandardowe wskaźniki aplikacji. Kubernetes może obsłużyć wszystkie te scenariusze za pomocą **poziomego autoskalowania kapsuł** (ang. *horizontal pod autoscaling* — **HPA**).

HPA wymaga obecności w klastrze kapsuły `metrics-server`. Ta kapsuła śledzi wskaźniki i zapewnia interfejs API do korzystania ze wskaźników, których HPA używa podczas podejmowania decyzji dotyczących skalowania. Większość instalacji Kubernetes domyślnie zawiera kapsułę `metrics-server`. Możesz potwierdzić jej obecność poprzez wylistowanie kapsuł w przestrzeni nazw `kube-system`:

```
$ kubectl get pods --namespace=kube-system
```

Gdzieś na tej liście powinieneś zobaczyć kapsułę o nazwie `metrics-server`. Jeżeli jej tam nie ma, autoskalowanie nie będzie działać poprawnie.

„Poziome autoskalowanie kapsuł” to dość długa nazwa i możesz się zastanawiać, dlaczego nie nazywamy tego procesu po prostu „autoskalowaniem”. Kubernetes dokonuje rozróżnienia między skalowaniem **poziomym**, co wiąże się z tworzeniem dodatkowych replik kapsuły, i skalowaniem **pionowym**, które obejmuje zwiększenie ilości zasobów wymaganych dla danej kapsuły (na przykład zwiększenie zasobów procesora wymaganych dla kapsuły). Wiele rozwiązań umożliwia także autoskalowanie **klastrów**, w którym liczba maszyn w klastrze jest skalowana w odpowiedzi na zapotrzebowanie na zasoby, ale to rozwiązanie nie zostało tutaj omówione.

Skalowanie na podstawie wykorzystania procesora jest najczęstszym przypadkiem użycia dla autoskalowania kapsuł. Ponadto można odnieść się do poziomu wykorzystania pamięci. Skalowanie na podstawie użycia procesora jest najbardziej przydatne dla systemów opartych na żądaniach, które wykorzystują procesor proporcjonalnie do liczby otrzymywanych żądań, wymagając przy tym stosunkowo stałej ilości pamięci.

Aby wykonać skalowanie obiektu `ReplicaSet`, można uruchomić na przykład takie polecenie:

```
$ kubectl autoscale rs kuard --min=2 --max=5 --cpu-percent=80
```

To polecenie tworzy autoskaler, który skaluje między dwoma i pięcioma replikami za pomocą progu wykorzystania procesora wynoszącego 80%. Aby wyświetlić, zmodyfikować lub usunąć ten zasób, możesz użyć standardowych poleceń `kubectl` i zasobu `horizontalpodautoscalers`. Oznacza to trochę znaków do wpisania, ale można to skrócić do `hpa`:

```
$ kubectl get hpa
```

Z powodu „rozłącznej natury” Kubernetes nie ma bezpośredniego połączenia między poziomym autoskalerem kapsuł i obiektem `ReplicaSet`. Chociaż to rozwiązanie doskonale sprawdza się w zakresie modułowości i kompozycji, umożliwia także stosowanie niektórych antywzorców. Złym pomysłem jest w szczególności połączenie autoskalowania z imperatywnym lub deklaratywnym zarządzaniem liczbą replik. Jeśli zarówno Ty, jak i autoskaler próbujecie zmodyfikować liczbę replik, jest bardzo prawdopodobne, że dojdzie do konfliktu powodującego nieoczekiwane zachowanie.

Usuwanie obiektów ReplicaSet

Gdy kontroler `ReplicaSet` nie jest już potrzebny, można go usunąć za pomocą polecenia `kubectl delete`. Domyślnie usuwa ono również kapsuły zarządzane przez dany kontroler `ReplicaSet`:

```
$ kubectl delete rs kuard
replicaset "kuard" deleted
```

Uruchomienie polecenia `kubectl get pods` pokazuje, że wszystkie kapsuły `kuard` utworzone przez ten kontroler `ReplicaSet` serwera `kuard` również zostały usunięte:

```
$ kubectl get pods
```

Jeśli nie chcesz usuwać kapsuł zarządzanych przez dany kontroler `ReplicaSet`, możesz ustawić flagę `--cascade` na wartość `false`, aby upewnić się, że usunięty zostanie tylko obiekt `ReplicaSet`, a kapsuły zostaną zachowane:

```
$ kubectl delete rs kuard --cascade=false
```

Podsumowanie

Komponowanie kapsuł z kontrolerami `ReplicaSet` stanowi podstawę budowania solidnych aplikacji z automatycznym przełączaniem awaryjnym i sprawia, że wdrażanie tych aplikacji jest bardzo proste dzięki umożliwieniu stosowania skalowalnych i rozsądnych wzorców wdrażania. Kontrolerów `ReplicaSet` powinieneś używać dla każdej kapsuły, na której Ci zależy, nawet jeśli jest to pojedyncza kapsuła! Niektórzy nawet domyślnie używają obiektów `ReplicaSet` zamiast obiektów `Pod`. Typowy klaster będzie miał wiele kontrolerów `ReplicaSet`, stosuj je więc szczodrze tam, gdzie potrzeba.

ROZDZIAŁ 10.

Obiekt Deployment

Do tej pory dowiedziałeś się, jak spakować aplikację jako kontener, utworzyć zestaw zreplikowanych kontenerów i korzystać z usług do równoważenia obciążenia ruchu skierowanego do Twojej usługi. Wszystkie omówione obiekty są używane do budowania pojedynczej instancji aplikacji. Nie są one zbyt pomocne w zarządzaniu dziennym lub tygodniowym rytmem wydawania nowych wersji aplikacji. Tak naprawdę zarówno obiekty `Pod`, jak i `ReplicaSet` mają być związane z konkretnymi obrazami kontenerów, które się nie zmieniają.

Zadaniem obiektu `Deployment` (wdrożenie) jest zarządzanie wydawaniem nowych wersji. Obiekty `Deployment` reprezentują wdrożone aplikacje w sposób wykraczający poza jakiekolwiek konkretne wersje oprogramowania aplikacji. Ponadto wdrożenia umożliwiają łatwe przechodzenie z jednej wersji kodu do następnej. Ten proces wdrażania nowych wersji jest konfigurowalny i cechuje go ostrożność. Pomiędzy aktualizacjami poszczególnych kapsuł odczekiwany jest czas skonfigurowany przez użytkownika. Wykorzystywane są również kontrole poprawności działania, aby zapewnić, że nowa wersja aplikacji będzie działać prawidłowo, a wdrożenie jest przerywane, jeśli pojawi się zbyt wiele błędów.

Korzystając z wdrożeń, można łatwo i niezawodnie wprowadzać nowe wersje oprogramowania bez przestojów i błędów. Tak naprawdę wdrażanie oprogramowania jest wykonywane przez obiekt `Deployment` i kontrolowane przez kontroler `Deployment`, który działa w klastrze Kubernetes. Oznacza to, że można pozwolić, by obiekt `Deployment` działał bez nadzoru, i to działanie będzie poprawne i bezpieczne. Ułatwia to integrację obiektów `Deployment` z licznymi narzędziami i usługami ciągłego dostarczania oprogramowania. Ponadto fakt, że obiekt `Deployment` działa po stronie serwera, umożliwia bezpieczne wykonywanie wdrożeń z lokalizacji o słabym lub przerywanym połączeniu z internetem. Wyobraź sobie wprowadzanie nowej wersji oprogramowania z telefonu podczas jazdy metrem. Obiekty `Deployment` sprawiają, że jest to możliwe i bezpieczne!

Po pierwszym wydaniu Kubernetes jedną z najbardziej popularnych demonstracji jego dużych możliwości była „ciągła aktualizacja” (ang. *rolling update*), która pokazywała, jak można użyć pojedynczego polecenia, aby bezproblemowo zaktualizować działającą aplikację bez żadnych przestojów lub utraty żądań. To oryginalne demo zostało oparte na poleceniu `kubectl rolling-update`, które jest nadal dostępne w narzędziu wiersza poleceń, ale jego funkcjonalność została w znacznym stopniu włączona do obiektu `Deployment`.

Twoje pierwsze wdrożenie

Wdrożenie, jak wszystkie obiekty w Kubernetes, można reprezentować jako deklaratywny obiekt YAML dostarczający informacji na temat tego, co ma zostać uruchomione. Poniższe wdrożenie na przykład zawiera żądanie jednej instancji aplikacji `kuard`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kuard
  labels:
    run: kuard
spec:
  selector:
    matchLabels:
      run: kuard
  replicas: 1
  template:
    metadata:
      labels:
        run: kuard
    spec:
      containers:
      - name: kuard
        image: gcr.io/kuar-demo/kuard-amd64:blue
```

Zapisz ten kod w pliku YAML o nazwie *kuard-deployment.yaml*, a następnie utwórz wdrożenie za pomocą polecenia:

```
$ kubectl create -f kuard-deployment.yaml
```

Przyjrzyjmy się, jak w rzeczywistości działają obiekty `Deployment`. Podobnie jak obiekty `ReplicaSet` zarządzają kapsułami, tak obiekty `Deployment` zarządzają obiektami `ReplicaSet`. Tak jak wszystkie relacje w Kubernetes, również ta relacja jest definiowana przez etykiety i selektor etykiet. Możesz zobaczyć selektor etykiet, gdy przyjrzysz się obiektowi `Deployment`:

```
$ kubectl get deployments kuard \
  -o jsonpath --template {.spec.selector.matchLabels}

map[run:kuard]
```

Widać tutaj, że dany obiekt `Deployment` zarządza `ReplicaSet` z etykietami `run=kuard`. Możemy to wykorzystać w zapytaniu selektora etykiet dla obiektów `ReplicaSet`, aby znaleźć określony obiekt `ReplicaSet`:

```
$ kubectl get replicasets --selector=run=kuard

NAME               DESIRED   CURRENT   READY   AGE
kuard-1128242161   1         1         1       13m
```

Zobaczmy teraz tę relację między `Deployment` i `ReplicaSet` w akcji. Możemy zmienić rozmiar obiektu przy użyciu imperatywnego polecenia `scale`:

```
$ kubectl scale deployments kuard --replicas=2

deployment.apps/kuard scaled
```

Gdy ponownie wyświetlimy ten `ReplicaSet`, powinniśmy zobaczyć następujące dane wyjściowe:

```
$ kubectl get replicasets --selector=run=kuard

NAME               DESIRED   CURRENT   READY   AGE
nginx-1128242161   2         2         2       13m
```

Wykonanie skalowania obiektu `Deployment` spowodowało przeskalowanie także kontrolowanego przez niego `ReplicaSet`.

Teraz spróbujmy działania odwrotnego, skalując obiekt `ReplicaSet`:

```
$ kubectl scale replicasets kuard-1128242161 --replicas=1

replicaset.apps/kuard-1128242161 scaled
```

Pobierzmy jeszcze raz ten `ReplicaSet`:

```
$ kubectl get replicasets --selector=run=kuard

NAME               DESIRED   CURRENT   READY   AGE
nginx-1128242161   2         2         2       13m
```

To dziwne. Pomimo naszego skalowania obiektu `ReplicaSet` do jednej repliki wciąż ma on dwie repliki, tak jak jego żądany stan. Co się dzieje?

Pamiętaj, że Kubernetes jest internetowym, samonaprawiającym się systemem. Obiektem `ReplicaSet` zarządza obiekt najwyższego poziomu, `Deployment`. Gdy ograniczysz liczbę replik do jednej, przestaje ona odpowiadać żądanemu stanowi obiektu `Deployment`, który ma `replicas` ustawione na `2`. Kontroler `Deployment` zauważa tę sytuację i podejmuje działania, aby upewnić się, że obserwowany stan pasuje do pożądanego stanu, w tym przypadku ponownie zwiększając liczbę replik do dwóch.

Jeśli kiedykolwiek będziesz chciał zarządzać obiektem `ReplicaSet` bezpośrednio, musisz usunąć dany obiekt `Deployment` (pamiętaj, aby ustawić `--cascade` na `false`, inaczej usuniesz także `ReplicaSet` i kapsuły!).

Tworzenie obiektów Deployment

Oczywiście, jak zostało powiedziane we wprowadzeniu, powinieneś preferować deklaratywne zarządzanie swoimi konfiguracjami Kubernetes. Oznacza to utrzymywanie stanu wdrożeń w plikach YAML lub JSON na dysku.

Jako punkt wyjścia pobierz to wdrożenie do pliku YAML:

```
$ kubectl get deployments nginx --export -o yaml > kuard-deployment.yaml
$ kubectl replace -f nginx-deployment.yaml --save-config
```

Jeśli zajrzysz do pliku, zobaczysz coś takiego (dla zwiększenia czytelności usunęliśmy wiele pól tylko do odczytu i domyślnych). Zwróć uwagę na pola `annotations`, `selector` i `strategy`, ponieważ zawierają informacje pozwalające zrozumieć funkcjonalność obiektu `Deployment`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
```

```
    creationTimestamp: null
    generation: 1
    labels:
      run: kuard
    name: kuard
  spec:
    progressDeadlineSeconds: 600
    replicas: 1
    revisionHistoryLimit: 10
    selector:
      matchLabels:
        run: kuard
    strategy:
      rollingUpdate:
        maxSurge: 25%
        maxUnavailable: 25%
      type: RollingUpdate
    template:
      metadata:
        creationTimestamp: null
        labels:
          run: kuard
      spec:
        containers:
        - image: gcr.io/kuar-demo/kuard-amd64:blue
          imagePullPolicy: IfNotPresent
          name: kuard
          resources: {}
          terminationMessagePath: /dev/termination-log
          terminationMessagePolicy: File
        dnsPolicy: ClusterFirst
        restartPolicy: Always
        schedulerName: default-scheduler
        securityContext: {}
        terminationGracePeriodSeconds: 30
  status: {}
```

Musimy również uruchomić polecenie `kubectl replace --save-config`. Dodaje ono adnotację, aby podczas wprowadzania zmian w przyszłości narzędzie `kubectl` „wiedziało”, jaka jest ostatnia zastosowana konfiguracja, co zapewni sensowniejsze scalanie konfiguracji. Jeśli zawsze używasz polecenia `kubectl apply`, ten krok jest wymagany tylko po pierwszym utworzeniu wdrożenia za pomocą polecenia `kubectl create -f`.

Specyfikacja obiektu `Deployment` ma bardzo podobną strukturę do specyfikacji `ReplicaSet`. Jest w niej szablon kapsuły, który zawiera liczbę kontenerów tworzonych dla każdej repliki zarządzanej przez `Deployment`. Oprócz szablonu kapsuły jest tam również obiekt `strategy`:

```
  ...
    strategy:
      rollingUpdate:
        maxSurge: 25%
        maxUnavailable: 25%
      type: RollingUpdate
  ...
```

Obiekt `strategy` określa różne sposoby postępowania przy wprowadzaniu nowego oprogramowania. Istnieją dwie różne strategie obsługiwane przez `Deployment`: `Recreate` i `RollingUpdate`. Zostaną one omówione szczegółowo w dalszej części tego rozdziału.

Zarządzanie obiektami Deployment

Podobnie jak w przypadku wszystkich obiektów Kubernetes, szczegółowe informacje na temat obiektu `Deployment` można uzyskać za pomocą polecenia `kubectl describe`. Po użyciu tego polecenia otrzymujemy informacje o konfiguracji obiektu `Deployment`, które zawierają interesujące pola, takie jak `Selector`, `Replicas` i `Events`:

```
$ kubectl describe deployments kuard

Name:                   kuard
Namespace:              default
CreationTimestamp:      Tue, 01 Jun 2021 21:19:46 -0700
Labels:                 run=kuard
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               run=kuard
Replicas:               1 desired | 1 updated | 1 total | 1 available | 0 ...
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
    Labels: run=kuard
    Containers:
     kuard:
      Image: gcr.io/kuar-demo/kuard-amd64:blue
      Port: <none>
      Host Port: <none>
      Environment: <none>
      Mounts: <none>
    Volumes: <none>
  Conditions:
    Type        Status  Reason
    ----        ------  ------
    Available   True    MinimumReplicasAvailable
  OldReplicaSets: <none>
  NewReplicaSet: kuard-6d69d9fc5c (2/2 replicas created)
  Events:
   Type   Reason            Age                   From               Message
   ----   ------            ----                  ----               -------
   Normal ScalingReplicaSet 4m6s                  deployment-con... ...
   Normal ScalingReplicaSet 113s (x2 over 3m20s) deployment-con... ...
```

W danych wyjściowych z polecenia `describe` znajduje się wiele ważnych informacji. Dwoma najważniejszymi elementami tych informacji wyjściowych są `OldReplicaSets` i `NewReplicaSet`. Te pola wskazują obiekty `ReplicaSet`, którymi obecnie zarządza to wdrożenie. Jeśli wdrożenie jest w trakcie wprowadzania nowej wersji, oba pola będą ustawione na jakąś wartość. Jeżeli wprowadzanie nowej wersji zostało ukończone, pole `OldReplicaSets` będzie ustawione na `<none>`.

Oprócz polecenia `describe` dostępne jest również polecenie `kubectl rollout` dla wdrożeń. Zajmiemy się tym poleceniem szczegółowo nieco dalej, na razie możesz używać `kubectl rollout history`,

aby uzyskać historię wprowadzania nowych wersji powiązaną z konkretnym obiektem `Deployment`. Jeśli masz bieżące wdrożenie w toku, możesz użyć polecenia `kubectl rollout status`, by uzyskać bieżący status wprowadzania nowej wersji.

Aktualizowanie obiektów Deployment

Wdrożenia są obiektami deklaratywnymi opisującymi wdrożoną aplikację. Dwie najczęstsze operacje na obiekcie `Deployment` to skalowanie i aktualizowanie aplikacji.

Skalowanie obiektu Deployment

Chociaż wcześniej pokazaliśmy, w jaki sposób można imperatywnie skalować `Deployment` za pomocą polecenia `kubectl scale`, najlepiej jest zarządzać wdrożeniami deklaratywnie za pośrednictwem plików YAML, a następnie używać tych plików do aktualizowania wdrożeń. Aby wykonać skalowanie obiektu `Deployment` w górę, należy zmodyfikować plik YAML w celu zwiększenia liczby replik:

```
...
  spec:
    replicas: 3
...
```

Po zapisaniu i zatwierdzeniu tej zmiany można zaktualizować obiekt `Deployment` za pomocą polecenia `kubectl apply`:

```
$ kubectl apply -f kuard-deployment.yaml
```

Spowoduje to zaktualizowanie żądanego stanu wdrożenia, zwiększenie rozmiaru zarządzanego przez nie obiektu `ReplicaSet` i ostatecznie utworzona zostanie nowa kapsuła zarządzana przez to wdrożenie:

```
$ kubectl get deployments kuard

NAME      READY   UP-TO-DATE   AVAILABLE   AGE
kuard     3/3     3            3           10m
```

Aktualizowanie obrazu kontenera

Innym częstym przypadkiem użycia dla aktualizowania obiektu `Deployment` jest wprowadzanie nowej wersji oprogramowania działające w jednym lub w kilku kontenerach. W tym celu również należy edytować plik YAML wdrożenia, ale w tym przypadku aktualizuje się obraz kontenera, a nie liczbę replik:

```
...
      containers:
      - image: gcr.io/kuar-demo/kuard-amd64:green
        imagePullPolicy: Always
...
```

Umieścimy adnotację w szablonie dla tego wdrożenia, aby zarejestrować niektóre informacje o aktualizacji:

```
...
spec:
  ...
  template:
    metadata:
      annotations:
        kubernetes.io/change-cause: "Aktualizacja do kuard green"
...
```

Upewnij się, że dodajesz tę adnotację do szablonu, a nie do samego obiektu `Deployment`, ponieważ polecenie `kubectl apply` wykorzystuje to pole w obiekcie `Deployment`. Ponadto nie aktualizuj adnotacji `change-cause` podczas wykonywania prostych operacji skalowania. Modyfikacja `change-cause` jest znaczącą zmianą w szablonie i uruchamia wprowadzanie nowej wersji.

Ponownie możesz użyć polecenia `kubectl apply`, aby zaktualizować `Deployment`:

```
$ kubectl apply -f kuard-deployment.yaml
```

Po aktualizacji wdrożenia uruchomione zostanie wprowadzanie nowej wersji, które następnie można monitorować za pomocą polecenia `kubectl rollout`:

```
$ kubectl rollout status deployments kuard
deployment kuard successfully rolled out
```

Możesz zobaczyć stare i nowe obiekty `ReplicaSet` zarządzane przez dane wdrożenie wraz z używanymi obrazami. Zarówno stare, jak i nowe obiekty `ReplicaSet` są przechowywane na wypadek, gdybyś chciał wycofać zmiany:

```
$ kubectl get replicasets -o wide

NAME               DESIRED   CURRENT   READY   ...   IMAGE(S)            ...
kuard-1128242161   0         0         0       ...   gcr.io/kuar-demo/   ...
kuard-1128635377   3         3         3       ...   gcr.io/kuar-demo/   ...
```

Jeśli jesteś w trakcie wprowadzania nowej wersji i chcesz chwilowo zatrzymać je z jakiegoś powodu (na przykład jeżeli zaczniesz obserwować dziwne zachowanie w swoim systemie i chcesz to sprawdzić), możesz użyć polecenia `pause`:

```
$ kubectl rollout pause deployments kuard
deployment.apps/kuard paused
```

Jeżeli po przeprowadzeniu dochodzenia uznasz, że wprowadzanie nowej wersji może być bezpiecznie kontynuowane, możesz użyć polecenia `resume`, aby zacząć od miejsca, w którym skończyłeś:

```
$ kubectl rollout resume deployments kuard
deployment.apps/kuard resumed
```

Historia wersji

Obiekty `Deployment` Kubernetes utrzymują historię wprowadzania nowych wersji, co może być przydatne zarówno do zrozumienia poprzedniego stanu wdrożenia, jak i przywrócenia konkretnej wersji.

Historię wdrażania można wyświetlić, uruchamiając następujące polecenie:

```
$ kubectl rollout history deployment kuard

deployment.apps/kuard
REVISION        CHANGE-CAUSE
1               <none>
2               Aktualizacja do kuard green
```

Historia zmian jest podawana w kolejności od najstarszej do najnowszej. Unikatowy numer wersji jest zwiększany dla każdego nowego wdrożenia. Jak dotąd mamy dwie wersje: wstępne wdrożenie i aktualizację obrazu do `kuard:green`.

Jeśli chcesz uzyskać więcej szczegółów na temat konkretnej wersji, możesz dodać flagę `--revision`:

```
$ kubectl rollout history deployment kuard --revision=2

deployment.apps/kuard with revision #2
Pod Template:
  Labels: pod-template-hash=54b74ddcd4
        run=kuard
  Annotations: kubernetes.io/change-cause: Update to green kuard
  Containers:
    kuard:
      Image: gcr.io/kuar-demo/kuard-amd64:green
      Port: <none>
      Host Port: <none>
      Environment: <none>
      Mounts: <none>
    Volumes: <none>
```

Zróbmy jeszcze jedną aktualizację dla tego przykładu. Zaktualizuj wersję `kuard` z powrotem do `blue`, modyfikując numer wersji kontenera i aktualizując adnotację `change-couse`. Zrób to za pomocą polecenia `kubectl apply`. Nasza historia powinna teraz zawierać trzy wpisy:

```
$ kubectl rollout history deployment kuard

deployment.apps/kuard
REVISION        CHANGE-CAUSE
1               <none>
2               Aktualizacja do kuard green
3               Aktualizacja do kuard blue
```

Powiedzmy, że jest jakiś problem z najnowszą wersją i chcesz ją wycofać na czas badania sprawy. Możesz po prostu cofnąć ostatnie wdrożenie:

```
$ kubectl rollout undo deployments kuard
deployment.apps/kuard rolled back
```

Polecenie `undo` działa niezależnie od etapu wdrożenia. Można cofać zarówno częściowo ukończone, jak i całkowicie ukończone wdrożenia. Cofnięcie wdrożenia to w rzeczywistości po prostu wdrożenie w odwrotnej kolejności (na przykład z *v2* do *v1* zamiast z *v1* do *v2*) i wszystkie zasady, które obowiązują dla strategii wdrażania, mają również zastosowanie do strategii wycofania. Możesz zobaczyć, że obiekt `Deployment` po prostu dostosowuje żądaną liczbę replik w zarządzanych obiektach `ReplicaSet`:

```
$ kubectl get replicasets -o wide

NAME                DESIRED   CURRENT   READY   ...   IMAGE(S)            ...
kuard-1128242161    0         0         0       ...   gcr.io/kuar-demo/   ...
kuard-1570155864    0         0         0       ...   gcr.io/kuar-demo/   ...
kuard-2738859366    3         3         3       ...   gcr.io/kuar-demo/   ...
```

Gdy używasz plików deklaratywnych do kontrolowania systemów produkcyjnych, w miarę możliwości powinieneś upewnić się, że manifesty przesłane do systemu kontroli wersji odpowiadają temu, co rzeczywiście działa w Twoim klastrze. Kiedy wykonujesz polecenie `kubectl rollout undo`, aktualizujesz stan produkcyjny w sposób, który nie jest odzwierciedlony w systemie kontroli wersji.

Alternatywnym (i być może preferowanym) sposobem cofnięcia wdrożenia jest przywrócenie pliku YAML i zastosowanie poprzedniej wersji za pomocą polecenia `kubectl apply`. W ten sposób Twoja „konfiguracja ze śledzeniem zmian" będzie lepiej śledzić, co naprawdę działa w klastrze.

Spójrzmy ponownie na naszą historię wdrażania:

```
$ kubectl rollout history deployment kuard

deployment.apps/kuard
REVISION              CHANGE-CAUSE
1                     <none>
3                     Aktualizacja do kuard blue
4                     Aktualizacja do kuard green
```

Brak wersji 2.! Okazuje się, że po przywróceniu poprzedniej wersji obiekt `Deployment` po prostu ponownie korzysta z szablonu i zmienia jego numerację, tak aby był najnowszą wersją. To, co było wcześniej wersją 2., teraz jest wersją 4.

Wcześniej widzieliśmy, że do powrotu do poprzedniej wersji wdrożenia można użyć polecenia `kubectl rollout undo`. Dodatkowo można wrócić do konkretnej wersji z historii przy użyciu flagi `--to-revision`:

```
$ kubectl rollout undo deployments kuard --to-revision=3
deployment.apps/kuard rolled back
$ kubectl rollout history deployment kuard
deployment.apps/kuard
REVISION        CHANGE-CAUSE
1               <none>
4               Aktualizacja do kuard green
5               Aktualizacja do kuard blue
```

Ponownie w wyniku użycia polecenia `undo` została zastosowana wersja 3., przenumerowana na wersję 5.

Określenie wersji `0` jest skróconą metodą określenia poprzedniej wersji. Dlatego polecenie `kubectl rollout undo` jest równoważne z `kubectl rollout undo --to-revision=0`.

Domyślnie do obiektu `Deployment` jest dołączonych 10 ostatnich wersji, zaleca się więc, aby w przypadku wdrożeń, które prawdopodobnie będą istniały przez długi czas, ustawić maksymalny rozmiar dla historii wersji. Jeżeli przeprowadzasz na przykład codzienne aktualizacje, możesz ograniczyć

historię wersji do 14, aby przechowywać maksymalnie 2 tygodnie danych (jeśli nie przewidujesz konieczności przywrócenia poprzedniej wersji po 2 tygodniach).

W tym celu użyj właściwości `revisionHistoryLimit` w specyfikacji obiektu `Deployment`:

```
...
spec:
  # Przeprowadzamy codzienne wdrażania i ograniczamy historię wersji do dwóch tygodni,
  # ponieważ po upływie tego czasu nie przewidujemy przywracania wcześniejszych wersji.
  revisionHistoryLimit: 14
...
```

Strategie wdrażania

Kiedy przychodzi czas na zmianę wersji oprogramowania implementującego usługę, obiekt `Deployment` Kubernetes obsługuje dwie różne strategie wdrażania: `Recreate` (odtworzenie) i `Rolling Update` (ciągła aktualizacja). Przyjrzymy się im bliżej.

Strategia Recreate

Strategia odtworzenia jest prostszą z tych dwóch strategii wdrażania. Obiekt `Deployment` aktualizuje po prostu zarządzany przez siebie obiekt `ReplicaSet`, aby użył nowego obrazu, i usuwa wszystkie kapsuły powiązane z tym wdrożeniem. `ReplicaSet` zauważa, że nie ma już żadnych replik, i ponownie tworzy wszystkie kapsuły za pomocą nowego obrazu. Po ponownym utworzeniu kapsuły działają z nową wersją.

Chociaż ta strategia jest szybka i prosta, wiążą się z nią przestoje spowodowane obciążeniem. Z tego powodu strategia odtworzenia powinna być używana tylko dla wdrożeń testowych, w przypadku których przestój jest akceptowalny.

Strategia RollingUpdate

Strategia ciągłej aktualizacji jest, ogólnie rzecz biorąc, preferowaną strategią dla każdej usługi skierowanej do użytkownika. Choć jest wolniejsza niż `Recreate`, jest również znacznie bardziej wyrafinowana i solidna. Korzystając z `RollingUpdate`, możesz wdrożyć nową wersję usługi, a usługa w tym czasie nadal będzie otrzymywać ruch od użytkowników bez żadnych przestojów.

Jak możesz wywnioskować z nazwy, strategia ciągłej aktualizacji działa poprzez aktualizowanie kilku kapsuł naraz, poruszających się stopniowo, dopóki we wszystkich kapsułach nie będzie działać nowa wersja oprogramowania.

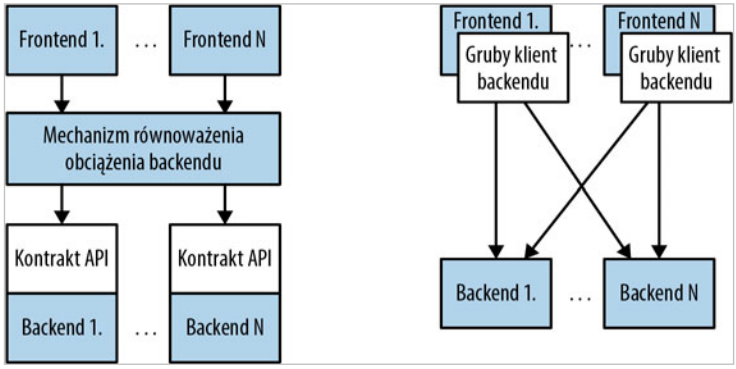
Zarządzanie wieloma wersjami usługi

Co ważne, oznacza to, że przez pewien czas zarówno nowa, jak i stara wersja usługi będzie odbierać żądania i obsługiwać ruch. Ma to ważne implikacje dla sposobu budowania oprogramowania. Mianowicie, niezwykle ważne jest, aby każda wersja oprogramowania i wszystkie jej klienty mogły komunikować się zarówno z nieco starszą, jak i nieco nowszą wersją oprogramowania.

Rozważmy następujący scenariusz: jesteś w trakcie wprowadzania nowej wersji Twojego oprogramowania frontendowego; na połowie Twoich serwerów działa wersja 1., a na drugiej połowie działa wersja 2. Użytkownik wysyła wstępne żądanie do usługi i pobiera kliencką bibliotekę JavaScript, która implementuje Twój interfejs użytkownika. To żądanie jest obsługiwane przez serwer z wersją 1., a zatem użytkownik otrzymuje wersję 1. biblioteki klienta. Ta biblioteka klienta działa w przeglądarce użytkownika i wysyła kolejne żądania API do Twojej usługi. Te żądania API są akurat kierowane do serwera z wersją 2. oprogramowania; w ten sposób wersja 1. klienckiej biblioteki JavaScript komunikuje się z wersją 2. Twojego serwera API. Jeżeli nie zapewniłeś zgodności między tymi wersjami, Twoja aplikacja nie będzie działać poprawnie.

Na początku może to wydawać się dodatkowym obciążeniem. Ale tak naprawdę zawsze miałeś ten problem; może po prostu go nie zauważyłeś. Mówiąc konkretnie, użytkownik może wysłać żądanie w czasie *t* tuż przed rozpoczęciem aktualizacji. To żądanie jest obsługiwane przez serwer z wersją 1. W czasie *t1* aktualizujesz usługę do wersji 2. W czasie *t2* kod klienta w wersji 1. działający w przeglądarce użytkownika uruchamia się i napotyka punkt końcowy interfejsu API obsługiwany przez serwer z wersją 2. Bez względu na to, jak będziesz aktualizował swoje oprogramowanie, musisz zachować kompatybilność wstecz i w przód w celu zagwarantowania niezawodnych aktualizacji. Charakter strategii ciągłej aktualizacji po prostu czyni to zagadnienie bardziej przejrzystym i wyraźnym, należy więc wziąć to pod uwagę.

Pamiętaj, że nie dotyczy to tylko klientów JavaScriptowych — to samo odnosi się do bibliotek klienckich skompilowanych w innych usługach, które wywołują Twoją usługę. Fakt, że Ty dokonałeś aktualizacji, nie oznacza, że pozostali również zaktualizowali swoje biblioteki klienckie. Ten rodzaj kompatybilności wstecznej ma kluczowe znaczenie dla oddzielenia usługi od systemów, które od niej zależą. Jeżeli nie sformalizujesz swoich interfejsów API i nie oddzielisz się od pozostałych systemów, będziesz zmuszony ostrożnie zarządzać wdrożeniami razem ze wszystkimi innymi systemami wywołującymi Twoją usługę. Ten rodzaj ścisłego powiązania sprawia, że niezwykle trudno jest osiągnąć niezbędną zwinność, aby być w stanie wysyłać nowe oprogramowanie co tydzień, nie mówiąc już o robieniu tego co godzinę lub codziennie. W rozłącznej architekturze pokazanej na rysunku 10.1 frontend jest odizolowany od backendu poprzez kontrakt API i system równoważenia obciążenia, a w architekturze ściśle powiązanej gruby klient wkompilowany we frontend jest używany do bezpośredniego łączenia się z backendami.



Rysunek 10.1. Diagramy rozłącznej (po lewej) i ściśle powiązanej (po prawej) architektury aplikacji

Konfigurowanie ciągłej aktualizacji

`RollingUpdate` to dość ogólna strategia; może być używana do aktualizacji rozmaitych aplikacji w wielu różnych ustawieniach. W związku z tym sama ciągła aktualizacja jest konfigurowalna; można dostosować jej zachowanie do własnych potrzeb. Do dostrajania działania ciągłej aktualizacji można używać dwóch parametrów: `maxUnavailable` i `maxSurge`.

Parametr `maxUnavailable` ustawia maksymalną liczbę kapsuł, które mogą być niedostępne podczas ciągłej aktualizacji. Może być ustawiony na liczbę bezwzględną (na przykład 3, co oznacza, że maksymalnie trzy kapsuły mogą być niedostępne) lub jako wartość procentowa (na przykład 20%, co oznacza, że maksymalnie niedostępnych może być 20% z żądanej liczby replik). Ogólnie rzecz biorąc, w przypadku większości usług dobrym podejściem jest użycie wartości procentowej, ponieważ jest ona poprawnie stosowana niezależnie od żądanej liczby replik w obiekcie `Deployment`. Czasami jednak należy użyć liczby bezwzględnej (na przykład ograniczając maksymalną liczbę niedostępnych kapsuł do jednej).

Zasadniczo parametr `maxUnavailable` pomaga dostroić szybkość przeprowadzania ciągłej aktualizacji. Jeżeli ustawisz na przykład `maxUnavailable` na `50%`, wówczas ciągła aktualizacja natychmiast przeskaluje stary `ReplicaSet` do 50% jego oryginalnego rozmiaru. Jeśli masz cztery repliki, zmniejszy ich liczbę do dwóch. Następnie ciągła aktualizacja zastąpi usunięte kapsuły poprzez skalowanie nowego obiektu `ReplicaSet` do dwóch replik, co da łącznie cztery repliki (dwie stare i dwie nowe). Potem przeskaluje stary `ReplicaSet` do zerowej liczby replik, co da łącznie dwie nowe repliki. Na koniec przeskaluje nowy `ReplicaSet` do czterech replik, kończąc wdrażanie. Tak więc przy ustawieniu `maxUnavailable` na `50%` wprowadzanie nowej wersji zostanie ukończone w czterech krokach, ale zapewnione będzie w tym czasie tylko 50% naszej zdolności do świadczenia usług.

Zastanów się, co stanie się, jeśli ustawimy `maxUnavailable` na `25%`. W tej sytuacji każdy krok będzie wykonywany tylko z pojedynczą repliką naraz, a więc do ukończenia wdrażania wymagane będzie wykonanie dwa razy większej liczby kroków, ale dostępność podczas wdrażania spadnie minimalnie, do 75%. To ilustruje, w jaki sposób odpowiednie ustawienie parametru `maxUnavailable` pozwala wyważyć prędkość wdrażania i dostępność.

Spostrzegawczy czytelnicy mogą zauważyć, że strategia odtworzenia jest w rzeczywistości tożsama ze strategią ciągłej aktualizacji z parametrem `maxUnavailable` ustawionym na `100%`.

Zmniejszanie pojemności usługi w celu zapewnienia skutecznego wdrażania jest przydatne, gdy usługa charakteryzuje się cyklicznymi wzorcami obciążenia ruchem (na przykład znacznie mniejszy ruch w nocy) lub gdy masz ograniczone zasoby, nie jest więc możliwe skalowanie do większej niż obecna maksymalna liczba replik.

Są jednak sytuacje, w których nie chcesz schodzić poniżej 100% możliwości, ale jesteś gotów tymczasowo skorzystać z dodatkowych zasobów w celu przeprowadzenia wdrożenia nowej wersji. W takich sytuacjach można ustawić parametr `maxUnavailable` na 0% i kontrolować wdrożenie za pomocą parametru `maxSurge`. Podobnie jak w przypadku `maxUnavailable`, wartość parametru `maxSurge` może być określona jako wartość liczbowa lub procentowa.

Parametr `maxSurge` kontroluje, ile dodatkowych zasobów można utworzyć w celu przeprowadzenia wdrożenia nowej wersji. Aby zilustrować, jak to działa, wyobraź sobie, że mamy usługę z 10 replikami. Ustawiliśmy `maxUnavailable` na `0`, a `maxSurge` na `20%`. To wdrożenie najpierw spowoduje przeskalowanie nowego `ReplicaSet` do dwóch replik, co da łącznie 12 (120%) replik w usłudze. Następnie przeskalowany zostanie stary obiekt `ReplicaSet` do 8 replik, co da łącznie 10 (8 starych, 2 nowe) replik w usłudze. Ten proces potrwa, dopóki wprowadzanie aktualizacji nie zostanie ukończone. W każdym dowolnym momencie pojemność usługi ma gwarantowane minimum 100%, a maksymalne dodatkowe zasoby wykorzystywane do wdrożenia są ograniczone do dodatkowych 20% wszystkich zasobów.

Ustawienie `maxSurge` na `100%` jest równoważne z zastosowaniem metody wdrażania typu *blue-green*. Kontroler `Deployment` najpierw skaluje nową wersję do 100% starej wersji. Gdy nowa wersja jest zdrowa, natychmiast skaluje starą wersję do 0%.

Spowalnianie wdrażania w celu zapewnienia poprawnego działania usługi

Celem wdrażania etapowego jest zapewnienie, że otrzymamy zdrową i stabilną usługę uruchamiającą nową wersję oprogramowania. W tym celu zanim kontroler `Deployment` przejdzie do aktualizacji kolejnej kapsuły, zawsze czeka, aż dana kapsuła zgłosi, że jest gotowa.

Kontroler `Deployment` określa status kapsuły na podstawie kontroli gotowości. Kontrole gotowości są częścią sondowania poprawności działania kapsuły i zostały opisane szczegółowo w rozdziale 5. Jeżeli chcesz używać kontrolerów `Deployment` do niezawodnego wdrażania oprogramowania, *musisz* określić kontrole gotowości dla kontenerów w kapsule. Jeśli tego nie zrobisz, kontroler `Deployment` będzie działał bez wiedzy na temat stanu kapsuły.

Czasami jednak stwierdzenie po prostu, że kapsuła stała się gotowa, nie daje dostatecznej pewności, iż rzeczywiście zachowuje się poprawnie. Niektóre stany błędów nie pojawiają się natychmiast. Możesz mieć na przykład poważne wycieki pamięci, które ujawniają się po kilku minutach, lub błąd wywołany tylko przez 1% wszystkich żądań. W większości scenariuszy w świecie rzeczywistym należy odczekać pewien czas, aby przed przejściem do aktualizacji następnej kapsuły mieć pewność, że nowa wersja działa poprawnie.

W przypadku wdrożeń ten czas oczekiwania jest określany przez parametr `minReadySeconds`:

```
...
spec:
  minReadySeconds: 60
...
```

Ustawienie wartości `minReadySeconds` na `60` powoduje, że `Deployment` *po* stwierdzeniu, że kapsuła działa poprawnie, musi odczekać 60 sekund, zanim przejdzie do aktualizacji następnej kapsuły.

Oprócz czasu oczekiwania, aż kapsuła okaże się zdrowa, należy również ustawić limit czasu, który ograniczy czas oczekiwania systemu. Załóżmy na przykład, że nowa wersja usługi ma błąd prowadzący do jej natychmiastowego zakleszczenia. Ta kapsuła nigdy nie stanie się gotowa, a przy braku limitu czasu oczekiwania kontroler `Deployment` utknie z wdrożeniem na zawsze.

Prawidłowe zachowanie w takiej sytuacji wiąże się z upłynięciem limitu czasu dla wdrożenia. To z kolei powoduje oznaczenie wdrożenia jako nieudanego. Status awarii może być używany do wyzwalania alertów wskazujących operatorowi, że wystąpił problem z wprowadzaniem nowej wersji.

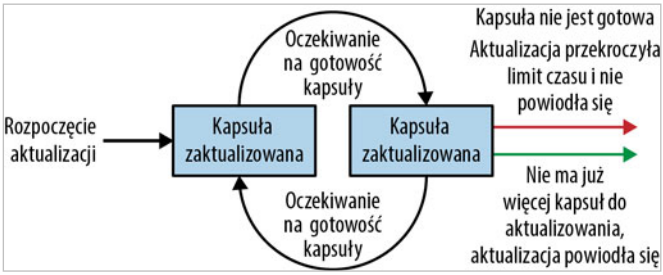
Na pierwszy rzut oka ograniczenie czasu wdrożenia może wydawać się niepotrzebną komplikacją. Jednak coraz częściej takie procesy jak wprowadzanie nowej wersji oprogramowania są uruchamiane przez w pełni zautomatyzowane systemy przy niewielkim lub zerowym zaangażowaniu czynnika ludzkiego. W takiej sytuacji upłynięcie limitu czasu staje się krytycznym wyjątkiem, który może spowodować automatyczne wycofanie wersji lub utworzenie biletu (zdarzenia) wyzwalającego interwencję człowieka.

Aby ustawić limit czasu, należy określić parametr `progressDeadlineSeconds` kontrolera `Deployment`:

```
...
spec:
  progressDeadlineSeconds: 600
...
```

W tym przykładzie ustalono termin realizacji na 10 minut. Jeśli jakiś konkretny etap we wdrożeniu nie zostanie przeprowadzony w ciągu 10 minut, wdrożenie zostanie oznaczone jako nieudane i wstrzymane zostaną wszystkie próby przejścia do kolejnego etapu.

Należy pamiętać, że ten limit czasu jest podawany w odniesieniu do *postępu* wdrażania, a nie całkowitej długości wdrożenia. W tym kontekście postęp jest definiowany jako każdy moment, kiedy podczas wdrożenia tworzona jest lub usuwana kapsuła. Gdy tak się dzieje, licznik limitu czasu jest resetowany do zera. Rysunek 10.2 przedstawia cykl życia wdrożenia.



Rysunek 10.2. Cykl życia wdrożenia Kubernetes

Usuwanie wdrożenia

Jeśli kiedykolwiek będziesz chciał usunąć wdrożenie, możesz to zrobić za pomocą polecenia imperatywnego:

```
$ kubectl delete deployments kuard
```

Możesz też użyć deklaratywnego pliku YAML, który utworzyliśmy wcześniej:

```
$ kubectl delete -f kuard-deployment.yaml
```

W obu przypadkach usunięcie obiektu `Deployment` spowoduje domyślnie usunięcie całej usługi. Zostanie usunięty sam obiekt `Deployment`, a także wszystkie zarządzane przez niego obiekty `ReplicaSet`, jak również wszystkie kapsuły zarządzane przez te obiekty `ReplicaSet`. Podobnie jak w przypadku `ReplicaSet`, jeśli nie jest to pożądane zachowanie, można użyć flagi `--cascade=false`, aby usunąć wyłącznie obiekt `Deployment`.

Monitorowanie wdrożenia

W odniesieniu do wdrożeń należy pamiętać, że jeśli przez pewien czas nie będzie postępu operacji, nastąpi przekroczenie limitu czasu i stan wdrożenia zostanie ustawiony na niepowodzenie. Wartość tę można pobrać z tablicy `status.conditions`, w której można znaleźć warunek typu `Progressing` ze stanem `False`. Wdrożenie w takim stanie jest nieudane i nie będzie kontynuowane. Czas oczekiwania przez kontroler wdrożenia na przejście w ten stan można ustawić w polu `spec.progressDeadlineSeconds`.

Podsumowanie

Możemy podsumować, że głównym celem systemu Kubernetes jest ułatwienie budowania i wdrażania niezawodnych systemów rozproszonych. Oznacza to nie tylko jednorazowe utworzenie instancji aplikacji, ale także zarządzanie regularnym wdrażaniem nowych wersji tej programowej usługi. Wdrożenia są kluczowym elementem niezawodnego wprowadzania nowych wersji i zarządzania wprowadzaniem tych wersji dla Twoich usług. W następnym rozdziale opisujemy obiekty `DaemonSet`, które pozwalają dopilnować, aby na zestawie węzłów w klastrze Kubernetes działała tylko jedna kopia kapsuły.

ROZDZIAŁ 11.
Obiekt DaemonSet

Obiekty `ReplicaSet` służą zasadniczo do tworzenia usługi (na przykład serwera WWW) z wieloma replikami dla zapewnienia redundancji. Ale chęć zapewnienia redundancji nie jest jedynym powodem replikowania zestawu kapsuł w klastrze. Kolejnym powodem jest zamiar rozplanowania pojedynczej kapsuły na wszystkie węzły w klastrze. Ogólnie rzecz biorąc, celem replikacji kapsuły na wszystkie węzły jest umieszczenie na każdym z nich pewnego rodzaju agenta lub demona, a służy do tego obiekt `DaemonSet` Kubernetes.

`DaemonSet` gwarantuje działanie kopii kapsuły w całym zestawie węzłów w klastrze Kubernetes. Obiekty `DaemonSet` służą do wdrażania demonów systemowych, takich jak programy do gromadzenia dzienników i agenty monitorowania, które zwykle muszą działać na każdym węźle. Obiekty `DaemonSet` mają podobne funkcje do obiektów `ReplicaSet`; oba typy obiektów tworzą kapsuły, które mają być długo działającymi usługami, i zapewniają, że żądany stan będzie odpowiadać obserwowanemu.

Biorąc pod uwagę podobieństwa między obiektami `DaemonSet` i `ReplicaSet`, ważne jest zrozumienie, kiedy używać jednego, a kiedy drugiego obiektu. Z kontrolerów `ReplicaSet` należy korzystać wtedy, gdy aplikacja jest całkowicie oddzielona od danego węzła i można uruchomić na nim wiele kopii tej aplikacji bez żadnych specjalnych uwzględnień. Obiekty `DaemonSet` natomiast powinny być używane w sytuacji, gdy pojedyncza kopia aplikacji musi działać na wszystkich węzłach w klastrze lub na pewnym podzbiorze węzłów.

Aby zagwarantować, że kapsuły nie będą kolokowane na tym samym węźle, zasadniczo nie należy używać ograniczeń rozplanowywania ani innych parametrów. Jeśli będziesz chciał mieć jedną kapsułę na węzeł, wtedy odpowiednim zasobem Kubernetes, którego należy użyć, jest obiekt `DaemonSet`. Odpowiednio, gdy będziesz budował jednorodną zreplikowaną usługę do obsługi ruchu użytkowników, prawdopodobnie do tego celu właściwym zasobem Kubernetes będzie obiekt `ReplicaSet`.

Do uruchamiania kapsuł `DaemonSet` na wybranych węzłach można używać etykiet. W ten sposób można na przykład uruchomić specjalne oprogramowanie wykrywające próby włamań na węzłach brzegowych sieci.

Za pomocą obiektów `DaemonSet` można też instalować oprogramowanie na węzłach w klastrach chmurowych. W wielu usługach chmurowych uaktualnienie lub skalowanie klastra może spowodować usunięcie albo ponowne utworzenie maszyn wirtualnych. Taka dynamiczna i *niezmienna*

infrastruktura może sprawiać trudności w przypadkach, gdy chcemy (lub jesteśmy zmuszeni przez dział IT) na każdym węźle mieć określone oprogramowanie. `DaemonSet` jest właśnie rozwiązaniem w takiej sytuacji — kiedy chcemy mieć pewność, że na każdym komputerze będzie znajdowało się konkretne oprogramowanie niezależnie od aktualizacji i skalowania. Można nawet zamontować system plików hosta i wykonywać skrypty instalujące pakiety RPM/DEB w systemie operacyjnym. W ten sposób otrzymujemy klaster chmurowy spełniający wszystkie wymagania działu IT.

Planista DaemonSet

Domyślnie `DaemonSet` tworzy kopię kapsuły na każdym węźle, chyba że użyty zostanie selektor węzłów, który ograniczy kwalifikujące się węzły do tych, które odpowiadają zestawowi etykiet. Podczas tworzenia kapsuły obiekty `DaemonSet` określają, w którym węźle będzie działać dana kapsuła, definiując pole `nodeName` w specyfikacji kapsuły. W wyniku tego kapsuły tworzone przez obiekty `DaemonSet` są ignorowane przez planistę Kubernetes.

Podobnie jak kontrolery `ReplicaSet`, obiekty `DaemonSet` są zarządzane przez pętlę kontroli uzgadniania, która porównuje żądany stan (kapsuła jest obecna na wszystkich węzłach) ze stanem obserwowanym (czy kapsuła jest obecna na danym węźle?). Biorąc pod uwagę te informacje, kontroler `DaemonSet` tworzy kapsułę na każdym węźle, który w danym momencie nie ma odpowiedniej kapsuły.

Jeżeli do klastra zostanie dodany nowy węzeł, kontroler `DaemonSet` zauważy, że brakuje na nim kapsuły, i doda ją do tego węzła.

Obiekty `DaemonSet` i `ReplicaSet` doskonale ilustrują wartość rozłącznej architektury systemu. Mogłoby się wydawać, że właściwe byłoby, gdyby kontroler `ReplicaSet` był właścicielem kapsuł, którymi zarządza, a kapsuły były podzasobami `ReplicaSet`. Analogicznie, kapsuły zarządzane przez `DaemonSet` byłyby podzasobami tego obiektu. Jednak ten rodzaj hermetyzacji wymagałby dwukrotnego napisania narzędzi służących do obsługi kapsuł: raz dla `DaemonSet` i drugi raz dla `ReplicaSet`. Zamiast tego Kubernetes korzysta z podejścia opartego na rozłączności, w którym obiektami najwyższego poziomu są kapsuły. Oznacza to, że każde narzędzie, z którego nauczyłeś się korzystać przy inspekcji kapsuł w kontekście obiektów `ReplicaSet` (na przykład `kubectl logs <nazwa_kapsuły>`), ma również zastosowanie do kapsuł tworzonych przez obiekty `DaemonSet`.

Tworzenie obiektów DaemonSet

Obiekty `DaemonSet` są tworzone poprzez przesłanie konfiguracji `DaemonSet` do serwera API Kubernetes. `DaemonSet` pokazany na listingu 11.1 utworzy agenta gromadzenia dzienników `fluentd` na każdym węźle w klastrze docelowym.

Listing 11.1. fluentd.yaml

```
apiVersion: apps/v1
kind: DaemonSet
```

```
metadata:
  name: fluentd
  namespace: kube-system
  labels:
    app: fluentd
spec:
  selector:
    matchLabels:
      app: fluentd
  template:
    metadata:
      labels:
        app: fluentd
    spec:
      containers:
      - name: fluentd
        image: fluent/fluentd:v0.14.10
        resources:
          limits:
            memory: 200Mi
          requests:
            cpu: 100m
            memory: 200Mi
        volumeMounts:
        - name: varlog
          mountPath: /var/log
        - name: varlibdockercontainers
          mountPath: /var/lib/docker/containers
          readOnly: true
      terminationGracePeriodSeconds: 30
      volumes:
      - name: varlog
        hostPath:
          path: /var/log
      - name: varlibdockercontainers
        hostPath:
          path: /var/lib/docker/containers
```

Poszczególne obiekty `DaemonSet` wymagają unikatowych nazw w danej przestrzeni nazw Kubernetes. Każdy `DaemonSet` musi zawierać specyfikację szablonu kapsuły, która będzie używana w razie potrzeby utworzenia kapsuły. To właśnie tutaj kończą się podobieństwa między obiektami `ReplicaSet` i `DaemonSet`. W przeciwieństwie do `ReplicaSet` kontrolery `DaemonSet` domyślnie tworzą kapsułę na każdym węźle w klastrze, chyba że użyty zostanie selektor węzłów.

Gdy masz już poprawną konfigurację `DaemonSet`, możesz użyć polecenia `kubectl apply`, aby przesłać dany obiekt `DaemonSet` do interfejsu API Kubernetes. W tym podrozdziale utworzymy kontroler `DaemonSet`, aby zapewnić, że serwer HTTP `fluentd` będzie działał na każdym węźle w naszym klastrze:

```
$ kubectl apply -f fluentd.yaml
daemonset.apps/fluentd created
```

Po pomyślnym przesłaniu obiektu `DaemonSet` serwera `fluentd` do interfejsu API Kubernetes możesz zapytać o jego bieżący stan za pomocą polecenia `kubectl describe`:

```
$ kubectl describe daemonset fluentd
Name:           fluentd
Selector:       app=fluentd
Node-Selector:  <none>
Labels:         app=fluentd
Annotations: deprecated.daemonset.template.generation: 1
Desired Number of Nodes Scheduled: 3
Current Number of Nodes Scheduled: 3
Number of Nodes Scheduled with Up-to-date Pods: 3
Number of Nodes Scheduled with Available Pods: 3
Number of Nodes Misscheduled: 0
Pods Status: 3 Running / 0 Waiting / 0 Succeeded / 0 Failed
```

Te dane wyjściowe wskazują, że kapsuła `fluentd` została pomyślnie wdrożona na wszystkich trzech węzłach w naszym klastrze. Możemy to sprawdzić za pomocą polecenia `kubectl get pods` z flagą `-o`, aby wydrukować węzły, do których przypisano każdą kapsułę `fluentd`:

```
$ kubectl get pods -l app=fluentd -o wide
NAME           READY  STATUS   RESTARTS  AGE     IP              NODE
fluentd-1q6c6  1/1    Running  0         13m     10.240.0.101    k0-default...
fluentd-mwi7h  1/1    Running  0         13m     10.240.0.80     k0-default...
fluentd-zr6l7  1/1    Running  0         13m     10.240.0.44     k0-default...
```

Po wprowadzeniu kontrolera `DaemonSet` serwera `fluentd` dodanie nowego węzła do klastra spowoduje, że kapsuła `fluentd` zostanie na tym węźle wdrożona automatycznie:

```
$ kubectl get pods -o wide
NAME           READY  STATUS     RESTARTS   AGE     IP              NODE
fluentd-1q6c6  1/1    Running    0          13m     10.240.0.101    k0-default...
fluentd-mwi7h  1/1    Running    0          13m     10.240.0.80     k0-default...
fluentd-oipmq  1/1    Running    0          43s     10.240.0.96     k0-default...
fluentd-zr6l7  1/1    Running    0          13m     10.240.0.44     k0-default...
```

Właśnie takie zachowanie chcemy osiągnąć, gdy zarządzamy demonami gromadzenia dzienników i innymi usługami obejmującymi cały klaster. Z naszej strony nie było wymagane żadne działanie. Tak właśnie kontroler `DaemonSet` Kubernetes uzgadnia obserwowany stan z naszym stanem żądanym.

Ograniczanie użycia kontrolerów DaemonSet do określonych węzłów

Najczęstszym przypadkiem użycia dla obiektów `DaemonSet` jest uruchamianie kapsuły na każdym węźle w klastrze Kubernetes. Są jednak pewne przypadki, w których potrzebujemy wdrożyć kapsułę tylko na podzbiorze węzłów. Możesz mieć na przykład obciążenie robocze wymagające GPU lub szybkiego magazynu danych dostępnego tylko w podzbiorze węzłów w klastrze. W takich przypadkach można użyć etykiet węzłów, aby oznaczyć określone węzły, które spełniają wymagania danego obciążenia roboczego.

Dodawanie etykiet do węzłów

Pierwszym krokiem podczas ograniczania użycia kontrolerów `DaemonSet` do określonych węzłów jest dodanie żądanego zestawu etykiet do podzbioru węzłów. Można to zrobić za pomocą polecenia `kubectl label`.

Poniższe polecenie powoduje dodanie etykiety `ssd=true` do pojedynczego węzła:

```
$ kubectl label nodes k0-default-pool-35609c18-z7tb ssd=true
node "k0-default-pool-35609c18-z7tb" labeled
```

Podobnie jak w przypadku innych zasobów Kubernetes, wylistowanie węzłów bez selektora etykiet zwraca wszystkie węzły w klastrze:

```
$ kubectl get nodes
NAME                            STATUS    ROLES    AGE     VERSION
k0-default-pool-35609c18-0xnl   Ready     agent    23m     v1.21.1
k0-default-pool-35609c18-pol3   Ready     agent    1d      v1.21.1
k0-default-pool-35609c18-ydae   Ready     agent    1d      v1.21.1
k0-default-pool-35609c18-z7tb   Ready     agent    1d      v1.21.1
```

Za pomocą selektora etykiet możemy filtrować węzły na podstawie etykiet. Aby wyświetlić tylko te węzły, które mają ustawioną etykietę `ssd` na `true`, użyj polecenia `kubectl get nodes` z flagą `--selector`:

```
$ kubectl get nodes --selector ssd=true
NAME                            STATUS      ROLES     AGE      VERSION
k0-default-pool-35609c18-z7tb   Ready       agent     1d       v1.21.1
```

Selektory węzłów

Selektory węzłów mogą być używane do ograniczenia węzłów, na których może działać kapsuła w danym klastrze Kubernetes. Selektory węzłów są definiowane jako część specyfikacji kapsuły podczas tworzenia obiektu `DaemonSet`. Konfiguracja `DaemonSet` przedstawiona na listingu 11.2 ogranicza uruchamianie serwera `nginx` tylko do węzłów z ustawioną etykietą `ssd=true`.

Listing 11.2. nginx-fast-storage.yaml

```
apiVersion: apps/v1
kind: "DaemonSet"
metadata:
  labels:
    app: nginx
    ssd: "true"
  name: nginx-fast-storage
spec:
  selector:
    matchLabels:
      app: nginx
      ssd: "true"
  template:
    metadata:
      labels:
        app: nginx
        ssd: "true"
```

```
spec:
  nodeSelector:
    ssd: "true"
  containers:
    - name: nginx
      image: nginx:1.10.0
```

Zobaczmy, co się stanie, gdy prześlemy obiekt `DaemonSet` o nazwie `nginx-fast-storage` do interfejsu API Kubernetes:

```
$ kubectl apply -f nginx-fast-storage.yaml
daemonset.apps/nginx-fast-storage created
```

Ponieważ istnieje tylko jeden węzeł z etykietą `ssd=true`, kapsuła `nginx-fast-storage` zostanie uruchomiona tylko na tym węźle:

```
$ kubectl get pods -l app=nginx -o wide
NAME                        READY   STATUS     RESTARTS    AGE   IP           NODE
nginx-fast-storage-7b90t    1/1     Running    0           44s   10.240.0.48  ...
```

Dodanie etykiety `ssd=true` do kolejnych węzłów sprawi, że kapsuła `nginx-fast-storage` zostanie wdrożona również na tych węzłach. Działa to także w drugą stronę: jeśli wymagana etykieta zostanie usunięta z węzła, kapsuła zostanie usunięta przez kontroler `DaemonSet`.

Usunięcie z węzła etykiet wymaganych przez selektor węzłów kontrolera `DaemonSet` spowoduje, że kapsuła zarządzana przez ten `DaemonSet` zostanie usunięta z węzła.

Aktualizowanie obiektu DaemonSet

Kontrolery `DaemonSet` doskonale nadają się do wdrażania usług w całym klastrze, ale co z aktualizacjami? Przed wprowadzeniem Kubernetes 1.6 jedynym sposobem na aktualizację kapsuł zarządzanych przez kontroler `DaemonSet` było zaktualizowanie tego kontrolera, a następnie ręczne usunięcie każdej kapsuły, która była przez niego zarządzana, aby mogła zostać ponownie utworzona przy użyciu nowej konfiguracji. Wraz z wydaniem wersji Kubernetes 1.6 obiekty `DaemonSet` uzyskały odpowiednik obiektu `Deployment`, który zarządza wdrażaniem nowych wersji `ReplicaSet` wewnątrz klastra.

Nowe wersje obiektów `DaemonSet` można wprowadzać przy użyciu tej samej strategii ciągłej aktualizacji, której używają wdrożenia. Można skonfigurować tę strategię za pomocą pola `spec.updateStrategy.type`. To pole powinno mieć wartość `RollingUpdate`. Kiedy `DaemonSet` ma ustawioną strategię aktualizacji `RollingUpdate`, każda zmiana pola `spec.template` (lub podpól) w `DaemonSet` inicjuje ciągłą aktualizację.

Podobnie jak w przypadku ciągłych aktualizacji wdrożeń (zobacz rozdział 10.), w ramach strategii ciągłej aktualizacji składowe `DaemonSet` są stopniowo aktualizowane, dopóki wszystkie kapsuły nie uruchomią nowej konfiguracji. Istnieją dwa parametry kontrolujące ciągłą aktualizację obiektu `DaemonSet`:

`spec.minReadySeconds`
: określa, jak długo kapsuła musi być „gotowa”, zanim w ramach ciągłej aktualizacji będzie można przejść do aktualizacji kolejnych kapsuł.

`spec.updateStrategy.rollingUpdate.maxUnavailable`
: wskazuje, jak dużo kapsuł może być jednocześnie aktualizowanych przez ciągłą aktualizację.

Parametr `spec.minReadySeconds` z reguły ustawia się na rozsądnie dużą wartość, na przykład 30 – 60 sekund, aby upewnić się, że kapsuła jest naprawdę zdrowa, zanim aktualizacja będzie kontynuowana.

Wartość ustawiana dla `spec.updateStrategy.rollingUpdate.maxUnavailable` zwykle zależy od aplikacji. Ustawienie tego parametru na 1 to bezpieczna strategia ogólnego przeznaczenia, ale ukończenie aktualizacji zajmuje również trochę czasu (liczba węzłów × `minReadySeconds`). Zwiększenie wartości `maxUnavailable` spowoduje, że wdrożenie będzie szybsze, ale zwiększy również „promień rażenia” w przypadku nieudanej aktualizacji. Charakterystyka aplikacji i środowisko klastra określają wartości prędkości względem bezpieczeństwa. Dobrym podejściem może być ustawienie wartości `maxUnavailable` na 1 i zwiększanie jej tylko wtedy, gdy użytkownicy lub administratorzy będą narzekać na szybkość wdrażania `DaemonSet`.

Po uruchomieniu ciągłej aktualizacji można użyć poleceń `kubectl rollout`, aby zobaczyć aktualny status wdrożenia obiektu `DaemonSet`. Uruchomienie na przykład polecenia `kubectl rollout status daemonSets` *`mój_daemon_set`* powoduje pokazanie bieżącego statusu aktualizacji obiektu `DaemonSet` o nazwie *`mój_daemon_set`*.

Usuwanie obiektu DaemonSet

Obiekt `DaemonSet` usuwa się dość łatwo przy użyciu polecenia `kubectl delete`. Upewnij się jedynie, że podałeś poprawną nazwę obiektu `DaemonSet`, który chcesz usunąć:

```
$ kubectl delete -f fluentd.yaml
```

Usunięcie obiektu `DaemonSet` spowoduje również usunięcie wszystkich zarządzanych przez niego kapsuł. Ustaw flagę `--cascade` na wartość `false`, aby zapewnić usunięcie samego obiektu `DaemonSet`, a nie kapsuł.

Podsumowanie

Obiekty `DaemonSet` zapewniają łatwą w użyciu abstrakcję służącą do uruchamiania zestawu kapsuł na każdym węźle w klastrze Kubernetes lub, jeśli wymaga tego dany przypadek, na podzbiorze węzłów na podstawie etykiet. `DaemonSet` ma własny kontroler i planistę, aby zagwarantować, że kluczowe usługi, takie jak agenty monitorowania, będą zawsze uruchomione i będą działać na właściwych węzłach w klastrze.

W przypadku niektórych aplikacji wystarczy rozplanować określoną liczbę replik. Nie obchodzi nas, gdzie działają, o ile mają wystarczające zasoby i dystrybucję umożliwiające niezawodne działanie. Istnieje jednak pewna klasa aplikacji, takich jak agenty i aplikacje monitorujące, które dla

poprawnego działania muszą być obecne na każdym komputerze w klastrze. Te obiekty `DaemonSet` nie są tak naprawdę tradycyjnymi aplikacjami obsługującymi, ale raczej dodają kolejne możliwości i funkcjonalności do samego klastra Kubernetes. Ponieważ `DaemonSet` jest aktywnym obiektem deklaratywnym zarządzanym przez kontroler, ułatwia deklarowanie zamiaru, że agent ma działać na każdym komputerze bez konieczności bezpośredniego umieszczania go na nim. Jest to szczególnie przydatne w kontekście autoskalowanego klastra Kubernetes, w którym węzły mogą być nieustannie dodawane i usuwane bez interwencji użytkownika. W takich przypadkach `DaemonSet` automatycznie dodaje odpowiednie agenty do każdego węzła, który jest wprowadzany do klastra przez autoskaler.

ROZDZIAŁ 12.
Obiekt Job

Do tej pory skupialiśmy się na długo działających procesach, takich jak bazy danych i aplikacje internetowe. Te typy obciążeń roboczych działają, dopóki nie zostaną zaktualizowane lub usługa nie przestanie być potrzebna. Chociaż długo działające procesy stanowią większość obciążeń roboczych uruchamianych w klastrze Kubernetes, często istnieje także potrzeba wykonywania krótkotrwałych zadań jednorazowych. Do obsługi tego typu zadań przeznaczony jest obiekt `Job`.

`Job` tworzy kapsuły, które działają aż do pomyślnego zakończenia (to znaczy wyjścia z kodem 0). Zwykła kapsuła będzie natomiast ciągle restartowana bez względu na jej kod wyjścia. Obiekty `Job` są przydatne dla czynności, które chcesz wykonać tylko raz, takie jak migracja bazy danych lub zadania wsadowe. Gdyby na przykład zadanie migracji bazy danych było uruchamiane z wykorzystaniem regularnych kapsuł, zadanie działałoby w pętli i baza danych byłaby ponownie zapełniana po każdym wyjściu.

W tym rozdziale omówimy najbardziej popularne wzorce `Job`, jakie oferuje Kubernetes. Będziemy również wykorzystywać te wzorce w rzeczywistych scenariuszach.

Obiekt Job

Obiekt `Job` odpowiada za tworzenie kapsuł zdefiniowanych w szablonie w specyfikacji tego obiektu i zarządzanie nimi. Te kapsuły zwykle działają aż do pomyślnego zakończenia zadania. Obiekt `Job` koordynuje uruchamianie równolegle wielu kapsuł.

Jeśli kapsuła ulegnie awarii przed pomyślnym zakończeniem zadania, kontroler `Job` utworzy nową kapsułę na podstawie szablonu kapsuły ze specyfikacji obiektu `Job`. Biorąc pod uwagę to, że kapsuły muszą być rozplanowywane, istnieje ryzyko, że zadanie nie zostanie wykonane, jeśli planista nie znajdzie wymaganych zasobów. Ponadto ze względu na charakter systemów rozproszonych podczas niektórych scenariuszy awarii istnieje pewne małe prawdopodobieństwo, że dla konkretnego zadania zostaną utworzone duplikaty kapsuły.

Wzorce obiektu Job

Obiekty `Job` są przeznaczone do zarządzania obciążeniami typu wsadowego, w których elementy pracy są przetwarzane przez jedną kapsułę lub większą liczbę kapsuł. Domyślnie każdy obiekt

`Job` uruchamia pojedynczą kapsułę, która działa do momentu pomyślnego zakończenia zadania. Ten wzorzec jest definiowany przez dwa podstawowe atrybuty obiektu `Job`, a mianowicie liczbę ukończeń zadania (`completions`) i liczbę kapsuł uruchamianych równolegle (`parallelism`). W przypadku wzorca „działanie do ukończenia zadania" atrybuty `completions` i `parallelism` są ustawiane na 1. Tabela 12.1 przedstawia wzorce obiektu `Job` oparte na kombinacji atrybutów `completions` i `parallelism` dla konfiguracji zadania.

Tabela 12.1. Wzorce obiektu Job

Typ	Przypadek użycia	Zachowanie	completions	parallelism
Zadanie jednorazowe	Migracje baz danych	Pojedyncza kapsuła działa aż do pomyślnego wykonania zadania	1	1
Określona liczba równoległych zadań do ukończenia	Wiele kapsuł przetwarza równolegle zestaw elementów pracy	Co najmniej jedna kapsuła uruchamiana co najmniej raz, dopóki nie zostanie osiągnięta ustalona liczba ukończonych zadań	1+	1+
Kolejka robocza: równoległe zadania	Wiele kapsuł przetwarza elementy ze scentralizowanej kolejki roboczej	Co najmniej jedna kapsuła działająca do pomyślnego zakończenia	1	2+

Zadania jednorazowe

Zadania jednorazowe zapewniają sposób na uruchomienie pojedynczej kapsuły, która działa aż do pomyślnego zakończenia zadania. Choć może się to wydawać łatwe, uruchomienie takiego zadania wymaga nieco pracy. Najpierw należy utworzyć kapsułę i przesłać ją do interfejsu API Kubernetes. Odbywa się to przy użyciu szablonu kapsuły zdefiniowanego w konfiguracji obiektu `Job`. Gdy zadanie zostanie uruchomione, obsługująca je kapsuła musi być monitorowana pod kątem pomyślnego zakończenia zadania. Wykonanie zadania może się nie powieść z wielu powodów, takich jak błąd aplikacji, nieprzechwycony wyjątek podczas wykonywania lub awaria węzła przed ukończeniem zadania. We wszystkich przypadkach kontroler `Job` jest odpowiedzialny za odtwarzanie kapsuły aż do pomyślnego zakończenia.

Istnieje wiele sposobów tworzenia jednorazowych zadań w Kubernetes. Najłatwiej jest użyć narzędzia wiersza poleceń `kubectl`:

```
$ kubectl run -i oneshot \
 --image=gcr.io/kuar-demo/kuard-amd64:blue \
 --restart=OnFailure \
--command /kuard \
 -- --keygen-enable \
    --keygen-exit-on-complete \
    --keygen-num-to-gen 10

...
(ID 0) Workload starting
(ID 0 1/10) Item done: SHA256:nAsUsG54XoKRkJwyN+OShkUPKew3mwq7OCc
(ID 0 2/10) Item done: SHA256:HVKX1ANns6SgF/er1lyo+ZCdnB8geFGt0/8
(ID 0 3/10) Item done: SHA256:irjCLRov3mTT0POJfsvUyhKRQ1TdGR8H1jg
(ID 0 4/10) Item done: SHA256:nbQAIVY/yrhmEGk3Ui2sAHuxb/o6mYOOqRk
(ID 0 5/10) Item done: SHA256:CCpBoXNlXOMQvR2v38yqimXGAa/w2Tym+aI
```

```
(ID 0 6/10) Item done: SHA256:wEY2TTIDz4ATjcr1iimxavCzZzNjRmbOQp8
(ID 0 7/10) Item done: SHA256:t3JSrCt7sQweBgqG5CrbMoBulwk4lfDWiTI
(ID 0 8/10) Item done: SHA256:E84/Vze7KKyjCh9OZhO2MkXJGoty9PhaCec
(ID 0 9/10) Item done: SHA256:UOmYex79qqbI1MhcIfG4hDnGKonlsij2k3s
(ID 0 10/10) Item done: SHA256:WCR8wIGOFag84Bsa8f/9QHuKqF+OmEnCADY
(ID 0) Workload exiting
```

Należy tu zwrócić uwagę na kilka rzeczy:

- Użycie opcji `-i` z `kubectl` wskazuje, że jest to polecenie interaktywne. `kubectl` poczeka, aż zadanie zostanie uruchomione, a następnie wyświetli zapis dziennika z pierwszej (i w tym przypadku jedynej) kapsuły w zadaniu.
- Opcja `--restart=OnFailure` wskazuje narzędziu `kubectl`, że ma utworzyć obiekt `Job`.
- Wszystkie opcje poprzedzone podwójnym myślnikiem (`--`) są argumentami wiersza poleceń dla obrazu kontenera. Instruuje to nasz serwer testowy (`kuard`), żeby wygenerować 10 kluczy SSH o 4096-bitowej mocy szyfrowania, a następnie zakończyć działanie.
- Twoje dane wyjściowe mogą się trochę różnić od przedstawionych. Przy opcji `-i` narzędzie `kubectl` często pomija kilka początkowych linii danych wyjściowych.

Po zakończeniu zadania obiekt `Job` i powiązana kapsuła pozostają dostępne. Jest tak dlatego, żeby można było sprawdzić dane wyjściowe dziennika. Zwróć uwagę, że ten obiekt `Job` nie pojawi się w danych wyjściowych polecenia `kubectl get jobs`, dopóki nie przekażesz flagi `-a`. Bez tej flagi `kubectl` ukrywa ukończone zadania. Zanim przejdziemy dalej, usuńmy to zadanie:

```
$ kubectl delete jobs oneshot
```

Inną opcją tworzenia jednorazowego zadania jest użycie pliku konfiguracyjnego, tak jak pokazano na listingu 12.1.

Listing 12.1. job-oneshot.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: oneshot
spec:
  template:
    spec:
      containers:
      - name: kuard
        image: gcr.io/kuar-demo/kuard-amd64:blue
        imagePullPolicy: Always
        command:
          - "/kuard"
        args:
        - "--keygen-enable"
        - "--keygen-exit-on-complete"
        - "--keygen-num-to-gen=10"
      restartPolicy: OnFailure
```

Prześlij zadanie za pomocą polecenia `kubectl apply`:

```
$ kubectl apply -f job-oneshot.yaml
job.batch created
```

Następnie użyj polecenia `describe`, aby uzyskać informacje o tym jednorazowym zadaniu:

```
$ kubectl describe jobs oneshot

Name: oneshot
Namespace:      default
Selector:       controller-uid=a2ed65c4-cfda-43c8-bb4a-707c4ed29143
Labels:         controller-uid=a2ed65c4-cfda-43c8-bb4a-707c4ed29143
                job-name=oneshot
Annotations:    <none>
Parallelism:    1
Completions:    1
Start Time:     Wed, 02 Jun 2021 21:23:23 -0700
Completed At:   Wed, 02 Jun 2021 21:23:51 -0700
Pods Statuses:  0 Running / 1 Succeeded / 0 Failed
Pod Template:
  Labels:  controller-uid=a2ed65c4-cfda-43c8-bb4a-707c4ed2914
           job-name=oneshot
Events:
  ... Reason              Message
  ... ------              -------
  ... SuccessfulCreate    Created pod: oneshot-4kfdt
```

Wyniki wykonania zadania możesz sprawdzić, przeglądając dzienniki utworzonej kapsuły:

```
$ kubectl logs oneshot-4kfdt

...
Serving on :8080
(ID 0) Workload starting
(ID 0 1/10) Item done: SHA256:+r6b4W81DbEjxMcD3LHjU+EIGnLEzbpxITKn8IqhkPI
(ID 0 2/10) Item done: SHA256:mzHewajaY1KA8VluSLOnNMk9fDE5zdn7vvBS5Ne8AxM
(ID 0 3/10) Item done: SHA256:TRtEQHfflJmwkqnNyGgQm/IvXNykSBIg8cO3hOg3onE
(ID 0 4/10) Item done: SHA256:tSwPYH/J347il/mgqTxRRdeZcOazEtgZlA8A3/HWbro
(ID 0 5/10) Item done: SHA256:IP8XtguJ6GbWwLHqjKecVfdS96B17nnO21I/TNc1j9k
(ID 0 6/10) Item done: SHA256:ZfNxdQvuST/6ZzEVkyxdRG98p73c/5TM99SEbPeRWfc
(ID 0 7/10) Item done: SHA256:tH+CNl/IUl/HUuKdMsq2XEmDQ8oAvmhMO6Iwj8ZEOjO
(ID 0 8/10) Item done: SHA256:3GfsUaALVEHQcGNLBOu4Qd1zqqqJ8j738i5r+I5XwVI
(ID 0 9/10) Item done: SHA256:5wV4L/xEiHSJXwLUT2fHfOSCKM2g3XH3sVtNbgskCXw
(ID 0 10/10) Item done: SHA256:bPqqOonwSbjzLqe9ZuVRmZkz+DBjaNTZ9HwmQhbdWLI
(ID 0) Workload exiting
```

Gratulacje, Twoje zadanie zostało pomyślnie wykonane!

Być może zauważyłeś, że podczas tworzenia obiektu `Job` nie określiliśmy żadnych etykiet. Podobnie jak w przypadku innych kontrolerów (takich jak `DaemonSet`, `ReplicaSet`, `Deployment` itp.), które korzystają z etykiet do identyfikacji zestawu kapsuł, jeśli kapsuła zostanie ponownie użyta w różnych obiektach, może się zdarzyć nieoczekiwane zachowanie.

Ponieważ obiekty `Job` mają określony początek i koniec, użytkownicy często tworzą wiele tych obiektów. To sprawia, że wybieranie unikatowych etykiet staje się trudniejsze i kluczowe. Z tego powodu obiekt `Job` automatycznie wybiera unikatową etykietę i używa jej do identyfikacji tworzonych przez siebie kapsuł. W zaawansowanych scenariuszach (takich jak zamiana uruchomionego obiektu `Job` bez usuwania zarządzanych przez niego kapsuł) użytkownicy mogą decydować się na wyłączenie tego automatycznego zachowania i ręcznie określać etykiety i selektory.

Właśnie widzieliśmy, jak obiekt `Job` może pomyślnie zakończyć wykonywanie zadania. Ale co się stanie, jeśli coś pójdzie nie tak? Spróbujmy sprowokować taką sytuację i zobaczmy, co będzie.

Zmodyfikujmy argumenty serwera `kuard` w naszym pliku konfiguracyjnym, aby uległ awarii z niezerowym kodem wyjścia po wygenerowaniu trzech kluczy, tak jak pokazano na listingu 12.2.

Listing 12.2. job-oneshot-failure1.yaml

```
...
spec:
  template:
    spec:
      containers:
        ...
        args:
        - "--keygen-enable"
        - "--keygen-exit-on-complete"
        - "--keygen-exit-code=1"
        - "--keygen-num-to-gen=3"
...
```

Teraz uruchom polecenie `kubectl -f job-oneshot-failure1.yaml`. Pozwól mu podziałać przez chwilę, a następnie spójrz na status kapsuły:

```
$ kubectl get pod -a -l job-name=oneshot

NAME             READY  STATUS             RESTARTS    AGE
oneshot-3ddk0    0/1    CrashLoopBackOff   4           3m
```

Widzimy, że ta sama kapsuła uruchomiła się ponownie cztery razy. Kubernetes ma ustawiony dla tej kapsuły status `CrashLoopBackOff`. Nierzadko w jakimś miejscu przydarza się błąd, który powoduje, że program ulega awarii zaraz po uruchomieniu. W takim przypadku Kubernetes jeszcze trochę poczeka przed ponownym uruchomieniem kapsuły, aby uniknąć pochłaniania zasobów w węźle przez pętle awarii. To wszystko jest obsługiwane lokalnie dla węzła przez `kubelet` bez angażowania obiektu `Job`.

Usuń to zadanie (`kubectl delete jobs oneshot`) i spróbujmy czegoś innego. Zmodyfikuj ponownie plik konfiguracyjny i zmień wartość parametru `restartPolicy` z `OnFailure` na `Never`. Uruchom to zadanie za pomocą polecenia `kubectl apply -f jobs-oneshot-failure2.yaml`.

Jeśli pozwolimy, żeby to zadanie chwilę podziałało, a potem spojrzymy na powiązane kapsuły, znajdziemy coś interesującego:

```
$ kubectl get pod -l job-name=oneshot -a

NAME             READY      STATUS    RESTARTS    AGE
oneshot-0wm49    0/1        Error     0           1m
oneshot-6h9s2    0/1        Error     0           39s
oneshot-hkzw0    1/1        Running   0           6s
oneshot-k5swz    0/1        Error     0           28s
oneshot-m1rdw    0/1        Error     0           19s
oneshot-x157b    0/1        Error     0           57s
```

Widzimy tutaj, że jest wiele kapsuł, które przestały działać w wyniku błędu. Przez ustawienie `restartPolicy: Never` instruujemy `kubelet`, aby nie restartował kapsuły w przypadku awarii i po

prostu zadeklarował, że kapsuła przestała działać. Zostaje to odnotowane przez obiekt `Job`, który tworzy zamienną kapsułę. Jeżeli nie będziesz ostrożny, utworzysz w ten sposób dużo „śmieci" w klastrze. Dlatego sugerujemy używanie `restartPolicy: OnFailure`, aby kapsuły, które uległy awarii, były restartowane. Wyczyść to zadanie za pomocą polecenia `kubectl delete jobs oneshot`.

Jak dotąd widzieliśmy awarię programu, który kończył działanie z niezerowym kodem wyjścia. Jednak wątki robocze mogą ulegać awarii również na inne sposoby. W szczególności mogą utknąć i nie wykonywać żadnych postępów. Aby obsłużyć ten przypadek, można z obiektami `Job` używać sond żywotności. Jeśli sonda żywotności ustali, że kapsuła jest martwa, zostanie ona ponownie uruchomiona lub zastąpiona.

Równoległość

Generowanie kluczy może być powolne. Uruchommy jednocześnie kilka wątków roboczych, aby przyspieszyć generowanie kluczy. Użyjemy kombinacji parametrów `completions` i `parallelism`. Naszym celem jest wygenerowanie 100 kluczy poprzez 10 uruchomień serwera `kuard`, który za każdym razem będzie generował 10 kluczy. Nie chcemy jednak zawalić pracą naszego klastra, ograniczymy się więc tylko do pięciu kapsuł naraz.

Przekłada się to na ustawienie `completions` na wartość `10` i `parallelism` na `5`. Ta konfiguracja została pokazana na listingu 12.3.

Listing 12.3. job-parallel.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: parallel
  labels:
    chapter: jobs
spec:
  parallelism: 5
  completions: 10
  template:
    metadata:
      labels:
        chapter: jobs
    spec:
      containers:
      - name: kuard
        image: gcr.io/kuar-demo/kuard-amd64:blue
        imagePullPolicy: Always
        command:
        - "/kuard"
        args:
        - "--keygen-enable"
        - "--keygen-exit-on-complete"
        - "--keygen-num-to-gen=10"
      restartPolicy: OnFailure
```

Uruchom to:

```
$ kubectl apply -f job-parallel.yaml
job.batch/parallel created
```

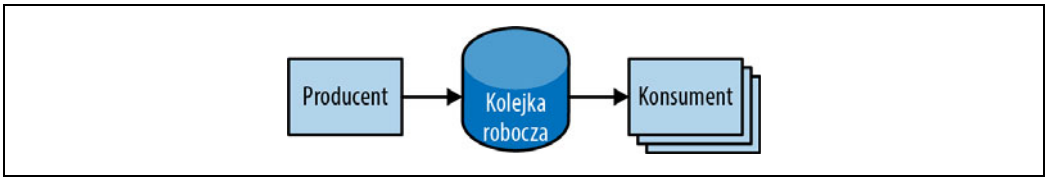
Teraz obserwuj, jak kapsuły są uruchamiane, wykonują swoje zadania i kończą działanie. Nowe kapsuły są tworzone do momentu, aż wszystkie 10 kapsuł nie zakończy swoich zadań. Używamy flagi `-watch`, aby polecenie `kubectl` pozostało uruchomione i na bieżąco wyświetlane były zmiany:

```
$ kubectl get pods -w
NAME            READY    STATUS             RESTARTS    AGE
parallel-55tlv  1/1      Running            0           5s
parallel-5s7s9  1/1      Running            0           5s
parallel-jp7bj  1/1      Running            0           5s
parallel-lssmn  1/1      Running            0           5s
parallel-qxcxp  1/1      Running            0           5s
NAME            READY    STATUS             RESTARTS    AGE
parallel-jp7bj  0/1      Completed          0           26s
parallel-tzp9n  0/1      Pending            0           0s
parallel-tzp9n  0/1      Pending            0           0s
parallel-tzp9n  0/1      ContainerCreating  0           1s
parallel-tzp9n  1/1      Running            0           1s
parallel-tzp9n  0/1      Completed          0           48s
parallel-x1kmr  0/1      Pending            0           0s
...
```

Poprzeglądaj zakończone zadania i sprawdź ich dzienniki, aby zobaczyć cyfrowe odciski wygenerowanych kluczy. Wykonaj czyszczenie, usuwając ukończony obiekt `Job` za pomocą polecenia `kubectl delete job parallel`.

Kolejki robocze

Typowym przykładem użycia dla obiektów `Job` jest przetwarzanie pracy z kolejki roboczej. W tym scenariuszu niektóre zadania powodują utworzenie kilku elementów pracy i publikowanie ich w kolejce roboczej. Do przetworzenia poszczególnych elementów pracy uruchamiane są robocze obiekty `Job`, dopóki kolejka robocza nie będzie pusta (zobacz rysunek 12.1).



Rysunek 12.1. Równoległe zadania

Uruchamianie kolejki roboczej

Zaczynamy od uruchomienia scentralizowanej usługi kolejki roboczej. `kuard` ma wbudowany prosty system kolejki roboczej oparty na pamięci. Uruchomimy instancję serwera `kuard`, która będzie działać jako koordynator całej pracy do wykonania.

Utwórz prosty obiekt `ReplicaSet` do zarządzania demonem singletonowej kolejki roboczej. Używamy obiektu `ReplicaSet`, aby być pewnym, że w przypadku awarii maszyny utworzona zostanie nowa kapsuła, tak jak pokazano na listingu 12.4.

Listing 12.4. rs-queue.yaml

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  labels:
    app: work-queue
    component: queue
    chapter: jobs
  name: queue
spec:
  replicas: 1
  selector:
    matchLabels:
      app: work-queue
      component: queue
      chapter: jobs
  template:
    metadata:
      labels:
        app: work-queue
        component: queue
        chapter: jobs
    spec:
      containers:
      - name: queue
        image: "gcr.io/kuar-demo/kuard-amd64:blue"
        imagePullPolicy: Always
```

Uruchom kolejkę roboczą za pomocą następującego polecenia:

```
$ kubectl apply -f rs-queue.yaml
replicaset.apps/queue created
```

W tym momencie demon kolejki roboczej powinien być uruchomiony. Użyjmy przekierowania portów, aby się z nim połączyć. Pozostaw to polecenie uruchomione w oknie terminala:

```
$ kubectl port-forward rs/queue 8080:8080
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
```

Możesz otworzyć w przeglądarce stronę *http://localhost:8080* i zobaczyć interfejs serwera `kuard`. Przejdź do zakładki *MemQ Server*, aby obserwować, co się dzieje.

Gdy mamy uruchomiony serwer kolejki roboczej, powinniśmy udostępnić go za pomocą usługi. Ułatwi to producentom i konsumentom znalezienie kolejki roboczej za pośrednictwem DNS, co pokazano na listingu 12.5.

Listing 12.5. service-queue.yaml

```
apiVersion: v1
kind: Service
metadata:
  labels:
    app: work-queue
    component: queue
    chapter: jobs
  name: queue
spec:
```

```
  ports:
  - port: 8080
    protocol: TCP
    targetPort: 8080
  selector:
    app: work-queue
    component: queue
```

Utwórz usługę kolejki za pomocą polecenia `kubectl`:

```
$ kubectl apply -f service-queue.yaml
service "queue" created
```

Ładowanie do kolejki

Jesteśmy teraz gotowi, aby umieścić w kolejce kilka elementów pracy. Dla uproszczenia użyjemy `curl` do sterowania API dla serwera kolejki roboczej i wstawimy kilka elementów pracy. `curl` będzie komunikować się z kolejką roboczą przez `kubectl port-forward`, co ustawiliśmy wcześniej, tak jak pokazano na listingu 12.6.

Listing 12.6. load-queue.sh

```
# Tworzy kolejkę roboczą o nazwie 'keygen'
curl -X PUT localhost:8080/memq/server/queues/keygen

# Tworzy 100 elementów pracy i ładuje je do kolejki
for i in work-item-{0..99}; do
  curl -X POST localhost:8080/memq/server/queues/keygen/enqueue \
    -d "$i"
done
```

Gdy uruchomisz te polecenia, powinieneś zobaczyć w oknie terminala wyświetlonych 100 obiektów JSON z unikatowym identyfikatorem komunikatu dla każdego elementu pracy. Status kolejki możesz potwierdzić, zerkając do zakładki *MemQ Server* w interfejsie użytkownika, lub możesz bezpośrednio zapytać API kolejki roboczej:

```
$ curl 127.0.0.1:8080/memq/server/stats
{
    "kind": "stats",
    "queues": [
        {
            "depth": 100,
            "dequeued": 0,
            "drained": 0,
            "enqueued": 100,
            "name": "keygen"
        }
    ]
}
```

Teraz jesteśmy gotowi uruchomić zadanie, aby rozpocząć przetwarzanie kolejki roboczej trwające dopóty, dopóki nie będzie pusta.

Tworzenie zadania konsumenta

Tutaj zaczyna robić się ciekawie! Serwer `kuard` może również działać w trybie konsumenta. Konfigurujemy go do pobierania elementów pracy z kolejki roboczej, tworzenia klucza, a następnie zakończenia działania, gdy kolejka będzie pusta, tak jak pokazano na listingu 12.7.

Listing 12.7. job-consumer.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  labels:
    app: message-queue
    component: consumer
    chapter: jobs
  name: consumers
spec:
  parallelism: 5
  template:
    metadata:
      labels:
        app: message-queue
        component: consumer
        chapter: jobs
    spec:
      containers:
      - name: worker
        image: "gcr.io/kuar-demo/kuard-amd64:blue"
        imagePullPolicy: Always
        command:
        - "/kuard"
        args:
        - "--keygen-enable"
        - "--keygen-exit-on-complete"
        - "--keygen-memq-server=http://queue:8080/memq/server"
        - "--keygen-memq-queue=keygen"
      restartPolicy: OnFailure
```

Instruujemy obiekt `Job`, aby równolegle uruchomił pięć kapsuł. Ponieważ parametr `completions` jest nieustawiony, ustawiamy `Job` w trybie puli roboczej. Gdy pierwsza kapsuła zakończy działanie z zerowym kodem wyjścia, obiekt `Job` zacznie kończyć działanie i nie będzie uruchamiał nowych kapsuł. To oznacza, że żadna z kapsuł roboczych nie powinna kończyć działania, dopóki praca nie zostanie zakończona i wszystkie kapsuły nie przejdą do etapu końcowego.

Utwórz zadanie `consumers`:

```
$ kubectl apply -f job-consumers.yaml
job.batch/consumers created
```

Następnie możesz przejrzeć obsługujące je kapsuły:

```
$ kubectl get pods
NAME             READY    STATUS    RESTARTS    AGE
queue-43s87      1/1      Running   0           5m
consumers-6wjxc  1/1      Running   0           2m
consumers-7l5mh  1/1      Running   0           2m
consumers-hvz42  1/1      Running   0           2m
consumers-pc8hr  1/1      Running   0           2m
consumers-w2Occ  1/1      Running   0           2m
```

Zwróć uwagę, że równolegle działa pięć kapsuł. Te kapsuły będą działać do momentu, aż kolejka robocza będzie pusta. Możesz to obserwować w interfejsie użytkownika na serwerze kolejki roboczej. W miarę stopniowego opróżniania kolejki kapsuły konsumenckie będą czysto kończyć działanie, a zadanie `consumers` zostanie ostatecznie uznane za ukończone.

Czyszczenie

Za pomocą etykiet możemy uprzątnąć wszystkie elementy, które utworzyliśmy w tym podrozdziale:

```
$ kubectl delete rs,svc,job -l chapter=jobs
```

Obiekt CronJob

Czasami trzeba zaplanować wykonywanie zadania w określonych odstępach czasu. W Kubernetes służy do tego obiekt `CronJob`, który tworzy nowy obiekt `Job` co określony czas. Na listingu 12.8 znajduje się przykładowa deklaracja obiektu `CronJob`:

Listing 12.8. job-cronjob.yaml

```
apiVersion: batch/v1beta1
kind: CronJob
metadata:
  name: example-cron
spec:
  # Wykonywanie co pięć godzin
  schedule: "0 */5 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: batch-job
            image: my-batch-image
          restartPolicy: OnFailure
```

Zwróć uwagę na pole `spec.schedule`, które zawiera definicję przedziału czasu w standardowym formacie crona.

Możesz zapisać ten kod w pliku o nazwie *job-cronjob.yaml* i utworzyć obiekt `CronJob` za pomocą polecenia `kubectl create -f cron-job.yaml`. Aktualny stan zadania `CronJob` można sprawdzić przy użyciu polecenia `kubectl describe <zadanie-crona>`.

Podsumowanie

W pojedynczym klastrze Kubernetes może obsługiwać zarówno długo działające obciążenia robocze, takie jak aplikacje internetowe, jak i krótkotrwałe obciążenia robocze, takie jak zadania wsadowe. Abstrakcja `Job` pozwala modelować wzorce zadań wsadowych od prostych zadań jednorazowych po zadania równoległe, które przetwarzają wiele elementów aż do wyczerpania pracy.

Zadania to podstawowe obiekty niskopoziomowe, które mogą być używane bezpośrednio dla prostych obciążeń roboczych. Jednak Kubernetes jest w całości zbudowany w taki sposób, aby mógł być rozszerzany przez obiekty wyższego poziomu. Proste zadania nie są wyjątkiem; mogą być z łatwością wykorzystywane przez systemy orkiestracji wyższego poziomu do wykonywania bardziej złożonych zadań.

ROZDZIAŁ 13.

Obiekty ConfigMap i tajne dane

Dobrą praktyką jest tworzenie takich obrazów kontenerów, które będą zdatne do wielokrotnego użytku. Ten sam obraz powinien nadawać się do wykorzystania w środowisku rozwojowym, testowym i produkcyjnym. Jeszcze lepiej, jeśli ten sam obraz będzie miał na tyle ogólne zastosowanie, aby można go było wykorzystać w różnych aplikacjach i usługach. Testowanie i numerowanie wersji oprogramowania staje się bardziej ryzykowne i skomplikowane, jeżeli dla każdego nowego środowiska obrazy muszą być tworzone od nowa. Ale w jaki sposób możemy wyspecjalizować korzystanie z danego obrazu w czasie wykonywania?

Tutaj do gry wkraczają obiekty `ConfigMap` i tajne dane. Obiekty `ConfigMap` są używane do zapewnienia informacji konfiguracyjnych dla obciążeń roboczych. Mogą to być informacje szczegółowe (krótkie łańcuchy znaków) lub wartości złożone w postaci pliku. Tajne dane są podobne do obiektów `ConfigMap`, ale wiążą się z udostępnianiem wrażliwych informacji obciążeniom roboczym. Mogą być używane do poświadczeń lub certyfikatów TLS.

Obiekty ConfigMap

Obiekty `ConfigMap` można potraktować jako obiekty Kubernetes, które definiują niewielki system plików. Można również traktować je jako zestaw zmiennych, które mogą być używane podczas definiowania środowiska lub wiersza poleceń dla kontenerów. Najważniejsze jest to, że obiekt `ConfigMap` jest łączony z kapsułą tuż przed jej uruchomieniem. Oznacza to, że obraz kontenera i definicja samej kapsuły mogą być ponownie wykorzystywane w wielu obciążeniach roboczych dzięki zmianie jedynie używanego obiektu `ConfigMap`.

Tworzenie obiektów ConfigMap

Przystąpmy od razu do utworzenia obiektu `ConfigMap`. Podobnie jak w przypadku wielu obiektów w Kubernetes, obiekty `ConfigMap` można tworzyć w bezpośredni, imperatywny sposób lub na podstawie manifestu zapisanego na dysku. Zaczniemy od metody imperatywnej.

Po pierwsze, załóżmy, że mamy na dysku plik (o nazwie *my-config.txt*), który chcemy udostępnić danej kapsule, tak jak pokazano na listingu 13.1.

Listing 13.1. my-config.txt

```
# Jest to przykładowy plik konfiguracyjny, którego mógłbym użyć do skonfigurowania aplikacji
parameter1 = value1
parameter2 = value2
```

Następnie za pomocą tego pliku utwórzmy obiekt `ConfigMap`. Dodajmy również kilka prostych par klucz-wartość. Odwołujemy się do nich w wierszu poleceń jako do wartości literalnych:

```
$ kubectl create configmap my-config \
  --from-file=my-config.txt \
  --from-literal=extra-param=extra-value \
  --from-literal=another-param=another-value
```

Odpowiednik YAML dla właśnie utworzonego obiektu `ConfigMap` jest następujący:

```
$ kubectl get configmaps my-config -o yaml

apiVersion: v1
data:
  another-param: another-value
  extra-param: extra-value
  my-config.txt: |
    #Jest to przykładowy plik konfiguracyjny, którego mógłbym użyć  do skonfigurowania aplikacji
    parameter1 = value1
    parameter2 = value2
kind: ConfigMap
metadata:
  creationTimestamp: ...
  name: my-config
  namespace: default
  resourceVersion: "13556"
  selfLink: /api/v1/namespaces/default/configmaps/my-config
uid: 3641c553-f7de-11e6-98c9-06135271a273
```

Jak widać, `ConfigMap` to tak naprawdę kilka par klucz-wartość przechowywanych w obiekcie. Ciekawie robi się, gdy próbujemy *użyć* obiektu `ConfigMap`.

Używanie obiektów ConfigMap

Istnieją trzy główne sposoby korzystania z `ConfigMap`:

System plików

Można zamontować `ConfigMap` w kapsule. Dla każdego wpisu tworzony jest plik na podstawie nazwy klucza. Zawartość tego pliku jest ustawiana na wartość.

Zmienna środowiskowa

Obiekt `ConfigMap` może być używany do dynamicznego ustawiania wartości zmiennej środowiskowej.

Argument wiersza poleceń

Kubernetes obsługuje dynamiczne tworzenie wiersza poleceń dla kontenera na podstawie wartości `ConfigMap`.

Utwórzmy manifest dla serwera `kuard`, który zbierze to wszystko razem, tak jak pokazano na listingu 13.2.

Listing 13.2. kuard-config.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard-config
spec:
  containers:
    - name: test-container
      image: gcr.io/kuar-demo/kuard-amd64:blue
      imagePullPolicy: Always
      command:
        - "/kuard"
        - "$(EXTRA_PARAM)"
      env:
       # Przykładowa zmienna środowiskowa używana w kontenerze
        - name: ANOTHER_PARAM
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: another-param
        # Przykładowa zmienna środowiskowa przekazywana do polecenia uruchamiającego kontener
        - name: EXTRA_PARAM
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: extra-param
      volumeMounts:
        # Montowanie ConfigMap jako zbioru plików
        - name: config-volume
          mountPath: /config
  volumes:
    - name: config-volume
      configMap:
        name: my-config
  restartPolicy: Never
```

W przypadku metody systemu plików tworzymy nowy wolumin wewnątrz kapsuły i nadajemy mu nazwę `config-volume`. Następnie definiujemy ten wolumin jako wolumin `ConfigMap` i wskazujemy mu obiekt `ConfigMap` do zamontowania. Za pomocą `volumeMount` musimy określić, w którym miejscu w kontenerze `kuard` zostanie on zamontowany. W tym przypadku montujemy go w `/config`.

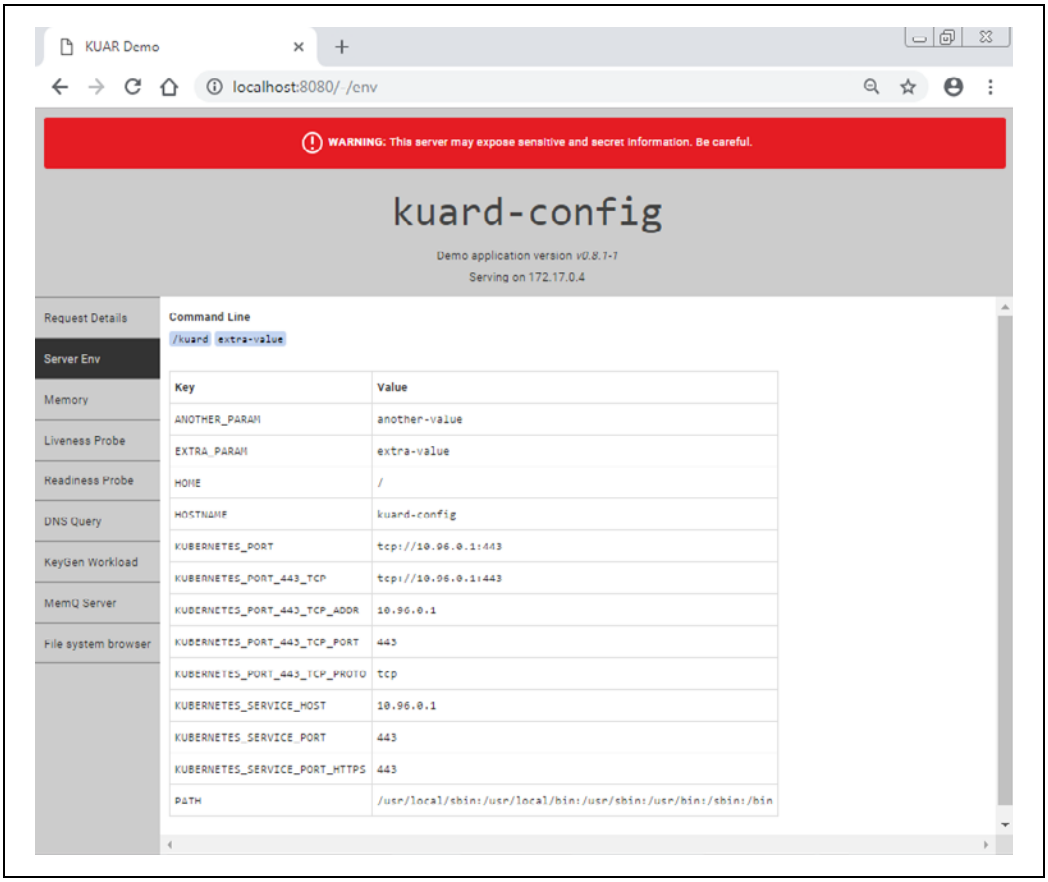
Zmienne środowiskowe są określane za pomocą specjalnej składowej `valueFrom`. Odwołuje się ona do obiektu `ConfigMap` i klucza danych, który ma być użyty w tym `ConfigMap`.

Argumenty wiersza poleceń bazują na zmiennych środowiskowych. Kubernetes wykona poprawne podstawienie za pomocą specjalnej składni `$(<nazwa_zmiennej_środowiskowej>)`.

Uruchom tę kapsułę i przekieruj porty, aby sprawdzić, jak aplikacja widzi świat:

```
$ kubectl apply -f kuard-config.yaml
$ kubectl port-forward kuard-config 8080
```

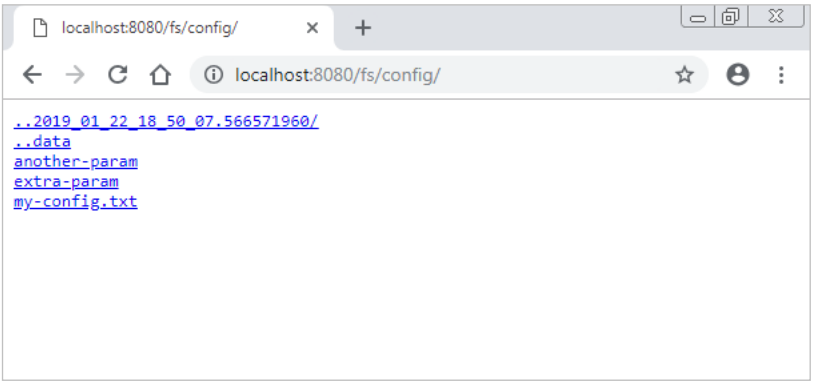
Teraz otwórz w przeglądarce stronę *http://localhost:8080*. Możemy zobaczyć, jak wstrzyknęliśmy wartości konfiguracyjne do programu na wszystkie trzy sposoby. Kliknij zakładkę *Server Env* po lewej stronie. Pokazany zostanie wiersz poleceń, za pomocą którego uruchomiona została aplikacja, wraz ze środowiskiem, tak jak widać na rysunku 13.1.



Rysunek 13.1. kuard pokazuje swoje środowisko

Widzimy tutaj, że dodaliśmy dwie zmienne środowiskowe (`ANOTHER_PARAM` i `EXTRA_PARAM`), których wartości zostały ustawione za pomocą obiektu `ConfigMap`. Ponadto dodaliśmy argument do wiersza poleceń serwera `kuard` na podstawie wartości `EXTRA_PARAM`.

Następnie kliknij zakładkę *File system browser* (zobacz rysunek 13.2). Pozwoli Ci to zbadać system plików tak, jak widzi go aplikacja. Powinieneś zobaczyć wpis o nazwie `/config`. To jest wolumin utworzony na podstawie naszego obiektu `ConfigMap`. Jeśli przejdziesz do niego, zobaczysz, że dla każdego wpisu w `ConfigMap` utworzony został plik. Zobaczysz również kilka ukrytych plików (poprzedzonych `..`), które są używane do podmiany wartości na nowe, gdy obiekt `ConfigMap` jest aktualizowany.



Rysunek 13.2. Katalog /config widziany poprzez kuard

Tajne dane

Chociaż obiekty `ConfigMap` świetnie nadają się dla większości danych konfiguracyjnych, istnieją pewne dane, które są wyjątkowo wrażliwe. Mogą to być hasła, tokeny zabezpieczające lub innego rodzaju prywatne klucze. Tego typu dane zbiorczo nazywamy „tajnymi danymi" (ang. *secrets*). Kubernetes oferuje natywne wsparcie przechowywania i przetwarzania tych danych z zachowaniem odpowiedniej ostrożności.

Tajne dane umożliwiają tworzenie obrazów kontenerów bez załączania w nich wrażliwych danych. Dzięki temu kontenery zachowują przenośność między różnymi środowiskami. Tajne dane są udostępniane kapsułom poprzez bezpośrednią deklarację w manifestach kapsuł oraz interfejs API Kubernetes. API tajnych danych Kubernetes zapewnia ukierunkowany na aplikację mechanizm służący do udostępniania aplikacjom wrażliwych informacji konfiguracyjnych w taki sposób, że możliwy jest łatwy audyt, i z wykorzystaniem natywnych podstawowych elementów izolacji systemu operacyjnego.

W pozostałej części tego podrozdziału dowiesz się, jak tworzyć tajne dane Kubernetes i zarządzać nimi, oraz poznasz najlepsze praktyki udostępniania tajnych danych kapsułom, które ich wymagają.

Domyślnie tajne dane Kubernetes są przechowywane w postaci zwykłego tekstu w magazynie *etcd* klastra. W niektórych przypadkach taki poziom bezpieczeństwa może być niewystarczający. Przykładowo każdy, kto ma uprawnienia administracyjne w danym klastrze, będzie mógł odczytać w nim wszystkie tajne dane.
W nowych wersjach Kubernetes dodano możliwość szyfrowania tajnych danych za pomocą dostarczonego przez użytkownika klucza, który jest zintegrowany z chmurowym magazynem kluczy. Ponadto większość chmurowych magazynów kluczy jest zintegrowana z woluminami Kubernetes Secrets Store CSI Driver (*https://oreil.ly/DRHt6*), co pozwala na całkowite pominięcie tajnych danych Kubernetes i korzystanie wyłącznie z magazynu kluczy dostawcy chmury. Wszystkie te opcje powinny wystarczyć do zbudowania profilu bezpieczeństwa spełniającego każde wymagania.

Tworzenie tajnych danych

Tajne dane są tworzone przy użyciu interfejsu API Kubernetes lub narzędzia wiersza poleceń `kubectl`. Tajne dane przechowują co najmniej jeden element danych jako zbiór par klucz-wartość.

W tym punkcie utworzymy tajne dane do przechowywania klucza TLS i certyfikaty dla aplikacji `kuard` spełniające powyższe wymagania dotyczące przechowywania.

Obraz kontenera `kuard` nie zawiera certyfikatu TLS ani klucza. Dzięki temu kontener `kuard` jest przenośny między różnymi środowiskami i może być dystrybuowany za pośrednictwem publicznych repozytoriów Dockera.

Pierwszym krokiem tworzenia tajnych danych jest uzyskanie surowych danych, które chcemy przechowywać. Klucz TLS i certyfikat dla aplikacji `kuard` można pobrać, uruchamiając poniższe polecenia:

```
$ curl -o kuard.crt https://storage.googleapis.com/kuar-demo/kuard.crt
$ curl -o kuard.key https://storage.googleapis.com/kuar-demo/kuard.key
```

Te certyfikaty są ogólnodostępne, więc nie zapewniają żadnej ochrony. Nie należy ich używać nigdzie poza przykładami z tej książki.

Po zapisaniu lokalnie plików *kuard.crt* i *kuard.key* jesteśmy gotowi, aby utworzyć tajne dane. Utwórz tajne dane o nazwie `kuard-tls` za pomocą polecenia `create secret`:

```
$ kubectl create secret generic kuard-tls \
  --from-file=kuard.crt \
  --from-file=kuard.key
```

Utworzone zostały tajne dane `kuard-tls` z dwoma elementami danych. Aby uzyskać szczegóły, uruchom następujące polecenie:

```
$ kubectl describe secrets kuard-tls

Name:         kuard-tls
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:         Opaque

Data
====
kuard.crt:    1050 bytes
kuard.key:    1679 bytes
```

Mamy już gotowe tajne dane `kuard-tls`, możemy więc ich używać z poziomu kapsuły za pomocą woluminu tajnych danych.

Korzystanie z tajnych danych

Tajne dane mogą być wykorzystywane za pomocą interfejsu REST API Kubernetes przez aplikacje, które wiedzą, jak bezpośrednio wywołać ten interfejs API. Naszym celem jest jednak utrzymanie przenośności aplikacji. Powinny one nie tylko działać dobrze w Kubernetes, ale być także uruchamiane w niezmodyfikowanej postaci na innych platformach.

Zamiast uzyskiwać dostęp do tajnych danych za pośrednictwem serwera API, możemy użyć **woluminu tajnych danych** (ang. *sercets volume*). Woluminy tajnych danych są zarządzane przez `kubelet` i tworzone w momencie tworzenia kapsuł. Tajne dane są przechowywane na woluminach *tmpfs* (znanych także jako dyski RAM) i jako takie nie są zapisywane na dysku w węzłach.

Każdy element tajnych danych jest przechowywany w osobnym pliku w docelowym punkcie montowania określonym w uchwycie woluminu. Tajne dane `kuard-tls` zawierają dwa elementy danych: *kuard.crt* i *kuard.key*. Po zamontowaniu woluminu tajnych danych `kuard-tls` w ścieżce `/tls` otrzymamy następujące pliki:

```
/tls/kuard.crt
/tls/kuard.key
```

Manifest kapsuły z listingu 13.3 pokazuje, jak zadeklarować wolumin tajnych danych, który udostępnia tajne dane `kuard-tls` kontenerowi `kuard` w `/tls`.

Listing 13.3. kuard-secret.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard-tls
spec:
  containers:
    - name: kuard-tls
      image: gcr.io/kuar-demo/kuard-amd64:blue
      imagePullPolicy: Always
      volumeMounts:
      - name: tls-certs
        mountPath: "/tls"
        readOnly: true
  volumes:
    - name: tls-certs
      secret:
        secretName: kuard-tls
```

Utwórz kapsułę `kuard-tls` za pomocą polecenia `kubectl` i obserwuj dane wyjściowe dziennika z uruchomionej kapsuły:

```
$ kubectl apply -f kuard-secret.yaml
```

Połącz się z kapsułą, uruchamiając następujące polecenie:

```
$ kubectl port-forward kuard-tls 8443:8443
```

Teraz otwórz w przeglądarce stronę *https://localhost:8443*. Powinieneś zobaczyć ostrzeżenia dotyczące nieprawidłowych certyfikatów, ponieważ jest to certyfikat samopodpisany dla witryny *kuard.example.com*. Jeśli pominiesz te ostrzeżenia, powinieneś zobaczyć serwer `kuard` hostowany przez HTTPS. Użyj zakładki *File system browser*, aby znaleźć te certyfikaty na dysku w katalogu */tls*.

Prywatne rejestry kontenera

Specjalnym przypadkiem użycia dla tajnych danych jest przechowywanie poświadczeń dostępu do prywatnych rejestrów kontenera. Kubernetes obsługuje obrazy przechowywane w prywatnych rejestrach, ale dostęp do tych obrazów wymaga poświadczeń. Prywatne obrazy mogą być przechowywane w jednym lub w kilku prywatnych rejestrach. Stwarza to wyzwanie związane z zarządzaniem poświadczeniami dla każdego prywatnego rejestru na każdym możliwym węźle w klastrze.

Tajne dane pobierania obrazów (ang. *image pull secrets*) wykorzystują API tajnych danych do automatyzacji dystrybucji poświadczeń prywatnych rejestrów. Są one przechowywane tak jak zwykłe tajne dane, ale korzysta się z nich za pomocą pola `spec.imagePullSecrets` specyfikacji kapsuły.

Użyj polecenia `kubectl create secret docker-registry`, aby utworzyć ten specjalny rodzaj tajnych danych:

```
$ kubectl create secret docker-registry my-image-pull-secret \
  --docker-username=<nazwa_użytkownika> \
  --docker-password=<hasło> \
  --docker-email=<adres_email>
```

Włącz dostęp do prywatnego repozytorium, odwołując się do tajnych danych pobierania obrazów w pliku manifestu kapsuły, tak jak pokazano na listingu 13.4.

Listing 13.4. kuard-secret-ips.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard-tls
spec:
  containers:
    - name: kuard-tls
      image: gcr.io/kuar-demo/kuard-amd64:blue
      imagePullPolicy: Always
      volumeMounts:
      - name: tls-certs
        mountPath: "/tls"
        readOnly: true
  imagePullSecrets:
    - name: my-image-pull-secret
  volumes:
    - name: tls-certs
      secret:
        secretName: kuard-tls
```

Jeśli często pobierasz dane z tego samego rejestru, możesz dodać tajne dane do domyślnego konta usługi powiązanego z każdą kapsułą, aby nie musieć ich definiować w każdej tworzonej kapsule.

Ograniczenia dotyczące nazewnictwa

Nazwy kluczy dla elementów danych wewnątrz tajnych danych lub obiektów `ConfigMap` są definiowane w taki sposób, aby mapować się na poprawne nazwy zmiennych środowiskowych. Mogą zaczynać się od kropki, po której następuje litera lub cyfra. Kolejnymi znakami mogą być kropki,

kreski i podkreślniki. Kropki nie mogą być powtórzone, poza tym kropki, podkreślniki lub kreski nie mogą ze sobą sąsiadować. Mówiąc bardziej formalnie, oznacza to, że nazwy kluczy muszą być zgodne z wyrażeniem regularnym `^[.[?[a-zAZ0-9[([.[?[a-zA-Z0-9[+[-_a-zA-Z0-9[?)*$`. W tabeli 13.1 podano kilka przykładów poprawnych i niepoprawnych nazw dla kluczy obiektów `ConfigMap` lub tajnych danych.

Tabela 13.1. Przykłady nazw kluczy obiektów ConfigMap i tajnych danych

Prawidłowa nazwa klucza	Nieprawidłowa nazwa klucza
`.auth_token`	`Token..properties`
`Key.pem`	`auth file.json`
`config_file`	`_password.txt`

Przy wyborze nazwy klucza należy wziąć pod uwagę, że te klucze mogą być udostępniane kapsułom za pomocą montowania woluminów. Wybieraj nazwy, które będą miały jakiś sens, gdy zostaną określone w wierszu poleceń lub w pliku konfiguracyjnym. Przy konfigurowaniu aplikacji do uzyskiwania dostępu do tajnych danych klucz TLS przechowywany jako `key.pem` jest bardziej zrozumiały niż taki o nazwie `tls-key`.

Wartości danych obiektów `ConfigMap` to zwykły tekst UTF-8 określony bezpośrednio w manifeście. Wartości tajnych danych zawierają dowolne dane zakodowane przy użyciu base64. Zastosowanie kodowania base64 umożliwia przechowywanie danych binarnych. Jednak przez to trudniej jest zarządzać tajnymi danymi przechowywanymi w plikach YAML, ponieważ wartości zakodowane w base64 muszą być umieszczane w YAML. Pamiętaj, że maksymalny rozmiar obiektu `ConfigMap` lub tajnych danych wynosi 1 MB.

Zarządzanie obiektami ConfigMap i tajnymi danymi

Tajne dane i obiekty `ConfigMap` są zarządzane za pośrednictwem interfejsu API Kubernetes. Tymi obiektami można manipulować za pomocą zwykłych poleceń `create`, `delete`, `get` i `describe`.

Wyświetlanie obiektów

Aby wyświetlić wszystkie obiekty tajnych danych w bieżącej przestrzeni nazw, możesz użyć polecenia `kubectl get secrets`:

```
$ kubectl get secrets

NAME                  TYPE                                  DATA        AGE
default-token-f5jq2   kubernetes.io/service-account-token   3           1h
kuard-tls             Opaque                                2           20m
```

W podobny sposób możesz wyświetlić wszystkie obiekty `ConfigMap` w przestrzeni nazw:

```
$ kubectl get configmaps

NAME         DATA        AGE
my-config    3           1m
```

Aby uzyskać więcej szczegółowych informacji na temat pojedynczego obiektu, możesz użyć polecenia `kubectl describe`:

```
$ kubectl describe configmap my-config

Name:           my-config
Namespace:      default
Labels:         <none>
Annotations:    <none>

Data
====
another-param:  13 bytes
extra-param:    11 bytes
my-config.txt:  116 bytes
```

Możesz także zobaczyć surowe dane (w tym wartości przechowywane w tajnych danych!), używając na przykład polecenia `kubectl get configmap my-config -o yaml` lub `kubectl get secret kuard-tls -o yaml`.

Tworzenie obiektów

Najłatwiejszym sposobem tworzenia tajnych danych lub `ConfigMap` jest użycie odpowiednio polecenia `kubectl create secret generic` lub `kubectl create configmap`. Istnieje wiele sposobów określania elementów danych, które mają znaleźć się w tajnych danych lub w `ConfigMap`. Można je łączyć w pojedynczym poleceniu:

`--from-file=<nazwa_pliku>`
: Ładowanie z pliku za pomocą klucza tajnych danych, który jest taki sam jak nazwa pliku.

`--from-file=<klucz>=<nazwa_pliku>`
: Ładowanie z pliku za pomocą bezpośrednio określonego klucza tajnych danych.

`--from-file=<katalog>`
: Ładowanie wszystkich plików z określonego katalogu, gdzie nazwa pliku jest akceptowalną nazwą klucza.

`--from-literal=<klucz>=<wartość>`
: Bezpośrednie użycie określonej pary klucz-wartość.

Aktualizowanie obiektów

Można dokonywać aktualizacji obiektów `ConfigMap` lub tajnych danych, która będzie odzwierciedlana w działających aplikacjach. Nie ma potrzeby restartowania aplikacji, jeśli jest skonfigurowana do ponownego odczytywania wartości konfiguracyjnych. Poniżej przedstawiliśmy trzy sposoby aktualizacji obiektów `ConfigMap` lub tajnych danych.

Aktualizacja z pliku

Jeśli masz manifest dla `ConfigMap` lub tajnych danych, możesz po prostu edytować go bezpośrednio i zastąpić nową wersją za pomocą polecenia `kubectl replace -f <nazwa_pliku>`. Możesz także użyć polecenia `kubectl apply -f <nazwa_pliku>`, jeśli wcześniej utworzyłeś dany zasób z wykorzystaniem `kubectl apply`.

Ze względu na sposób kodowania plików danych w tych obiektach aktualizowanie konfiguracji może być nieco kłopotliwe, ponieważ nie ma polecenia `kubectl` obsługującego ładowanie danych z pliku zewnętrznego. Dane muszą być przechowywane bezpośrednio w manifeście YAML.

Najczęstszym przypadkiem użycia jest sytuacja, gdy obiekt `ConfigMap` jest zdefiniowany jako część katalogu lub lista zasobów i wszystko jest tworzone oraz aktualizowane razem. Często manifesty są przesyłane do systemu kontroli wersji.

Przesyłanie plików YAML tajnych danych do systemu kontroli wersji zwykle nie jest dobrym pomysłem. Zbyt łatwo jest przesłać te pliki do jakiejś publicznej lokalizacji i spowodować wyciek tajnych danych.

Odtworzenie i aktualizacja

Jeśli dane wejściowe dla obiektów `ConfigMap` lub tajnych danych przechowujesz jako oddzielne pliki na dysku (w przeciwieństwie do osadzonych bezpośrednio w YAML), możesz użyć `kubectl` do odtworzenia manifestu, a następnie do zaktualizowania obiektu.

To będzie wyglądało mniej więcej tak:

```
$ kubectl create secret generic kuard-tls \
  --from-file=kuard.crt --from-file=kuard.key \
  --dry-run -o yaml | kubectl replace -f -
```

Ten wiersz poleceń tworzy najpierw nowe tajne dane o tej samej nazwie co nasze istniejące tajne dane. Gdybyśmy zatrzymali się tutaj, serwer API Kubernetes zwróciłby błąd, narzekając, że próbujemy utworzyć tajne dane, które już istnieją. Zamiast tego instruujemy `kubectl`, aby nie wysyłał danych do serwera, tylko zrzucił do `stdout` plik YAML, który *normalnie zostałby wysłany* na serwer API. Następnie przekazujemy go za pomocą potoku do `kubectl replace` i używamy `-f`, by nakazać odczyt ze `stdin`. W ten sposób możemy aktualizować tajne dane z plików znajdujących się na dysku bez konieczności ręcznego kodowania danych w base64.

Edytowanie bieżącej wersji

Ostatnim sposobem aktualizacji obiektu `ConfigMap` jest użycie polecenia `kubectl edit` w celu wyświetlenia wersji `ConfigMap` w edytorze, dzięki czemu będzie go można zmodyfikować (mógłbyś również zrobić tak z tajnymi danymi, ale utknąłbyś na zarządzaniu na własną rękę kodowaniem wartości w base64):

```
$ kubectl edit configmap my-config
```

Powinieneś zobaczyć w swoim edytorze definicję `ConfigMap`. Wprowadź żądane zmiany, a następnie je zapisz i zamknij edytor. Nowa wersja obiektu zostanie przesłana do serwera interfejsu API Kubernetes.

Aktualizacje na żywo

Po zaktualizowaniu obiektu `ConfigMap` lub tajnych danych za pomocą interfejsu API zostaną one automatycznie przekazane do wszystkich korzystających z nich woluminów. Może to potrwać kilka sekund, ale lista plików i zawartość plików, te widziane przez `kuard`, zostaną zaktualizowane o te nowe wartości. Korzystając z funkcji aktualizacji na żywo, możesz aktualizować konfiguracje aplikacji bez ponownego uruchamiania tych aplikacji.

Obecnie nie ma żadnego wbudowanego sposobu sygnalizowania aplikacji, że wdrażana jest nowa wersja `ConfigMap`. To do zadań aplikacji (lub jakiegoś skryptu pomocniczego) należy wyszukanie plików konfiguracyjnych, aby je zmienić i przeładować.

Świetnym sposobem na interaktywną zabawę z dynamicznie aktualizującymi się tajnymi danymi i obiektami `ConfigMap` jest użycie przeglądarki plików na serwerze `kuard` (do którego dostęp uzyskujemy poprzez przekierowanie portów za pomocą polecenia `kubectl port-forward`).

Podsumowanie

Wykorzystanie obiektów `ConfigMap` i tajnych danych to świetny sposób na zapewnienie dynamicznej konfiguracji w aplikacji. Umożliwiają one jednorazowe utworzenie obrazu kontenera (i definicji kapsuły) i wielokrotne wykorzystywanie go w różnych kontekstach. Może to oznaczać używanie dokładnie tego samego obrazu przy przechodzeniu ze środowiska rozwojowego do testowego, a następnie do produkcyjnego. Może również obejmować użycie jednego obrazu w wielu zespołach i usługach. Oddzielenie konfiguracji od kodu aplikacji sprawi, że Twoje aplikacje będą bardziej niezawodne i będą oferować większe możliwości wielokrotnego użytku.

ROZDZIAŁ 14.

Model kontroli dostępu oparty na rolach w Kubernetes

Obecnie praktycznie wszystkie istniejące klastry Kubernetes korzystają z modelu kontroli dostępu opartego na rolach (RBAC), więc z pewnością już nie raz miałeś z nim styczność. Możliwe, że dostęp do własnego klastra udało Ci się uzyskać dopiero po użyciu magicznego zaklęcia dodającego obiekt `RoleBinding` wiążący użytkownika z rolą. Mimo to, nawet jeśli spotkałeś się już z techniką RBAC, niekoniecznie musisz wiedzieć, jak ona działa w Kubernetes, do czego służy oraz jak się nią poprawnie posługiwać.

Model kontroli dostępu oparty na rolach pozwala ograniczyć możliwość dostępu i wykonywania operacji na API Kubernetes tylko do wybranej grupy użytkowników. RBAC odgrywa podwójną rolę. Z jednej strony zwiększa bezpieczeństwo dostępu do klastra Kubernetes, w którym jest wdrażana aplikacja, a z drugiej (co ważniejsze) zapobiega takim wypadkom, jak spowodowanie przestoju przez nieumyślne usunięcie klastra produkcyjnego zamiast testowego przez osobę, która pomyliła przestrzenie nazw.

Choć RBAC całkiem skutecznie ogranicza dostęp do API Kubernetes, należy pamiętać, że każdy, kto ma możliwość wykonywania kodu bez ograniczeń w klastrze Kubernetes, może uzyskać uprawnienia `root` w całym tym klastrze. Istnieją techniki utrudniające przeprowadzanie takich ataków, a prawidłowa konfiguracja RBAC jest ich elementem. Sama funkcja RBAC wystarczy, aby ochronić się przed wrogim wykorzystywaniem modelu dzierżawy. Aby zapewnić bezpieczeństwo w środowisku z wieloma dzierżawcami, należy odizolować od siebie kapsuły działające w klastrze. Zazwyczaj w tym celu wykorzystuje się kontener nadzorowany przez hipernadzorcę lub jakiś rodzaj piaskownicy.

Zanim zagłębimy się w szczegóły modelu RBAC w Kubernetes, omówimy tę technikę z szerszej perspektywy oraz przyjrzymy się ogólnie koncepcjom uwierzytelniania i autoryzacji.

Każde żądanie wysyłane do Kubernetes musi zostać **uwierzytelnione**. Uwierzytelnianie ma na celu potwierdzenie tożsamości nadawcy żądania. Realizujący je mechanizm może być bardzo prosty i może informować tylko, że dane żądanie nie zostało uwierzytelnione, lub może mieć postać zintegrowanego rozwiązania (na przykład Azure Active Directory) pozwalającego ustalić tożsamość w zewnętrznym systemie. Co ciekawe, Kubernetes nie ma wbudowanego magazynu tożsamości, tylko koncentruje się na integracji innych rozwiązań.

Po pomyślnym uwierzytelnieniu użytkowników następuje faza autoryzacji, w trakcie której sprawdzane jest, czy mają oni uprawnienia do wykonywania określonych czynności. Autoryzacja łączy tożsamość użytkownika, zasób (ostatecznie ścieżkę HTTP) oraz czasownik lub czynność, którą użytkownik próbuje wykonać. Jeśli ma on autoryzację do wykonywania danej czynności na danym zasobie, to realizowanie żądania może być kontynuowane. W przeciwnym przypadku zostaje zwrócony błąd HTTP 403. W kolejnych podrozdziałach proces ten jest objaśniony bardziej szczegółowo.

Kontrola dostępu oparta na rolach

Aby prawidłowo zarządzać dostępem w Kubernetes, należy zrozumieć relacje występujące między tożsamością, rolami i powiązaniami ról w zakresie kontroli dostępu do zasobów. Początkowo może być trudno pojąć abstrakcyjne łączące się koncepcje składające się na technikę RBAC, ale każdy, kto się z tym upora, będzie mógł bezpiecznie i wygodnie zarządzać dostępem do klastrów.

Tożsamość w Kubernetes

Każde żądanie przychodzące do Kubernetes ma jakąś tożsamość. Nawet takie żądania, z którymi nie została powiązana żadna tożsamość, należą do grupy `system:unauthenticated`. Kubernetes rozróżnia tożsamości użytkowników i usług. Konta usług są tworzone i zarządzane przez sam Kubernetes i są generalnie powiązane z komponentami działającymi w klastrze. Natomiast konta użytkowników to wszystkie pozostałe konta powiązane z rzeczywistymi użytkownikami klastra. Często zawierają mechanizmy automatyzacji, takie jak ciągłe dostarczanie jako usługa działająca poza klastrem.

Do uwierzytelniania dostawców w Kubernetes używany jest ogólny interfejs. Każdy dostawca dostarcza nazwę użytkownika i opcjonalnie zbiór grup, do których ten użytkownik należy. Kubernetes obsługuje kilku dostawców usług uwierzytelniania. Te usługi to między innymi:

- podstawowe uwierzytelnianie HTTP (wycofywane z użytku);
- certyfikaty klientów x509;
- statyczne pliki tokenów na hoście;
- uwierzytelnianie chmurowe przez takich dostawców, jak Azure Active Directory, AWS Identity czy Access Management (IAM);
- elementy uwierzytelniania webhook.

Większość zarządzanych instalacji Kubernetes konfiguruje uwierzytelnianie za użytkownika, więc jeśli chcesz wdrożyć własny system uwierzytelniania, musisz odpowiednio skonfigurować flagi na serwerze API Kubernetes.

Różne aplikacje w klastrze zawsze powinny mieć różne tożsamości. Na przykład inną tożsamość powinny mieć produkcyjne aplikacje frontendowe, a inną backendowe, dodatkowo wszystkie tożsamości produkcyjne powinny różnić się od testowych. Ponadto także różne klastry powinny mieć różne tożsamości. Wszystkie one powinny być tożsamościami komputerów, które nie są udostępniane użytkownikom. Aby osiągnąć ten cel, można użyć Kubernetes Service Accounts lub skorzystać z dostawcy tożsamości kapsuł udostępnianego przez używany system tożsamości.

Na przykład Azure Active Directory zapewnia dostawcę tożsamości typu open source dla kapsuł (*https://oreil.ly/YLymu*). Podobnie wygląda to w przypadku innych popularnych systemów tożsamości.

Role i powiązania ról

Sprawdzanie tożsamości stanowi pierwszy stopień do autoryzacji w Kubernetes. Kiedy system zweryfikuje tożsamość żądania, przechodzi do sprawdzenia, czy dany użytkownik ma prawo wykonywać określoną operację. W tym celu wykorzystuje ogólną koncepcję ról i powiązań ról.

Rola to zbiór abstrakcyjnych uprawnień. Na przykład rola `appdev` może reprezentować opcję tworzenia kapsuł i usług. **Powiązanie roli** to jej przypisanie do jednej lub większej liczby tożsamości. Na przykład powiązanie roli `appdev` do tożsamości użytkownika `alice` oznacza, że użytkownik ten może tworzyć kapsuły i usługi.

Role i powiązania ról w Kubernetes

W Kubernetes są dwie pary spokrewnionych zasobów, które reprezentują role i powiązania ról. Jedna dotyczy tylko przestrzeni nazw (`Role` i `RoleBinding`), a druga ma zasięg na cały klaster (`ClusterRole` i `ClusterRoleBinding`).

Najpierw przyjrzymy się `Role` i `RoleBinding`. Zasoby `Role` są objęte przez przestrzeń nazw i reprezentują uprawnienia w tej przestrzeni. Nie można ich mieszać z zasobami nieobjętymi przez przestrzeń nazw (na przykład `CustomResourceDefinitions`), a powiązanie `RoleBinding` z rolą zapewnia autoryzację tylko w tej przestrzeni nazw Kubernetes, która zawiera zarówno rolę, jak i jej powiązanie.

Poniżej znajduje się przykład prostej roli, która daje tożsamości możliwość tworzenia i modyfikowania kapsuł oraz usług:

```
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  namespace: default
  name: pod-and-services
rules:
- apiGroups: [""]
  resources: ["pods", "services"]
  verbs: ["create", "delete", "get", "list", "patch", "update", "watch"]
```

Aby powiązać tę rolę z użytkownikiem `alice`, musimy utworzyć przedstawione poniżej powiązanie roli. Dodatkowo wiąże ono grupę `mydevs` z tą samą rolą:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  namespace: default
  name: pods-and-services
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: alice
```

```
  - apiGroup: rbac.authorization.k8s.io
    kind: Group
    name: mydevs
  roleRef:
    apiGroup: rbac.authorization.k8s.io
    kind: Role
    name: pod-and-services
```

Oczywiście czasami potrzebna jest rola mająca zastosowanie do całego klastra lub trzeba ograniczyć dostęp do zasobów na jego poziomie. W takim przypadku należy skorzystać z obiektów `ClusterRole` i `ClusterRoleBinding`. Są praktycznie identyczne ze swoimi odpowiednikami o zasięgu przestrzeni nazw, tylko mają zasięg klastrowy.

Czasowniki określające możliwości w Kubernetes

Definicje ról określają zarówno zasoby (na przykład kapsuły), jak i czasowniki informujące, co można z tymi zasobami robić. Czasowniki z grubsza odpowiadają nazwom metod HTTP. W tabeli 14.1 znajduje się lista najczęściej używanych czasowników w funkcji RBAC w Kubernetes.

Tabela 14.1. Często używane czasowniki w funkcji RBAC w Kubernetes

Czasownik	Metoda HTTP	Opis
`create`	`POST`	Tworzy nowy zasób
`delete`	`DELETE`	Usuwa zasób
`get`	`GET`	Pobiera zasób
`list`	`GET`	Pobiera listę dostępnych zasobów
`patch`	`PATCH`	Modyfikuje istniejący zasób za pomocą częściowej zmiany
`update`	`PUT`	Modyfikuje istniejący zasób za pomocą kompletnego obiektu
`watch`	`GET`	Monitoruje napływ aktualizacji do zasobu
`proxy`	`GET`	Łączy z zasobem przez strumieniowy serwer proxy WebSocket

Role wbudowane

Projektowanie własnych ról może być skomplikowane i czasochłonne. Ponadto w Kubernetes jest duża liczba wbudowanych ról klastrowych dla powszechnie znanych tożsamości systemowych (na przykład planista), które wymagają znanego zbioru możliwości. W konsekwencji Kubernetes dysponuje dużą liczbą wbudowanych ról klastrowych. Poniższe polecenie zwraca ich listę:

```
$ kubectl get clusterroles
```

Większość z nich jest przeznaczona dla narzędzi systemowych, ale cztery zaprojektowano do ogólnego użytku:

- Rola `cluster-admin` daje pełny dostęp do całego klastra.
- Rola `admin` daje pełny dostęp do całej przestrzeni nazw.
- Rola `edit` umożliwia użytkownikowi modyfikowanie zasobów w przestrzeni nazw.
- Rola `view` daje możliwość odczytu w przestrzeni nazw.

Większość klastrów ma wiele powiązań `ClusterRole`, które można wyświetlić za pomocą polecenia `kubectl get clusterrolebindings`.

Automatyczne uzgadnianie wbudowanych ról

W chwili uruchomienia serwera API Kubernetes w jego kodzie automatycznie instalowana jest pewna liczba domyślnych ról klastrowych. To oznacza, że jeśli zmodyfikujemy którąś z nich, zmiany te nie będą trwałe. Po ponownym uruchomieniu serwera (na przykład w związku z aktualizacją) wprowadzone zmiany zostaną z powrotem zastąpione domyślnymi ustawieniami.

Aby temu zapobiec, należy przypisać wbudowanemu obiektowi `ClusterRole` adnotację `rbac.authorization.kubernetes.io/autoupdate` o wartości `false` przed wprowadzeniem jakichkolwiek innych modyfikacji. Będzie to stanowić sygnał dla serwera API, aby nie nadpisywać zmodyfikowanego obiektu `ClusterRole`.

Domyślnie serwer API Kubernetes instaluje rolę klastrową dającą użytkownikom `system:unauthenticated` dostęp do punktu końcowego odnajdywania API serwera API. To zły pomysł w przypadku klastrów mających styczność z wrogimi środowiskami (na przykład z publicznym internetem), zwłaszcza że wykryto przynajmniej jedną poważną lukę bezpieczeństwa w tej funkcji. Jeśli udostępniasz usługę Kubernetes w internecie lub w innym wrogim środowisku, ustaw flagę `--anonymous-auth=false` na serwerze API.

Techniki zarządzania funkcją RBAC

Zarządzanie funkcją RBAC w klastrze może być skomplikowane i nużące. Najgorsze jednak jest to, że błąd konfiguracji może doprowadzić do poważnych problemów z bezpieczeństwem. Na szczęście istnieją narzędzia i techniki, które ułatwiają zarządzanie funkcją RBAC.

Testowanie autoryzacji za pomocą narzędzia can-i

Pierwszym narzędziem, które warto poznać, jest polecenie `kubectl can-i` pozwalające sprawdzić, czy dany użytkownik może wykonać określoną czynność. Za jego pomocą można skontrolować poprawność ustawień konfiguracji klastra, można też polecić użytkownikom skorzystanie z niego przy wypełnianiu raportów o błędach związanych z dostępem.

W najprostszej formie polecenie `can-i` przyjmuje jako parametry czasownik i zasób.

Poniższe polecenie na przykład informuje, czy bieżący użytkownik `kubectl` może tworzyć kapsuły:

```
$ kubectl auth can-i create pods
```

Dodatkowo przy użyciu flagi `--subresource` można testować podzasoby, na przykład dzienniki czy przekierowania portów:

```
$ kubectl auth can-i get pods --subresource=logs
```

Zarządzanie funkcją RBAC w kontroli źródła

Tak jak wszystkie zasoby w Kubernetes, zasoby RBAC są reprezentowane w formacie YAML. Zważywszy, że są to formaty tekstowe, dobrym pomysłem wydaje się wprowadzenie ich do systemu kontroli wersji, który umożliwia ścisłą kontrolę i cofanie wprowadzonych zmian.

Polecenie `kubectl reconcile` działa podobnie do `kubectl apply` i uzgadnia zbiór ról i ich powiązań z bieżącym stanem klastra. Możesz wykonać poniższe polecenie:

```
$ kubectl auth reconcile -f some-rbac-config.yaml
```

Jeśli chcesz sprawdzić, jakie zmiany zostaną wprowadzone, przed ich wprowadzeniem, możesz dodać flagę `--dry-run`, aby tylko wydrukować listę zmian, ale ich nie zatwierdzać.

Tematy zaawansowane

Znając podstawy modelu kontroli dostępu opartego na rolach, z łatwością można kontrolować dostęp do klastrów Kubernetes. Gdy jednak liczba użytkowników i ról jest znaczna, powstają nowe wyzwania, z którymi można sobie poradzić z pomocą dodatkowych zaawansowanych funkcji RBAC.

Agregowanie ról klastrowych

Czasami trzeba zdefiniować rolę będącą kombinacją innych ról. Jedną z możliwości jest skopiowanie wszystkich reguł z jednej roli do innej, ale to skomplikowane rozwiązanie, w przypadku którego łatwo popełnić błędy, ponieważ zmiany w jednej roli nie są automatycznie odzwierciedlane w drugiej. Dlatego lepiej jest skorzystać z **reguły agregacji** funkcji RBAC Kubernetes, która umożliwia tworzenie jednej roli z kilku innych ról. Ta nowa rola ma wszystkie możliwości składowych, a wszelkie zmiany w nich wprowadzone są automatycznie odzwierciedlane także w niej.

Role `ClusterRole`, jak wszystkie inne agregacje i grupy w Kubernetes, tworzy się za pomocą selektorów etykiet. W tym przypadku pole `aggregationRule` obiektu `ClusterRole` zawiera pole `clusterRoleSelector`, które z kolei jest selektorem etykiety. Wszystkie obiekty `ClusterRole` pasujące do tego selektora są dynamicznie agregowane w tablicy `rules` agregacyjnego obiektu `ClusterRole`.

Najlepszym sposobem zarządzania obiektami `ClusterRole` jest utworzenie kilku szczegółowych ról klastrowych, a z nich szerszych ról klastrowych. Tak są zdefiniowane wbudowane role klastrowe. Na przykład wbudowana rola `edit` wygląda tak:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: edit
  ...
aggregationRule:
  clusterRoleSelectors:
  - matchLabels:
    rbac.authorization.k8s.io/aggregate-to-edit: "true"
...
```

To oznacza, że rola `edit` jest agregacją wszystkich obiektów `ClusterRole` mających etykietę `rbac.authorization.k8s.io/aggregate-to-edit` ustawioną na `true`.

Wykorzystywanie grup do wiązań

Kiedy trzeba zarządzać dużą liczbą osób należących do różnych organizacji o podobnych prawach dostępu do klastra, najlepszym rozwiązaniem jest zarządzanie rolami definiującymi dostęp do klastra za pomocą grup zamiast pojedynczego dodawania wiązania do konkretnych tożsamości. Gdy przypiszemy grupę do obiektu `ClusterRole` lub `Role` ograniczonego przestrzenią nazw, każdy członek tej grupy będzie miał dostęp do zasobów i czasowników zdefiniowanych przez tę rolę. Aby zatem dać wybranej osobie dostęp do roli grupy, należy osobę tę dodać do tej grupy.

Zarządzanie dostępem na szerszą skalę za pomocą grup jest preferowaną strategią z kilku powodów. Pierwszy z nich jest taki, że w każdej dużej organizacji dostęp do klastra uzyskuje się raczej w ramach zespołu, do którego ktoś należy, a nie za pomocą indywidualnej tożsamości. Na przykład osoba należąca do zespołu zajmującego się frontendem potrzebuje zarówno możliwości oglądania, jak i edytowania zasobów związanych z frontendami, natomiast wystarczą jej uprawnienia przeglądania backendu. Udzielenie uprawnień całej grupie wyraźnie ukazuje związek między danym zespołem i jego możliwościami. Gdy role przypisuje się poszczególnym osobom, trudniej jest określić odpowiedni (czyli minimalny) zestaw uprawnień dla każdego zespołu, zwłaszcza gdy jakaś osoba należy do kilku różnych zespołów.

Inną zaletą wiązania ról z grupami zamiast z indywidualnymi osobami jest prostota i spójność tego rozwiązania. Kiedy ktoś dołącza do zespołu lub go opuszcza, to wystarczy go dodać do odpowiedniej grupy lub z niej usunąć za pomocą jednej operacji. Gdyby zamiast tego trzeba było pozbawić wybraną tożsamość wielu powiązań ról, to istniałoby ryzyko omyłkowego usunięcia zbyt wielu powiązań lub pominięcia niektórych, w wyniku czego dana osoba miałaby zbyt szerokie lub zbyt wąskie uprawnienia. Ponadto istnienie jednej grupy powiązań ról eliminuje mnóstwo pracy związanej z zapewnianiem spójnego zestawu uprawnień wszystkim członkom zespołu.

Wielu dostawców chmury obsługuje integrację ze swoją platformą zarządzania tożsamościami i dostępem, aby z użytkownikami i grupami z tych platform można było pracować w połączeniu z funkcją Kubernetes RBAC.

Dodatkowo w wielu systemach można włączyć dostęp na czas (JIT), czyli dodawanie osób do grup w odpowiedzi na jakieś zdarzenie (na przykład wezwanie w środku nocy) na pewien czas, a nie na stałe. Umożliwia to kontrolowanie, kto miał dostęp o określonej godzinie, a także wyklucza ryzyko, że tożsamość przejęta przez hakera będzie w stanie uzyskać dostęp do infrastruktury produkcyjnej.

W końcu często te same grupy wykorzystuje się do zarządzania dostępem do innych zasobów, od obiektów przez dokumenty po loginy do urządzeń. Dlatego grupowe kontrolowanie dostępu do Kubernetes radykalnie upraszcza zarządzanie.

Aby powiązać grupę z rolą klastrową, należy użyć rodzaju `Group` w temacie wiązania:

```
...
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: Group
  name: my-great-groups-name
...
```

W Kubernetes grupy tworzą dostawcy usług uwierzytelniania. W tym systemie nie ma ściśle zdefiniowanego pojęcia grupy, jedynie takie, że tożsamość może należeć do jednej grupy lub większej liczby grup, które mogą być powiązane z obiektem `Role` lub `ClusterRole`.

Podsumowanie

Kiedy klaster i zespół są niewielkie, wystarczy każdemu członkowi nadać takie same uprawnienia dostępu do tego klastra. Jednak w miarę jak zespoły rosną, a produkty ich pracy stają się coraz bardziej istotne dla organizacji, znaczenia nabiera sprawa ograniczania dostępu do określonych części klastra. W dobrze zaprojektowanym klastrze dostęp ma tylko ograniczona grupa ludzi, którzy dysponują uprawnieniami pozwalającymi na efektywne zarządzanie aplikacjami.

Znajomość implementacji funkcji RBAC w Kubernetes i zasad posługiwania się uprawnieniami w celu kontrolowania dostępu do klastra jest niezbędna zarówno dla programistów, jak i administratorów. Tak jak w przypadku budowy infrastruktury testowej, im szybciej prawidłowo skonfiguruje się funkcje RBAC, tym lepiej. O wiele łatwiej jest zacząć pracę od solidnych podstaw, niż dorabiać je później. Mamy nadzieję, że informacje zawarte w tym rozdziale są wystarczające do tego, by każdy mógł skonfigurować podstawowe funkcje ochrony dostępu do klastra na bazie ról.

ROZDZIAŁ 15.
Siatki usług

Pojęcie **siatki usług** (ang. *service mesh*), podobnie jak pojęcie kontenera, stało się synonimem tworzenia aplikacji chmurowych. Jednak — znów analogicznie do kontenera — siatka usług jest szerokim pojęciem obejmującym różne projekty open source, a także produkty komercyjne. Warto znać ogólną rolę siatki usług w architekturze chmurowej. W tym rozdziale dowiesz się, czym jest siatka usług, jak jest implementowana w różnych projektach oprogramowania oraz (co najważniejsze), kiedy warto wybrać ją zamiast prostszej architektury do budowy aplikacji.

Na wielu abstrakcyjnych schematach architektury chmurowej siatka usług wydaje się niezbędna. To jednak nieprawda. Zanim zdecydujesz się na adaptację siatki usług, musisz rozważyć poziom złożoności operacji dodawania nowego komponentu (zazwyczaj dostarczanego przez inny podmiot) do swojej listy zależności. W wielu przypadkach prościej i bezpieczniej jest skorzystać z istniejących zasobów Kubernetes, jeśli spełniają wymogi Twojej aplikacji.

W poprzednich rozdziałach opisaliśmy inne podstawowe składniki sieciowe Kubernetes, takie jak usługi i Ingress. Skoro istnieją one w rdzeniu Kubernetes, dlaczego powstaje potrzeba dodawania kolejnych elementów (które zwiększają poziom złożoności) do warstwy sieciowej? Zasadniczo wszystko sprowadza się do wymagań aplikacji, która będzie z nich korzystać.

Warstwa sieciowa rdzenia Kubernetes „wie" o aplikacji tylko jako o celu. Zarówno zasoby usługowe, jak i Ingress mają selektory etykiet, które kierują ruch do odpowiednich kapsuł, ale poza tym te zasoby zapewniają niewiele więcej dodatkowych możliwości. Ingress, jako usługa równoważenia obciążenia HTTP, idzie o krok dalej, ale konieczność zdefiniowania wspólnego interfejsu API, który pasuje do szerokiego spektrum różnych istniejących implementacji, ogranicza jego możliwości. Jak naprawdę „chmurowy" interfejs API routingu HTTP może być zgodny z usługami równoważenia obciążenia i serwerami proxy obejmującymi zarówno zwykłe urządzenia sieciowe, jak i publiczne API chmurowe stworzone bez uwzględnienia kwestii związanych z tworzeniem aplikacji chmurowych?

Powstanie i rozwój interfejsów API siatek usług poza rdzeniem Kubernetes jest właśnie odpowiedzią na te pytania. API Ingress wprowadzają ruch HTTP(S) ze świata zewnętrznego do aplikacji chmurowych. W aplikacji chmurowej w Kubernetes, która nie musi być zgodna z istniejącą infrastrukturą, API siatki usług zapewniają dodatkowe funkcje w zakresie obsługi sieci. Co to za funkcje? Większość implementacji siatki usług obsługuje trzy obszary: szyfrowanie sieciowe i uwierzytelnianie, kształtowanie ruchu oraz obserwowalność. Poniżej opisujemy każdy z nich.

Szyfrowanie i uwierzytelnianie przy użyciu Mutual TLS

Szyfrowanie ruchu sieciowego między kapsułami ma kluczowe znaczenie dla bezpieczeństwa architektury mikrousług. Możliwość skorzystania z szyfrowania zapewnianego przez Mutual Transport Layer Security (mTLS) jest jednym z najczęstszych powodów, które decydują o wyborze siatki usług. Programista może wprawdzie samodzielnie zaimplementować tego rodzaju szyfrowanie, ale obsługa certyfikatów i szyfrowanie ruchu to trudne i skomplikowane dziedziny. Pozostawianie implementacji szyfrowania indywidualnym zespołom programistycznym sprawia, że programiści albo całkowicie zapominają dodać szyfrowanie, albo robią to nieporadnie. Źle zaimplementowane szyfrowanie może mieć negatywny wpływ na stabilność aplikacji, a nawet pozbawić ją realnej ochrony. Natomiast instalacja siatki usług w klastrze Kubernetes automatycznie zapewnia szyfrowanie ruchu sieciowego między wszystkimi kapsułami. Siatka usług wprowadza do każdej kapsuły dodatkowy kontener, który transparentnie przechwytuje cały ruch sieciowy. Ponadto mTLS dodaje tożsamość do szyfrowania dzięki wykorzystaniu certyfikatów, a to sprawia, że aplikacja „zna" tożsamość każdego klienta sieciowego.

Kształtowanie ruchu

Pierwsze koncepcje projektowe aplikacji mają zazwyczaj postać prostego diagramu zawierającego po jednej ramce dla każdej mikrousługi lub warstwy w systemie (na przykład usługa frontendowa, usługa preferencji użytkownika itd.). Po implementacji często okazuje się, że każda mikrousługa działająca w aplikacji występuje w kilku egzemplarzach. Na przykład kiedy zmieniamy wersję X swojej usługi na wersję Y, to na pewnym etapie tego procesu mamy dwie jednocześnie działające jej wersje. Ten etap ma charakter przejściowy, ale często trzeba utworzyć dłuższy eksperyment, który jest potrzebny przez pewien czas. Modelem powszechnie stosowanym w branży produkcji oprogramowania jest „podjadanie własnej karmy" (ang. *dog-fooding*), czyli wypróbowywanie nowej wersji programu wewnętrznie przed jej wprowadzeniem gdziekolwiek indziej. W modelu tym możesz uruchomić wersję Y swojej usługi na dzień lub na tydzień (albo na dłużej) dla wybranej grupy użytkowników przed wdrożeniem jej dla wszystkich.

Takie eksperymenty wymagają umiejętności **kształtowania ruchu** (ang. *traffic shaping*), czyli kierowania żądań do różnych implementacji usługi na podstawie ich cech. W tym przykładzie utworzylibyśmy eksperyment, w którym cały ruch pochodzący od użytkowników wewnętrznych firmy trafiałby do usługi Y, a cały ruch z reszty świata byłby kierowany do usługi X.

Eksperymenty są przydatne w wielu sytuacjach, także związanych z tworzeniem aplikacji. W ich ramach programista może wysłać ograniczony zbiór (zazwyczaj 1% lub mniej) prawdziwego ruchu do eksperymentalnego backendu lub wykonać testy A/B, w ramach których połowa użytkowników korzysta z jednej wersji, a połowa — z drugiej wersji. To pozwala na budowę modeli statystycznych pozwalających stwierdzić, które podejście jest efektywniejsze. Eksperymenty to niezwykle cenne narzędzie umożliwiające dopracowanie aplikacji pod wieloma względami, ale często trudno je zaimplementować, przez co są wykorzystywane rzadziej, niż mogłyby być.

Siatki usług zmieniają to, ponieważ eksperymenty są w nie wbudowane. Zamiast pisać kod implementujący eksperyment lub wdrażać całkiem nowy egzemplarz aplikacji w nowej infrastrukturze,

deklarujemy tylko parametry eksperymentu (10% ruchu do wersji Y, 90% ruchu do wersji X), a siatka implementuje go za nas. Choć jako programiści jesteśmy zaangażowani w definiowanie eksperymentu, implementacja jest transparentna i automatyczna, dzięki czemu wykonujemy więcej eksperymentów, a to z kolei pociąga za sobą zwiększenie niezawodności, zwinności oraz zasobu informacji.

Introspekcja

Jeśli jesteś podobny do większości programistów, to po napisaniu programu przystępujesz do jego debugowania, usuwając po kolei pojawiające się błędy. Czynność wyszukiwania błędów zajmuje dużą część czasu pracy programisty. Ich usuwanie jest jeszcze trudniejsze, kiedy aplikacja jest rozproszona między kilka mikrousług. Trudno jest złożyć jedno żądanie, gdy ma ono dotyczyć wielu kapsuł. Informacje potrzebne do debugowania trzeba poskładać z różnych źródeł, oczywiście przy założeniu, że w ogóle udało się je zgromadzić.

Automatyczna introspekcja to kolejna ważna funkcja siatek usług. Taka siatka bierze udział we wszystkich zdarzeniach komunikacyjnych między kapsułami, więc „wie", jaką drogą przebiegają żądania, i dysponuje informacjami niezbędnymi do odtworzenia ich w całości. Zamiast lawiny żądań do różnych mikrousług programista widzi jedno żądanie zbiorcze, które definiuje interakcję użytkownika z aplikacją. Ponadto siatkę usług implementuje się raz dla całego klastra. To znaczy, że konkretny trop żądania jest zawsze taki sam bez względu na to, który zespół opracował usługę. Dane monitoringu są spójne w odniesieniu do wszystkich usług połączonych przez obejmującą cały klaster siatkę usług.

Czy naprawdę potrzebujesz siatki usług

Opisane tutaj zalety siatek usług mogą zachęcić Cię do jak najszybszej instalacji ich w swoim klastrze. Zanim jednak to zrobisz, zastanów się, czy naprawdę tego potrzebujesz. Siatka usług to rozproszony system, który zwiększa poziom złożoności aplikacji i jest ściśle zintegrowany z rdzeniem komunikacyjnym mikrousług. Jeśli w siatce pojawi się błąd, to cała aplikacja przestanie działać. Zanim zaadaptujesz siatkę usług, musisz mieć pewność, że będziesz w stanie rozwiązywać problemy, gdy się pojawią. Ponadto musisz się przygotować na monitorowanie wersji oprogramowania siatki, aby zawsze pobierać najnowsze poprawki funkcjonalności i bezpieczeństwa i oczywiście wdrażać je w sposób niezakłócający pracy aplikacji. Ten dodatkowy narzut pracy sprawia, że w przypadku wielu małych aplikacji siatka usług jest niepotrzebnym naddatkiem.

Jeśli używasz Kubernetes jako usługi zarządzanej, która udostępnia także siatkę usług, to skorzystanie z niej jest znacznie łatwiejsze, ponieważ wiadomo, że dostawca usługi będzie zapewniał wsparcie, usuwał błędy oraz bezproblemowo wprowadzał nowe wersje. Jednak nawet w przypadku siatki usług udostępnianej przez dostawcę usług chmurowych programiści muszą opanować dodatkowe źródło złożoności. Dlatego każdy zespół pracujący nad aplikacją lub platformą musi zważyć koszty i korzyści płynące z użycia siatki usług na poziomie klastra. W maksymalizacji korzyści pomaga adaptacja siatki dla wszystkich mikrousług w klastrze jednocześnie.

Introspekcja implementacji siatki usług

W ekosystemie chmurowym istnieje wiele projektów i implementacji siatek usług, ale większość z nich opiera się na takich samych założeniach projektowych i technologiach. Ponieważ siatka usług transparentnie przechwytuje ruch sieciowy z kapsuły aplikacji, odpowiednio go modyfikując i przekierowując przez klaster, pewna jej część musi być obecna w każdej kapsule. Zmuszenie programistów do dodania czegoś do każdego kontenera wywołałoby poważne opory i znacznie utrudniłoby centralne zarządzanie wersją siatki usług. By tego uniknąć, większość implementacji siatki usług dodaje poboczny kontener do każdej kapsuły aplikacji. Jako że ten kontener poboczny znajduje się w tej samej sieci, co kapsuła aplikacji, może używać takich narzędzi, jak `iptables` czy `eBPF`, w celu introspekcji i przechwytywania ruchu sieciowego pochodzącego od kontenera aplikacji oraz wprowadzania go do siatki usług.

Oczywiście wymaganie od każdego programisty, aby dodał obraz kontenera do swojej definicji kapsuły, wywołuje prawie taki sam opór jak żądanie, by zmodyfikował swój obraz kontenera. Dlatego większość implementacji siatki usług bazuje na mutacyjnym kontrolerze dostępu (ang. *mutating admission controller*), który automatycznie dodaje poboczny kontener siatki usług do wszystkich kapsuł utworzonych w wybranym klastrze. Każde REST-owe żądanie API utworzenia kapsuły jest najpierw kierowane do tego kontrolera. Modyfikuje on definicję kapsuły przez dodanie kontenera pobocznego. Ponieważ mutacyjny kontroler dostępu jest zainstalowany przez administratora klastra, implementuje on transparentnie i spójnie siatkę usług dla całego klastra.

Jednak funkcjonalność siatki usług nie sprowadza się jedynie do modyfikacji sieci kapsuł. Musimy mieć także możliwość kontrolowania tego, jak się zachowuje, na przykład definiując reguły routingu dla eksperymentów lub ograniczania dostępu dla usług. Jak ze wszystkim innym w Kubernetes, te definicje zasobów deklaruje się za pomocą definicji obiektów JSON lub YAML, które tworzy się przy użyciu polecenia `kubectl` lub innych narzędzi komunikujących się z serwerem API Kubernetes. Implementacje siatki usług wykorzystują **niestandardowe definicje zasobów** (ang. *Custom Resource Definition* — CRD), aby dodawać specjalistyczne zasoby do klastra Kubernetes, które nie należą do domyślnej instalacji. W większości przypadków specyficzne niestandardowe zasoby są ściśle powiązane z samą siatką usług. W CNCF nieustannie trwają prace nad definicją standardowego niezależnego od dostawców interfejsu siatek usług (ang. *Service Mesh Interface* — SMI), który będą mogły implementować różne siatki usług.

Krajobraz siatek usług

Najbardziej przytłaczającym zadaniem w odniesieniu do siatek usług jest wybór jednej z nich. Jak na razie żadna nie zdobyła de facto pozycji standardu. Trudno jest uzyskać konkretne dane statystyczne, ale najpopularniejszą siatką usług może być projekt Istio. Oprócz niego istnieje wiele siatek open source, takich jak Consul Connect, Open Service Mesh i inne. Ponadto dostępne są również zastrzeżone siatki, takie jak AWS App Mesh. Spodziewamy się, że te interfejsy będą w najbliższej przyszłości ulegały standaryzacji w społeczności chmurowej.

W jaki sposób programista lub administrator klastra ma dokonać wyboru? Prawda jest taka, że najlepsza siatka usług dla Ciebie to ta, którą udostępnia Twój dostawca chmury. Dodawanie

obowiązków związanych z obsługą siatki usług do i tak skomplikowanych zadań administratora klastra zazwyczaj nie ma uzasadnienia. O wiele lepiej jest pozwolić dostawcy chmury zająć się wszystkim za nas.

Jeśli takie rozwiązanie nie wchodzi w grę, musisz przeprowadzić badania. Nie daj się zwieść efektownym demonstracjom i obietnicom funkcjonalności. Siatka usług głęboko integruje się z infrastrukturą i wszelkie błędy mogą znacząco wpłynąć na dostępność Twoich aplikacji. Ponadto interfejsy API siatek usług są z reguły powiązane z implementacją, przez co trudno jest zmienić raz wybraną siatkę. Dlatego może się okazać, że najlepszą siatką usług dla Ciebie jest brak siatki.

Podsumowanie

Siatki usług znacznie rozszerzają funkcjonalność w zakresie bezpieczeństwa i elastyczności aplikacji. Jednocześnie utrudniają jednak obsługę klastra i są kolejnym potencjalnym źródłem błędów, które mogą spowodować niedostępność aplikacji. Bardzo dokładnie przemyśl zalety i wady dodania siatki usług do swojej infrastruktury. Jeśli masz wybór, skorzystaj z siatki zarządzanej — dzięki temu ktoś inny będzie zajmował się jej obsługą, a Ty będziesz tylko używać jej w swoich aplikacjach.

ROZDZIAŁ 16.

Integracja rozwiązań do przechowywania danych i Kubernetes

W wielu przypadkach oddzielenie stanu od aplikacji i budowanie możliwie jak najbardziej bezstanowych mikrousług skutkuje uzyskaniem maksymalnie niezawodnych i łatwych w zarządzaniu systemów.

Jednak prawie każdy system, który ma jakąś złożoność, ma gdzieś w systemie pewien stan, od rekordów w bazie danych po pofragmentowane indeksy, które serwują wyniki dla wyszukiwarki internetowej. W pewnym momencie musisz mieć gdzieś zapisane dane.

Integracja tych danych z kontenerami i rozwiązaniami orkiestracji kontenerów jest często najbardziej skomplikowanym aspektem budowy rozproszonego systemu. Ta złożoność w dużej mierze wynika z faktu, że przejście na architektury kontenerowe jest również krokiem w kierunku rozłącznego, niemutowalnego i deklaratywnego rozwoju aplikacji. Te wzorce są względnie łatwe do zastosowania do bezstanowych aplikacji internetowych, ale nawet „chmurowo natywne" rozwiązania do przechowywania danych, takie jak Cassandra lub MongoDB, wymagają wykonania pewnego rodzaju ręcznych lub imperatywnych kroków w celu skonfigurowania niezawodnego rozwiązania opartego na replikach.

Jako przykład rozważmy skonfigurowanie `ReplicaSet` w MongoDB, co obejmuje wdrożenie demona Mongo, a następnie uruchomienie imperatywnego polecenia w celu określenia węzła głównego i pozostałych węzłów klastra Mongo. Oczywiście te kroki mogą być zapisane w skrypcie, ale w świecie kontenerowym trudno jest dostrzec, jak zintegrować takie polecenia ze wdrożeniem. Podobnie wyzwaniem jest uzyskanie rozpoznawalnych nazw DNS dla poszczególnych kontenerów w zestawie zreplikowanych kontenerów.

Dodatkowa trudność wynika z faktu, że istnieje grawitacja danych. Większość systemów kontenerowych nie została zbudowana w próżni; zwykle są one adaptowane z istniejących systemów wdrożonych na maszynach wirtualnych, a te systemy najprawdopodobniej obejmują dane, które należy importować lub przenosić.

Ponadto ewolucja do chmury oznacza, że często magazyn danych jest w rzeczywistości zewnętrzną usługą w chmurze i w tym kontekście nigdy nie może istnieć wewnątrz klastra Kubernetes.

Ten rozdział omawia różne podejścia do integracji magazynów danych ze skonteneryzowanymi mikrousługami w Kubernetes. Najpierw omówimy, jak zaimportować do Kubernetes istniejące

zewnętrzne rozwiązania do przechowywania danych (usługi chmurowe lub działające na maszynach wirtualnych). Następnie zobaczysz, jak uruchamiać wewnątrz Kubernetes niezawodne singletony, które pozwalają stworzyć środowisko w dużej mierze odpowiadające maszynom wirtualnym, na jakich wcześniej wdrażałeś rozwiązania do przechowywania danych. Na koniec przyjrzymy się obiektom `StatefulSet`, które są najczęściej używanym zasobem do obsługi stanowych obciążeń roboczych w Kubernetes.

Importowanie usług zewnętrznych

W wielu przypadkach masz już działającą w sieci maszynę, na której uruchomiona jest jakaś baza danych. W takiej sytuacji możesz nie chcieć przenosić tej bazy danych do kontenerów i Kubernetes. Być może jest ona prowadzona przez inny zespół, dokonujesz stopniowego przejścia lub zadanie migracji danych jest po prostu na tyle kłopotliwe, że nie jest warte zachodu.

Bez względu na powody starszy serwer i usługa nie zostaną przeniesione do Kubernetes, ale mimo to nadal warto reprezentować ten serwer w Kubernetes. Gdy to zrobisz, będziesz mógł skorzystać ze wszystkich wbudowanych podstawowych elementów nazewnictwa i wykrywania usług dostarczanych przez Kubernetes. Dodatkowo pozwoli Ci to skonfigurować wszystkie aplikacje w taki sposób, aby baza danych uruchomiona na jakieś maszynie gdzieś w sieci była w rzeczywistości usługą Kubernetes. Oznacza to, że trywialne stanie się zastąpienie jej bazą danych, która jest usługą Kubernetes. W środowisku produkcyjnym możesz na przykład polegać na starszej bazie danych uruchomionej na komputerze, ale do ciągłego testowania możesz wdrożyć testową bazę danych jako tymczasowy kontener. Ponieważ jest ona tworzona i niszczona dla każdego uruchomienia testu, w przypadku testowania ciągłego utrwalanie danych nie jest istotne. Reprezentowanie obu baz danych jako usług Kubernetes umożliwia utrzymywanie identycznych konfiguracji zarówno podczas testowania, jak i produkcji. Wysoki poziom zgodności między środowiskiem testowym i produkcyjnym zapewnia, że pomyślne przechodzenie testów będzie prowadzić do udanego wdrożenia w produkcji.

Aby nauczyć się, jak zachować dużą zgodność między środowiskiem rozwojowym i produkcyjnym, pamiętaj, że wszystkie obiekty Kubernetes są wdrażane w *przestrzeniach nazw*. Wyobraź sobie, że mamy zdefiniowane przestrzenie nazw `test` i `production`. Usługa testowa jest importowana za pomocą następującego obiektu:

```
kind: Service
metadata:
  name: my-database
  # zwróć uwagę na przestrzeń nazw 'test'
  namespace: test
...
```

Usługa produkcyjna wygląda tak samo, z tym wyjątkiem, że używa innej przestrzeni nazw:

```
kind: Service
metadata:
  name: my-database
  # zwróć uwagę na przestrzeń nazw 'prod'
  namespace: prod
...
```

Gdy kapsuła zostanie wdrożona w przestrzeni nazw `test` i wyszuka usługę o nazwie `my-database`, otrzyma wskaźnik do usługi `my-database.test.svc.cluster.internal`, co wskazuje na testową bazę danych. Gdy z kolei tę samą nazwę (`my-database`) wyszuka kapsuła wdrożona w przestrzeni nazw `prod`, otrzyma wskaźnik do usługi `my-database.prod.svc.cluster.internal`, która jest produkcyjną bazą danych. A zatem ta sama nazwa usługi w dwóch różnych przestrzeniach nazw rozwiązuje się na dwie różne usługi. Więcej informacji o tym, jak to działa, znajdziesz w rozdziale 7.

Wszystkie opisane poniżej techniki wykorzystują bazę danych lub inne usługi przechowywania, ale te podejścia mogą być równie dobrze stosowane w połączeniu z innymi usługami, które nie działają w Twoim klastrze Kubernetes.

Usługi bez selektorów

Kiedy po raz pierwszy pisaliśmy o usługach, omawialiśmy szeroko zapytania etykietowe i sposób ich używania do identyfikacji dynamicznego zestawu kapsuł stanowiących backendy dla konkretnej usługi. Jednak w przypadku usług zewnętrznych nie ma takiego zapytania etykietowego. Zamiast tego zazwyczaj mamy nazwę DNS, która wskazuje konkretny serwer z uruchomioną bazą danych. W naszym przykładzie załóżmy, że ten serwer ma nazwę `database.company.com`. Aby zaimportować tę zewnętrzną usługę bazy danych do Kubernetes, na listingu 16.1 zaczynamy od utworzenia usługi bez selektora kapsuł, która odwołuje się do nazwy DNS serwera bazy danych.

Listing 16.1. dns-service.yaml

```
kind: Service
apiVersion: v1
metadata:
  name: external-database
spec:
  type: ExternalName
  externalName: database.company.com
```

Gdy tworzona jest typowa usługa Kubernetes, tworzony jest również adres IP, a usługa DNS Kubernetes jest zapełniana rekordem A wskazującym na ten adres IP. Z kolei podczas tworzenia usługi typu `ExternalName` usługa DNS Kubernetes jest zamiast tego zapełniana rekordem CNAME wskazującym na określoną zewnętrzną nazwę (w tym przypadku `database.company.com`). Gdy aplikacja w klastrze przeprowadza wyszukiwanie DNS dla nazwy hosta `external-database.svc.default.cluster`, protokół DNS powiązuje tę nazwę z `database.company.com`. Jest ona następnie rozwiązywana na adres IP zewnętrznego serwera bazy danych. Dzięki temu wszystkie kontenery w Kubernetes są „przekonane", że komunikują się z usługą, za którą stoją inne kontenery, w rzeczywistości natomiast są przekierowywane do zewnętrznej bazy danych.

Należy zwrócić uwagę, że nie ogranicza się to do baz danych uruchamianych we własnej infrastrukturze. Wiele baz danych w chmurze i innych usług udostępnia nazwę DNS, z której można korzystać podczas uzyskiwania do nich dostępu (na przykład `my-database.databases.cloudprovider.com`). Możesz użyć tej nazwy DNS jako `externalName`. Spowoduje to zaimportowanie dostarczonej przez chmurę bazy danych do przestrzeni nazw Twojego klastra Kubernetes.

Czasami jednak nie masz adresu DNS dla zewnętrznej usługi bazy danych, tylko adres IP. W takich przypadkach nadal można zaimportować tę usługę jako usługę Kubernetes, ale operacja jest trochę inna. Tak jak pokazano na listingu 16.2, najpierw tworzy się obiekt `Service` bez selektora etykiet, ale także bez typu `ExternalName`, którego używaliśmy wcześniej.

Listing 16.2. external-ip-service.yaml

```
kind: Service
apiVersion: v1
metadata:
  name: external-ip-database
```

W tym momencie Kubernetes przydziela wirtualny adres IP dla tej usługi i zapełnia dla niego rekord A. Ponieważ jednak nie ma selektora dla usługi, nie zostaną zapełnione żadne punkty końcowe, do których mechanizm równoważenia obciążenia mógłby przekierowywać ruch.

Biorąc pod uwagę, że jest to usługa zewnętrzna, to użytkownik jest odpowiedzialny za ręczne zapełnianie punktów końcowych za pomocą zasobu `Endpoints`, tak jak pokazano na listingu 16.3.

Listing 16.3. external-ip-endpoints.yaml

```
kind: Endpoints
apiVersion: v1
metadata:
  name: external-ip-database
subsets:
  - addresses:
    - ip: 192.168.0.1
    ports:
      - port: 3306
```

Jeśli dla redundancji masz więcej niż jeden adres IP, możesz powtórzyć te adresy w tablicy `addresses`. Po zapełnieniu punktów końcowych system równoważenia obciążenia zacznie przekierowywać ruch z Twojej usługi Kubernetes do punktu końcowego (lub punktów końcowych) w postaci adresu IP.

Ponieważ odpowiedzialność za utrzymanie aktualnego adresu IP serwera spada na użytkownika, musisz upewnić się, że ten adres nigdy się nie zmieni, lub zastosować jakiś zautomatyzowany proces do aktualizowania rekordu `Endpoints`.

Ograniczenia usług zewnętrznych: sprawdzanie poprawności działania

Usługi zewnętrzne w Kubernetes mają jedno istotne ograniczenie: nie przeprowadzają żądnych kontroli poprawności działania. To użytkownik jest odpowiedzialny za zapewnienie, że punkt końcowy lub nazwa DNS dostarczone do Kubernetes będą odpowiednio niezawodne dla aplikacji.

Uruchamianie niezawodnych singletonów

Wyzwanie polegające na uruchamianiu rozwiązań do przechowywania danych w Kubernetes często polega na tym, że podstawowe obiekty, takie jak na przykład `ReplicaSet`, oczekują, że każdy kontener

będzie identyczny i wymienny, ale w przypadku magazynów tak nie jest. Jedną z opcji rozwiązania tego problemu jest wykorzystanie podstawowych obiektów Kubernetes, ale bez replikowania magazynu danych. Zamiast tego wystarczy uruchomić pojedynczą kapsułę z bazą danych lub innym rozwiązaniem do przechowywania danych. Dzięki temu nie pojawiają się wyzwania związane z uruchamianiem zreplikowanej pamięci w Kubernetes, ponieważ nie ma replikacji.

Na pierwszy rzut oka może się to wydawać sprzeczne z zasadami budowania niezawodnych systemów rozproszonych, ale ogólnie rzecz biorąc, jest to nie mniej niezawodne rozwiązanie niż uruchomienie bazy danych lub infrastruktury pamięci masowej na pojedynczej maszynie wirtualnej lub fizycznej, a w taki właśnie sposób często do tej pory budowano systemy. Tak naprawdę, jeśli system będzie odpowiednio zorganizowany, właściwie jedyną rzeczą, którą ryzykujesz, jest potencjalny przestój w przypadku aktualizacji lub awarii urządzenia. Może to nie być akceptowalne dla dużych lub kluczowych systemów, dla wielu mniejszych aplikacji ten rodzaj ograniczonych przestojów jest jednak rozsądnym kompromisem, gwarantującym mniejszą złożoność. Jeżeli w Twojej sytuacji takie rozwiązanie nie wchodzi w grę, możesz pominąć tę cześć rozdziału i zaimportować istniejące usługi zgodnie z opisem z poprzedniego podrozdziału lub przejść od razu do natywnych obiektów `StatefulSet` systemu Kubernetes opisanych w kolejnym podrozdziale. Pozostali czytelnicy mogą teraz dowiedzieć się, jak budować niezawodne singletony do przechowywania danych.

Uruchamianie singletona MySQL

W tym punkcie opiszemy, jak uruchomić niezawodną pojedynczą instancję bazy danych MySQL jako kapsułę w Kubernetes i jak udostępnić ten singleton innym aplikacjom w klastrze. W tym celu utworzymy trzy podstawowe obiekty:

- Wolumin trwały do zarządzania czasem życia magazynu danych na dysku niezależnie od długości życia działającej aplikacji MySQL.
- Kapsułę MySQL, która będzie uruchamiać aplikację MySQL.
- Usługę, która będzie udostępniać tę kapsułę innym kontenerom w klastrze.

W rozdziale 5. opisaliśmy trwałe woluminy: miejsca przechowywania danych, których czas życia nie jest zależny od żadnych kapsuł lub kontenerów. Trwały wolumin jest przydatny w przypadku trwałych rozwiązań pamięci masowej, w których reprezentacja bazy danych na dysku powinna przetrwać, nawet jeśli kontenery obsługujące aplikację bazy danych ulegną awarii lub zostaną przeniesione na inne komputery. Jeżeli aplikacja zostanie przeniesiona na inną maszynę, wolumin powinien zostać przeniesiony razem z nią, a dane powinny zostać zachowane. Umożliwia to wydzielenie magazynu danych jako trwałego woluminu.

Na początek utworzymy trwały wolumin dla naszej bazy danych MySQL. W tym przykładzie wykorzystano NFS w celu zapewnienia maksymalnej przenośności, ale Kubernetes obsługuje wiele różnych typów napędów dla trwałych woluminów. Dostępne są na przykład napędy trwałych woluminów dla wszystkich głównych dostawców chmury publicznej, a także dla wielu prywatnych dostawców usług w chmurze. Aby użyć tych rozwiązań, po prostu zamień `nfs` na odpowiedni typ woluminu dostawcy chmury (na przykład `azure`, `awsElasticBlockStore` lub `gcePersistentDisk`).

We wszystkich przypadkach wystarczy tylko ta zmiana. Kubernetes wie, jak utworzyć odpowiedni dysk pamięci u odpowiedniego dostawcy usług w chmurze. Jest to świetny przykład tego, jak Kubernetes upraszcza tworzenie niezawodnych systemów rozproszonych. Listing 16.4 pokazuje obiekt woluminu trwałego.

Listing 16.4. nfs-volume.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: database
  labels:
    volume: my-volume
spec:
  accessModes:
  - ReadWriteMany
  capacity:
    storage: 1Gi
  nfs:
    server: 192.168.0.1
    path: "/exports"
```

Powyżej zdefiniowany został obiekt woluminu trwałego NFS z 1 GB przestrzeni dyskowej. Jak zwykle, możemy utworzyć ten trwały wolumin za pomocą następującego polecenia:

```
$ kubectl apply -f nfs-volume.yaml
```

Gdy mamy już utworzony trwały wolumin, musimy zażądać go dla naszej kapsuły. Robimy to przy użyciu obiektu `PersistentVolumeClaim`, tak jak pokazano na listingu 16.5.

Listing 16.5. nfs-volume-claim.yaml

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: database
spec:
  accessModes:
  - ReadWriteMany
  resources:
    requests:
      storage: 1Gi
  selector:
    matchLabels:
      volume: my-volume
```

Pole `selector` wykorzystuje etykiety do znalezienia pasującego woluminu, który zdefiniowaliśmy uprzednio.

Taki rodzaj pośrednictwa może wydawać się zbyt skomplikowany, ale ma swój cel — służy odizolowaniu definicji kapsuły od definicji magazynu danych. Woluminy można deklarować bezpośrednio w specyfikacji kapsuły, ale to ogranicza specyfikację kapsuły do konkretnego dostawcy woluminu (na przykład określonej chmury publicznej lub prywatnej). Korzystając z żądań woluminów, można zachować niezależność specyfikacji kapsuły od chmury. Po prostu utwórz różne woluminy charakterystyczne dla poszczególnych chmur i używaj żądania `PersistentVolumeClaim`, aby powiązać je z kapsułami. Ponadto w wielu przypadkach kontroler woluminu trwałego automatycznie utworzy wolumin — więcej na ten temat przeczytasz w następnym podrozdziale.

Skoro pobraliśmy już wolumin, możemy użyć obiektu `ReplicaSet` do skonstruowania naszej singletonowej kapsuły. Może wydawać się dziwne, że używamy `ReplicaSet` do zarządzania pojedynczą kapsułą, ale jest to konieczne dla zapewnienia niezawodności. Pamiętaj, że po rozplanowaniu kapsuły na maszynę sama kapsuła jest związana z tą maszyną na zawsze. Jeśli maszyna ulegnie awarii, wszystkie znajdujące się na tej maszynie kapsuły, które nie są zarządzane przez kontroler wyższego poziomu, taki jak `ReplicaSet`, znikają wraz z maszyną i nie są ponownie rozplanowywane w innym miejscu. W związku z tym, aby upewnić się, że nasza kapsuła bazy danych zostanie ponownie rozplanowana w przypadku awarii maszyny, do zarządzania bazą danych używamy kontrolera wyższego poziomu, `ReplicaSet`, z ustawioną jedną repliką, tak jak pokazano na listingu 16.6.

Listing 16.6. mysql-replicaset.yaml

```
apiVersion: extensions/v1
kind: ReplicaSet
metadata:
  name: mysql
  # etykiety, żebyśmy mogli powiązać obiekt Service z tą kapsułą
  labels:
    app: mysql
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
      - name: database
        image: mysql
        resources:
          requests:
            cpu: 1
            memory: 2Gi
        env:
        # Zmienne środowiskowe nie są najlepszym wyborem pod względem bezpieczeństwa,
        # ale używamy ich tutaj dla zachowania zwięzłości.
        # Lepsze opcje znajdziesz w rozdziale 11.
        - name: MYSQL_ROOT_PASSWORD
          value: some-password-here
        livenessProbe:
          tcpSocket:
            port: 3306
        ports:
        - containerPort: 3306
        volumeMounts:
          - name: database
            # MySQL przechowuje swoje bazy danych w /var/lib/mysql
            mountPath: "/var/lib/mysql"
      volumes:
      - name: database
        persistentVolumeClaim:
          claimName: database
```

Gdy utworzymy obiekt `ReplicaSet`, on z kolei utworzy kapsułę z uruchomioną bazą danych MySQL, korzystając z utworzonego przez nas wcześniej woluminu trwałego. Ostatnim krokiem jest udostępnienie tej bazy danych jako usługi Kubernetes, tak jak pokazano na przykładzie 16.7.

Listing 16.7. mysql-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  ports:
  - port: 3306
    protocol: TCP
  selector:
    app: mysql
```

Teraz mamy niezawodną pojedynczą instancję MySQL działającą w naszym klastrze i udostępnioną jako usługa o nazwie `mysql`, do której mamy dostęp pod pełną nazwą domenową `mysql.svc.default.cluster`.

Podobne instrukcje można stosować dla wielu różnych magazynów danych, a jeśli masz proste potrzeby i możesz przetrwać ograniczone przestoje w obliczu awarii lub konieczności zaktualizowania oprogramowania bazodanowego, użycie niezawodnego singletonu może być właściwym podejściem do przechowywania danych w Twojej aplikacji.

Dynamiczne przydzielanie woluminów

Wiele klastrów obsługuje także **dynamiczne przydzielanie woluminów** (ang. *dynamic volume provisioning*). W przypadku dynamicznego przydzielania woluminów operator klastra tworzy co najmniej jeden obiekt `StorageClass`. W Kubernetes obiekt `StorageClass` definiuje cechy określonego typu magazynu danych. Klaster może mieć zainstalowanych wiele różnych klas magazynowych. Jedną z nich może być na przykład klasa dla serwera NFS w sieci, a inną — klasa pamięci blokowej iSCSI. Ponadto klasy magazynowe mogą reprezentować różne poziomy niezawodności lub wydajności. Listing 16.8 pokazuje domyślną klasę magazynu danych, która automatycznie przydziela obiekty dysków na platformie Microsoft Azure.

Listing 16.8. storageclass.yaml

```
apiVersion: storage.k8s.io/v1beta1
kind: StorageClass
metadata:
  name: default
  annotations:
    storageclass.beta.kubernetes.io/is-default-class: "true"
  labels:
    kubernetes.io/cluster-service: "true"
provisioner: kubernetes.io/azure-disk
```

Po utworzeniu klasy magazynu danych dla klastra możesz odwoływać się do niej w żądaniu przydzielenia trwałego woluminu, zamiast odnosić się do konkretnego trwałego woluminu. Gdy dynamiczny dostawca woluminów zauważy to żądanie, użyje odpowiedniego napędu woluminu, aby utworzyć ten wolumin i powiązać go z żądaniem dotyczącym trwałego woluminu.

Listing 16.9 pokazuje żądanie `PersistentVolumeClaim`, które używa właśnie zdefiniowanej przez nas klasy `default` magazynu danych, aby żądać nowo utworzonego trwałego woluminu.

Listing 16.9. dynamic-volume-claim.yaml

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: my-claim
  annotations:
    volume.beta.kubernetes.io/storage-class: default
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

Adnotacja `volume.beta.kubernetes.io/storage-class` jest tym, co łączy to żądanie z utworzoną przez nas klasą magazynu danych.

Automatyczne dostarczanie trwałego woluminu bardzo ułatwia tworzenie aplikacji stanowych w Kubernetes i zarządzanie nimi. Jednak długość życia tych woluminów zależy od zasad odzyskiwania pamięci obiektu `PersistentVolumeClaim`, które domyślnie wiążą ją z czasem istnienia kapsuły, która utworzyła wolumin.

Oznacza to, że jeśli usuniemy kapsułę (na przykład w ramach redukcji lub innej akcji), wraz z nią zostanie usunięty wolumin. W niektórych przypadkach właśnie o to chodzi, ale trzeba uważać, aby przypadkowo nie usunąć potrzebnych trwałych woluminów.

Trwałe woluminy są świetne w przypadku tradycyjnych aplikacji, które wymagają przechowywania danych, ale jeśli musisz opracować wysoce dostępny, skalowalny magazyn danych w sposób natywny dla Kubernetes, możesz użyć nowo wprowadzonego obiektu `StatefulSet`. Mając to na uwadze, w następnym podrozdziale opiszemy, jak wdrożyć MongoDB za pomocą `StatefulSet`.

Natywne magazyny danych Kubernetes z wykorzystaniem obiektów StatefulSet

Gdy system Kubernetes został opracowany, duży nacisk położono na homogeniczność wszystkich replik w zbiorze. W tym projekcie żadna replika nie miała indywidualnej tożsamości lub konfiguracji. Od poszczególnych programistów aplikacji zależało określenie projektu, który mógłby ustalić tę tożsamość dla aplikacji.

Chociaż to podejście zapewnia dużą izolację systemu orkiestracji, utrudnia także tworzenie aplikacji stanowych. Po znaczącym wkładzie ze strony społeczności i wielu eksperymentach z różnymi istniejącymi aplikacjami stanowymi do systemu Kubernetes w wersji 1.5 wprowadzone zostały obiekty `StatefulSet`.

Właściwości obiektów StatefulSet

Obiekty `StatefulSet` to grupy zreplikowanych kapsuł podobne do obiektów `ReplicaSet`, ale w przeciwieństwie do `ReplicaSet` mają pewne wyjątkowe właściwości:

- Każda replika otrzymuje stałą nazwę hosta z unikatowym indeksem (na przykład `database-0`, `database-1` itd.).
- Każda replika jest tworzona w kolejności od najmniejszego do największego indeksu, a tworzenie kolejnej jest wstrzymywane do momentu, aż kapsuła z poprzednim indeksem okaże się zdrowa i dostępna. To dotyczy także skalowania w górę.
- Przy usuwaniu obiektu `StatefulSet` poszczególne repliki są usuwane w kolejności od największego do najmniejszego indeksu. To dotyczy również skalowania liczby replik w dół.

Ten prosty zbiór wymagań radykalnie ułatwia wdrażanie aplikacji do przechowywania w Kubernetes. Na przykład połączenie stabilnego nazewnictwa hostów (jak `database-0`) i warunków porządkowania sprawia, że wszystkie repliki, z wyjątkiem pierwszej, mogą odnosić się do `database-0` w celu odkrywania i ustalania kworum replikacji.

Ręcznie zreplikowany klaster MongoDB z wykorzystaniem obiektów StatefulSet

W tym punkcie wdrożymy zreplikowany klaster MongoDB. Na razie konfiguracja replikacji zostanie wykonana ręcznie, abyś mógł się zorientować, jak działa obiekt `StatefulSet`. Ostatecznie zautomatyzujemy również tę konfigurację.

Na początek utworzymy zestaw trzech zreplikowanych kapsuł MongoDB przy użyciu obiektu `StatefulSet`, tak jak pokazano na listingu 16.10.

Listing 16.10. mongo-simple.yaml

```
apiVersion: apps/v1beta1
kind: StatefulSet
metadata:
  name: mongo
spec:
  serviceName: "mongo"
  replicas: 3
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
      - name: mongodb
        image: mongo:3.4.24
        command:
        - mongod
        - --replSet
        - rs0
```

```
        ports:
        - containerPort: 27017
          name: peer
```

Jak widać, ta definicja jest podobna do definicji `ReplicaSet` z poprzednich punktów. Jedyne zmiany to pola `apiVersion` i `kind`. Utwórz obiekt `StatefulSet`:

```
$ kubectl apply -f mongo-simple.yaml
```

Po utworzeniu różnice między `ReplicaSet` i `StatefulSet` stają się oczywiste. Uruchom polecenie `kubectl get pods`; zobaczysz następujące dane wyjściowe:

```
NAME      READY   STATUS              RESTARTS   AGE
mongo-0   1/1     Running             0          1m
mongo-1   0/1     ContainerCreating   0          10s
```

Istnieją dwie ważne różnice między tymi danymi wyjściowymi a tym, co można zobaczyć w przypadku obiektu `ReplicaSet`. Pierwszą z nich jest to, że każda zreplikowana kapsuła ma indeks numeryczny (0, 1, …) zamiast losowego przyrostka dodawanego przez kontroler `ReplicaSet`. Drugą jest to, że kapsuły są tworzone powoli w kolejności, a nie wszystkie naraz, tak jak w przypadku `ReplicaSet`.

Po utworzeniu obiektu `StatefulSet` musimy również utworzyć „bezgłową” usługę do zarządzania wpisami DNS dla `StatefulSet`. W systemie Kubernetes usługa jest nazywana „bezgłową”, jeśli nie ma wirtualnego adresu IP klastra. Ponieważ w przypadku obiektów `StatefulSet` każda kapsuła ma unikatową tożsamość, nie ma sensu posiadanie adresu IP dla celów równoważenia obciążenia dla zreplikowanej usługi. Bezgłową usługę możesz również utworzyć przy użyciu `clusterIP: None` w specyfikacji usługi, tak jak pokazano na listingu 16.11.

Listing 16.11. mongo-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: mongo
spec:
  ports:
  - port: 27017
    name: peer
  clusterIP: None
  selector:
    app: mongo
```

Po utworzeniu tej usługi istnieją zwykle cztery wpisy DNS, które są zapełniane. Jak zwykle tworzony jest wpis `mongo.default.svc.cluster.local`, ale w przeciwieństwie do standardowej usługi wyszukiwanie DNS na tej nazwie hosta zapewni wszystkie adresy ze `StatefulSet`. Dodatkowo tworzone są wpisy dla `mongo-0.mongo.default.svc.cluster.local` oraz dla `mongo-1.mongo` i `mongo-2.mongo`. Każdy z nich rozwiązuje się na określony adres IP indeksu repliki w `StatefulSet`. Tak więc w przypadku obiektów `StatefulSet` otrzymujesz dobrze zdefiniowane, trwałe nazwy dla każdej repliki w zestawie. Jest to często bardzo przydatne podczas konfigurowania opartego na replikach rozwiązania do przechowywania danych. Możesz zobaczyć te wpisy DNS, uruchamiając polecenia w jednej z replik Mongo:

```
$ kubectl run -it --rm --image busybox busybox ping mongo-1.mongo
```

Następnie ręcznie skonfigurujemy replikację Mongo przy użyciu tych nazw hostów dla poszczególnych kapsuł. Wybieramy `mongo-0.mongo` jako początkową replikę główną. Uruchom narzędzie `mongo` w tej kapsule:

```
$ kubectl exec -it mongo-0 mongo
> rs.initiate( {
  _id: "rs0",
  members:[ { _id: 0, host: "mongo-0.mongo:27017" } ]
});
OK
```

To polecenie instruuje `mongodb`, aby zainicjować `ReplicaSet` o nazwie `rs0` z `mongo-0.mongo` jako główną repliką.

Nazwa `rs0` jest przykładowa. Możesz użyć takiej nazwy, jak chcesz, ale musisz zmienić ją również w definicji *mongo-simple.yaml* obiektu `StatefulSet`.

Po zainicjowaniu obiektu `ReplicaSet` Mongo możesz dodać pozostałe repliki poprzez uruchomienie następujących poleceń w narzędziu `mongo` w kapsule `mongo-0.mongo`:

```
> rs.add("mongo-1.mongo:27017");
> rs.add("mongo-2.mongo:27017");
```

Jak widać, w celu dodania ich jako replik do naszego klastra Mongo używamy nazw DNS charakterystycznych dla replik. W tym momencie skończyliśmy. Nasz zreplikowany klaster MongoDB jest gotowy i uruchomiony. Ale tak naprawdę nie jest zautomatyzowany tak, jak byśmy tego chcieli. W następnym punkcie zobaczysz, jak używać skryptów do automatyzacji instalacji.

Automatyzacja tworzenia klastra MongoDB

Aby zautomatyzować wdrażanie naszego klastra MongoDB opartego na obiekcie `StatefulSet`, trzeba dodać do kapsuł jeszcze jeden kontener do przeprowadzania inicjowania. By skonfigurować tę kapsułę bez konieczności tworzenia nowego obrazu Dockera, użyjemy `ConfigMap` w celu dodania skryptu do istniejącego obrazu MongoDB.

Wykonamy ten skrypt jako **kontener inicjalizacji**. Kontenery inicjalizacji (lub kontenery „init”) to specjalne jednostki uruchamiane raz podczas uruchamiania kapsuły. Generalnie są używane w takich przypadkach jak ten, gdy trzeba przeprowadzić nieskomplikowane czynności konfiguracyjne przed uruchomieniem głównej aplikacji. W definicji kapsuły znajduje się osobna lista `initContainers`, w której można definiować kontenery inicjalizacyjne. Poniżej znajduje się przykład:

```
...
      initContainers:
      - name: init-mongo
        image: mongo:3.4.24
        command:
        - bash
        - /config/init.sh
        volumeMounts:
        - name: config
          mountPath: /config
```

```
...
      volumes:
      - name: config
      configMap:
        name: "mongo-init"
```

Zwróć uwagę, że jest to montowanie woluminu `ConfigMap`, którego nazwa to `mongo-init`. Ten obiekt `ConfigMap` przechowuje skrypt, który wykonuje inicjowanie. Najpierw skrypt określa, czy działa na `mongo-0`, czy nie. Jeśli jest na `mongo-0`, tworzy `ReplicaSet`, używając tego samego polecenia, które wcześniej wykonywaliśmy w sposób imperatywny. Jeśli jest na innej replice Mongo, czeka, aż `ReplicaSet` zostanie utworzony, a następnie rejestruje się jako składowa tego obiektu `ReplicaSet`.

Listing 16.12 pokazuje kompletny obiekt `ConfigMap`.

Listing 16.12. mongo-configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mongo-init
data:
  init.sh: |
    #!/bin/bash

    # Trzeba poczekać na przejście kontroli działania, żeby rozwiązane zostały
    # nazwy mongo. To nie jest najlepsze rozwiązanie.
    until ping -c 1 ${HOSTNAME}.mongo; do
      echo "czekanie na DNS (${HOSTNAME}.mongo)..."
      sleep 2
    done

    until /usr/bin/mongo --eval 'printjson(db.serverStatus())'; do
      echo "łączenie z lokalnym mongo..."
      sleep 2
    done
    echo "połączeno z local."

    HOST=mongo-0.mongo:27017

    until /usr/bin/mongo --host=${HOST} --eval 'printjson(db.serverStatus())'; do
      echo "łączenie ze zdalnym mongo..."
      sleep 2
    done
    echo "połączono z remote."

    if [[ "${HOSTNAME}" != 'mongo-0' ]]; then
      until /usr/bin/mongo --host=${HOST} --eval="printjson(rs.status())" \
            | grep -v "nie otrzymano konfiguracji replset"; do
        echo "oczekiwanie, aż replikacja ustawi inicjowanie"
        sleep 2
      done
      echo "dodawanie siebie do mongo-0"
      /usr/bin/mongo --host=${HOST} \
          --eval="printjson(rs.add('${HOSTNAME}.mongo'))"
    fi

    if [[ "${HOSTNAME}" == 'mongo-0' ]]; then
```

```
        echo "inicjowanie zestawu replik"
        /usr/bin/mongo --eval="printjson(rs.initiate(\
            {'_id': 'rs0', 'members': [{'_id': 0, \
             'host': 'mongo-0.mongo:27017'}]}))"
    fi
    echo "zainicjowano"
```

Zwróć uwagę, że ten skrypt od razu kończy działanie. Jest to ważne w przypadku używania kontenerów inicjalizacyjnych. Każdy taki kontener przed rozpoczęciem działania czeka, aż poprzedni zakończy działanie. Główny kontener aplikacji czeka na zakończenie działania wszystkich kontenerów inicjalizacyjnych. Gdyby ten skrypt nie zakończył działania, główny serwer Mongo nigdy by się nie uruchomił.

Łącząc to wszystko razem na listingu 16.13, otrzymamy kompletny obiekt `StatefulSet`, który używa `ConfigMap`.

Listing 16.13. mongo.yaml

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongo
spec:
  serviceName: "mongo"
  replicas: 3
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
      - name: mongodb
        image: mongo:3.4.24
        command:
        - mongod
        - --replSet
        - rs0
        ports:
        - containerPort: 27017
          name: web
      # Ten kontener inicjuje serwer mongodb, a następnie zostaje uśpiony.
      - name: init-mongo
        image: mongo:3.4.24
        command:
        - bash
        - /config/init.sh
        volumeMounts:
        - name: config
          mountPath: /config
      volumes:
      - name: config
        configMap:
          name: "mongo-init"
```

Biorąc pod uwagę wszystkie te pliki, możesz utworzyć klaster Mongo za pomocą następujących poleceń:

```
$ kubectl apply -f mongo-config-map.yaml
$ kubectl apply -f mongo-service.yaml
$ kubectl apply -f mongo-simple.yaml
```

Możesz też, jeśli chcesz, połączyć je wszystkie w jednym pliku YAML, w którym poszczególne obiekty są oddzielone znakami `---`. Upewnij się, że zachowujesz tę samą kolejność, ponieważ definicja `StatefulSet` opiera się na istniejącej definicji `ConfigMap`.

Trwałe woluminy i obiekty StatefulSet

W przypadku trwałego magazynu danych należy zamontować trwały wolumin w katalogu */data/db*. Aby zamontować żądanie trwałego woluminu w tym katalogu, musisz dokonać aktualizacji w szablonie kapsuły:

```
...
      volumeMounts:
      - name: database
        mountPath: /data/db
```

Chociaż to podejście jest podobne do tego, jakie stosowaliśmy w przypadku niezawodnych singletonów, ponieważ `StatefulSet` replikuje więcej niż jedną kapsułę, nie można po prostu odwołać się do żądania trwałego woluminu. Zamiast tego trzeba dodać **szablon żądania trwałego woluminu**. Możesz potraktować szablon żądania jako identyczny z szablonem kapsuły, ale zamiast tworzyć kapsuły, tworzy on żądania woluminów. Musisz dodać poniższy kod na dole definicji `StatefulSet`:

```
volumeClaimTemplates:
- metadata:
    name: database
    annotations:
      volume.alpha.kubernetes.io/storage-class: anything
  spec:
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: 100Gi
```

Po dodaniu szablonu żądania woluminu do definicji obiektu `StatefulSet` za każdym razem, gdy kontroler `StatefulSet` utworzy kapsułę będącą częścią `StatefulSet`, utworzy jako cześć tej kapsuły żądanie trwałego woluminu oparte na tym szablonie.

Aby zreplikowane trwałe woluminy działały poprawnie, musisz mieć ustawione automatyczne przydzielanie dla trwałych woluminów lub musisz wstępnie zapełnić kolekcję obiektów trwałych woluminów, z których będzie pobierał kontroler `StatefulSet`. Jeśli nie ma żądań, które można utworzyć, kontroler `StatefulSet` nie będzie w stanie wygenerować odpowiednich kapsuł.

Ostatnia rzecz: sondy gotowości

Ostatnim elementem przy wprowadzaniu do produkcji naszego klastra MongoDB jest dodanie kontroli żywotności do kontenerów serwowania Mongo. Jak wspomnieliśmy w rozdziale 5., w podrozdziale „Kontrole działania”, sonda żywotności służy do określenia, czy kontener działa poprawnie.

Do kontrolowania żywotności możemy użyć samego narzędzia `mongo`, dodając następujący kod do szablonu kapsuły w obiekcie `StatefulSet`:

```
...
  livenessProbe:
    exec:
      command:
        - /usr/bin/mongo
        - --eval
        - db.serverStatus()
      initialDelaySeconds: 10
      timeoutSeconds: 10
...
```

Podsumowanie

Po połączeniu obiektów `StatefulSet`, żądań trwałych woluminów i sond żywotności mamy zaimplementowaną skalowalną, opartą na chmurze instalację MongoDB działającą w Kubernetes. Chociaż ten przykład dotyczy MongoDB, kroki tworzenia obiektów `StatefulSet` do zarządzania innymi rozwiązaniami do przechowywania danych są bardzo podobne i można stosować analogiczne wzorce.

ROZDZIAŁ 17.

Rozszerzanie Kubernetes

Od samego początku było jasne, że Kubernetes nie zostanie na poziomie podstawowego zestawu interfejsów API. Po wprowadzeniu aplikacji do klastra użytkownik ma do wyboru mnóstwo dodatkowych narzędzi, które może wdrożyć w Kubernetes jako obiekty API. Trzeba było tylko zastanowić się, jak zapanować nad tą obfitością obiektów i zastosowań, nie dopuszczając do rozrostu API do gigantycznych rozmiarów.

Aby rozwiązać problem związany z puchnięciem API przy dodawaniu rozszerzeń, poświęcono wiele pracy na to, by interfejs API Kubernetes uczynić rozszerzalnym. Umożliwiło to operatorom klastrów rozszerzanie funkcjonalności za pomocą komponentów spełniających ich wymagania. Wprowadzony model pozwala na samodzielne rozszerzanie funkcjonalności klastrów, wykorzystywanie dodatków udostępnianych przez członków społeczności oraz tworzenie rozszerzeń, które można sprzedawać w postaci wtyczek. Ponadto możliwość wprowadzania rozszerzeń przyczyniła się do powstania nowych wzorców zarządzania systemami, takich jak na przykład wzorzec operatora.

Znajomość mechanizmów rozszerzania API serwera Kubernetes oraz technik tworzenia i dostarczania rozszerzeń jest niezbędna do optymalnego wykorzystania możliwości tego systemu i jego środowiska niezależnie od tego, czy tworzy się własne rozszerzenia, czy korzysta się tylko z gotowych rozwiązań. W miarę jak dzięki tym funkcjom powstają coraz bardziej zaawansowane narzędzia i platformy na bazie Kubernetes, praktyczna znajomość sposobu ich działania staje się niezbędna dla każdego, kto chce budować aplikacje w nowoczesnym klastrze Kubernetes.

Co znaczy rozszerzanie Kubernetes

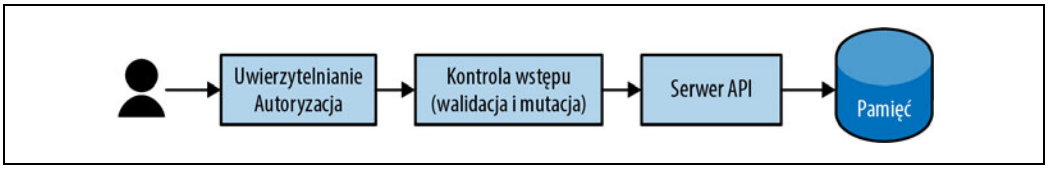
Generalnie rozszerzenia serwera API Kubernetes dodają nowe funkcje do klastra lub ograniczają i modyfikują sposoby interakcji użytkowników z klastrami. Administratorzy klastrów mają do dyspozycji szeroki wybór wtyczek, które rozszerzają ich klastry o nowe funkcje i usługi. Należy podkreślić, że klaster mogą rozszerzać tylko użytkownicy o bardzo wysokich uprawnieniach. Nie jest to czynność dostępna dla przypadkowych użytkowników lub programów, ponieważ wymaga uprawnień administratora. Nawet administratorzy klastra powinni być bardzo ostrożni w trakcie instalowania narzędzi zewnętrznych. Niektóre rozszerzenia, takie jak kontrolery wstępu, pozwalają przeglądać wszystkie obiekty tworzone w klastrze, przez co mogą zostać wykorzystane do kradzieży tajnych kodów i uruchamiania złośliwego kodu. Ponadto klaster z rozszerzeniami różni się od podstawowego klastra Kubernetes. Jeśli liczba klastrów jest większa, warto opracować narzędzia umożliwiające zapewnienie spójności sposobu pracy z nimi. Dotyczy to także instalowanych rozszerzeń.

Punkty rozszerzalności

Kubernetes można rozszerzać na wiele sposobów, na przykład przy użyciu własnych definicji zasobów (`CustomResourceDefinition`) lub wtyczek Container Network Interface (CNI). W tym rozdziale skupiamy się na dodawaniu nowych typów zasobów do serwera API lub kontrolerów wstępu do żądań API. Nie opisujemy rozszerzeń CNI/CSI/CRI (ang. *Container Network Interface/Container Storage Interface/Container Runtime Interface*), ponieważ z nich częściej korzystają twórcy klastrów Kubernetes, a książka ta jest przeznaczona dla użytkowników końcowych tego systemu.

Oprócz kontrolerów wstępu i rozszerzeń API istnieją też narzędzia „rozszerzania" klastra w ogóle niewymagające modyfikacji serwera API. Zaliczają się do nich na przykład zasoby `DaemonSet` instalujące automatyczne funkcje zapisu danych w dzienniku i monitoringu, narzędzia skanujące usługi w poszukiwaniu luk bezpieczeństwa umożliwiających przeprowadzanie ataków XSS (ang. *cross-site scripting*) i wiele więcej. Zanim jednak zaczniesz rozszerzać klaster, dobrze zapoznaj się z możliwościami podstawowych API Kubernetes.

W zrozumieniu roli kontrolerów wstępu i zasobów `CustomResourceDefinition` pomocna jest wiedza na temat przepływu żądań przez serwer API Kubernetes, który jest pokazany na rysunku 17.1.



Rysunek 17.1. Przepływ żądań przez serwer API

Kontrolery wstępu, które są wywoływane przed zapisaniem obiektu API w pamięci, mogą odrzucać lub modyfikować żądania API. W serwer API Kubernetes jest wbudowanych kilka kontrolerów wstępu. Na przykład kontroler ograniczeń zakresu określa domyślne limity dla kapsuł, które ich nie mają. Wiele innych systemów wykorzystuje własne kontrolery wstępu do automatycznego wstrzykiwania pobocznych kontenerów do wszystkich kapsuł utworzonych w systemie w celu włączenia „magicznych automatyzacji".

Inny rodzaj rozszerzenia, który także może być stosowany w połączeniu z kontrolerami wstępu, to **zasoby własne użytkownika**. Przy ich użyciu dodaje się całe nowe obiekty do API Kubernetes. Obiekty te można wprowadzać do przestrzeni nazw, podlegają funkcji RBAC, a dostęp do nich jest możliwy zarówno poprzez standardowe narzędzia, jak `kubectl`, jak i przez API Kubernetes.

Dalej bardziej szczegółowo opisujemy te typy rozszerzeń klastrów oraz przedstawiamy praktyczne przykłady ich zastosowania.

Przygotowanie własnego zasobu należy zacząć od utworzenia obiektu `CustomResourceDefinition`, który jest tak zwanym metazasobem, czyli zasobem, który definiuje inny zasób.

W ramach przykładu zdefiniujemy nowy zasób reprezentujący testy obciążeniowe klastra. Gdy zostanie utworzony zasób `LoadTest`, w klastrze Kubernetes nastąpi uruchomienie testu obciążenia i skierowanie ruchu do usługi.

Tworzenie nowego zasobu zaczynamy od jego zdefiniowania za pomocą obiektu `CustomResourceDefinition`. Poniżej znajduje się przykładowa definicja:

```
apiVersion: apiextensions.k8s.io/v1beta1
kind: CustomResourceDefinition
metadata:
  name: loadtests.beta.kuar.com
spec:
  group: beta.kuar.com
  versions:
    - name: v1
      served: true
      storage: true
  scope: Namespaced
  names:
    plural: loadtests
    singular: loadtest
    kind: LoadTest
    shortNames:
    - lt
```

Jak widać, jest to typowy obiekt Kubernetes zawierający podobiekt `metadata`, w którym znajduje się nazwa zasobu. Jednak w zasobach użytkownika nazwa jest wyjątkowa, ponieważ musi mieć następujący format: *`<nazwawliczbiemnogiej>.<grupa-api>`*. Zgodność nazw wszystkich obiektów `CustomResourceDefinition` z tym wzorem zapewnia niepowtarzalność definicji zasobów, ponieważ żadne dwa obiekty w klastrze nie mogą mieć takiej samej nazwy. To daje nam gwarancję, że nie ma dwóch definicji `CustomResourceDefinitions` opisujących ten sam zasób.

Oprócz metadanych definicja `CustomResourceDefinition` zawiera też podobiekt `spec`, w którym znajduje się właśnie definicja samego zasobu. Należące do niego pole `apigroup` określa grupę API zasobu. Jak napisaliśmy wcześniej, musi ona odpowiadać sufiksowi nazwy obiektu `CustomResourceDefinition`. Ponadto w tej części znajduje się lista wersji zasobu. Zawiera ona nazwę wersji (na przykład `v1`, `v2` itd.) oraz pola określające, czy dana wersja jest obsługiwana przez serwer API i która wersja jest używana do zapisywania danych w pamięci serwera API. Pole `storage` może mieć wartość `true` tylko dla jednej wersji zasobu. Pole `scope` określa, czy dany zasób należy do przestrzeni nazw, czy nie (domyślnie tak), a w polu `names` można podać nazwy w liczbie pojedynczej, mnogiej i nazwę rodzajową. Można też zdefiniować „krótkie nazwy", którymi łatwiej jest posługiwać się w `kubectl` i w innych narzędziach.

Na podstawie tej definicji można utworzyć zasób w serwerze API Kubernetes. Najpierw jednak wyświetlimy listę naszych zasobów `loadtests`, aby pokazać prawdziwą naturę dynamicznych typów zasobów:

```
$ kubectl get loadtests
```

Okazuje się, że w tej chwili taki zasób jest niezdefiniowany. Teraz utworzymy go przy użyciu pliku *loadtest-resource.yaml*:

```
$ kubectl create -f loadtest-resource.yaml
```

Ponownie pobieramy listę zasobów `loadtests`:

```
$ kubectl get loadtests
```

Tym razem zostaje znaleziona definicja zasobu typu `LoadTest`, choć nadal nie ma żadnych jego egzemplarzy. Zmienimy to, tworząc nowy zasób `LoadTest`.

Własny zasób (w tym przypadku `LoadTest`), podobnie jak wszystkie wbudowane obiekty API Kubernetes, można zdefiniować w formacie YAML lub JSON. Spójrz na poniższy przykład:

```
apiVersion: beta.kuar.com/v1
kind: LoadTest
metadata:
  name: my-loadtest
spec:
  service: my-service
  scheme: https
  requestsPerSecond: 1000
  paths:
  - /index.html
  - /login.html
  - /shares/my-shares/
```

Zwróć uwagę, że w obiekcie `CustomResourceDefinition` naszego zasobu brak definicji schematu. Wprawdzie można zdefiniować specyfikację OpenAPI (wcześniej zwaną Swagger), ale w przypadku prostych typów zasobów gra nie jest warta świeczki. Jeśli potrzebna jest możliwość walidacji, można zarejestrować kontroler wstępu w sposób opisany w dalszej części tego rozdziału.

Teraz naszego pliku *loadtest.yaml* możemy użyć do utworzenia zasobu w taki sam sposób, w jaki tworzy się zasoby typów wbudowanych:

```
$ kubectl create -f loadtest.yaml
```

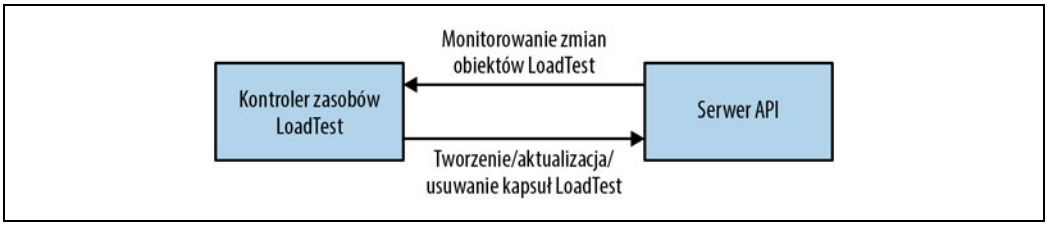
Aktualnie na liście zasobów `loadtests` pojawi się nasz nowo utworzony zasób:

```
$ kubectl get loadtests
```

Choć udało nam się już coś osiągnąć, na razie w naszym przykładzie nie dzieje się nic ciekawego. Oczywiście możemy użyć tego prostego API CRUD (tworzenie, odczyt, aktualizacja, usuwanie) do pracy z danymi obiektów `LoadTest`, ale na razie nie są tworzone żadne testy obciążeniowe w reakcji na ten nowy interfejs API. Przyczyną tego jest brak w klastrze kontrolera reagującego w odpowiedni sposób na pojawienie się definicji obiektu `LoadTest`. Zasób `LoadTest` jest tylko połową infrastruktury potrzebnej do dodawania obiektów `LoadTest` do naszego klastra. Druga połowa to kod, który będzie nieustannie monitorował zasoby oraz tworzył, modyfikował lub usuwał obiekty `LoadTest` według potrzeby.

Kontroler, jak przystało na użytkownika API, pobiera od serwera API listę obiektów `LoadTest` i obserwuje, czy zachodzą jakieś zmiany. Na rysunku 17.2 pokazaliśmy, jak odbywa się ta interakcja między kontrolerem i serwerem API.

Kod takiego kontrolera może być zarówno bardzo prosty, jak i skomplikowany. Najprostsze kontrolery zawierają pętlę `for`, za pomocą której nieustannie sprawdzają, czy pojawiły się nowe obiekty, i tworzą lub usuwają implementujące je zasoby (na przykład kapsuły robocze `LoadTest`).



Rysunek 17.2. Interakcje zasobu CustomResourceDefinition

Niestety to rozwiązanie oparte na ciągłym sprawdzaniu jest nieefektywne, ponieważ pętla stanowi dodatkowe obciążenie dla serwera API. Lepszym wyjściem jest użycie obserwacyjnego interfejsu API na serwerze API, który wysyła strumień aktualizacji, gdy się pojawią, eliminując w ten sposób zarówno opóźnienie, jak i narzut związane z ciągłym sprawdzaniem. Tylko że trudno jest bezbłędnie posłużyć się tym interfejsem API. Jeśli więc chcesz używać czujek, najlepiej skorzystaj z dobrze zaprojektowanego mechanizmu, takiego jak na przykład wzorzec Informer udostępniany w bibliotece `client-go` (*https://oreil.ly/L0QK2*).

Po utworzeniu własnego zasobu i zaimplementowaniu go przez kontroler w naszym klastrze możemy korzystać z nowej funkcji. Trzeba jednak przyznać, że nadal brakuje nam pewnych mechanizmów, które powinien mieć każdy dobrze działający zasób. Dwa najważniejsze z nich to walidacja i wybór wartości domyślnych. **Walidacja** to proces sprawdzania, czy obiekty `LoadTest` wysyłane do serwera API są poprawne i nadają się do tworzenia testów, natomiast **wybór wartości domyślnych** ułatwia korzystanie z naszych zasobów przez automatyczne dostarczanie typowych wartości domyślnych. Dodamy teraz te dwa składniki funkcjonalności do naszego zasobu.

Jak napisaliśmy wcześniej, mechanizm walidacji naszych obiektów możemy zdefiniować w postaci specyfikacji OpenAPI. Ta metoda sprawdzi się w przypadku podstawowych testów obecności wymaganych pól lub nieobecności nieznanych pól. Opis OpenAPI wykracza poza zakres tej książki, ale w internecie można znaleźć wiele materiałów na ten temat. Jednym z cennych źródeł jest pełna specyfikacja API Kubernetes (*https://oreil.ly/u3rRl*).

Generalnie rzecz biorąc, schemat API nie wystarczy do walidacji obiektów API. Na przykład w `loadtests` możemy chcieć sprawdzać, czy obiekt `LoadTest` ma prawidłowy schemat (przykładowo `http` lub `https`) lub czy wartość `requestsPerSecond` jest większa od zera.

W celu zaimplementowania takich testów użyjemy walidacyjnego kontrolera wstępu. Jak napisaliśmy wcześniej, kontrolery te przechwytują żądania do serwera API, zanim te żądania zostaną przetworzone, i mogą je odrzucać lub modyfikować w locie. Kontrolery wstępu można dodawać do klastrów przez dynamiczny system kontroli wstępu. Dynamiczny kontroler wstępu to prosta aplikacja HTTP. Serwer API łączy się z nim przez obiekt usługi Kubernetes lub dowolny adres URL. Oznacza to, że kontrolery wstępu mogą działać poza klastrem — na przykład jako funkcja usługowa w ramach oferty dostawcy chmury, jak Azure Functions czy AWS Lambda.

Aby zainstalować walidacyjny kontroler wstępu, należy go zdefiniować jako obiekt Kubernetes `ValidatingWebhookConfiguration`. Obiekt ten określa miejsce działania kontrolera oraz zasób (w tym przypadku `LoadTest`) i akcję (w tym przypadku `CREATE`), w której kontroler ten ma działać. Poniżej znajduje się kompletna definicja walidacyjnego kontrolera wstępu:

```
apiVersion: admissionregistration.k8s.io/v1beta1
kind: ValidatingWebhookConfiguration
metadata:
  name: kuar-validator
webhooks:
  - name: validator.kuar.com
    rules:
    - apiGroups:
      - "beta.kuar.com"
      apiVersions:
      - v1
      operations:
      - CREATE
    resources:
      - loadtests
    clientConfig:
      # Wpisz adres IP swojego elementu webhook
      url: https://192.168.1.233:8080
      # To powinien być certyfikat Twojego klastra w formacie base64
      # Znajduje się on w pliku ${KUBECONFIG}
      caBundle: ZAMIEŃMNIE
```

Na szczęście pod względem bezpieczeństwa i na nieszczęście pod kątem poziomu złożoności serwer API Kubernetes dostęp do elementów webhook może uzyskiwać tylko przez HTTPS. To znaczy, że musimy wygenerować certyfikat do obsługi elementu webhook. Najprostszym sposobem jest użycie funkcji generowania certyfikatów za pomocą własnej organizacji certyfikacyjnej klastra.

Najpierw potrzebujemy klucza prywatnego i żądania podpisania certyfikatu (CSR). Oto prosty program w języku Go, który je generuje:

```
package main

import (
        "crypto/rand"
        "crypto/rsa"
        "crypto/x509"
        "crypto/x509/pkix"
        "encoding/asn1"
        "encoding/pem"
        "net/url"
        "os"
)

func main() {
        host := os.Args[1]
        name := "server"

        key, err := rsa.GenerateKey(rand.Reader, 1024)
        if err != nil {
                panic(err)
        }
        keyDer := x509.MarshalPKCS1PrivateKey(key)
        keyBlock := pem.Block{
                Type: "KLUCZ PRYWATNY RSA",
                Bytes: keyDer,
        }
        keyFile, err := os.Create(name + ".key")
```

```
        if err != nil {
                panic(err)
        }
        pem.Encode(keyFile, &keyBlock)
        keyFile.Close()
        commonName := "myuser"
        emailAddress := "someone@myco.com"

        org := "My Co, Inc."
        orgUnit := "Widget Farmers"
        city := "Seattle"
        state := "WA"
        country := "US"

        subject := pkix.Name{
                CommonName:          commonName,
                Country:           []string{country},
                Locality:           []string{city},
                Organization:           []string{org},
                OrganizationalUnit:           []string{orgUnit},
                Province:           []string{state},
        }

        uri, err := url.ParseRequestURI(host)
        if err != nil {
                panic(err)
        }

        asn1, err := asn1.Marshal(subject.ToRDNSequence())
        if err != nil {
                panic(err)
        }
        csr := x509.CertificateRequest{
                RawSubject:          asn1,
                EmailAddresses:      []string{emailAddress},
                SignatureAlgorithm:  x509.SHA256WithRSA,
                URIs:                []*url.URL{uri},
        }
        bytes, err := x509.CreateCertificateRequest(rand.Reader, &csr, key)
        if err != nil {
                panic(err)
        }
        csrFile, err := os.Create(name + ".csr")
        if err != nil {
                panic(err)
        }
        pem.Encode(csrFile, &pem.Block{Type: "CERTIFICATE REQUEST", Bytes: bytes})
        csrFile.Close()
}
```

Ten program można uruchomić za pomocą następującego polecenia:

```
$ go run csr-gen.go <URL-elementu-webook>
```

Wygeneruje on dwa pliki — *server.csr* i *server-key.pem*.

Następnie możemy utworzyć żądanie podpisania certyfikatu do serwera API Kubernetes przy użyciu przedstawionego kodu YAML:

```
apiVersion: certificates.k8s.io/v1beta1
kind: CertificateSigningRequest
metadata:
  name: validating-controller.default
spec:
  groups:
  - system:authenticated
  request: ZAMIEŃMNIE
  usages:
  usages:
  - digital signature
  - key encipherment
  - key agreement
  - server auth
```

W polu `request` znajduje się wartość `ZAMIEŃMNIE`, którą należy zamienić na żądanie podpisania certyfikatu w formacie base64 wygenerowane w poprzednim kodzie:

```
$ perl -pi -e s/REPLACEME/$(base64 server.csr | tr -d '\n')/ \
  admission-controller-csr.yaml
```

Gdy ma się gotowe żądanie podpisania certyfikatu, można wysłać je do serwera API, aby uzyskać certyfikat:

```
$ kubectl create -f admission-controller-csr.yaml
```

Następnie należy zatwierdzić żądanie:

```
$ kubectl certificate approve validating-controller.default
```

Po zatwierdzeniu można pobrać nowy certyfikat:

```
$ kubectl get csr validating-controller.default -o json | \
  jq -r .status.certificate | base64 -d > server.crt
```

Jeśli masz certyfikat, możesz w końcu utworzyć kontroler wstępu oparty na SSL (uf!). Gdy kontroler ten otrzyma żądanie, zawiera obiekt typu `AdmissionReview`, w którym znajdują się metadane dotyczące żądania, jak również samą treść właściwą. Nasz walidacyjny kontroler wstępu zarejestrowaliśmy tylko dla jednego typu zasobów i jednej akcji (`CREATE`), więc nie musimy analizować metadanych żądania. Zamiast tego od razu zagłębiamy się w zasób i sprawdzamy, czy wartość `requestsPerSecond` jest dodatnia oraz czy schemat adresu URL jest poprawny. Jeśli coś jest nie tak, zwracamy odpowiedź odmowną w formacie JSON.

Implementacja domyślnych ustawień w kontrolerze wstępu jest podobna do poprzedniej implementacji, tylko zamiast konfiguracji `ValidatingWebhookConfiguration` użyjemy `MutatingWebhookConfiguration` i dostarczymy obiekt `JSONPatch` w celu modyfikacji obiektu żądania przed jego zapisaniem.

Poniżej znajduje się fragment kodu w języku TypeScript, który możesz dodać do swojego kontrolera, aby ustawiał wartości domyślne. Jeśli pole `paths` w `loadtest` ma zerową długość, dodajemy jedną ścieżkę do */index.html*:

```
if (needsPatch(loadtest)) {
    const patch = [
        { 'op': 'add', 'path': '/spec/paths', 'value': ['/index.html'] },
```

```
        ]
        response['patch'] = Buffer.from(JSON.stringify(patch))
            .toString('base64');
        response['patchType'] = 'JSONPatch';
    }
```

Następnie możemy zarejestrować ten element webhook jako konfigurację `MutatingWebhookConfiguration`, zmieniając pole `kind` w obiekcie YAML i zapisując plik jako *mutating-controller.yaml*. W dalszej kolejności tworzymy kontroler, wykonując poniższe polecenie:

```
$ kubectl create -f mutating-controller.yaml
```

W ten sposób stworzyliśmy kompletne rozszerzenie serwera API Kubernetes na bazie własnych zasobów i kontrolerów wstępu. W następnym podrozdziale opisujemy kilka ogólnych wzorców tworzenia rozszerzeń.

Wzorce tworzenia zasobów

Nie wszystkie własne zasoby użytkownika są takie same. Interfejs API Kubernetes rozszerza się w różnych celach, dlatego poniżej przedstawiamy kilka ogólnych wzorców, które warto znać.

Tylko dane

Najprostszy wzorzec rozszerzania API to „tylko dane". Polega on na wykorzystywaniu serwera API wyłącznie do zapisywania i pobierania danych dla aplikacji. Należy podkreślić, że serwera API Kubernetes **nie** powinno się używać do przechowywania danych aplikacji, ponieważ nie jest to magazyn par klucz-wartość. Zamiast tego należy używać rozszerzeń będących obiektami kontroli lub konfiguracji pomagającymi we wdrażaniu albo wykonywaniu aplikacji. Przykładowym zastosowaniem wzorca „tylko dane" jest konfiguracja wdrożeń canary aplikacji — na przykład kierowanie 10% całości ruchu do eksperymentalnego backendu. Teoretycznie takie informacje konfiguracyjne można przechowywać w obiekcie `ConfigMap`, ale obiekty te nie są typizowane, przez co w wielu przypadkach lepszym i prostszym rozwiązaniem jest użycie ściślej typizowanego obiektu rozszerzenia API.

Rozszerzenia reprezentujące tylko dane nie wymagają kontrolera do aktywacji, ale ewentualnie można zdefiniować kontroler walidacyjny lub mutacyjny w celu zapewnienia poprawności. Na przykład w przypadku canary kontroler walidacyjny mógłby sprawdzać, czy suma wartości procentowych wynosi 100%.

Kompilatory

Wzorzec „kompilator" lub „abstrakcja" jest odrobinę bardziej skomplikowany. W jego przypadku obiekt rozszerzenia API reprezentuje abstrakcję wyższego poziomu, która jest „wkompilowana" w kombinację obiektów Kubernetes niższego poziomu. Przykładem zastosowania tego wzorca w praktyce jest rozszerzenie `LoadTest` z poprzedniego przykładu. Użytkownik używa rozszerzenia jako koncepcji wysokiego poziomu, w tym przypadku `loadtest`, ale koncepcja ta powstaje poprzez wdrożenie jako kolekcji kapsuł i usług Kubernetes. Aby to osiągnąć, gdzieś w klastrze musi

działać kontroler API obserwujący bieżące obiekty `LoadTest` i tworzący „skompilowaną” reprezentację (oraz usuwający te obiekty, które już nie istnieją). Inaczej niż w przypadku wzorca „operator”, który jest opisany w następnym punkcie, w skompilowanych abstrakcjach nie ma mechanizmu sprawdzania stanu. Czynność ta zostaje oddelegowana do obiektów niższego poziomu (na przykład kapsuł).

Operatory

Podczas gdy rozszerzenia według wzorca „kompilator” dostarczają łatwych w użyciu abstrakcji, rozszerzenia typu „operator” umożliwiają aktywne zarządzanie zasobami tworzonymi przez rozszerzenia. Mogą one dostarczać abstrakcje wyższego poziomu (na przykład bazę danych), które są kompilowane do postaci reprezentacji niższego poziomu, ale dodatkowo zapewniają funkcjonalność online, na przykład wykonywanie migawek bazy danych czy wysyłanie powiadomień o dostępności nowych uaktualnień. To wymaga, aby kontroler nie tylko monitorował API rozszerzeń w celu dodawania lub usuwania pewnych elementów, ale również by monitorował informacje o stanie działania aplikacji przekazywane przez rozszerzenie (na przykład bazę danych) i podejmował właściwe działania zaradcze w przypadku problemów z bazami danych, wykonywał migawki lub przywracał je w razie awarii.

„Operator” to najbardziej skomplikowany, a jednocześnie dający największe możliwości wzorzec rozszerzeń API Kubernetes. Umożliwia on użytkownikowi tworzenie abstrakcji, które nie tylko wspomagają wdrożenia, ale dodatkowo potrafią sprawdzać stan i przeprowadzać naprawy.

Jak zacząć

Początki pracy z rozszerzeniami API Kubernetes mogą być nużące i wyczerpujące, ale na szczęście istnieje bardzo dużo pomocnego kodu, który można wykorzystać. Projekt Kubebuilder *(https://kubebuilder.io/)* zawiera bibliotekę kodu, za pomocą którego można z łatwością tworzyć solidne rozszerzenia API Kubernetes. To doskonałe źródło na początek dla każdego, kto chce nauczyć się tworzyć rozszerzenia.

Podsumowanie

Jedną z największych zalet systemu Kubernetes jest jego ekosystem, który prosperuje między innymi dzięki doskonałym możliwościom rozszerzania API Kubernetes. Nieważne, czy projektujesz własne rozszerzenia w celu modyfikacji sposobu działania klastra, czy korzystasz z gotowych rozszerzeń jako narzędzi, usług klastrowych lub operatorów, rozszerzenia API są podstawowym rozwiązaniem do budowy indywidualnie dostosowanych klastrów i środowiska odpowiedniego do szybkiego tworzenia niezawodnych aplikacji.

ROZDZIAŁ 18.

Dostęp do Kubernetes z poziomu popularnych języków programowania

Większa część tej książki jest poświęcona deklarowaniu konfiguracji w języku YAML za pomocą polecenia `kubectl` lub narzędzi takich jak Helm. Czasami konieczne jest jednak odwołanie się do API Kubernetes z poziomu języka programowania. Na przykład twórcy narzędzia Helm (*https://helm.sh*) musieli użyć języka programowania, aby napisać tę aplikację. Jest to typowe w przypadku, gdy trzeba stworzyć jakieś dodatkowe narzędzie, takie jak wtyczka `kubectl`, lub przygotować bardziej skomplikowany fragment kodu, taki jak na przykład operator Kubernetes.

Duża część ekosystemu Kubernetes jest napisana w języku programowania Go. W efekcie ten język ma najbogatszego i najbardziej rozbudowanego klienta. Istnieją jednak także wysokiej jakości klienty dla większości popularnych języków programowania (a nawet dla paru mniej znanych). Jako że powstała już obszerna, wzbogacona licznymi przykładami dokumentacja na temat klienta Go, w tym rozdziale opisujemy podstawy interakcji z serwerem API Kubernetes na bazie przykładów w Pythonie, Javie i językach platformy .NET.

API Kubernetes — perspektywa klienta

W ostatecznym rozrachunku serwer API Kubernetes to po prostu serwer HTTP(S), który dokładnie tak jest traktowany przez wszystkie biblioteki klienckie, choć każdy klient ma dodatkową logikę implementującą różne wywołania API oraz dokonującą serializacji do i z formatu JSON. W związku z tym możesz czuć pokusę, aby do pracy z API Kubernetes używać zwykłego klienta HTTP. Biblioteki klienckie opakowują jednak wywołania HTTP w nadające im znaczenie API, dzięki czemu kod jest czytelny (na przykład `readNamespacedPod(...)`), oraz typizowane modele obiektowe wykorzystujące statyczną kontrolę typów, co pozwala na zmniejszenie liczby błędów (na przykład `Deployment`). Co ważniejsze, biblioteki klientów implementują także specjalne funkcje Kubernetes, takie jak ładowanie informacji autoryzacyjnych z pliku *kubeconfig* lub ze środowiska kapsuły. Ponadto klienty dostarczają implementacji nie REST-owych części API Kubernetes, takich jak przekierowania portów, dzienniki i czujki. Opisujemy te elementy funkcjonalności w dalszej części rozdziału.

OpenAPI i generowane biblioteki klientów

Zbiór zasobów i funkcji w API Kubernetes jest ogromny. Zasoby są podzielone na wiele grup i dla każdego z nich istnieje dużo różnych operacji. Gdyby programiści mieli ręcznie pisać wszystkie te wywołania API, opanowanie zasobów i ich wersji byłoby bardzo trudnym (i niezwykle nudnym) zadaniem, zwłaszcza jeśli weźmie się pod uwagę fakt, że w każdym języku programowania klienta trzeba napisać samodzielnie. Dlatego klienty przyjmują inną strategię polegającą na tym, że podstawy interakcji z serwerem API Kubernetes są generowane przez program komputerowy przypominający nieco odwrotnie działający kompilator. Generator kodu dla klientów API pobiera specyfikację danych dla API Kubernetes i na jej podstawie tworzy klienta dla wybranego języka.

API Kubernetes jest zdefiniowany w formacie o nazwie OpenAPI, który jest najczęściej używany do reprezentowania API REST-owych. Abyś mógł uświadomić sobie rozmiar API Kubernetes, wiedz, że jego specyfikacja OpenAPI (*https://oreil.ly/3gRIW*) w portalu GitHub ma ponad 4 MB. To bardzo duży plik tekstowy! Wszystkie oficjalne biblioteki klienckie Kubernetes są generowane przy użyciu tej samej podstawowej logiki, którą można znaleźć w GitHub (*https://oreil.ly/F39uK*). Jest mało prawdopodobne, że będziesz musiał generować biblioteki klienckie samodzielnie, niemniej jednak warto rozumieć proces ich powstawania. Jako że większość kodu klienckiego jest generowana, aktualizacji i poprawek nie można wprowadzać bezpośrednio w tak powstałym kodzie, ponieważ zmiany te zostałyby utracone po następnym wygenerowaniu API. Kiedy więc w kliencie zostanie wykryty błąd, poprawki należy wprowadzić w specyfikacji OpenAPI (jeśli błąd dotyczy samej specyfikacji) lub w generatorze kodu (jeśli błąd znajduje się w kodzie). Choć może się to wydawać niezwykle skomplikowane, to jedyny sposób, aby niewielka liczba twórców klientów Kubernetes była w stanie opanować API Kubernetes.

Kwestia kubectl x

Prawdopodobnie niedługo po tym, jak zaczniesz implementować własną logikę interakcji z API Kubernetes, zaczniesz się zastanawiać, jak wykonać `kubectl x`. Większość osób zaczyna posługiwać się narzędziem `kubectl`, kiedy uczy się Kubernetes, i wydaje się im, że istnieje zależność 1:1 między możliwościami `kubectl` a API Kubernetes. Niektóre polecenia mają bezpośrednią reprezentację w API Kubernetes (na przykład `kubectl get pods`), ale większość bardziej zaawansowanych funkcji obejmuje większą liczbę wywołań API o skomplikowanej logice w narzędziu `kubectl`.

Ten stosunek funkcjonalności klienckiej do serwerowej stanowi źródło kompromisów projektowych od samego początku istnienia Kubernetes. Wiele funkcji, które obecnie znajdują się w API serwerowym, początkowo było klienckimi implementacjami w `kubectl`. Na przykład funkcje wdrożeniowe teraz zaimplementowane na serwerze przez zasób `Deployment` początkowo były zaimplementowane w kliencie. Analogicznie do niedawna polecenie `kubectl apply ...` było dostępne tylko w wierszu poleceń, ale przeniesiono je na serwer jako funkcjonalność `apply`, która jest opisana w dalszej części tego rozdziału.

Mimo ogólnej tendencji zwracania się ku implementacjom serwerowym wciąż pewne ważne funkcje pozostają w kliencie. Wszystkie muszą być zaimplementowane w każdej bibliotece klienckiej. W różnych językach inaczej wygląda stosunek między funkcjonalnością klienta i wiersza poleceń `kubectl`. W szczególności w przypadku Javy tłusty klient emuluje większość funkcjonalności `kubectl`.

Jeśli nie możesz znaleźć potrzebnej funkcji w swojej bibliotece klienckiej, dodaj flagę `--v=10` do polecenia `kubectl`. Spowoduje to włączenie rozszerzonych dzienników rejestrujących wszystkie żądania i odpowiedzi HTTP wysyłane do serwera API Kubernetes. Na podstawie tych informacji możesz odtworzyć działanie narzędzia `kubectl`. Jeśli potrzebujesz sięgnąć jeszcze głębiej, w repozytorium Kubernetes znajdziesz kod źródłowy tego programu.

Programowanie API Kubernetes

Wiesz już dokładniej, jak działa API Kubernetes oraz jak wygląda współpraca klienta z serwerem. W kolejnych podrozdziałach pokazujemy, jak dokonać uwierzytelnienia na serwerze API Kubernetes oraz jak posługiwać się zasobami. Na końcu poruszamy zaawansowane tematy, takie jak pisanie operatorów czy interakcja z kapsułami w interaktywnych operacjach.

Instalacja bibliotek klienckich

Aby móc zacząć programowanie przy użyciu API Kubernetes, należy znaleźć odpowiednie biblioteki klienckie. My korzystamy z oficjalnych bibliotek utworzonych przez sam projekt Kubernetes, ale istnieją także wysokiej jakości klienty przygotowane w ramach innych niezależnych inicjatyw. Wszystkie biblioteki klienckie znajdują się w repozytorium GitHub *kubernetes-client*:

- Python (*https://oreil.ly/ku6mT*)
- Java (*https://oreil.ly/aUSkD*)
- .NET (*https://oreil.ly/9J8iy*)

Każdy z tych projektów ma tabelę zgodności pokazującą, które wersje klienta współpracują z określonymi wersjami API Kubernetes, oraz zawiera instrukcje na temat instalacji bibliotek przy użyciu menedżerów pakietów (na przykład `npm`) powiązanych z danym językiem programowania[1].

Uwierzytelnianie w API Kubernetes

Serwer API Kubernetes nie byłby zbyt bezpieczny, gdyby każdy człowiek na świecie mógł uzyskać do niego dostęp oraz odczytać i zapisać jego zasoby. Dlatego pierwszym krokiem programisty, który chce korzystać z interfejsu API Kubernetes, jest połączenie się z nim i podanie swojej tożsamości. Jako że ten serwer API jest zasadniczo serwerem HTTP, uwierzytelnianie jest przeprowadzane standardowymi metodami uwierzytelniania HTTP. W pierwszych implementacjach Kubernetes używano podstawowego uwierzytelniania HTTP na podstawie nazwy użytkownika i hasła, ale odchodzi się od tego podejścia na rzecz nowocześniejszych rozwiązań.

Jeśli korzystasz z narzędzia wiersza poleceń `kubectl` do interakcji z Kubernetes, to być może nie zastanawiałeś się nad szczegółami implementacyjnymi uwierzytelniania. Na szczęście biblioteki klienckie z reguły ułatwiają łączenie się z API. Niemniej jednak warto mieć ogólną wiedzę na temat mechanizmów uwierzytelniania Kubernetes, ponieważ może być ona pomocna przy debugowaniu.

[1] Dla uproszczenia nie pokazujemy przykładów w JavaScripcie (*https://oreil.ly/8mw5F*), ale klient w tym języku też jest aktywnie rozwijany.

Narzędzie `kubectl` i klienty pobierają informacje uwierzytelniające na dwa podstawowe sposoby: z pliku konfiguracyjnego i z kontekstu kapsuły w klastrze Kubernetes.

Kod, który nie działa w klastrze Kubernetes, wymaga podania danych uwierzytelniających w pliku konfiguracyjnym. Domyślnie klient szuka go na ścieżce *${HOME}/.kube/config* lub w zmiennych środowiskowych `$KUBECONFIG`. Jeśli jest obecna zmienna `KUBECONFIG`, to ma ona pierwszeństwo przed plikami konfiguracyjnymi znajdującymi się w katalogu głównym. Plik konfiguracyjny zawiera komplet informacji potrzebnych do uzyskania dostępu do serwera API Kubernetes. Wszystkie klienty udostępniają łatwe w użyciu wywołania umożliwiające utworzenie klienta z domyślnych lokalizacji albo z pliku konfiguracyjnego określonego w samym kodzie:

Python

```
config.load_kube_config()
```

Java

```
ApiClient client = Config.defaultClient();
Configuration.setDefaultApiClient(client);
```

.NET

```
var config = KubernetesClientConfiguration.BuildDefaultConfig();
var client = new Kubernetes(config);
```

U wielu dostawców chmury uwierzytelnianie odbywa się przez zewnętrzny plik wykonywalny generujący token do klastra Kubernetes. Plik ten często jest zainstalowany jako jedno z narzędzi wiersza poleceń danego dostawcy. Jeśli chcesz pisać kod korzystający z API Kubernetes, musisz upewnić się, że ten plik jest dostępny także w kontekście, w którym tworzony przez Ciebie kod będzie działał, aby można go było wykonać w celu pobrania tokenu.

W kontekście kapsuły w klastrze Kubernetes kod działający w kapsule ma dostęp do konta usługi Kubernetes, które jest powiązane z tą kapsułą. Pliki zawierające odpowiedni token i certyfikat zostają umieszczone w kapsule przez Kubernetes jako wolumin podczas jej tworzenia. W klastrze Kubernetes serwer API zawsze jest dostępny pod stałą nazwą DNS, zwykle `kubernetes`. Ponieważ wszystkie potrzebne dane znajdują się w kapsule, plik konfiguracyjny jest zbędny i klient może utworzyć konfigurację ze swojego kontekstu. Wszystkie klienty mają łatwe w użyciu wywołania służące do tworzenia takiego klienta „w klastrze”:

Python

```
config.load_incluster_config()
```

Java

```
ApiClient client = ClientBuilder.cluster().build();
Configuration.setDefaultApiClient(client);
```

.NET

```
var config = KubernetesClientConfiguration.InClusterConfig()
var client = new Kubernetes(config);
```

Domyślne konto usługowe powiązane z kapsułami ma przydzielone minimalne role (RBAC). To znaczy, że domyślnie kod działający w kapsule nie może zbyt wiele zrobić z API Kubernetes. Jeśli pojawiają się błędy uwierzytelniania, może być konieczne dostosowanie konta usługowego do kodu oraz przydzielenie mu odpowiednich ról w klastrze.

Dostęp do API Kubernetes

Najczęstsze formy interakcji z API Kubernetes to podstawowe operacje, takie jak tworzenie, wyświetlanie listy czy usuwanie zasobów. Jako że wszystkie klienty są generowane na podstawie tej samej specyfikacji OpenAPI, są zbudowane z grubsza według jednego wzoru. Zanim przejdziemy do kodu, opowiemy jeszcze o dwóch istotnych elementach API Kubernetes, o których trzeba wiedzieć.

W Kubernetes rozróżnia się zasoby przynależne do przestrzeni nazw i zasoby na poziomie klastra. Zasoby *należące do przestrzeni nazw* egzystują w przestrzeni nazw Kubernetes. Na przykład kapsuła lub wdrożenie mogą należeć do przestrzeni `kube-system`. Zasoby na *poziomie klastra* istnieją tylko w pojedynczym egzemplarzu na przestrzeni całego klastra. Oczywistym przykładem takiego zasobu jest `Namespace`, ale są nimi także `CustomResourceDefinitions` i `ClusterRoleBindings`. To rozróżnienie jest ważne, ponieważ jest zachowane w wywołaniach funkcji używanych w celu uzyskiwania dostępu do zasobów. Aby na przykład wyświetlić listę kapsuł należących do domyślnej przestrzeni nazw Python, należałoby napisać `api.list_namespaced_pods('default')`. By otrzymać listę zasobów `Namespace`, należałoby napisać `api.list_namespaces()`.

Druga koncepcja, którą należy zrozumieć, to **grupa API**. W Kubernetes wszystkie zasoby są podzielone na różne zbiory API. Użytkownicy narzędzia `kubectl` raczej nie mają z tym do czynienia, ale z grupowaniem można się spotkać w polu `apiVersion` w specyfikacji YAML obiektu Kubernetes. Podczas programowania z użyciem API Kubernetes te grupy są ważne, ponieważ zwykle każda grupa ma własnego klienta do interakcji z danym zbiorem zasobów. Na przykład aby utworzyć klienta do interakcji z zasobem `Deployment` (który znajduje się w grupie API i wersji `apps/v1`), należy utworzyć nowy obiekt `AppsV1Api()`, który „wie", jak współpracować ze wszystkimi zasobami w grupie i wersji API `apps/v1`. Przykład utworzenia klienta dla grupy API jest pokazany w następnym punkcie.

Generowanie list i tworzenie kapsuł w Pythonie, Javie i .NET

Możemy przystąpić do pisania kodu. Zaczniemy od utworzenia obiektu klienta, a następnie użyjemy go do wygenerowania listy kapsuł znajdujących się w „domyślnej" przestrzeni nazw. Poniżej znajdują się odpowiednie fragmenty kodu w Pythonie, Javie i .NET:

Python

```
config.load_kube_config()
api = client.CoreV1Api()
pod_list = api.list_namespaced_pod('default')
```

Java

```
ApiClient client = Config.defaultClient();
Configuration.setDefaultApiClient(client);
CoreV1Api api = new CoreV1Api();
V1PodList list = api.listNamespacedPod("default");
```

.NET

```
var config = KubernetesClientConfiguration.BuildDefaultConfig();
var client = new Kubernetes(config);
var list = client.ListNamespacedPod("default");
```

Gdy już nauczysz się generować listy, odczytywać i usuwać obiekty, następną czynnością, którą zapewne będziesz chciał wykonać, jest tworzenie nowych obiektów. Wywołanie API tworzące nowy obiekt jest bardzo proste (na przykład `create_namespaced_pod` w Pythonie), definiowanie nowych zasobów kapsuły bywa natomiast bardziej skomplikowane.

Poniżej znajdują się przykłady ilustrujące tworzenie kapsuł w Pythonie, Javie i .NET:

Python

```
container = client.V1Container(
    name="myapp",
    image="my_cool_image:v1",
)

pod = client.V1Pod(
    metadata = client.V1ObjectMeta(
        name="myapp",
    ),
    spec=client.V1PodSpec(containers=[container]),
)
```

Java

```
V1Pod pod =
    new V1PodBuilder()
        .withNewMetadata().withName("myapp").endMetadata()
        .withNewSpec()
            .addNewContainer()
                .withName("myapp")
                .withImage("my_cool_image:v1")
            .endContainer()
        .endSpec()
        .build();
```

.NET

```
var pod = new V1Pod()
{
    Metadata = new V1ObjectMeta{ Name = "myapp", },
    Spec = new V1PodSpec
    {
        Containers = new[] {
            new V1Container() {
                Name = "myapp", Image = "my_cool_image:v1",
            },
        },
    }
};
```

Tworzenie i łatanie nowych obiektów

Kiedy przejrzysz API klienta dla Kubernetes, z pewnością zauważysz, że zasobami można manipulować na trzy sposoby: `create`, `replace` i `patch`. Te trzy czasowniki reprezentują różne metody interakcji z zasobami:

`create`
: Ta operacja tworzy nowy zasób, ale kończy się niepowodzeniem, jeśli dany zasób już istnieje.

`replace`
: Ta operacja powoduje całkowite zastąpienie istniejącego zasobu bez jego oglądania. Używając jej, należy określić kompletny zasób.

`patch`
: Ta operacja modyfikuje istniejący zasób, pozostawiając nietknięte te części, które nie uległy zmianie. Podczas jej używania korzysta się ze specjalnego zasobu `Patch`, zamiast wysyłać zasób (na przykład kapsułę), który jest modyfikowany.

Łatanie zasobu może być skomplikowane. W wielu przypadkach łatwiej jest wymienić zasób. Jednak w niektórych sytuacjach, zwłaszcza dotyczących dużych zasobów, łatanie może być bardziej efektywne pod względem transferu danych i przetwarzania na serwerze API. Ponadto poszczególne strony mogą łatać różne części zasobu jednocześnie bez ryzyka powstania konfliktów zapisu, co redukuje narzut.

Aby załatać zasób Kubernetes, należy utworzyć obiekt `Patch` reprezentujący zmianę, której chcemy dokonać w zasobie. Kubernetes obsługuje trzy formaty łat: JSON Patch, JSON Merge Patch i strategiczna łata scalana (ang. *strategic merge patch*). Pierwsze dwa formaty to standardy RFC używane także w innych miejscach. Trzeci natomiast to format specyficzny dla Kubernetes. Każdy z nich ma zalety i wady. W przykładach używamy formatu JSON Patch, ponieważ jest najłatwiejszy do zrozumienia.

Oto przykład łaty zasobu `Deployment` zwiększającej liczbę replik do trzech:

Python

```
deployment.spec.replicas = 3
api_response = api_instance.patch_namespaced_deployment(
    name="my-deployment",
    namespace="some-namespace",
    body=deployment)
```

Java

```
// Format JSON Patch
static String jsonPatch =
  "[{\"op\":\"replace\",\"path\":\"/spec/replicas\",\"value\":3}]";

V1Deployment patched =
          PatchUtils.patch(
              V1Deployment.class,
              () ->
                    api.patchNamespacedDeploymentCall(
```

```
                        "my-deployment",
                        "some-namespace",
                        new V1Patch(jsonPatchStr),
                        null,
                        null,
                        null,
                        null,
                        null),
                V1Patch.PATCH_FORMAT_JSON_PATCH,
                api.getApiClient());
```

.NET

```
var jsonPatch = @"
[{
    ""op"": ""replace"",
    ""path"": ""/spec/replicas"",
    ""value"": 3
}]";

client.PatchNamespacedPod(
  new V1Patch(patchStr, V1Patch.PatchType.JsonPatch),
  "my-deployment",
  "some-namespace");
```

W każdym z tych przykładów modyfikacja zasobu `Deployment` polega na zmianie liczby replik we wdrożeniu na trzy.

Obserwowanie zmian w API Kubernetes

Zasoby w Kubernetes są deklarowane i reprezentują żądany stan systemu. Program musi monitorować ten stan pod kątem zmian i podejmować działania, aby bieżący stan świata zgadzał się z żądanym.

Z tego powodu jednym z najczęstszych zadań podczas programowania z wykorzystaniem API Kubernetes jest monitorowanie zmian w zasobie i podejmowanie odpowiednich działań, gdy takie zmiany się pojawią. Najprostszym sposobem na to jest **sondowanie** (ang. *polling*). Polega ono na wywoływaniu funkcji generowania listy opisanej powyżej co określony czas (na przykład co 60 sekund) i wymienianiu wszystkich zasobów, które są istotne dla kodu. Choć ten kod łatwo napisać, ma on liczne wady zarówno w odniesieniu do kodu klienta, jak i serwera API. Sondowanie powoduje niepotrzebne opóźnienie, ponieważ oczekiwanie na cykl sondowania opóźnia wprowadzenie zmian, które nastąpiły krótko po jego zakończeniu. Ponadto zwiększa się obciążenie serwera API, bo wielokrotnie zwraca on zasoby, które nie uległy zmianie. Jeśli zbyt wiele klientów stosuje sondowanie, serwer API może ulec przeciążeniu, co spowoduje zwiększenie opóźnień.

Aby rozwiązać ten problem, w API Kubernetes dodano **czujkę** (ang. *watch*), czyli semantykę opartą na zdarzeniach. Za pomocą wywołania `watch` można zarejestrować zainteresowanie określonymi zmianami w API serwera. Wówczas sondowanie jest niepotrzebne, ponieważ serwer API generuje powiadomienia, gdy pojawią się jakiekolwiek zmiany. W praktyce klient wysyła wiszące żądanie `GET` do serwera API HTTP. Połączenie TCP, na którym bazuje to żądanie HTTP, pozostaje otwarte przez cały czas działania czujki i serwer zapisuje odpowiedź w tym strumieniu (ale go nie zamyka), kiedy tylko pojawi się jakaś zmiana.

Z programistycznego punktu widzenia semantyka `watch` umożliwia programowanie zdarzeniowe, zastępując pętlę `while` nieustannie wykonującą sondowanie kolekcją wywołań zwrotnych. Poniżej znajdują się przykłady obserwacji kapsuł pod kątem zmian:

Python

```
config.load_kube_config()
api = client.CoreV1Api()
w = watch.Watch()

for event in w.stream(v1.list_namespaced_pods, "some-namespace"):
  print(event)
```

Java

```
ApiClient client = Config.defaultClient();
CoreV1Api api = new CoreV1Api();

Watch<V1Namespace> watch =
    Watch.createWatch(
        client,
        api.listNamespacedPodCall(
            "some-namespace",
            null,
            null,
            null,
            null,
            null,
            Integer.MAX_VALUE,
            null,
            null,
            60,
            Boolean.TRUE);
        new TypeToken<Watch.Response<V1Pod>>() {}.getType());

try {
    for (Watch.Response<V1Pod> item : watch) {
        System.out.printf(
            "%s : %s%n", item.type, item.object.getMetadata().getName());
    }
} finally {
    watch.close();
}
```

.NET

```
var config = KubernetesClientConfiguration.BuildConfigFromConfigFile();
var client = new Kubernetes(config);

var watch =
    client.ListNamespacedPodWithHttpMessagesAsync("default", watch: true);
using (watch.Watch<V1Pod, V1PodList>((type, item) =>
{
    Console.WriteLine(item);
}
```

W prezentowanych przykładach wywołanie API `watch` przesyła każdą zmianę w zasobie do funkcji zwrotnej zdefiniowanej przez użytkownika. To zmniejsza opóźnienie oraz obciążenie serwera API Kubernetes.

Interakcja z kapsułami

API Kubernetes udostępnia także funkcje do bezpośredniej interakcji z aplikacjami działającymi w kapsułach. Narzędzie `kubectl` ma kilka poleceń do interakcji z kapsułami (`logs`, `exec` i `port-forward`) i każdego z nich można używać także we własnym kodzie.

Ponieważ API `logs`, `exec` i `port-forward` są niestandardowe w sensie REST, wymagają niestandardowej logiki w bibliotekach klienckich i są w związku z tym mniej spójne między różnymi klientami. Niestety nie ma innego wyjścia, jak nauczyć się implementacji w każdym języku.

Jeśli chodzi o pobieranie dzienników kapsuły, należy wybrać, czy chcemy je odczytać w celu uzyskania migawki obecnego stanu, czy zamierzamy je przesyłać strumieniowo, aby otrzymywać nowe dzienniki, kiedy będą się pojawiać. W drugim przypadku (ekwiwalent polecenia `kubectl logs -f...`) należy utworzyć otwarte połączenie z serwerem API i wówczas nowe wiersze dziennika będą zapisywane w tym strumieniu w trakcie ich zapisywania w kapsule. Jeśli tego nie zrobimy, po prostu otrzymamy obecną zawartość dzienników.

Poniżej znajduje się przykład odczytu i strumieniowania dzienników:

Python

```
config.load_kube_config()
api = client.CoreV1Api()
log = api_instance.read_namespaced_pod_log(
  name="my-pod", namespace="some-namespace")
```

Java

```
V1Pod pod = ...; // kod definiujący lub pobierający kapsułę
PodLogs logs = new PodLogs();
InputStream is = logs.streamNamespacedPodLog(pod);
```

.NET

```
IKubernetes client = new Kubernetes(config);
var response = await client.ReadNamespacedPodLogWithHttpMessagesAsync(
    "my-pod", "my-namespace", follow: true);
var stream = response.Body;
```

Innym popularnym zadaniem jest wykonywanie polecenia w kapsule i pobieranie jego wyniku. W tym celu możemy użyć polecenia `kubectl exec...` w wierszu poleceń. Interfejs API, który je implementuje, tworzy połączenie WebSocket z serwerem API. Połączenia WebSocket umożliwiają współistnienie wielu strumieni danych (w tym przypadku `stdin`, `stdout` i `stderr`) w tym samym połączeniu HTTP. Jeśli nigdy nie używałeś połączeń WebSocket, nie przejmuj się. Interakcje z nimi są obsługiwane przez biblioteki klienckie.

Poniżej znajduje się przykład wykonania polecenia `ls /foo` w kapsule:

Python

```
cmd = [ 'ls', '/foo' ]
response = stream(
    api_instance.connect_get_namespaced_pod_exec,
    "my-pod",
    "some-namespace",
    command=cmd,
    stderr=True,
    stdin=False,
    stdout=True,
    tty=False)
```

Java

```
ApiClient client = Config.defaultClient();
Configuration.setDefaultApiClient(client);
Exec exec = new Exec();
final Process proc =
exec.exec("some-namespace",
          "my-pod",
          new String[] {"ls", "/foo"},
          true,
          true /*tty*/);
```

.NET

```
var config = KubernetesClientConfiguration.BuildConfigFromConfigFile();
IKubernetes client = new Kubernetes(config);
var webSocket =
    await client.WebSocketNamespacedPodExecAsync(
        "my-pod", "some-namespace", "ls /foo", "my-container-name");
var demux = new StreamDemuxer(webSocket);
demux.Start();
var stream = demux.GetStream(1, 1);
```

Oprócz wykonywania poleceń w kapsułach możemy także przekazywać połączenia sieciowe z kapsuły do kodu działającego na komputerze lokalnym. Podobnie jak w przypadku `exec`, przekazany ruch przechodzi przez WebSocket. Od stworzonego przez Ciebie kodu zależy, co z nim zrobi. Możesz wysłać pojedyncze żądanie i odebrać odpowiedź jako łańcuch bajtów lub zbudować kompletny serwer proxy (tak jak to robi polecenie `kubectl port-forward`), aby przez niego serwować dowolne żądania.

Poniżej znajduje się przykład konfiguracji przekazywania portów:

Python

```
pf = portforward(
    api_instance.connect_get_namespaced_pod_portforward,
    'my-pod', 'some-namespace',
    ports='8080',
)
```

Java

```
PortForward fwd = new PortForward();

List<Integer> ports = new ArrayList<>();
int localPort = 8080;
int targetPort = 8080;
ports.add(targetPort);
final PortForward.PortForwardResult result =
    fwd.forward("some-namespace", "my-pod", ports);
```

.NET

```
var config = KubernetesClientConfiguration.BuildConfigFromConfigFile();
IKubernetes client = new Kubernetes(config);
var webSocket = await client.WebSocketNamespacedPodPortForwardAsync(
  "some-namespace", "my-pod", new int[] {8080}, "v4.channel.k8s.io");
var demux = new StreamDemuxer(webSocket, StreamType.PortForward);
demux.Start();
var stream = demux.GetStream((byte?)0, (byte?)0);
```

Każdy z tych fragmentów kodu tworzy połączenie z portu 8080 w kapsule do portu 8080 w programie. Kod zwraca strumienie bajtów, komunikując się przez ten kanał przekierowywania portów. Strumienie te można wykorzystać do wysyłania i odbierania wiadomości.

Podsumowanie

API Kubernetes zapewnia bogatą funkcjonalność dostępną podczas tworzenia własnego kodu. Pisanie aplikacji w języku, który jest najbardziej odpowiedni w danym przypadku, pozwala wykorzystać zalety API orkiestracji wspólnie z jak największą liczbą użytkowników Kubernetes. Kiedy będziesz gotowy przejść poza wywołania skryptowe do `kubectl`, biblioteki klienckie Kubernetes zapewnią Ci dostęp do API umożliwiającego budowę operatora, agenta monitoringu, nowego interfejsu użytkownika i wszystkiego, co sobie zamarzysz.

ROZDZIAŁ 19.

Zabezpieczanie aplikacji w Kubernetes

Zapewnienie bezpiecznej platformy do wykonywania aplikacji ma krytyczne znaczenie pod kątem szerokiego wykorzystywania Kubernetes w środowisku produkcyjnym. Na szczęście Kubernetes ma wiele różnych API zabezpieczeń, za pomocą których można stworzyć bezpieczne środowisko. To, że jest ich dużo, może się jednak okazać problemem, bo trzeba zadeklarować, których chce się używać. Posługiwanie się tymi API może być niewygodne i skomplikowane, co jeszcze bardziej utrudnia osiągnięcie celów w zakresie bezpieczeństwa.

Aby skutecznie zabezpieczyć kapsuły w Kubernetes, należy na wstępie zrozumieć dwa pojęcia: dogłębna obrona i zasada minimalnych uprawnień. **Dogłębna obrona** to koncepcja polegająca na wykorzystaniu kilku warstw środków bezpieczeństwa w systemach komputerowych zawierających także Kubernetes. **Zasada minimalnych uprawnień** oznacza natomiast, że obciążeniom należy udzielać dostępu tylko do tych zasobów, które są im niezbędne do działania. Obie te koncepcje są powszechnie stosowane w nieustannie zmieniającym się krajobrazie systemów komputerowych.

W tym rozdziale opisujemy interfejsy API Kubernetes zawierające narzędzia bezpieczeństwa, które można stopniowo stosować, aby zabezpieczyć obciążenia na poziomie kapsuły.

Kontekst zabezpieczeń

Podstawę zabezpieczeń kapsuł stanowi kontekst zabezpieczeń (SecurityContext). Jest to zbiór wszystkich pól dotyczących bezpieczeństwa, których można używać zarówno na poziomie kapsuły, jak i specyfikacji kontenera. Oto kilka przykładowych środków bezpieczeństwa dostępnych w kontekście zabezpieczeń:

- kontrola uprawnień i dostępu użytkowników (na przykład ustawianie identyfikatora użytkownika i grupy),
- główny system plików tylko do odczytu,
- zezwalanie na podnoszenie uprawnień,
- przypisania profili i etykiet Seccomp, AppArmor i SELinux,
- wykonywanie aplikacji jako użytkownik z uprawnieniami i bez uprawnień.

Na listingu 19.1 pokazano przykładową kapsułę ze zdefiniowanym kontekstem zabezpieczeń.

Listing 19.1. kuard-pod-securitycontext.yaml

```
apiVersion: v1
kind: Pod
metadata:
    name: kuard
spec:
    securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        runAsGroup: 3000
        fsGroup: 2000
    containers:
      - image: gcr.io/kuar-demo/kuard-amd64:blue
        name: kuard
        securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            privileged: false
        ports:
          - containerPort: 8080
            name: http
            protocol: TCP
```

W tym przykładzie kontekst zabezpieczeń jest zdefiniowany zarówno na poziomie kapsuły, jak i na poziomie kontenera. Wiele środków bezpieczeństwa można stosować na obu tych poziomach. W takim przypadku konfiguracja na poziomie kontenera jest ważniejsza. Przyjrzymy się polom, które zdefiniowaliśmy w tej specyfikacji kapsuły, oraz zastanowimy się, jaki mają wpływ na bezpieczeństwo naszego obciążenia.

`runAsNonRoot`

Kapsuła lub kontener muszą być uruchomione na koncie użytkownika innego niż `root`. Uruchomienie kontenera jako użytkownik `root` się nie powiedzie. Uruchamianie przy użyciu innego użytkownika niż `root` jest uważane za dobrą praktykę, ponieważ wiele błędów konfiguracji i exploitów wykorzystuje środowisko wykonawcze kontenera z uprawnieniami użytkownika głównego. To pole można ustawić zarówno w kontekście PodSecurityContext, jak i Security Context. Obraz kontenera `kuard` jest skonfigurowany w pliku *Dockerfile* (*https://oreil.ly/4IZI7*) tak, aby był uruchamiany jako użytkownik „nobody". Uruchamianie kontenerów jako inny użytkownik niż `root` zawsze jest dobrym wyborem. Jeśli jednak uruchamiasz kontener pobrany z innego, który nie ma wprost ustawionego użytkownika, może być konieczne rozszerzenie oryginalnego pliku *Dockerfile*, aby to zmienić. Ta metoda nie zawsze działa, ponieważ aplikacja może mieć jeszcze inne wymagania, które należy uwzględnić.

`runAsUser/runAsGroup`

To ustawienie zastępuje użytkownika i grupę wykorzystywane do uruchamiania procesu kontenera. Obrazy kontenerów mogą mieć to pole skonfigurowane w pliku *Dockerfile*.

`fsgroup`

Konfiguruje Kubernetes, aby zmieniał grupę wszystkich plików w woluminie podczas ich montowania w kapsule. Dodatkowo można użyć pola `fsGroupChangePolicy`, by dokładniej skonfigurować ustawienia.

`allowPrivilegeEscalation`

Określa, czy proces w kontenerze może uzyskać więcej uprawnień niż jego proces nadrzędny. To pole jest popularnym wektorem ataku i należy je ustawić na `false`. Ponadto trzeba mieć świadomość, że będzie ono miało wartość `true`, jeśli pole `privileged` zostanie ustawione na `true`.

`privileged`

Uruchamia kontener jako uprzywilejowany, co zwiększa jego uprawnienia do poziomu uprawnień hosta.

`readOnlyRootFilesystem`

Montuje główny system plików kontenera jako tylko do odczytu. Jest to popularny wektor ataku i dobrą praktyką jest włączenie tego ustawienia. Wszystkie dane lub dzienniki wymagające zapisu można zamontować przez wolumin.

Pola pokazane w tym miejscu nie wyczerpują listy wszystkich dostępnych środków bezpieczeństwa, ale wystarczą na początek pracy z kontekstem zabezpieczeń. W dalszej części tego rozdziału opisujemy jeszcze parę innych podobnych pól.

Teraz utworzymy kapsułę przez zapisanie przykładowego kodu w pliku o nazwie *kuard-pod-securitycontext.yaml*. Pokażemy, jak zastosować konfigurację kontekstu bezpieczeństwa do działającej kapsuły. Utwórz kapsułę za pomocą następującego polecenia:

```
$ kubectl create -f kuard-pod-securitycontext.yaml
pod/kuard created
```

Następnie uruchom powłokę w kontenerze `kuard` i sprawdź identyfikatory użytkownika i grupy, pod których kontrolą działają procesy:

```
$ kubectl exec -it kuard -- ash
/ $ id
uid=1000 gid=3000 groups=2000
/ $ ps
PID   USER       TIME    COMMAND
    1 1000         0:00  /kuard
   30 1000         0:00  ash
   37 1000         0:00  ps
/ $ touch file
touch: file: Read-only file system
```

Jak widać, uruchomiona powłoka, `ash`, działa pod kontrolą użytkownika o identyfikatorze (`uid`) 1000 i grupy o identyfikatorze (`gid`) 3000 oraz znajduje się w grupie 2000. Ponadto widzimy, że proces `kuard` działa jako użytkownik 1000 zgodnie z definicją SecurityContext w specyfikacji kapsuły. Potwierdziliśmy też, że nie możemy tworzyć żadnych nowych plików, ponieważ kontener jest tylko do odczytu. Jeśli zastosujesz opisane zmiany, masz wszystko, czego potrzeba na początek.

Teraz wprowadzimy kilka innych środków bezpieczeństwa należących do SecurityContext, które umożliwiają jeszcze dokładniejszą kontrolę dostępu i uprawnień obciążeń. Zaczniemy od środków bezpieczeństwa na poziomie systemu operacyjnego i pokażemy, jak je skonfigurować przez kontekst bezpieczeństwa. Należy pamiętać, że wiele z nich jest zależnych od systemu operacyjnego hosta. To znaczy, że mogą dotyczyć tylko kontenerów działających w systemach Linux, a nie w innych systemach obsługiwanych przez Kubernetes, takich jak Windows. Poniżej znajduje się lista podstawowych zabezpieczeń systemu operacyjnego, które obejmuje SecurityContext.

Umiejętności (ang. *capabilities*)

Umożliwiają dodanie lub usunięcie grup uprawnień, które mogą być potrzebne do działania obciążenia. Na przykład dane obciążenie może definiować konfigurację sieci hosta. Zamiast nadawać uprawnienia kapsule, co w istocie oznacza udzielenie dostępu na poziomie użytkownika `root` hosta, można dodać specjalną „umiejętność" konfiguracji sieci hosta (nazwa tej umiejętności to `NET_ADMIN`). Taki sposób postępowania jest zgodny z zasadą minimalnych uprawnień.

AppArmor

Określa pliki, do których procesy mają dostęp. Profile AppArmor można stosować do kontenerów przez dodanie adnotacji `container.apparmor.security.beta.kubernetes.io/`*`<nazwa_kontenera>`*`:` *`<ref_profilu>`* do specyfikacji kapsuły. Akceptowalne wartości *`<ref_profilu>`* to `runtime/default`, `localhost/`*`<ścieżka do profilu>`* i `unconfined`. Domyślną wartością jest `unconfined`, która oznacza brak profilu do zastosowania.

Seccomp

Profile Seccomp (ang. *secure computing* — bezpieczne obliczenia) umożliwiają tworzenie filtrów wywołań systemowych, za pomocą których można przepuszczać lub blokować wybrane wywołania systemowe, aby ograniczyć obszar jądra Linuksa dostępny dla procesów w kapsułach.

SELinux

Definiuje środki kontroli dostępu do plików i procesów. Operatory SELinux używają etykiet, które są podzielone na grupy tworzące kontekst bezpieczeństwa (nie mylić z kontekstem zabezpieczeń SecurityContext Kubernetes) ograniczający dostęp do procesu. Domyślnie Kubernetes przydziela losowy kontekst SELinux każdemu kontenerowi, ale można ustawić wybrany przez SecurityContext.

Zarówno AppArmor, jak i Seccomp mogą ustawiać domyślny profil środowiska wykonawczego, który ma być używany. Każde środowisko wykonawcze kontenera ma domyślne profile AppArmor i Seccomp — zostały one starannie skonfigurowane tak, aby ograniczać powierzchnię ataku przez usunięcie wywołań systemowych i dostępu do plików, które są znanymi wektorami ataku lub nie są powszechnie używane przez aplikacje. Te domyślne ustawienia rzadko wpływają na działanie obciążeń i stanowią doskonały punkt startowy.

Aby zademonstrować zastosowanie tych środków bezpieczeństwa do kapsuły, użyjemy narzędzia o nazwie amicontained („Am I contained", *https://oreil.ly/6ubkU*), napisanego przez Jess Frazelle. Zapisz specyfikację kapsuły z listingu 19.2 w pliku o nazwie *amicontained-pod.yaml*. Pierwsza kapsuła nie ma zastosowanego kontekstu SecurityContext i zostanie użyta do pokazania, które środki bezpieczeństwa są domyślnie wykorzystywane w kapsułach. Pamiętaj, że u Ciebie wyniki mogą być odmienne, jeśli używasz innej dystrybucji Kubernetes i innych usług zarządzanych.

Listing 19.2. amicontained-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
    name: amicontained
```

```
spec:
    containers:
      - image: r.j3ss.co/amicontained:v0.4.9
        name: amicontained
        command: [ "/bin/sh", "-c", "--" ]
        args: [ "amicontained" ]
```

Utwórz kapsułę `amicontainer`:

```
$ kubectl apply -f amicontained-pod.yaml
pod/amicontained created
```

Zajrzymy do dzienników kapsuły, aby obejrzeć wyniki działania narzędzia `amicontained`:

```
$ kubectl logs amicontained
Container Runtime: kube
Has Namespaces:
        pid: true
        user: false
AppArmor Profile: docker-default (enforce)
Capabilities:
        BOUNDING -> chown dac_override fowner fsetid kill setgid setuid
        setpcap net_bind_service net_raw sys_chroot mknod audit_write
        setfcap
Seccomp: disabled
Blocked Syscalls (21):
        SYSLOG SETPGID SETSID VHANGUP PIVOT_ROOT ACCT SETTIMEOFDAY UMOUNT2
        SWAPON SWAPOFF REBOOT SETHOSTNAME SETDOMAINNAME INIT_MODULE
        DELETE_MODULE LOOKUP_DCOOKIE KEXEC_LOAD FANOTIFY_INIT
        OPEN_BY_HANDLE_AT FINIT_MODULE KEXEC_FILE_LOAD
Looking for Docker.sock
```

Z powyższych danych wynika, że został zastosowany domyślny profil środowiska wykonawczego AppArmor. Ponadto widzimy umiejętności, które są dozwolone domyślnie, oraz dowiadujemy się, że wyłączono Seccomp. Na końcu znajduje się lista 21 domyślnie zablokowanych wywołań systemowych. Na tej podstawie możemy zastosować środki bezpieczeństwa Seccomp, AppArmor i Capabilities do specyfikacji kapsuły. Utwórz plik o nazwie *amicontained-pod-securitycontext.yaml* i zawartości przedstawionej na listingu 19.3.

Listing 19.3. amicontained-pod-securitycontext.yaml

```
apiVersion: v1
kind: Pod
metadata:
    name: amicontained
    annotations:
        container.apparmor.security.beta.kubernetes.io/amicontained: "runtime/default"
spec:
    securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        runAsGroup: 3000
        fsGroup: 2000
        seccompProfile:
            type: RuntimeDefault
    containers:
      - image: r.j3ss.co/amicontained:v0.4.9
```

```
        name: amicontained
        command: [ "/bin/sh", "-c", "--" ]
        args: [ "amicontained" ]
        securityContext:
            capabilities:
                add: ["SYS_TIME"]
                drop: ["NET_BIND_SERVICE"]
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            privileged: false
```

Najpierw musimy usunąć istniejącą kapsułę `amicontained`:

```
$ kubectl delete pod amicontained
pod "amicontained" deleted
```

Teraz możemy utworzyć nową kapsułę z zastosowanym kontekstem SecurityContext. Deklarujemy, że chcemy wykorzystać domyślne profile środowiska wykonawczego AppArmor i Seccomp. Ponadto dodaliśmy i usunęliśmy po jednej umiejętności:

```
$ kubectl apply -f amicontained-pod-securitycontext.yaml
pod/amicontained created
```

Jeszcze raz zajrzymy do dzienników kapsuły, aby obejrzeć wyniki działania narzędzia `amicontained`:

```
$ kubectl logs amicontained
Container Runtime: kube
Has Namespaces:
        pid: true
        user: false
AppArmor Profile: docker-default (enforce)
Capabilities:
        BOUNDING -> chown dac_override fowner fsetid kill setgid setuid setpcap
        net_raw sys_chroot sys_time mknod audit_write setfcap
Seccomp: filtering
Blocked Syscalls (67):
        SYSLOG SETUID SETGID SETPGID SETSID SETREUID SETREGID SETGROUPS
        SETRESUID SETRESGID USELIB USTAT SYSFS VHANGUP PIVOT_ROOT_SYSCTL ACCT
        SETTIMEOFDAY MOUNT UMOUNT2 SWAPON SWAPOFF REBOOT SETHOSTNAME
        SETDOMAINNAME IOPL IOPERM CREATE_MODULE INIT_MODULE DELETE_MODULE
        GET_KERNEL_SYMS QUERY_MODULE QUOTACTL NFSSERVCTL GETPMSG PUTPMSG
        AFS_SYSCALL TUXCALL SECURITY LOOKUP_DCOOKIE VSERVER MBIND SET_MEMPOLICY
        GET_MEMPOLICY KEXEC_LOAD ADD_KEY REQUEST_KEY KEYCTL MIGRATE_PAGES
        FUTIMESAT UNSHARE MOVE_PAGES PERF_EVENT_OPEN FANOTIFY_INIT
        NAME_TO_HANDLE_AT OPEN_BY_HANDLE_AT SETNS PROCESS_VM_READV
        PROCESS_VM_WRITEV KCMP FINIT_MODULE KEXEC_FILE_LOAD BPF USERFAULTFD
        PKEY_MPROTECT PKEY_ALLOC PKEY_FREE
Looking for Docker.sock
```

Wyzwania związane z kontekstem zabezpieczeń

Jak widać, posługiwanie się kontekstem zabezpieczeń SecurityContext jest dość skomplikowane i nie jest łatwo zastosować podstawowego zestawu środków bezpieczeństwa przez bezpośrednie skonfigurowanie wszystkich pól każdej kapsuły. Tworzenie profili i kontekstów AppArmor, Seccomp i SELinux oraz zarządzanie nimi jest trudne i można przy tym popełnić błąd. Błędy uniemożliwiają aplikacji spełnianie swojej funkcji. Istnieje kilka narzędzi pozwalających na wygenerowanie

z działającej kapsuły profilu Seccomp, który następnie można zastosować przy użyciu SecurityContext. Jednym z takich projektów jest Security Profiles Operator (*https://oreil.ly/grPCN*), który ułatwia generowanie profili Seccomp i zarządzanie nimi. Teraz przyjrzymy się innym interfejsom API zabezpieczeń, które pozwalają na spójne zastosowanie kontekstu SecurityContext w całym klastrze.

Pod Security

Wiesz już, jak zarządzać środkami bezpieczeństwa kapsuł i kontenerów za pomocą SecurityContext, więc teraz pokażemy Ci, jak zastosować zbiór wartości kontekstu zabezpieczeń na większą skalę. Kubernetes ma obecnie wycofywany interfejs API PodSecurityPolicy (PSP), który umożliwia zarówno walidację, jak i mutację. **Walidacja** nie pozwoli na utworzenie zasobów Kubernetes, jeśli nie będą miały zastosowanego określonego kontekstu SecurityContext. **Mutacja** natomiast zmienia zasoby Kubernetes i stosuje określony SecurityContext na podstawie kryteriów pochodzących od PSP. Zważywszy na to, że interfejs PSP jest wycofywany i zostanie usunięty w Kubernetes v1.25, nie opisujemy go szczegółowo, za to więcej miejsca poświęcamy jego następcy — Pod Security. Jedną z najważniejszych różnic między Pod Security a poprzednim API jest to, że Pod Security przeprowadza tylko walidację. Jeśli chcesz dowiedzieć się więcej o mutacji, przeczytaj rozdział 20.

Czym jest Pod Security

Pod Security umożliwia deklarowanie różnych profili bezpieczeństwa dla kapsuł. Są one nazywane standardami Pod Security (ang. *Pod Security Standards*) oraz stosowane na poziomie przestrzeni nazw. Pod Security Standards to kolekcja pól dotyczących bezpieczeństwa w specyfikacji kapsuły (wliczając między innymi SecurityContext) i powiązanych z nimi wartości. Istnieją trzy różne standardy obejmujące zakres od ścisłego do swobodnego poziomu zabezpieczeń. Idea jest taka, że można zastosować ogólny zestaw zabezpieczeń do wszystkich kapsuł w danej przestrzeni nazw. Poniżej znajduje się opis trzech wspomnianych standardów zabezpieczeń:

Baseline

Najczęściej używana eskalacja uprawnień ułatwiająca wdrażanie.

Restricted

Wysoce restrykcyjny standard uwzględniający najlepsze praktyki w zakresie bezpieczeństwa. Może spowodować awarię obciążenia.

Privileged

Otwarty i nieograniczony.

Interfejs Pod Security w Kubernetes v1.23 znajduje się w fazie beta i może ulec zmianom.

Każdy standard Pod Security definiuje listę pól w specyfikacji kapsuły oraz ich dozwolone wartości. Poniżej wymieniliśmy kilka pól obejmowanych przez te standardy:

- `spec.securityContext`
- `spec.containers[*].securityContext`
- `spec.containers[*].ports`
- `spec.volumes[*].hostPath`

Kompletna lista pól ze wszystkich standardów Pod Security znajduje się w oficjalnej dokumentacji (*https://oreil.ly/xPK2p*).

Każdy standard jest stosowany do przestrzeni nazw za pomocą określonego trybu. Są trzy tryby, do których zasady mogą zostać odniesione:

Enforce
: Wszystkie kapsuły łamiące zasady zostaną odrzucone.

Warn
: Kapsuły łamiące zasady nie zostaną odrzucone, ale zostanie wyświetlone ostrzeżenie.

Audit
: Kapsuły łamiące zasady będą generować komunikat audytowy w dzienniku audytów.

Stosowanie standardów Pod Security

Standardy Pod Security stosuje się do przestrzeni nazw za pomocą etykiet w następujący sposób:

- Wymagane: `pod-security.kubernetes.io/<TRYB>: <POZIOM>`.
- Opcjonalne: `pod-security.kubernetes.io/<TRYB>-version: <WERSJA>` (domyślnie najnowsza).

Przestrzeń nazw na listingu 19.4 pokazuje, jak zastosować tryb `enforce` w jednym standardzie (w tym przypadku `baseline`) oraz tryby `audit` i `warn` w innym (`restricted`). Wykorzystanie różnych trybów umożliwia wdrożenie zasad o niższym poziomie zabezpieczeń oraz przeprowadzenie audytu w celu wykrycia, które obciążenia łamią reguły standardu za pomocą bardziej rygorystycznych zasad. Następnie można rozwiązać znalezione problemy z zabezpieczeniami przed wdrożeniem bardziej restrykcyjnego standardu. Ponadto można przypisać tryb do konkretnej wersji, na przykład v1.22. To umożliwia zmianę standardów zasad z każdą wersją Kubernetes oraz pozwala na odniesienie się do określonej wersji. Na listingu 19.4 ustawiliśmy standard `baseline` na tryb `enforce`, a standardy `warn` i `audit` — na tryb `restricted`. Wszystkie tryby są powiązane z wersją v1.22 standardu.

Listing 19.4. baseline-ns.yaml

```
apiVersion: v1
kind: Namespace
metadata:
    name: baseline-ns
    labels:
        pod-security.kubernetes.io/enforce: baseline
        pod-security.kubernetes.io/enforce-version: v1.22
```

```
        pod-security.kubernetes.io/audit: restricted
        pod-security.kubernetes.io/audit-version: v1.22
        pod-security.kubernetes.io/warn: restricted
        pod-security.kubernetes.io/warn-version: v1.22
```

Pierwsze wdrożenie zasad może być przytłaczającym zadaniem. Na szczęście interfejs Pod Security pozwala na sprawdzenie w łatwy sposób, które istniejące obciążenia łamią zasady standardu, za pomocą polecenia `dry-run`:

```
$ kubectl label --dry-run=server --overwrite ns \
  --all pod-security.kubernetes.io/enforce=baseline
Warning: kuard: privileged
namespace/default labeled
namespace/kube-node-lease labeled
namespace/kube-public labeled
Warning: kube-proxy-vxjwb: host namespaces, hostPath volumes, privileged
Warning: kube-proxy-zxqzz: host namespaces, hostPath volumes, privileged
Warning: kube-apiserver-kind-control-plane: host namespaces, hostPath volumes
Warning: etcd-kind-control-plane: host namespaces, hostPath volumes
Warning: kube-controller-manager-kind-control-plane: host namespaces, ...
Warning: kube-scheduler-kind-control-plane: host namespaces, hostPath volumes
namespace/kube-system labeled
namespace/local-path-storage labeled
```

To polecenie sprawdza wszystkie kapsuły w klastrze Kubernetes pod kątem standardu Pod Security `baseline` i zgłasza w wyniku wszystkie przypadki złamania zasad jako ostrzeżenia.

Obejrzyjmy Pod Security w akcji. Utwórz plik o nazwie *baseline-ns.yaml* o zawartości pokazanej na listingu 19.5.

Listing 19.5. baseline-ns.yaml

```
apiVersion: v1
kind: Namespace
metadata:
    name: baseline-ns
    labels:
        pod-security.kubernetes.io/enforce: baseline
        pod-security.kubernetes.io/enforce-version: v1.22
        pod-security.kubernetes.io/audit: restricted
        pod-security.kubernetes.io/audit-version: v1.22
        pod-security.kubernetes.io/warn: restricted
        pod-security.kubernetes.io/warn-version: v1.22

        $ kubectl apply -f baseline-ns.yaml
        namespace/baseline-ns created
```

Utwórz plik o nazwie *kuard-pod.yaml* o zawartości pokazanej na listingu 19.6.

Listing 19.6. kuard-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
    name: kuard
    labels:
    app: kuard
spec:
```

```
    containers:
      - image: gcr.io/kuar-demo/kuard-amd64:blue
        name: kuard
        ports:
          - containerPort: 8080
            name: http
            protocol: TCP
```

Utwórz kapsułę i wyświetl wynik za pomocą następującego polecenia:

```
$ kubectl apply -f kuard-pod.yaml --namespace baseline-ns
Warning: would violate "v1.22" version of "restricted" PodSecurity profile:
allowPrivilegeEscalation != false (container "kuard" must set
securityContext.allowPrivilegeEscalation=false), unrestricted capabilities (container
"kuard" must set securityContext.capabilities.drop=["ALL"]), runAsNonRoot != true (pod or
container "kuard" must set securityContext.runAsNonRoot=true), seccompProfile (pod or
container "kuard" must set securityContext.seccompProfile.type to "RuntimeDefault" or
"Localhost")
pod/kuard created
```

W tych wynikach widać, że kapsuła została pomyślnie utworzona, ale złamała standard `restricted` Pod Security. W wynikach są podane szczegóły, dzięki czemu można poradzić sobie z tym problemem. Ponadto otrzymaliśmy komunikat w dzienniku audytów serwera API, ponieważ skonfigurowaliśmy tryb `audit`:

```
{"kind":"Event","apiVersion":"audit.k8s.io/v1","level":"Metadata","auditID":"...
```

Użycie interfejsu Pod Security to doskonały sposób na zarządzanie poziomem bezpieczeństwa obciążeń przez zastosowanie zasad w obrębie przestrzeni nazw i umożliwienie tworzenia kapsuł tylko w sytuacji, gdy nie łamią tych zasad. Ten elastyczny API zawiera różne gotowe zasady, od bardzo swobodnych po wyjątkowo restrykcyjne, oraz narzędzia ułatwiające wdrażanie zmian zasad bez ryzyka wywołania awarii obciążeń.

Zarządzanie kontami usług

Konta usług to zasoby Kubernetes nadające tożsamość obciążeniom, które działają w kapsułach. Za pomocą RBAC można kontrolować, przez API Kubernetes, do jakich zasobów dana tożsamość ma dostęp. Więcej informacji na ten temat znajduje się w rozdziale 14. Jeśli Twoja aplikacja nie wymaga dostępu do API Kubernetes, wyłącz dostęp zgodnie z zasadą minimalnych uprawnień. Kubernetes tworzy domyślne konto usług w każdej przestrzeni, które zostaje automatycznie ustawione jako konto usług dla wszystkich kapsuł. Zawiera ono token, który jest montowany w każdej kapsule i używany w celu uzyskiwania dostępu do API Kubernetes. Aby to wyłączyć, należy dodać ustawienie `automountServiceAccountToken: false` do konfiguracji konta usług. Na listingu 19.7 pokazaliśmy, jak to zrobić dla domyślnego konta usług. Czynność tę należy wykonać w każdej przestrzeni nazw.

Listing 19.7. service-account.yaml

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: default
automountServiceAccountToken: false
```

Konta usług są często pomijane przy konfigurowaniu zabezpieczeń kapsuł. Jednak umożliwiają one bezpośrednie używanie API Kubernetes i bez odpowiedniej kontroli RBAC mogą pozwolić hakerowi na włamanie. Dlatego należy wiedzieć, jak ograniczyć dostęp do API przez wprowadzenie prostej zmiany w sposobie posługiwania się tokenami kont usług.

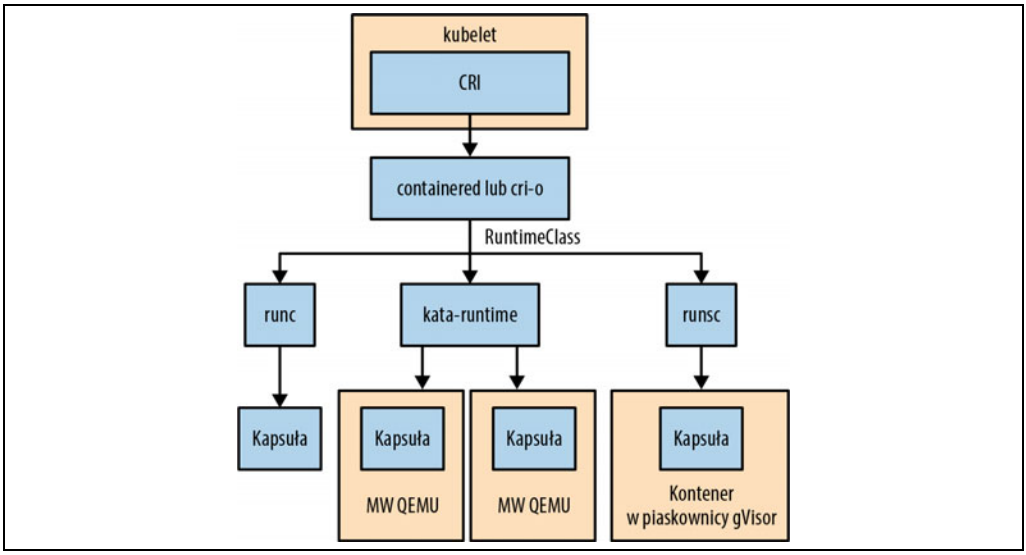
Kontrola dostępu oparta na rolach

W rozdziale na temat zabezpieczeń kapsuł nie możemy pominąć tematu kontroli opartej na rolach Kubernetes (RBAC). Wszystkie potrzebne informacje o RBAC znajdują się w rozdziale 14. i można je wykorzystać w celu uzupełnienia zabezpieczeń swojego obciążenia.

RuntimeClass

Kubernetes współpracuje ze środowiskiem wykonawczym kontenera w systemie operacyjnym węzła przez *Container Runtime Interface* (CRI). Utworzenie i standaryzacja tego interfejsu umożliwiły powstanie ekosystemu środowisk wykonawczych kontenerów. Środowiska te mogą różnić się poziomem izolacji, co daje pewniejsze gwarancje bezpieczeństwa w zależności od sposobu ich implementacji. Projekty takie jak Kata Containers, Firecracker czy gVisor bazują na różnych mechanizmach izolacji, od wirtualizacji zagnieżdżonej, po bardziej zaawansowane filtrowanie wywołań systemowych. Te gwarancje bezpieczeństwa i izolacji dają administratorowi Kubernetes elastyczność w zakresie zezwalania użytkownikom na wybór środowiska wykonawczego na podstawie typu obciążenia. Na przykład jeśli Twoje obciążenie potrzebuje silniejszych gwarancji bezpieczeństwa, możesz wybrać kapsułę używającą innego środowiska wykonawczego kontenerów.

API RuntimeClass wprowadzono, aby umożliwić wybór środowiska wykonawczego kontenerów. Pozwala on użytkownikowi wybrać jedno środowisko z listy środowisk obsługiwanych w klastrze. Na rysunku 19.1 przedstawiono schemat działania API RuntimeClass.



Rysunek 19.1. Diagram RuntimeClass

Różne klasy RuntimeClass muszą być skonfigurowane przez administratora klastra i mogą wymagać, aby obciążenie miało określone selektory `nodeSelector` lub tolerancje (`toleration`) zaplanowane dla odpowiedniego węzła.

Aby użyć interfejsu RuntimeClass, można zdefiniować `runtimeClassName` w specyfikacji kapsuły. Na listingu 19.8 jest pokazana przykładowa kapsuła określająca RuntimeClass.

Listing 19.8. kuard-pod-runtimeclass.yaml

```
apiVersion: v1
kind: Pod
metadata:
    name: kuard
    labels:
      app: kuard
spec:
    runtimeClassName: firecracker
    containers:
      - image: gcr.io/kuar-demo/kuard-amd64:blue
        name: kuard
        ports:
          - containerPort: 8080
            name: http
            protocol: TCP
```

Interfejs RuntimeClass umożliwia użytkownikom wybranie różnych środowisk wykonawczych kontenerów, które mogą mieć inne poziomy izolacji zabezpieczeń. Wykorzystanie interfejsu RuntimeClass może pomóc w uzupełnieniu zabezpieczeń obciążeń, zwłaszcza jeśli przetwarzają one wrażliwe informacje lub wykonują niezaufany kod.

Network Policy

Kubernetes ma także interfejs API Network Policy, który umożliwia tworzenie zarówno wejściowych, jak i wyjściowych zasad sieciowych dla obciążeń. Zasady te są konfigurowane przy użyciu etykiet, które pozwalają na wybór kapsuł oraz definiują dopuszczalny sposób komunikacji z innymi kapsułami i punktami końcowymi. Zasada bezpieczeństwa taka jak Ingress nie jest dostarczana z powiązanym kontrolerem Kubernetes. To znaczy, że można tworzyć zasoby Network Policy, ale jeśli nie zainstaluje się kontrolera włączającego się w reakcji na utworzenie tych zasobów, to nie zostaną one wyegzekwowane. Zasoby Network Policy są implementowane przez wtyczki sieciowe, takie jak Calico, Cilium i Weave Net.

Zasób Network Policy znajduje się w przestrzeni nazw oraz jest podzielony na sekcje `podSelector`, `policyTypes`, `ingress` oraz `egress`, z których wymagane jest tylko pole `podSelector`. Jeśli to pole nie zostanie zdefiniowane, polityka odpowiada wszystkim kapsułom w przestrzeni nazw. Może ono także zawierać sekcję `matchLabels`, która działa tak samo, jak zasób `Service`, umożliwiając dodanie zbioru etykiet odpowiadających określonemu zbiorowi kapsuł.

Z używaniem interfejsu Network Policy wiąże się kilka specyficznych kwestii, o których należy wiedzieć. Jeśli kapsule odpowiada jakikolwiek zasób Network Policy, to wszelka komunikacja wejściowa lub wyjściowa musi być wprost zdefiniowana albo zostanie zablokowana. Gdy kapsule

odpowiada kilka zasobów Network Policy, to zasady się sumują. Jeżeli kapsule nie odpowiada żaden zasób Network Policy, to ruch jest dozwolony. Takie rozwiązanie ma na celu ułatwienie wdrażania nowych obciążeń. Jeśli jednak chcesz domyślnie blokować wszelki ruch, utwórz domyślną regułę odmowy, którą można zastosować do przestrzeni nazw. Na listingu 19.9 pokazaliśmy przykład takiej domyślnej reguły.

Listing 19.9. networkpolicy-default-deny.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
    name: default-deny-ingress
spec:
    podSelector: {}
    policyTypes:
    - Ingress
```

Przeanalizujemy przykładowy zestaw zasad sieciowych, aby pokazać, jak za ich pomocą zabezpieczyć obciążenia. Najpierw utwórz przestrzeń nazw do testów, korzystając z następującego polecenia:

```
$ kubectl create ns kuard-networkpolicy
namespace/kuard-networkpolicy created
```

Przygotuj plik o nazwie *kuard-pod.yaml* o zawartości przedstawionej na listingu 19.10.

Listing 19.10. kuard-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
    name: kuard
    labels:
        app: kuard
spec:
    containers:
      - image: gcr.io/kuar-demo/kuard-amd64:blue
        name: kuard
        ports:
          - containerPort: 8080
            name: http
            protocol: TCP
```

Utwórz kapsułę `kuard` w przestrzeni nazw `kuard-networkpolicy`:

```
$ kubectl apply -f kuard-pod.yaml \
  --namespace kuard-networkpolicy
pod/kuard created
```

Udostępnij kapsułę `kuard` jako usługę:

```
$ kubectl expose pod kuard --port=80 --target-port=8080 \
  --namespace kuard-networkpolicy
pod/kuard created
```

Teraz możesz użyć polecenia `kubectl run`, aby uruchomić kapsułę do przetestowania jako naszego zasobu oraz w celu sprawdzenia dostępu do kapsuły `kuard` bez stosowania jakiejkolwiek polityki Network Policy:

```
$ kubectl run test-source --rm -ti --image busybox /bin/sh \
  --namespace kuard-networkpolicy
If you don't see a command prompt, try pressing enter.
/ # wget -q kuard -O -
<!doctype html>

<html lang="en">
<head>
  <meta charset="utf-8">

  <title><KUAR Demo></title>
...
```

Udało Ci się nawiązać połączenie z kapsułą `kuard` z kapsuły testowej. Teraz zastosuj domyślną zasadę odmowy i ponownie wykonaj test. Utwórz plik o nazwie *networkpolicy-default-deny.yaml* o zawartości pokazanej na listingu 19.11.

Listing 19.11. networkpolicy-default-deny.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
    name: default-deny-ingress
spec:
    podSelector: {}
    policyTypes:
    - Ingress
```

Zastosuj domyślną sieciową zasadę odmowy:

```
$ kubectl apply -f networkpolicy-default-deny.yaml \
  --namespace kuard-networkpolicy
networkpolicy.networking.k8s.io/default-deny-ingress created
```

W dalszej kolejności przetestuj dostęp do kapsuły `kuard` z kapsuły `test-source`:

```
$ kubectl run test-source --rm -ti --image busybox /bin/sh \
  --namespace kuard-networkpolicy
If you don't see a command prompt, try pressing enter.
/ # wget -q --timeout=5 kuard -O -
wget: download timed out
```

Domyślna zasada odmowy Network Policy spowodowała, że nie masz już dostępu do kapsuły `kuard` z kapsuły `test-source`. Utwórz plik o nazwie *networkpolicy-kuard-allow-test-source.yaml* o treści pokazanej na listingu 19.12.

Listing 19.12. networkpolicy-kuard-allow-test-source.yaml

```
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
    name: access-kuard
spec:
```

```
    podSelector:
        matchLabels:
            app: kuard
    ingress:
        - from:
            - podSelector:
                matchLabels:
                    run: test-source
```

Zastosuj zasadę Network Policy:

```
$ kubectl apply \
  -f code/chapter-security/networkpolicy-kuard-allow-test-source.yaml \
  --namespace kuard-networkpolicy
networkpolicy.networking.k8s.io/access-kuard created
```

Ponownie zweryfikuj, czy kapsuła `test-source` ma dostęp do kapsuły `kuard`:

```
$ kubectl run test-source --rm -ti --image busybox /bin/sh \
  --namespace kuard-networkpolicy
If you don't see a command prompt, try pressing enter.
/ # wget -q kuard -O -
<!doctype html>

<html lang="en">
<head>
  <meta charset="utf-8">

  <title><KUAR Demo></title>
...
```

Wyczyść przestrzeń nazw przez wykonanie następującego polecenia:

```
$ kubectl delete namespace kuard-networkpolicy
namespace "kuard-networkpolicy" deleted
```

Zastosowanie zasady Network Policy powoduje dodanie kolejnej warstwy zabezpieczeń obciążeń oraz stanowi realizację reguł dogłębnej obrony i minimalnych uprawnień.

Siatka usług

Poziom bezpieczeństwa obciążenia można zwiększyć także za pomocą siatki usług. Siatki takie udostępniają zasady dostępu, które umożliwiają konfigurację rozpoznających protokół reguł opartych na usługach. Na przykład nasza zasada dostępu może deklarować, że usługa A łączy się z usługą B przez port HTTPS 443. Ponadto siatki usług zazwyczaj implementują wzajemny TLS w całej komunikacji między usługami, dzięki czemu jest ona szyfrowana oraz są weryfikowane tożsamości usług. Jeśli chcesz dowiedzieć się więcej na temat siatek usług oraz ich zastosowania w zabezpieczeniach obciążeń, przeczytaj rozdział 15.

Narzędzia do weryfikacji zabezpieczeń

Istnieje kilka narzędzi open source, które umożliwiają wykonywanie pakietów testów zabezpieczeń klastra Kubernetes, aby sprawdzić, czy konfiguracja spełnia określony zestaw wymagań. Jedno

z nich nazywa się `kube-bench` (*https://oreil.ly/TnUlm*). Za jego pomocą można przeprowadzać testy CIS (*https://oreil.ly/VvUe5*) dla Kubernetes. Narzędzia takie jak `kube-bench` wykonujące testy CIS nie są specjalnie nastawione na bezpieczeństwo kapsuł, ale potrafią wykryć błędy konfiguracji klastra oraz pomagają w rozwiązywaniu problemów. Narzędzie `kube-bench` można uruchomić za pomocą poniższego polecenia:

```
$ kubectl apply -f https://raw.githubusercontent.com/aquasecurity/kube-bench...
job.batch/kube-bench created
```

Następnie można przejrzeć wyniki testu i zalecane rozwiązania w dziennikach kapsuły:

```
$ kubectl logs job/kube-bench
[INFO] 4 Worker Node Security Configuration
[INFO] 4.1 Worker Node Configuration Files
[PASS] 4.1.1 Ensure that the kubelet service file permissions are set to 644...
[PASS] 4.1.2 Ensure that the kubelet service file ownership is set to root ...
[PASS] 4.1.3 If proxy kubeconfig file exists ensure permissions are set to ...
[PASS] 4.1.4 Ensure that the proxy kubeconfig file ownership is set to root ...
[PASS] 4.1.5 Ensure that the --kubeconfig kubelet.conf file permissions are ...
[PASS] 4.1.6 Ensure that the --kubeconfig kubelet.conf file ownership is set...
[PASS] 4.1.7 Ensure that the certificate authorities file permissions are ...
[PASS] 4.1.8 Ensure that the client certificate authorities file ownership ...
[PASS] 4.1.9 Ensure that the kubelet --config configuration file has permiss...
[PASS] 4.1.10 Ensure that the kubelet --config configuration file ownership ...
[INFO] 4.2 Kubelet
[PASS] 4.2.1 Ensure that the anonymous-auth argument is set to false (Automated)
[PASS] 4.2.2 Ensure that the --authorization-mode argument is not set to ...
[PASS] 4.2.3 Ensure that the --client-ca-file argument is set as appropriate...
[PASS] 4.2.4 Ensure that the --read-only-port argument is set to 0 (Manual)
[PASS] 4.2.5 Ensure that the --streaming-connection-idle-timeout argument is...
[FAIL] 4.2.6 Ensure that the --protect-kernel-defaults argument is set to ...
[PASS] 4.2.7 Ensure that the --make-iptables-util-chains argument is set to ...
[PASS] 4.2.8 Ensure that the --hostname-override argument is not set (Manual)
[WARN] 4.2.9 Ensure that the --event-qps argument is set to 0 or a level ...
[WARN] 4.2.10 Ensure that the --tls-cert-file and --tls-private-key-file arg...
[PASS] 4.2.11 Ensure that the --rotate-certificates argument is not set to ...
[PASS] 4.2.12 Verify that the RotateKubeletServerCertificate argument is set...
[WARN] 4.2.13 Ensure that the Kubelet only makes use of Strong Cryptographic...

== Remediations node ==
4.2.6 If using a Kubelet config file, edit the file to set protectKernel...
If using command line arguments, edit the kubelet service file
/etc/systemd/system/kubelet.service.d/10-kubeadm.conf on each worker node and
set the below parameter in KUBELET_SYSTEM_PODS_ARGS variable.
--protect-kernel-defaults=true
Based on your system, restart the kubelet service. For example:
systemctl daemon-reload
systemctl restart kubelet.service

4.2.9 If using a Kubelet config file, edit the file to set eventRecordQPS...
If using command line arguments, edit the kubelet service file
/etc/systemd/system/kubelet.service.d/10-kubeadm.conf on each worker node and
set the below parameter in KUBELET_SYSTEM_PODS_ARGS variable.
Based on your system, restart the kubelet service. For example:
systemctl daemon-reload
systemctl restart kubelet.service
...
```

Za pomocą takich narzędzi jak `kube-bench` wykonujących testy CIS można sprawdzić, czy klaster Kubernetes spełnia wymogi bezpieczeństwa. Ponadto dostarczają one podpowiedzi, jak rozwiązać ewentualne problemy.

Bezpieczeństwo obrazów

Innym ważnym aspektem związanym z bezpieczeństwem kapsuł jest zapewnienie bezpieczeństwa kodu i aplikacji w kapsule. Zabezpieczanie kodu aplikacji to skomplikowany temat, którego omówienie wykracza poza ramy tego rozdziału. Podstawowe zasady bezpieczeństwa kontenera obejmują **skanowanie statyczne** rejestru obrazu pod kątem znanych luk w kodzie. Ponadto należy używać narzędzia **skanującego w trakcie działania** obrazu, które będzie znajdowało luki odkryte po jego uruchomieniu oraz będzie szukało potencjalnie szkodliwej działalności, na przykład włamań. Istnieje wiele narzędzi do skanowania, zarówno typu open source, jak i opracowanych przez różne firmy. Ponadto należy minimalizować zawartość obrazu kontenera, aby pozbyć się niepotrzebnych zależności i zredukować szum podczas skanowania. W końcu dbanie o bezpieczeństwo obrazów to kolejny dobry powód, aby zainwestować w ciągłe dostarczanie, które umożliwia szybkie wprowadzanie poprawek i wdrażanie obrazów po wykryciu luk bezpieczeństwa.

Podsumowanie

W tym rozdziale opisaliśmy wiele różnych API i zasobów zabezpieczeń, za pomocą których można zwiększyć poziom bezpieczeństwa obciążeń. Przestrzegając zasad dogłębnej obrony i minimalnych uprawnień, można stopniowo poprawiać bezpieczeństwo klastra Kubernetes. Nigdy nie jest za późno, aby zacząć stosować lepsze zabezpieczenia, a w tym rozdziale znajdują się wszystkie informacje potrzebne do tego, by zrozumieć środki bezpieczeństwa dostępne w Kubernetes.

ROZDZIAŁ 20.

Polityki i zarządzanie klastrami Kubernetes

W poprzednich rozdziałach opisaliśmy wiele różnych typów zasobów Kubernetes, z których każdy ma określone przeznaczenie. Liczba zasobów w klastrze Kubernetes bardzo szybko rośnie z paru (w przypadku pojedynczej aplikacji mikrousługowej) do setek i tysięcy (w przypadku kompletnej aplikacji rozproszonej). W kontekście klastra produkcyjnego nietrudno sobie wyobrazić, jakie wyzwania stoją przed osobą, która musi zarządzać tysiącami zasobów.

W tym rozdziale przedstawiamy koncepcje polityki i zarządzania. **Polityka** (ang. *policy*) to zbiór ograniczeń warunków dotyczących sposobu konfiguracji zasobów Kubernetes. **Zarządzanie** (ang. *governance*) to weryfikacja i egzekwowanie polityk organizacyjnych dla wszystkich zasobów wdrożonych w klastrze Kubernetes, na przykład sprawdzanie, czy używają one obecnych najlepszych praktyk, są zgodne z polityką bezpieczeństwa lub z firmowymi konwencjami. Twój zestaw narzędzi musi być elastyczny i skalowalny, aby wszystkie zasoby istniejące w klastrze spełniały warunki polityk zdefiniowanych w Twojej organizacji.

Dlaczego polityka i zarządzanie są ważne

W Kubernetes jest wiele różnych typów polityk. Na przykład NetworkPolicy umożliwia określenie usług sieciowych i punktów końcowych, z którymi może się łączyć kapsuła. `PodSecurityPolicy` umożliwia precyzyjną kontrolę nad elementami zabezpieczeń kapsuły. Obu można używać do konfiguracji sieci lub środowisk wykonawczych kontenerów.

Użytkownik może jednak chcieć wymusić przestrzeganie polityki, zanim zasoby Kubernetes zostaną w ogóle utworzone. Rozwiązaniem są właśnie polityka i zarządzanie. Być może zadajesz sobie pytanie, czy nie do tego służy kontrola dostępu oparta na rolach. Jak przekonasz się w tym rozdziale, RBAC nie jest wystarczająco precyzyjna, aby uniemożliwić ustawianie niektórych pól w zasobach.

Poniżej znajduje się kilka przykładów polityk, które są często konfigurowane przez administratorów klastrów:

- Wszystkie kontenery *muszą* pochodzić wyłącznie z określonego rejestru kontenerów.
- Wszystkie kapsuły *muszą* mieć określone nazwę działu i dane kontaktowe.
- Wszystkie kapsuły *muszą* mieć ustawione limity wykorzystania procesora i pamięci.

- Wszystkie nazwy hostów Ingress *muszą* być unikalne na przestrzeni klastra.
- Dana usługa *nie może* być dostępna w internecie.
- Kontenery *nie mogą* nasłuchiwać na uprzywilejowanych portach.

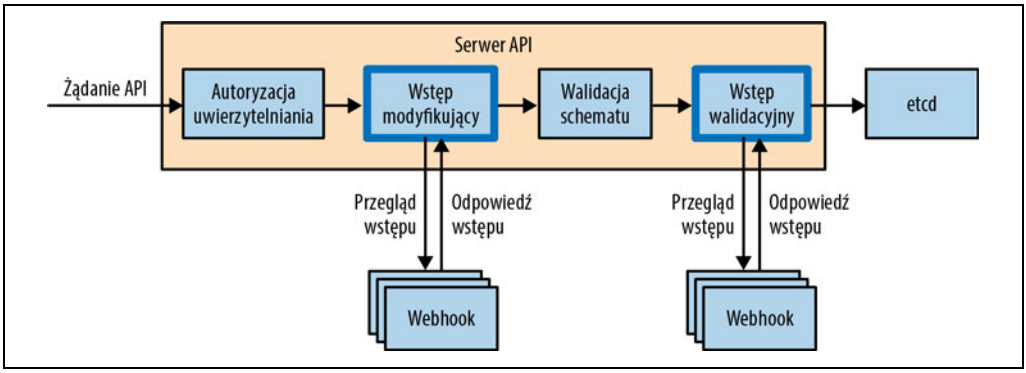
Administratorzy klastrów mogą też chcieć przeprowadzać audyty istniejących zasobów w klastrze, sprawdzać polityki „na sucho", a nawet modyfikować zasoby na podstawie zestawów warunków — na przykład przez nadanie etykiet kapsule, jeśli ich nie ma.

Dla administratorów klastrów bardzo ważna jest możliwość definiowania polityk i przeprowadzania audytów zgodności bez przeszkadzania programistom we wdrażaniu aplikacji w Kubernetes. Jeśli programiści tworzą zasoby niespełniające warunków, niezbędny jest system dostarczający informacji zwrotnych i wskazówek potrzebnych, aby to zmienić.

Zobaczmy, jak zastosować polityki i zarządzanie przez wykorzystanie podstawowej rozszerzalności komponentów Kubernetes.

Przepływ wstępu

Aby zrozumieć, jak polityka i zarządzanie zapewniają zgodność zasobów, jeszcze zanim zostaną one utworzone w klastrze Kubernetes, należy przeanalizować przepływ żądań przez serwer API Kubernetes — przedstawiliśmy go na rysunku 20.1. Skupimy się na wstępie modyfikującym, wstępie walidacyjnym oraz elementach webhook.



Rysunek 20.1. Przepływ żądań API przez serwer API Kubernetes

Kontrolery wstępu działają jako przepływy żądań API przez serwer API Kubernetes i są używane do modyfikacji lub do walidacji zasobu żądania API przed jego zapisaniem w magazynie. Kontrolery wstępu modyfikującego umożliwiają zmienianie zasobu. Kontrolery wstępu walidacyjnego tego nie robią. Istnieje wiele typów kontrolerów wstępu. W tym rozdziale skupiamy się na elementach webhook wstępu, które są konfigurowalne dynamicznie. Umożliwiają administratorowi klastra konfigurację punktu końcowego, do którego serwer API może wysyłać żądania do ewaluacji, przez utworzenie zasobu `MutatingWebhookConfiguration` lub `ValidatingWebhookConfiguration`. Walidacyjny element webhook odpowie dyrektywą „wstępu" lub „odmowy", aby poinformować serwer, czy ma zapisać zasób w magazynie.

Polityka i zarządzanie z narzędziem Gatekeeper

Teraz opowiemy, jak konfigurować polityki oraz zapewniać, aby zasoby Kubernetes ich przestrzegały. Projekt Kubernetes nie dostarcza żadnych kontrolerów włączających politykę i zarządzanie, ale istnieją rozwiązania open source tego typu. W tym podrozdziale skupimy się na takim projekcie o nazwie Gatekeeper (*https://oreil.ly/u0deR*).

Gatekeeper to kontroler polityki w Kubernetes, który ewaluuje zasoby na podstawie zdefiniowanej polityki i określa, czy zezwolić na utworzenie lub modyfikację danego zasobu. Te ewaluacje odbywają się na serwerze podczas przepływu żądania API przez serwer API Kubernetes, co oznacza, że każdy klaster ma jeden punkt przetwarzania. Przetwarzanie ewaluacji polityk na serwerze pozwala na zainstalowanie narzędzia Gatekeeper w istniejących klastrach Kubernetes bez zmieniania narzędzi programistycznych, przepływów pracy i potoków ciągłego dostarczania.

Gatekeeper używa **niestandardowych definicji zasobów** (ang. *custom resource definition* — CRD) do zdefiniowania nowego zestawu zasobów Kubernetes specyficznych dla jego konfiguracji, co umożliwia administratorom klastrów operowanie Gatekeeperem za pomocą znanych narzędzi, takich jak `kubectl`. Ponadto dostarcza użytkownikowi w czasie rzeczywistym cennych informacji zwrotnych na temat tego, dlaczego dany zasób został odrzucony oraz jak rozwiązać ten problem. Te specyficzne dla Gatekeepera niestandardowe zasoby mogą być przechowywane w kontroli zasobu i można nimi zarządzać przy użyciu przepływów pracy GitOps.

Gatekeeper wykonuje też *modyfikacje* (na podstawie zdefiniowanych warunków) i audyty zasobów. Ma wiele opcji konfiguracji oraz umożliwia precyzyjne wskazanie, jakie zasoby sprawdzić i w których przestrzeniach nazw to zrobić.

Czym jest Open Policy Agent

Rdzeń narzędzia Gatekeeper stanowi Open Policy Agent (*https://oreil.ly/nbR5d*), chmurowy silnik polityk open source, który jest rozszerzalny i umożliwia przenoszenie polityk między różnymi aplikacjami. Open Policy Agent (OPA) odpowiada za wykonywanie wszystkich ewaluacji polityk i wydawanie zezwolenia lub zakazu. Daje to Gatekeeperowi dostęp do ekosystemu narzędzi do pracy z politykami, takich jak Conftest (*https://oreil.ly/ElWYE*), co umożliwia pisanie testów polityk i implementowanie ich w potokach ciągłej integracji przed wdrożeniem.

Open Policy Agent do definiowania polityk używa specjalnego języka zapytań o nazwie Rego (*https://oreil.ly/Ar55f*). Jednym z podstawowych założeń Gatekeepera jest przejęcie wewnętrznych mechanizmów Rego od administratora klastra i prezentacja strukturalnego API w formie CRD Kubernetes w celu tworzenia i stosowania polityk. To umożliwia nam udostępnianie parametryzowanych polityk w organizacji i społeczności. Projekt Gatekeeper obejmuje bibliotekę polityk przeznaczoną wyłącznie do tego celu (została opisana w dalszej części tego rozdziału).

Instalacja Gatekeepera

Przed rozpoczęciem konfigurowania polityk należy zainstalować Gatekeepera. Jego komponenty działają jako kapsuły w przestrzeni nazw `gatekeeper-system` i konfigurują kontroler wstępu w postaci elementu webhook.

Nie instaluj Gatekeepera w klastrze Kubernetes, jeśli nie wiesz, jak bezpiecznie utworzyć i wyłączyć politykę. Ponadto przejrzyj kod YAML instalacji przed zainstalowaniem tego narzędzia, aby upewnić się, że nie masz problemu z tworzonymi przez nie zasobami.

Gatekeepera można zainstalować za pomocą menedżera pakietów Helm:

```
$ helm repo add gatekeeper https://open-policy-agent.github.io/gatekeeper/charts
$ helm install gatekeeper/gatekeeper --name-template=gatekeeper \
  --namespace gatekeeper-system --create
```

Instalacja Gatekeepera wymaga uprawnień administratora klastra, a jej przebieg jest zależny od wersji. Informacje na temat najnowszej wersji znajdują się w dokumentacji narzędzia (*https://oreil.ly/GvLHc*).

Po zakończeniu instalacji sprawdź, czy Gatekeeper działa:

```
$ kubectl get pods -n gatekeeper-system
NAME                                            READY    STATUS    RESTARTS    AGE
gatekeeper-audit-54c9759898-ljwp8               1/1      Running   0           1m
gatekeeper-controller-manager-6bcc7f8fb5-4nbkt  1/1      Running   0           1m
gatekeeper-controller-manager-6bcc7f8fb5-d85rn  1/1      Running   0           1m
gatekeeper-controller-manager-6bcc7f8fb5-f8m8j  1/1      Running   0           1m
```

Za pomocą następującego polecenia można także przejrzeć konfigurację elementu webhook:

```
$ kubectl get validatingwebhookconfiguration -o yaml
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  labels:
    gatekeeper.sh/system: "yes"
  name: gatekeeper-validating-webhook-configuration
webhooks:
- admissionReviewVersions:
  - v1
  - v1beta1
  clientConfig:
    service:
      name: gatekeeper-webhook-service
      namespace: gatekeeper-system
      path: /v1/admit
  failurePolicy: Ignore
  matchPolicy: Exact
  name: validation.gatekeeper.sh
  namespaceSelector:
    matchExpressions:
    - key: admission.gatekeeper.sh/ignore
      operator: DoesNotExist
  rules:
  - apiGroups:
    - '*'
    apiVersions:
    - '*'
    operations:
    - CREATE
```

```
      - UPDATE
      resources:
      - '*'
    sideEffects: None
    timeoutSeconds: 3
        ...
```

W sekcji `rules` powyżej widzimy, że wszystkie zasoby są wysyłane do kontrolera wstępu webhook, który działa jako usługa o nazwie `gatekeeper-webhook-service` w przestrzeni nazw `gatekeeper-system`. Do ewaluacji pod kątem polityki będą brane pod uwagę tylko zasoby z przestrzeni nazw, które nie mają oznaczenia `admission.gatekeeper.sh/ignore`. Pole `failurePolicy` zostało ustawione na `Ignore`, co wskazuje, że jest to **konfiguracja otwarta na niepowodzenie** (ang. *fail open configuration*), czyli jeśli usługa Gatekeeper nie odpowie w ciągu określonego w konfiguracji czasu trzech sekund, żądanie zostanie przepuszczone.

Konfigurowanie polityk

Po zainstalowaniu Gatekeepera możesz zacząć konfigurować polityki. Najpierw przestudiujemy kanoniczny przykład i pokażemy, jak administrator klastra tworzy polityki. Następnie przyjrzymy się tworzeniu zgodnych i niezgodnych zasobów z perspektywy programisty. Później szerzej omówimy każdy krok, abyś mógł lepiej wszystko zrozumieć, oraz pokażemy proces tworzenia przykładowej polityki stanowiącej, że obrazy kontenerów mogą pochodzić tylko z jednego określonego rejestru. Ten przykład bazuje na bibliotece polityk Gatekeepera (*https://oreil.ly/ikfZk*).

Aby skonfigurować politykę, najpierw należy utworzyć niestandardowy zasób o nazwie **szablon ograniczeń** (ang. *constraint template*). Zazwyczaj robi to administrator klastra. Szablon ograniczeń przedstawiony na listingu 20.1 wymaga podania listy repozytoriów kontenerów jako parametrów, których mogą używać zasoby Kubernetes.

Listing 20.1. allowedrepos-constraint-template.yaml

```
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sallowedrepos
  annotations:
    description: Requires container images to begin with a repo string from a
      specified list.
spec:
  crd:
    spec:
      names:
        kind: K8sAllowedRepos
      validation:
        # Schemat pola parameters
        openAPIV3Schema:
          properties:
            repos:
              type: array
              items:
                type: string
targets:
  - target: admission.k8s.gatekeeper.sh
```

```
    rego: |
      package k8sallowedrepos

      violation[{"msg": msg}] {
        container := input.review.object.spec.containers[_]
        satisfied := [good | repo = input.parameters.repos[_] ; good = starts...
        not any(satisfied)
        msg := sprintf("container <%v> has an invalid image repo <%v>, allowed...
    }

    violation[{"msg": msg}] {
      container := input.review.object.spec.initContainers[_]
      satisfied := [good | repo = input.parameters.repos[_] ; good = starts...
      not any(satisfied)
      msg := sprintf("container <%v> has an invalid image repo <%v>, allowed...)
    }
```

Utwórz szablon ograniczeń za pomocą takiego polecenia:

```
$ kubectl apply -f allowedrepos-constraint-template.yaml
constrainttemplate.templates.gatekeeper.sh/k8sallowedrepos created
```

Teraz możesz utworzyć zasób ograniczeń, aby wdrożyć politykę (także jako administrator klastra). Ograniczenie przedstawione na listingu 20.2 przepuszcza wszystkie kontenery z przedrostkiem `gcr.io/kuar-demo/` należące do przestrzeni nazw `default`. Pole `enforcementAction` jest ustawione tak, aby odrzucać (`deny`) niezgodne zasoby.

Listing 20.2. allowedrepos-constraint.yaml

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sAllowedRepos
metadata:
  name: repo-is-kuar-demo
spec:
  enforcementAction: deny
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
    namespaces:
      - "default"
  parameters:
    repos:
      - "gcr.io/kuar-demo/"

$ kubectl create -f allowedrepos-constraint.yaml
k8sallowedrepos.constraints.gatekeeper.sh/repo-is-kuar-demo created
```

Następnym krokiem jest utworzenie kilku kapsuł, aby sprawdzić, czy polityka rzeczywiście działa. Na listingu 20.3 przy użyciu obrazu kontenera, `gcr.io/kuar-demo/kuard-amd64:blue`, jest tworzona kapsuła, która jest zgodna z ograniczeniem zdefiniowanym w poprzednim kroku. Tworzenie zasobu obciążenia to zazwyczaj zadanie programisty odpowiedzialnego za operowanie usługą lub potokiem ciągłego dostarczania.

Listing 20.3. compliant-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard
spec:
  containers:
    - image: gcr.io/kuar-demo/kuard-amd64:blue
      name: kuard
      ports:
        - containerPort: 8080
          name: http
          protocol: TCP

$ kubectl apply -f compliant-pod.yaml
pod/kuard created
```

Co się dzieje, kiedy uzyskamy niezgodną kapsułę? Na listingu 20.4 przy użyciu obrazu kontenera, `nginx`, została utworzona kapsuła, która jest *niezgodna* z ograniczeniem zdefiniowanym przez nas w poprzednim kroku. Tworzenie zasobu obciążenia jest zazwyczaj zadaniem programisty lub potoku ciągłego dostarczania odpowiedzialnego za operowanie usługą. Spójrz na wyniki pokazane na listingu 20.4.

Listing 20.4. noncompliant-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-noncompliant
spec:
  containers:
    - name: nginx
      image: nginx

$ kubectl apply -f noncompliant-pod.yaml
  Error from server ([repo-is-kuar-demo] container <nginx> has an invalid image
  repo <nginx>, allowed repos are ["gcr.io/kuar-demo/"]): error when creating
  "noncompliant-pod.yaml": admission webhook "validation.gatekeeper.sh" denied
  the request: [repo-is-kuar-demo] container <nginx> has an invalid image
  repo <nginx>, allowed repos are ["gcr.io/kuar-demo/"]
```

Na listingu 20.4 widać, że został zwrócony błąd ze szczegółowymi informacjami na temat tego, dlaczego nie utworzono zasobu oraz jak rozwiązać problem. Administratorzy klastra mogą skonfigurować komunikat o błędzie w szablonie ograniczeń.

Jeśli zakresem naszego ograniczenia są kapsuły i utworzymy zasób generujący kapsuły, taki jak `ReplicaSet`, Gatekeeper zwróci błąd. Nie zostanie on jednak zwrócony do nas, użytkowników, tylko do kontrolera próbującego utworzyć kapsułę. Aby przeczytać komunikaty o błędach, otwórz dziennik zdarzeń odpowiedniego zasobu.

Szablony ograniczeń

Po analizie kanonicznego przykładu przyjrzymy się dokładnie szablonowi ograniczeń z listingu 20.1, który pobiera listę repozytoriów kontenerów dozwolonych w zasobach Kubernetes.

Ten szablon ograniczeń ma pola `apiVersion` i `kind`, które należą do niestandardowych zasobów używanych tylko przez Gatekeepera. W sekcji `spec` znajduje się nazwa `K8sAllowedRepos`: zapamiętaj ją, ponieważ użyjesz jej jako rodzaju ograniczeń podczas ich tworzenia. Ponadto zobaczysz schemat definiujący tablicę łańcuchów do skonfigurowania przez administratora klastra. Robi się to poprzez dostarczenie listy dozwolonych rejestrów kontenerów. Dodatkowo mamy definicję polityki `rego` (w sekcji `target`). Ta polityka ewaluuje kontenery i kontenery `initContainer` pod kątem tego, czy nazwa repozytorium kontenerów zaczyna się od wartości dostarczonych przez ograniczenie. Sekcja `msg` definiuje komunikat, który jest wysyłany do użytkownika w przypadku złamania zasad polityki.

Tworzenie ograniczeń

Aby utworzyć egzemplarz polityki, należy zdefiniować ograniczenie wskazujące parametry wymagane przez szablon. Może istnieć wiele ograniczeń odpowiadających rodzajowi określonego szablonu ograniczeń. Przyjrzymy się ograniczeniu, którego użyliśmy na listingu 20.2, przepuszczającemu tylko obrazy kontenerów pochodzące z *gcr.io/kuar-demo/*.

Można zauważyć, że to ograniczenie jest rodzaju `K8sAllowedRepos`, który został zdefiniowany jako część szablonu ograniczeń. Ponadto definiuje ono ustawienie `enforcementAction` o wartości `deny`, co znaczy, że niezgodne zasoby będą odrzucane. Pole `enforcementAction` może mieć także wartości `dryrun` i `warn`. Pierwsza używa funkcji audytu w celu testowania polityk i weryfikacji ich wpływu. Wartość `warn` wysyła do użytkownika ostrzeżenie z powiązanym komunikatem, ale pozwala mu na wykonywanie operacji tworzenia i aktualizacji. Człon `match` definiuje zakres danego ograniczenia, wszystkie kapsuły w przestrzeni nazw `default`. W końcu w szablonie ograniczeń musi być obecna sekcja `parameters` (tablica łańcuchów). Poniżej pokazujemy, co widzi użytkownik, gdy ustawimy `enforcementAction` na `warn`:

```
$ kubectl apply -f noncompliant-pod.yaml
Warning: [repo-is-kuar-demo] container <nginx> has an invalid image repo...
pod/nginx-noncompliant created
```

Ograniczenia są egzekwowane tylko w odniesieniu do zdarzeń `CREATE` i `UPDATE` zasobów. Jeśli masz już obciążenia działające w klastrze, Gatekeeper dokona ich ewaluacji dopiero po zdarzeniu `CREATE` lub `UPDATE`.

Oto prawdziwy przykład: powiedzmy, że tworzymy politykę zezwalającą tylko na kontenery z określonego rejestru. Wszystkie obciążenia, które już działają w klastrze, nadal będą działać. Jeśli zmienisz skalę wdrożenia z 1 na 2, to `ReplicaSet` spróbuje utworzyć kolejną kapsułę. Jeżeli ta kapsuła nie będzie miała kontenera z dozwolonego repozytorium, to zostanie odrzucona. Należy ustawić pole `enforcementAction` na `dryrun` i przeprowadzić audyt, aby dowiedzieć się o wszelkich złamaniach zasad polityki przed ustawieniem `enforcementAction` na `deny`.

Audyt

Możliwość egzekwowania przestrzegania polityki przez nowe zasoby to tylko część historii polityk i zarządzania. Polityki często zmieniają się po pewnym czasie, a za pomocą narzędzia Gatekeeper można sprawdzić, czy wszystko, co jest obecnie wdrożone, wciąż spełnia ich warunki. Ponadto możesz już mieć klaster pełen usług i chcieć zainstalować Gatekeepera, aby zapewnić ich zgodność z polityką. Funkcje audytu Gatekeepera umożliwiają administratorom klastrów uzyskanie listy zasobów klastra, które w danym momencie nie spełniają wymogów polityki.

Poniżej przedstawiamy przykład ilustrujący sposób wykonywania audytu. Zaktualizujemy ograniczenie `repo-is-kuar-demo`, zmieniając ustawienie pola `enforcementAction` na `dryrun` (listing 20.5). To umożliwi użytkownikom tworzenie niezgodnych zasobów. Następnie dzięki audytowi dowiemy się, które zasoby są niezgodne z polityką.

Listing 20.5. allowedrepos-constraint-dryrun.yaml

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sAllowedRepos
metadata:
  name: repo-is-kuar-demo
spec:
  enforcementAction: dryrun
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
    namespaces:
      - "default"
  parameters:
    repos:
      - "gcr.io/kuar-demo/"
```

Zaktualizuj ograniczenie przez wykonanie następującego polecenia:

```
$ kubectl apply -f allowedrepos-constraint-dryrun.yaml
k8sallowedrepos.constraints.gatekeeper.sh/repo-is-kuar-demo configured
```

Utwórz niespełniającą warunków kapsułę za pomocą poniższego polecenia:

```
$ kubectl apply -f noncompliant-pod.yaml
pod/nginx-noncompliant created
```

Aby przeprowadzić audyt listy zasobów niespełniających warunków dla danego ograniczenia, wykonaj na nim polecenie `kubectl get constraint` i określ, że chcesz otrzymać wynik w formacie YAML:

```
$ kubectl get constraint repo-is-kuar-demo -o yaml
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sAllowedRepos
...
spec:
  enforcementAction: dryrun
  match:
    kinds:
    - apiGroups:
      - ""
```

```
        kinds:
        - Pod
      namespaces:
      - default
    parameters:
      repos:
      - gcr.io/kuar-demo/
  status:
    auditTimestamp: "2021-07-14T20:05:38Z"
        ...
    totalViolations: 1
    violations:
    - enforcementAction: dryrun
      kind: Pod
      message: Kontener <nginx> ma nieprawidłowe repozytorium obrazów <nginx>,
      ↪dozwolone repozytoria to ["gcr.io/kuar-demo/"]
      name: nginx-noncompliant
      namespace: default
```

W sekcji `status` znajduje się pole `auditTimestamp`, które określa datę i godzinę ostatniego przeprowadzenia audytu. Pole `totalViolations` pokazuje liczbę zasobów łamiących dane ograniczenie. Sekcja `violations` zawiera listę przypadków jego złamania. Dowiadujemy się, że kapsuła `nginxnoncompliant` łamie ograniczenie, oraz otrzymujemy szczegółowe informacje na ten temat.

Ustawienie ograniczenia `enforcementAction` o wartości `dryrun` w połączeniu z audytem jest doskonałym sposobem na sprawdzenie, czy nasza polityka daje pożądany efekt. Ponadto technika ta pozwala utworzyć przepływ pracy umożliwiający zapewnienie zgodności zasobów.

Modyfikacja

Do tej pory pokazywaliśmy, jak używać ograniczeń w celu sprawdzania, czy dany zasób spełnia warunki. A jak modyfikuje się zasoby, aby je spełniały? Do tego służy funkcja modyfikacji Gatekeepera. Wcześniej opisaliśmy dwa typy elementów webhook wstępu — modyfikujący i walidujący. Domyślnie Gatekeeper jest wdrażany tylko jako webhook wstępu walidującego, ale można go skonfigurować, aby działał jako webhook wstępu modyfikującego.

Funkcje modyfikacji Gatekeepera są w fazie beta i mogą ulec zmianie. Pokazujemy je, aby zademonstrować przyszłe możliwości tego narzędzia. Kroki instalacji w tym rozdziale nie obejmują włączania funkcji modyfikacji. Więcej szczegółów na ten temat znajduje się na stronie projektu Gatekeeper (*https://oreil.ly/DQKhl*).

Przestudiujemy przykład demonstrujący możliwości modyfikacji. W tym przykładzie ustawimy `imagePullPolicy` na `Always` we wszystkich kapsułach. Przyjmiemy założenie, że Gatekeeper jest poprawnie skonfigurowany do obsługi modyfikacji. Na listingu 20.6 znajduje się definicja przypisania modyfikującego, które pasuje do wszystkich kapsuł z wyjątkiem tych należących do przestrzeni nazw `system` i przypisuje wartość `Always` polu `imagePullPolicy`.

Listing 20.6. imagepullpolicyalways-mutation.yaml

```
apiVersion: mutations.gatekeeper.sh/v1alpha1
kind: Assign
metadata:
  name: demo-image-pull-policy
spec:
  applyTo:
  - groups: [""]
    kinds: ["Pod"]
    versions: ["v1"]
  match:
    scope: Namespaced
    kinds:
    - apiGroups: ["*"]
      kinds: ["Pod"]
  excludedNamespaces: ["system"]
location: "spec.containers[name:*].imagePullPolicy"
parameters:
  assign:
    value: Always
```

Utwórz przypisanie modyfikujące:

```
$ kubectl apply -f imagepullpolicyalways-mutation.yaml
assign.mutations.gatekeeper.sh/demo-image-pull-policy created
```

Teraz utwórz kapsułę. Ta kapsuła nie ma ustawienia użytkownika dla pola `imagePullPolicy`, więc domyślnie ma ono wartość `IfNotPresent`. My jednak oczekujemy, że Gatekeeper zmieni tę wartość na `Always`:

```
$ kubectl apply -f compliant-pod.yaml
pod/kuard created
```

Sprawdź, czy wartość pola `imagePullPolicy` została zmieniona na `Always`, przez wykonanie następującego polecenia:

```
$ kubectl get pods kuard -o=jsonpath="{.spec.containers[0].imagePullPolicy}"
Always
```

Przypisanie modyfikujące ma miejsce przed walidacyjnym, więc należy tworzyć ograniczenia sprawdzające poprawność modyfikacji, które mają być zastosowane do określonego zasobu.

Usuń kapsułę za pomocą następującego polecenia:

```
$ kubectl delete -f compliant-pod.yaml
pod/kuard deleted
```

Usuń przypisanie modyfikujące, korzystając z takiego polecenia:

```
$ kubectl delete -f imagepullpolicyalways-mutation.yaml
assign.mutations.gatekeeper.sh/demo-image-pull-policy deleted
```

W odróżnieniu od walidacji modyfikacja przedstawia rozwiązanie pozwalające na automatyczną naprawę niezgodnych zasobów w imieniu administratora klastra.

Replikacja danych

Podczas pisania ograniczeń możemy chcieć porównać wartość pewnego pola z wartością pola w innym zasobie. Tak jest na przykład w sytuacji, gdy zamierzamy się upewnić, czy nazwy hostów Ingress są niepowtarzalne na przestrzeni całego klastra. Domyślnie Gatekeeper może ewaluować tylko pola znajdujące się w obecnym zasobie. Jeśli polityka wymaga dokonywania porównań między różnymi zasobami, należy odpowiednio skonfigurować narzędzie. Gatekeeper może buforować wybrane zasoby w Open Policy Agent, co umożliwia dokonywanie porównań między zasobami. Zasób przedstawiony na listingu 20.7 konfiguruje Gatekeepera tak, aby buforował zasoby `Namespace` i `Pod`.

Listing 20.7. config-sync.yaml

```
apiVersion: config.gatekeeper.sh/v1alpha1
kind: Config
metadata:
  name: config
  namespace: "gatekeeper-system"
spec:
  sync:
    syncOnly:
      - group: ""
        version: "v1"
        kind: "Namespace"
      - group: ""
        version: "v1"
        kind: "Pod"
```

Należy buforować tylko konkretne zasoby, które są potrzebne do wykonania ewaluacji polityki. Buforowanie setek lub tysięcy zasobów w OPA wymaga dużej ilości pamięci oraz może wpływać na poziom bezpieczeństwa.

Szablon ograniczeń przedstawiony na listingu 20.8 demonstruje sposób porównania elementów z sekcji `rego` (w tym przypadku niepowtarzalnych nazw hostów Ingress). Mówiąc dokładnie, `data.inventory` odnosi się do zasobów buforowanych, `input` natomiast jest zasobem wysyłanym do ewaluacji z serwera API Kubernetes w ramach przepływu wstępu. Ten przykład bazuje na bibliotece polityk Gatekeepera (*https://oreil.ly/gGrts*).

Listing 20.8. uniqueingresshost-constraint-template.yaml

```
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8suniqueingresshost
  annotations:
    description: Requires all Ingress hosts to be unique.
spec:
  crd:
    spec:
      names:
        kind: K8sUniqueIngressHost
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8suniqueingresshost
```

```
identical(obj, review) {
  obj.metadata.namespace == review.object.metadata.namespace
  obj.metadata.name == review.object.metadata.name
}

violation[{"msg": msg}] {
  input.review.kind.kind == "Ingress"
  re_match("^(extensions|networking.k8s.io)$", input.review.kind.group)
  host := input.review.object.spec.rules[_].host
  other := data.inventory.namespace[ns][otherapiversion]["Ingress"][name]
  re_match("^(extensions|networking.k8s.io)/.+$", otherapiversion)
  other.spec.rules[_].host == host
  not identical(other, input.review)
  msg := sprintf("ingress host conflicts with an existing ingress <%v>"...
}
```

Replikacja danych to potężne narzędzie umożliwiające wykonywanie porównań między zasobami Kubernetes. Zalecamy korzystanie z niej tylko wtedy, kiedy wymagają tego polityki. Używając tej techniki, ograniczaj ją wyłącznie do odpowiednich zasobów.

Metryki

Gatekeeper emituje metryki w formacie Prometheus, aby umożliwić ciągłe monitorowanie zasobów pod kątem zgodności z politykami. Dostępne są proste dane dotyczące ogólnego stanu Gatekeepera, takie jak liczba ograniczeń, szablony ograniczeń i żądania do tego narzędzia.

Ponadto można uzyskać szczegółowe informacje na temat zgodności z politykami i zarządzania:

- Suma przypadków złamania zasad wykrytych w audycie.
- Liczba ograniczeń według `enforcementAction`.
- Czas trwania audytu.

Ostatecznym celem jest pełna automatyzacja procesów związanych z politykami i zarządzaniem, dlatego gorąco zachęcamy do monitorowania Gatekeepera za pomocą zewnętrznego systemu monitoringu oraz do ustawienia alarmów na podstawie zgodności zasobów.

Biblioteka polityk

Jednym z podstawowych założeń projektu Gatekeeper jest stworzenie bibliotek polityk wielokrotnego użytku, które mogą być przekazywane między organizacjami. Możliwość udostępniania polityk pozwala zredukować ilość powtarzalnej pracy i umożliwia administratorom klastrów skupienie się na stosowaniu polityk, zamiast na ich pisaniu. Projekt Gatekeeper ma bogatą bibliotekę polityk (*https://oreil.ly/uBY2h*). Zawiera ona ogólną bibliotekę z najczęściej używanymi politykami oraz bibliotekę polityk bezpieczeństwa kapsuł, która modeluje możliwości API `PodSecurity Policy` jako politykę Gatekeepera. Jej wielką zaletą jest to, że ciągle się rozwija i jest typu open source, dzięki czemu każdy może przekazać do niej własne polityki.

Podsumowanie

W tym rozdziale opisaliśmy koncepcje polityk i zarządzania oraz wyjaśniliśmy, dlaczego są ważne w obliczu rosnącej liczby zasobów wdrażanych w Kubernetes. Opisaliśmy projekt Gatekeeper, czyli kontroler polityk Kubernetes stworzony na bazie Open Policy Agent, i pokazaliśmy, jak za jego pomocą zapewnić przestrzeganie wymogów polityk i zarządzania. Teraz znasz już cały proces umożliwiający spełnienie wymogów zgodności, od pisania polityk po przeprowadzanie audytów.

ROZDZIAŁ 21.

Wdrożenia aplikacji w wielu klastrach

Po dwudziestu rozdziałach tej książki na pewno już wiesz, że zagadnienia dotyczące Kubernetes mogą być skomplikowane, choć oczywiście mamy nadzieję, że skoro dotarłeś tutaj, to wiele z nich się rozjaśniło. Zważywszy na poziom złożoności procesu budowania i uruchamiania aplikacji w jednym klastrze Kubernetes, po co mielibyśmy jeszcze bardziej komplikować sobie życie i wdrażać aplikację w wielu klastrach?

Fakty są takie, że wymagania prawdziwego świata wymuszają wdrażanie większości aplikacji w wielu klastrach. Jest ku temu sporo powodów i istnieje duże prawdopodobieństwo, że Twoja aplikacja również będzie wymagała takiego wdrożenia.

Pierwszy wymóg dotyczy redundancji i odporności. Jedno centrum danych, czy to w chmurze, czy lokalne, stanowi pojedynczą domenę awarii. Wystarczy myśliwy, który postanowi poćwiczyć celność strzału na przewodzie optycznym, awaria zasilania spowodowana śnieżycą albo nieudane wdrożenie oprogramowania, aby aplikacja wdrożona tylko w jednym miejscu przestała działać i pozostawiła użytkowników bez żadnej pomocy. Nierzadko pojedynczy klaster Kubernetes jest powiązany z jednym miejscem i przez to stanowi pojedynczą domenę awarii.

W niektórych przypadkach, zwłaszcza w środowiskach chmurowych, klaster Kubernetes jest **regionalny**. Klasy regionalne obejmują wiele niezależnych stref, dzięki czemu są odporne na opisane wcześniej problemy z infrastrukturą. Kuszące może się wydawać przyjęcie założenia, że takie regionalne klastry są wystarczające do zapewnienia odporności, nawet mimo faktu, że sam Kubernetes może stanowić pojedynczy punkt awarii. Każdy klaster Kubernetes jest jednak powiązany z określoną wersją Kubernetes (na przykład 1.21.3) i jest wysoce prawdopodobne, że aktualizacja klastra spowoduje awarię aplikacji. Od czasu do czasu Kubernetes wycofuje interfejsy API albo zmienia ich sposób działania. Takie zmiany są rzadkie, a społeczność Kubernetes informuje o nich zawczasu. Ponadto mimo wielu testów niekiedy pojawiają się błędy. Choć istnieje małe ryzyko, że dany problem wpłynie na Twoją aplikację, jeśli weźmie się pod uwagę czas życia większości aplikacji (lata), prawdopodobne staje się to, że wcześniej czy później coś zakłóci jej działanie. W przeważającej części przypadków taki poziom ryzyka jest nieakceptowalny.

Oprócz wymagań dotyczących odporności kolejnym ważnym czynnikiem zachęcającym do wdrożeń wieloklastrowych jest potrzeba, by pozostawać w bliskości firmy lub aplikacji. Na przykład serwery gier powinny znajdować się blisko graczy, aby można było zredukować skalę opóźnień i poprawić komfort grania. Inne aplikacje mogą podlegać prawnym lub przepisowym regulacjom,

które wymagają przechowywania danych w obrębie określonego obszaru geograficznego. Ponieważ każdy klaster Kubernetes jest powiązany z określonym miejscem, wymogi dotyczące wdrażania aplikacji w konkretnych lokalizacjach sprawiają, że trzeba je wdrażać w wielu klastrach.

W końcu, choć istnieje wiele sposobów na izolację użytkowników w obrębie jednego klastra (można wykorzystać do tego na przykład przestrzenie nazw, RBAC, pule węzłów — kolekcje węzłów Kubernetes, które są zorganizowane pod kątem różnych umiejętności lub obciążeń), klaster Kubernetes to wciąż w dużym stopniu pojedyncza przestrzeń współpracy. Dla niektórych zespołów ryzyko związane z wpływem innych zespołów na ich aplikację, nawet przypadkowym, jest nie do zaakceptowania i dlatego wybierają mniejsze zło — złożoność przy zarządzaniu wieloma klastrami.

Pewnie już rozumiesz, że teraz lub w bliskiej przyszłości Twoja aplikacja prawdopodobnie będzie musiała objąć kilka klastrów. W dalszej części tego rozdziału opisujemy, jak wdrażać aplikacje w wielu klastrach.

Zanim zaczniesz

Zanim zaczniesz wdrażać aplikację w wielu klastrach, musisz prawidłowo wszystko skonfigurować na jednym. Każdy ma jakąś listę rzeczy do zrobienia w swojej konfiguracji, ale wszelkie problemy i niedociągnięcia stają się jeszcze bardziej dotkliwe w skali wdrożenia wieloklastrowego. Analogicznie rozwiązanie podstawowych problemów we własnej infrastrukturze jest 10 razy trudniejsze, jeśli mamy 10 klastrów. Ponadto jeśli dodatkowy klaster powoduje znaczne zwiększenie ilości pracy, niechętnie dodasz kolejne klastry, nawet jeśli będzie to uzasadnione (ze wszystkich wcześniej opisanych powodów).

Co mamy na myśli, pisząc o „podstawowych problemach”? Najważniejsze, aby automatyzacja działała prawidłowo. Dotyczy to zarówno automatyzacji wdrażania nowych aplikacji, jak i tworzenia klastrów oraz zarządzania nimi. Kiedy mamy jeden klaster, to jest on z definicji spójny ze sobą. Kiedy jednak dodamy kolejne klastry, zwiększamy ryzyko wystąpienia niezgodności wersji pomiędzy wszystkimi elementami klastra. Możemy mieć klastry z różnymi wersjami Kubernetes, innymi wersjami agentów monitoringu i prowadzenia dzienników, a nawet z odmienną wersją czegoś tak podstawowego, jak środowisko wykonawcze kontenerów. Wszystkie te rozbieżności należy traktować jako coś, co utrudnia pracę. Różnice w infrastrukturze czynią system „dziwniejszym”. Wiedza zdobyta w jednym klastrze nie przydaje się w odniesieniu do następnych klastrów, a problemy czasami występują w pozornie losowych miejscach. Jednym z najważniejszych aspektów utrzymania stabilnego fundamentu jest zapewnienie spójności między wszystkimi klastrami.

Jedynym sposobem na osiągnięcie pełnej spójności jest automatyzacja. Możesz pomyśleć, że zawsze tworzysz klastry w taki sposób, ale doświadczenie nauczyło nas, że to po prostu nieprawda. W następnym rozdziale szczegółowo opisujemy zalety infrastruktury jako kodu do zarządzania aplikacjami, ale to samo dotyczy zarządzania klastrami. Nie używaj narzędzi GUI ani CLI do tworzenia klastrów. Początkowo wprowadzanie wszystkich zmian przez kontrolę źródła oraz CI/CD może się wydawać niewygodne, ale stabilny fundament z czasem okaże się na wagę złota.

To samo dotyczy podstawowych komponentów, które wdrażamy w klastrach. Zaliczają się do nich monitoring, rejestracja dzienników i skanery bezpieczeństwa, które muszą być obecne przed

wdrożeniem każdej aplikacji. Nimi także należy zarządzać za pomocą narzędzi infrastruktury jako kodu, takich jak Helm, oraz wdrażać je z wykorzystaniem automatyzacji.

Abstrahując od kształtu klastrów, istnieją jeszcze inne aspekty dotyczące spójności, które koniecznie należy uwzględnić. Pierwszy to używanie systemu pojedynczej tożsamości we wszystkich swoich klastrach. Choć Kubernetes obsługuje proste uwierzytelnianie oparte na certyfikatach, gorąco zachęcamy do korzystania z integracji z globalnym dostawcą tożsamości, takim jak Azure Active Directory czy inny dostawca zgodny z OpenID Connect. Dopilnowanie, aby każdy używał tej samej tożsamości podczas uzyskiwania dostępu do wszystkich klastrów, jest krytycznym aspektem stosowania najlepszych praktyk bezpieczeństwa i unikania niebezpiecznych zachowań, takich jak udostępnianie certyfikatów. Ponadto większość dostawców tożsamości oferuje dodatkowe zabezpieczenia, takie jak uwierzytelnianie dwuskładnikowe, które zwiększają poziom bezpieczeństwa klastrów.

Oprócz kwestii tożsamości bardzo ważne jest zapewnienie spójnej kontroli dostępu do klastrów. W większości chmur oznacza to używanie chmurowego systemu RBAC, w którym role RBAC i wiązania są przechowywane w centralnym repozytorium, a nie w samych klastrach. Zdefiniowanie RBAC w pojedynczej lokalizacji zapobiega powstawaniu błędów takich jak pozostawienie uprawnień w jednym z klastrów czy niedodanie uprawnień któremuś z nich. Niestety jeśli definiujesz RBAC dla klastrów lokalnych, sytuacja jest trochę bardziej skomplikowana niż w przypadku tożsamości. Istnieją pewne rozwiązania (na przykład Azure Arc dla Kubernetes), które mogą dostarczać RBAC dla klastrów lokalnych, ale jeśli taka usługa jest niedostępna w Twoim środowisku, spójność wszystkich uprawnień można zapewnić przez zdefiniowanie RBAC w kontroli źródła i używanie infrastruktury jako kodu w celu stosowania reguł do wszystkich klastrów.

Analogicznie kiedy myślimy o zdefiniowaniu polityki dla naszych klastrów, to musimy je zdefiniować w jednym miejscu oraz zapewnić pojedynczy pulpit do sprawdzania stanu zgodności wszystkich klastrów. Podobnie jak w przypadku RBAC, takie globalne usługi są często udostępniane przez dostawcę chmury, ale w odniesieniu do lokalnych systemów możliwości są ograniczone. Użycie infrastruktury jako kodu dla polityk może pomóc w zamknięciu tej luki oraz pozwolić na definiowanie polityk w jednym miejscu.

Tak jak utworzenie właściwych testów jednostkowych i budowa infrastruktury mają krytyczne znaczenie dla rozwoju aplikacji, tak konfiguracja odpowiedniego fundamentu do zarządzania wieloma klastrami Kubernetes stanowi podstawę stabilnych wdrożeń aplikacji w całej szerokiej infrastrukturze. W następnych podrozdziałach opisujemy, jak tworzyć aplikacje, które dobrze działają w środowisku wieloklastrowym.

Podejście od góry z równoważeniem obciążenia

Kiedy zaczniesz myśleć o wdrożeniu swojej aplikacji w wielu lokalizacjach, najistotniejszą sprawą stanie się to, w jaki sposób użytkownicy będą uzyskiwać do niej dostęp. Zazwyczaj służy do tego nazwa domeny (na przykład *ja.mojafirma.com*). Choć poświęcamy dużo miejsca na omówienie budowy aplikacji pod kątem działania w wielu lokalizacjach, na początek ważniejszy jest sposób implementacji dostępu. Wynika to oczywiście z faktu, że ludzie muszą mieć dostęp do aplikacji,

a także z tego, że konstrukcja mechanizmu dostępu do niej może zwiększyć nasze możliwości w zakresie szybkiego reagowania i przekierowywania ruchu w przypadku niespodziewanych wzrostów obciążenia lub awarii.

Dostęp do aplikacji zaczyna się od nazwy domeny. To znaczy, że początek naszej wieloklastrowej strategii równoważenia obciążenia wiąże się z wyszukiwaniem DNS. Jest ono pierwszym wyborem w tej strategii. W wielu tradycyjnych podejściach do równoważenia obciążenia wyszukiwanie DNS było używane do kierowania ruchu do określonych lokalizacji. Technikę tę określa się mianem GeoDNS. W jej ramach adres IP zwrócony przez wyszukiwanie DNS jest powiązany z fizyczną lokalizacją klienta. Generalnie adres IP jest regionalnym klastrem, który jest najbliżej klienta.

Choć technika GeoDNS wciąż przeważa w wielu aplikacjach i bywa jedynym możliwym podejściem w przypadku aplikacji lokalnych, ma ona kilka wad. Pierwsza jest taka, że DNS jest buforowany w wielu miejscach w internecie i choć można ustawić czas życia (ang. *time to live* — TTL) wyszukiwania DNS, w wielu lokalizacjach jest on ignorowany ze względu na optymalizację wydajności. W stabilnych warunkach to buforowanie nie stanowi wielkiego problemu, ponieważ DNS jest generalnie dość stabilny bez względu na TTL. Staje się jednak wielkim problemem, kiedy trzeba przenosić ruch z jednego klastra do innego, na przykład w odpowiedzi na przestój w określonym centrum danych. W takich nagłych przypadkach to, że wyszukiwania DNS są buforowane, może znacznie wydłużyć czas trwania przestoju i spotęgować jego skutki. Ponadto, ponieważ GeoDNS zgaduje fizyczną lokalizację na podstawie adresu IP klienta, często popełnia błędy, kiedy wiele różnych klientów generuje ruch z tego samego adresu IP zapory mimo przebywania w różnych lokalizacjach geograficznych.

Alternatywą dla wyboru klastra przez DNS jest technika równoważenia obciążenia zwana **anycast**. W technice tej pojedynczy statyczny adres IP jest ogłaszany z wielu lokalizacji w internecie za pomocą podstawowych protokołów routingu. Podczas gdy tradycyjnie mapujemy adres IP na pojedynczy komputer, w anycast adres IP jest wirtualny i zostaje routowany do różnych lokalizacji w zależności od lokalizacji sieciowej użytkownika. Nasz ruch jest kierowany do „najbliższej" lokalizacji na podstawie odległości w kategoriach wydajności sieci, a nie dystansu fizycznego. Technika anycast generalnie daje lepsze wyniki, ale nie jest dostępna we wszystkich środowiskach.

Ostatnią kwestią, jaką należy uwzględnić, gdy projektuje się równoważenie obciążenia, jest to, czy równoważenie ma odbywać się na poziomie TCP, czy HTTP. Do tej pory opisywaliśmy tylko równoważenie na poziomie TCP, ale w przypadku aplikacji sieciowych równoważenie w warstwie HTTP daje wiele istotnych korzyści. Jeśli piszesz aplikację opartą na HTTP (jak większość dzisiejszych aplikacji), to zastosowanie globalnego systemu równoważenia obciążenia opartego na HTTP przyniesie Ci więcej informacji na temat komunikacji z klientem. Na przykład decyzje dotyczące równoważenia obciążenia będziesz mógł podejmować na podstawie plików cookie zdefiniowanych w przeglądarce. Ponadto system równoważenia obciążenia rozpoznający ten protokół może podejmować bardziej inteligentne decyzje dotyczące routingu, ponieważ „widzi" każde żądanie HTTP, a nie tylko strumień bajtów przepływający w połączeniu TCP.

Bez względu na wybrane podejście ostatecznie lokalizacja usługi jest mapowana z globalnego punktu końcowego DNS do kolekcji regionalnych adresów IP reprezentującej punkt wejścia do tej usługi. Adresy te to generalnie adres IP usługi Kubernetes lub zasobu Ingress, o których była mowa w poprzednich rozdziałach. Kiedy ruch użytkownika dojdzie do tego punktu, przepływa przez klaster w sposób określony przez konstrukcję naszej aplikacji.

Budowa aplikacji dla wielu klastrów

Kiedy już uporasz się z kwestiami równoważenia obciążenia, stanie przed Tobą wyzwanie zaprojektowania wieloklastrowej aplikacji „świadomej” stanu. Najlepiej, aby ta aplikacja nie wymagała stanu lub by w całości był on tylko do odczytu. W takiej sytuacji do wdrożenia wieloklastrowego nie trzeba dużo pracy. Można wdrożyć aplikację osobno w każdym klastrze, dodać do tego system równoważenia obciążenia i to wszystko. Niestety w większości aplikacji istnieje stan, którym trzeba zarządzać w spójny sposób między ich replikami. Jeśli tego nie zapewnimy, korzystanie z tych aplikacji będzie utrudnione dla użytkowników.

Abyś mógł dobrze zrozumieć, jaki wpływ na wrażenia użytkownika ma zreplikowany stan, posłużymy się przykładem sklepu internetowego. To oczywiste, że jeśli zapiszemy zamówienie klienta tylko w jednym z wielu klastrów, ten sam klient ze zdziwieniem odkryje, że jego zamówienie znikło, kiedy system przeniesie jego żądania do innego regionu, na przykład w ramach równoważenia obciążenia albo z powodu zmiany lokalizacji geograficznej. Dlatego aplikacja musi replikować stan użytkownika między regionami. Mniej oczywiste może być to, że podejście do replikacji wpływa także na wrażenia klienta. Problemy związane z replikacją danych i wrażeniami klienta zwięźle podsumowuje następujące pytanie: „Czy mogę odczytać to, co sam zapisałem?”. Odpowiedź potwierdzająca wydaje się oczywista, ale osiągnięcie tego jest trudniejsze, niż się wydaje. Weźmy na przykład klienta składającego zamówienie na komputerze, a następnie próbującego je natychmiast obejrzeć na swoim telefonie komórkowym. Te dwa urządzenia mogą kontaktować się z naszą aplikacją przez dwie różne sieci i w efekcie mogą korzystać z dwóch różnych klastrów. Możliwość obejrzenia własnego zamówienia przez użytkownika świadczy o **spójności** danych.

Spójność dyktuje sposób myślenia o replikacji danych. Zakładamy, że chcemy, aby nasze dane były spójne, to znaczy, że będziemy w stanie odczytywać te same dane niezależnie od miejsca, w którym się znajdujemy. Czynnikiem komplikującym sprawę jest czas — jak szybko nasze dane muszą stawać się spójne? Czy będzie wyświetlana jakaś informacja wskazująca, że dane nie są jeszcze uspójnione? Wyróżnia się dwa podstawowe modele spójności: **spójność silną** (ang. *strong consistency*), która gwarantuje, że zapis nie powiedzie się, dopóki nie zostanie prawidłowo zreplikowany, oraz **spójność następczą** (ang. *eventual consistency*), w przypadku której zapis zawsze jest wykonywany od razu oraz mamy gwarancję, że później zostanie zreplikowany. Ponadto niektóre systemy umożliwiają klientowi wybór modelu spójności według żądań. Na przykład Azure Cosmos DB implementuje **spójność związaną** (ang. *bounded consistency*), w przypadku której istnieją pewne gwarancje dotyczące tego, jak bardzo przestarzałe dane mogą znajdować się w systemie, który ostatecznie zostanie uspójniony. Google Cloud Spanner umożliwia klientom zadeklarowanie, że są skłonni tolerować przestarzałe odczyty w zamian za zwiększenie wydajności.

Może się wydawać, że zawsze powinno się wybierać spójność silną, ponieważ jest to najprostszy model, w którym dane zawsze i wszędzie są takie same. Jednak spójność silna ma swoją cenę. Gwarancja replikacji w momencie zapisu wymaga znacznie więcej wysiłku i o wiele więcej zapisów nie powiedzie się, kiedy replikacja nie będzie możliwa. Spójność silna jest bardziej kosztowna i w jej przypadku może być obsługiwanych znacznie mniej jednoczesnych transakcji w porównaniu ze spójnością następczą. Spójność następcza jest tańsza i pozwala na obsługiwanie dużo większego obciążenia zapisem, ale ten model jest też bardziej skomplikowany dla programisty

aplikacji, a pewne warunki brzegowe mogą być eksponowane użytkownikowi końcowemu. Wiele systemów przechowywania danych obsługuje tylko jeden model współbieżności. Te, które obsługują ich kilka, wymagają określenia czasu utworzenia systemu przechowywania. Wybór modelu współbieżności ma także poważny wpływ na kształt aplikacji i trudno jest go zmienić. W efekcie wybór modelu spójności jest ważnym pierwszym krokiem przed przystąpieniem do projektowania aplikacji dla wielu środowisk.

Wdrażanie replikowanego magazynu danych z określonym stanem i zarządzanie nim to skomplikowane zadanie, które wymaga współpracy specjalnego zespołu posiadającego odpowiednią wiedzę pozwalającą na konfigurację, konserwację i monitorowanie całości. Należy poważnie zastanowić się nad możliwością użycia magazynu chmurowego do przechowywania zreplikowanych danych, aby te obowiązki spadły na duży zespół pracujący u dostawcy chmury, a nie na zespół programisty. W środowisku lokalnym można także zlecić obsługę magazynu firmie specjalizującej się w takich rozwiązaniach. Inwestycja w budowę własnego zespołu do zarządzania magazynem ma sens tylko w przypadku dużych organizacji.

Po wybraniu warstwy magazynowej kolejnym krokiem jest stworzenie projektu aplikacji.

Replikowane silosy — najprostszy model międzyregionalny

Najprostszym sposobem na replikację aplikacji do wielu klastrów i regionów jest po prostu kopiowanie tej aplikacji do każdego regionu. Każda instancja aplikacji jest dokładnym klonem i wygląda tak samo niezależnie od tego, w jakim klastrze działa. Ponieważ nad wszystkim operuje system równoważenia obciążenia, który rozdysponowuje żądania klientów, oraz zaimplementowaliśmy replikację danych w miejscach, w których potrzebujemy stanu, w naszej aplikacji nie musimy wiele zmieniać, aby obsługiwała ten model. Zależnie od wybranego przez nas modelu spójności danych musimy się liczyć z tym, że dane mogą nie być szybko replikowane między regionami, chociaż, szczególnie w przypadku spójności silnej, nie będzie to wymagało poważnych zmian w aplikacji.

Kiedy zaprojektujemy naszą aplikację w ten sposób, każdy region będzie swoim własnym silosem. Wszystkie dane, jakich potrzebuje aplikacja, są obecne w regionie i kiedy do tego regionu dotrze żądanie, aplikacja serwuje je przez kontener działający w danym klastrze. Daje to wymierne korzyści w zakresie redukcji poziomu złożoności, ale jak zwykle odbywa się to kosztem wydajności.

Aby zrozumieć, jak podejście silosowe wpływa na wydajność, wyobraź sobie aplikację rozproszoną po dużej liczbie regionów geograficznych na całym świecie w celu skrócenia opóźnień u użytkowników. Na świecie jest tak, że w pewnych regionach zagęszczenie ludności jest bardzo duże, a w innych bardzo małe. Gdyby każdy silos w każdym klastrze aplikacji był taki sam, to każdy musiałby mieć rozmiar spełniający wymogi najludniejszego regionu. W efekcie większość replik aplikacji w regionalnych klastrach byłaby znacznie przerośnięta, co pogarszałoby opłacalność całej aplikacji. Oczywistym rozwiązaniem w takim przypadku jest zmniejszenie rozmiaru zasobów wykorzystywanych przez aplikację w mniejszych obszarach. Choć zmiana rozmiaru aplikacji wydaje się łatwa, nie zawsze da się to zrobić z powodu wąskich gardeł i innych wymagań (na przykład konieczności utrzymywania przynajmniej trzech replik).

W szczególności pod kątem przenoszenia istniejącej aplikacji z jednego klastra do wielu klastrów konstrukcja oparta na replikowanych silosach jest najprostsza, ale warto pamiętać, że jej wykorzystanie może pociągać za sobą koszty, które początkowo będą znośne, ale z czasem wymuszą refaktoryzację aplikacji.

Fragmentowanie — dane regionalne

Kiedy Twoja aplikacja będzie się rozrastać, w przypadku zastosowania podejścia z silosami regionalnymi możesz napotkać problem polegający na tym, że globalna replikacja danych będzie stawać się coraz bardziej kosztowna i nieefektywna. Replikacja danych w celu zwiększenia niezawodności jest pożyteczna, ale jest mało prawdopodobne, że wszystkie dane aplikacji będą musiały być kolokowane w każdym klastrze, w którym wdrożyliśmy tę aplikację. Większość użytkowników korzysta z naszej aplikacji tylko z niewielkiej liczby regionów geograficznych.

Ponadto w miarę rozwoju naszej aplikacji na całym świecie mogą zacząć pojawiać się problemy prawne i inne związane z lokalnością danych. Mogą obowiązywać zewnętrzne zakazy dotyczące miejsc przechowywania danych użytkownika zależnie od jego narodowości lub innych czynników. Kombinacja tych wymagań sprawi, że w pewnym momencie będzie trzeba zacząć myśleć o regionalnym fragmentowaniu danych. Fragmentowanie danych między regionami oznacza, że nie wszystkie dane będą obecne we wszystkich klastrach, w których została wdrożona aplikacja, co oczywiście wpłynie na jej konstrukcję.

W ramach przykładu wyobraź sobie, że aplikacja jest wdrożona w sześciu klastrach regionalnych (A, B, C, D, E, F). Bierzemy zbiór danych naszej aplikacji i dzielimy go na trzy podzbiory, czyli **fragmenty** (1, 2, 3).

Wówczas nasza fragmentacja może wyglądać tak:

```
  A B C D E F
1 ✓ - - ✓ - -
2 - ✓ - - ✓ -
3 - - ✓ - - ✓
```

Każdy fragment jest obecny w dwóch regionach w celu zapewnienia redundancji, ale każdy klaster regionalny może serwować tylko jedną trzecią danych. To znaczy, że musimy wprowadzić dodatkową warstwę routingu do naszej usługi za każdym razem, gdy potrzebujemy dostępu do danych. Warstwa routingu odpowiada za określanie, czy konkretne żądanie należy skierować do lokalnego, czy do międzyregionalnego fragmentu danych.

Choć może Cię kusić, aby wdrożyć taki routing danych jako część biblioteki klienta, która jest połączona z główną aplikacją, gorąco zalecamy zaimplementowanie tego mechanizmu jako osobnej mikrousługi. Dodanie nowej mikrousługi może kojarzyć się ze zwiększaniem poziomu złożoności, ale w rzeczywistości wprowadza abstrakcję, która wszystko upraszcza. Wszystkie usługi w aplikacji nie muszą zajmować się routingiem danych, ponieważ mamy jedną usługę, która to robi, a pozostałe po prostu z niej korzystają. Aplikacje wydzielone do niezależnych mikrousług zapewniają elastyczność w środowiskach międzyklastrowych.

Większa elastyczność — routing mikrousług

Podczas omawiania podejścia z wykorzystaniem regionalnych silosów do tworzenia aplikacji wieloklastrowych przedstawiliśmy przykład ilustrujący, w jaki sposób może ono zmniejszyć opłacalność naszej wdrożonej aplikacji wieloklastrowej. Istnieją jednak jeszcze inne czynniki wpływające na elastyczność. Tworząc silos, tworzymy w większej skali monolit w rodzaju tych, które kontenery i Kubernetes starają się rozbić. Ponadto zmuszamy każdą mikrousługę w aplikacji do skalowania w tym samym czasie do tej samej liczby regionów.

Gdyby nasza aplikacja była mała i zwarta, miałoby to sens, ale ponieważ nasze usługi się rozrastają, a zwłaszcza jeśli mogą zacząć być używane przez różne aplikacje, monolityczne podejście do wieloklastrowości zaczyna znacząco wpływać na elastyczność. Jeżeli klaster jest jednostką wdrożenia i nasze CI/CD jest z nim w całości powiązane, to będziemy zmuszać każdy zespół do wykorzystywania tego samego procesu i przestrzegania harmonogramu wdrażania, nawet jeśli nie będzie to dobrym pomysłem.

W ramach konkretnego przykładu wyobraź sobie, że mamy jedną bardzo dużą aplikację, która jest wdrożona w obcych klastrach, i małą nową aplikację, która jest jeszcze nieukończona. Nie ma sensu zmuszać małego zespołu pracującego nad nową aplikacją do natychmiastowego osiągania skali większej aplikacji, ale jeśli będziemy mieć zbyt sztywne podejście podczas projektowania, tak to się skończy.

Lepszym wyborem pod względem projektowym jest traktowanie każdej mikrousługi w aplikacji jako usługi publicznej. Wspomniana mała usługa może nigdy nie będzie dostępna publicznie, ale powinna mieć własny globalny system równoważenia obciążenia zgodnie z opisem z poprzednich podrozdziałów oraz powinna zarządzać własną usługą replikacji danych. Różne mikrousługi powinny być bezwzględnie niezależne od siebie. Kiedy usługa odnosi się do innej usługi, jej obciążenie jest równoważone w taki sam sposób, w jaki byłoby równoważone obciążenie zewnętrzne. Przy takiej abstrakcji każdy zespół może niezależnie skalować i wdrażać swoją usługę wieloklastrową, tak jak to robią w przypadku implementacji jednoklastrowej.

Oczywiście zapewnienie tego w przypadku każdej mikrousługi w aplikacji byłoby dużym ciężarem dla zespołów oraz mogłoby zwiększać koszty z uwagi na konieczność utrzymywania systemu równoważenia obciążenia dla każdej usługi i obsługi potencjalnego ruchu międzyregionalnego. Jak to zawsze bywa przy projektowaniu oprogramowania, należy znaleźć równowagę między poziomem złożoności a wydajnością, określić odpowiednie miejsca do zastosowania granic izolacji usług, a także, tam gdzie to ma sens, pogrupować usługi w replikowanych silosach. Tak jak mikrousługi w jednoklastrowym kontekście, ten format może się zmienić wraz ze wzrostem i ewolucją aplikacji. Uwzględnienie takiej płynności podczas projektowania pomoże stworzyć aplikację, która będzie w stanie adaptować się bez konieczności wprowadzania dużych zmian.

Podsumowanie

Choć wdrożenie aplikacji w wielu klastrach zwiększa poziom złożoności, wymagania i oczekiwania użytkowników w prawdziwym świecie sprawiają, że w przypadku większości aplikacji nie da się tego uniknąć. Projektowanie aplikacji i infrastruktury od początku z uwzględnieniem wieloklastrowego wdrażania znacznie zwiększa niezawodność aplikacji i istotnie zmniejsza ryzyko konieczności przeprowadzenia kosztownej refaktoryzacji, gdy aplikacja się rozrośnie. Jednym z najważniejszych elementów wdrażania wieloklastrowego jest zarządzanie konfiguracją i wdrażaniem aplikacji w klastrze. Niezależnie od tego, czy Twoja aplikacja jest regionalna, czy wdrożona w wielu klastrach, zajrzyj do następnego rozdziału. Dowiesz się, jak szybko i niezawodnie ją wdrożyć.

ROZDZIAŁ 22.

Organizacja aplikacji

W poprzednich rozdziałach opisaliśmy różne komponenty aplikacji tworzonych na bazie Kubernetes. Pokazaliśmy, jak opakowywać programy w kontenery, umieszczać je w kapsułach, replikować te kapsuły za pomocą obiektów `ReplicaSet` oraz jak instalować oprogramowanie co tydzień przy użyciu wdrożeń. Przedstawiliśmy też metody wdrażania aplikacji rzeczywistych i stanowych, które łączą zbiory tych obiektów w jeden rozproszony system. Do tej pory nie napisaliśmy jednak nic na temat praktycznych aspektów pracy z taką aplikacją. Jak powinno się zbudować, udostępniać, obsługiwać i aktualizować różne konfiguracje, z których składa się aplikacja? To jest temat tego rozdziału.

Podstawowe zasady

Przed opisem szczegółowych zasad budowy aplikacji dobrze jest zastanowić się, czemu mają one służyć. Oczywiście aplikacje chmurowe w oparciu o Kubernetes tworzy się przede wszystkim ze względu na ich niezawodność i zwinność, ale warto też zastanowić się dokładniej nad tym, jaki ma to wpływ na budowę i wdrażanie aplikacji. W tym podrozdziale opisujemy różne zasady, którymi można się kierować podczas projektowania struktury programu spełniającej określone wymogi. Oto te zasady:

- Systemy plików są źródłem prawdy.
- Recenzje kodu pozwalają zapewnić wysoką jakość wprowadzanych zmian.
- Elementy funkcjonalności warto oznaczać flagami, aby umożliwić etapowe wprowadzanie i cofanie zmian.

Systemy plików jako źródło prawdy

Pierwsze doświadczenia z Kubernetes mają zazwyczaj charakter imperatywny, tak jak zaprezentowaliśmy na początku tej książki. Za pomocą poleceń `kubectl run` lub `kubectl edit` tworzy się lub modyfikuje kapsuły albo inne obiekty działające w klastrze. Nawet sposoby tworzenia plików YAML i JSON opisaliśmy dość wyrywkowo, tak jakby same pliki były tylko przystankiem na drodze do modyfikacji stanu klastra. A jednak w prawdziwej aplikacji przeznaczonej do użytku w produkcji powinno być odwrotnie.

Źródłem wiedzy na temat aplikacji nie powinien być stan klastra — dane w pliku *etcd* — lecz powinna nim być struktura obiektów YAML w systemie plików. Obiekty API wdrożone w klastrze Kubernetes są wówczas odzwierciedleniem informacji przechowywanych w systemie plików.

To podejście jest zasadne z kilku powodów. Przede wszystkim pozwala traktować klaster jako niezmienną infrastrukturę. Po przejściu na architektury chmurowe szybko przyzwyczailiśmy się do tego, że nasze aplikacje i ich kontenery są takimi strukturami, ale w odniesieniu do klastrów jest to mniej oczywiste. A jednak powody, dla których przenosimy nasze aplikacje do niezmiennej infrastruktury, są aktualne także w odniesieniu do klastrów. Jeśli Twój klaster to chaotyczna zbieranina przypadkowych plików YAML pobranych z internetu, to korzystanie z niego jest tak samo niebezpieczne jak z maszyny wirtualnej utworzonej przy użyciu imperatywnych skryptów bash.

Ponadto zarządzanie stanem klastra przez system plików znacznie ułatwia współpracę z wieloma członkami zespołu. Systemy kontroli kodu źródłowego są dobrze znane i pozwalają wielu osobom naraz na edytowanie stanu klastra, od razu wyraźnie ujawniając wszelkie konflikty, co pozwala na ich rozwiązywanie.

Każda aplikacja wdrażana w Kubernetes powinna być przede wszystkim opisana w kategoriach plików przechowywanych w systemie plików. Późniejsze obiekty API są już tylko odzwierciedleniem tej struktury w konkretnym klastrze.

Rola recenzji kodu

Jeszcze nie tak dawno temu recenzowanie kodu źródłowego aplikacji było nowym pomysłem, a dziś jest oczywiste, że najlepszym sposobem na uzyskanie wysokiej jakości niezawodnego kodu jest poddanie go analizie przez kilka osób przed wprowadzeniem go do aplikacji.

Dlatego dziwi fakt, że w przypadku konfiguracji służących do wdrażania tych programów to podejście już nie jest tak chętnie stosowane. Przecież wszystkie powody, dla których recenzuje się kod źródłowy, w równym stopniu dotyczą także konfiguracji. Co więcej, gdyby dokładnie się zastanowić, można by stwierdzić, że recenzja kodu tych konfiguracji ma krytyczne znaczenie dla niezawodności usług. Z naszego doświadczenia wynika, że większość przestojów w dostępności usług jest spowodowana przez niespodziewane skutki, literówki i inne proste błędy. Wystarczy jednak pokazać każdą zmianę konfiguracji przynajmniej dwóm osobom, aby radykalnie zmniejszyć ryzyko wystąpienia takich błędów.

Druga zasada budowy aplikacji głosi, że każda zmiana wprowadzana do zbioru plików będącego źródłem prawdy o naszym klastrze powinna przejść proces recenzji.

Bramy i flagi funkcji

Po wprowadzeniu kodu źródłowego aplikacji i plików konfiguracyjnych wdrożeń do systemu kontroli wersji często powstaje pytanie, jak mają się do siebie te repozytoria. Czy kod źródłowy i konfigurację powinno się przechowywać w tym samym repozytorium? W małych projektach może

to wystarczy, ale w większych zwykle lepiej jest je rozdzielić. Nawet jeśli za budowę i wdrażanie aplikacji odpowiadają te same osoby, te dwie perspektywy są na tyle różne, że ich całkowite rozdzielenie ma sens.

Jeśli zdecydujemy się na taki rozdział, pozostaje do rozwiązania problem wypełnienia luki między procesem tworzenia nowych elementów funkcjonalności w systemie kontroli kodu źródłowego a ich wdrażaniem w środowisku produkcyjnym. Tutaj z pomocą przychodzą bramy funkcji, które odgrywają bardzo ważną rolę.

Ten pomysł polega na tym, że prace nad nową funkcją są prowadzone za flagą lub *bramą* funkcji, która może wyglądać mniej więcej tak:

```
if (featureFlags.myFlag) {
  // implementacja funkcji
}
```

Podejście to ma wiele zalet. Po pierwsze, umożliwia wprowadzenie kodu do gałęzi produkcyjnej na długo przed tym, zanim dana funkcja będzie gotowa do oddania do użytku. Dzięki temu prace nad funkcją toczą się bliżej głównego repozytorium, co niejednokrotnie chroni przed gigantycznymi konfliktami podczas scalania z istniejącą od długiego czasu gałęzią.

Ponadto praca za flagą funkcji oznacza, że aby włączyć funkcję, wystarczy tylko aktywować odpowiednią flagę w konfiguracji. Dzięki temu zawsze wyraźnie widać, co zmieniło się w środowisku produkcyjnym, a w razie problemów łatwo można cofnąć aktywację danej funkcji.

Flagi funkcji ułatwiają zatem debugowanie aplikacji w produkcji oraz chronią przed koniecznością przywracania starszej wersji binarnej kodu pozbawionej poprawek błędów i innych ulepszeń wprowadzonych w nowszej wersji.

Trzecia zasada budowy aplikacji głosi, że kod źródłowy, domyślnie wyłączony, trafia do systemu kontroli źródła za flagą i jest aktywowany za pomocą recenzowanych zmian w plikach konfiguracyjnych.

Zarządzanie aplikacją w systemie kontroli źródła

Ustaliliśmy już, że system plików powinien być źródłem wiedzy na temat klastra, więc teraz czas zastanowić się, jaką konkretnie strukturę powinny reprezentować nasze pliki. Oczywiście systemy plików zawierają hierarchiczne struktury katalogów, a systemy kontroli źródła dodają takie koncepcje jak znaczniki i gałęzie. W tym podrozdziale opisujemy, jak je połączyć w jedną reprezentację do zarządzania aplikacją.

Układ systemu plików

W tym podrozdziale omówimy budowę egzemplarza naszej aplikacji dla pojedynczego klastra, a w następnych pokażemy, jak dodać parametry do tego układu, aby był odpowiedni dla wielu egzemplarzy aplikacji. Należy podkreślić, że o dobrą organizację warto dbać od samego początku.

Podobnie jak w przypadku pakietów w systemie kontroli źródła, modyfikowanie konfiguracji wdrożenia po czasie jest skomplikowaną i kosztowną czynnością, której każdy woli uniknąć.

Podstawową oś, wokół której powinna kręcić się organizacja aplikacji, stanowi komponent lub warstwa o charakterze semantycznym (na przykład *frontend*, *kolejka zadań* itd.). Choć na początku może się to wydawać przesadą, ponieważ wszystkimi komponentami zarządza jeden zespół, postępując w ten sposób, zapewniamy sobie podstawę do sprawnego rozwoju — z czasem odpowiedzialność za każdy komponent może przejąć inny zespół lub część zespołu.

Podsumowując, system plików aplikacji z frontendem wykorzystującym dwie usługi może wyglądać następująco:

```
frontend/
usługa-1/
usługa-2/
```

W każdym z tych katalogów przechowywane są konfiguracje każdej aplikacji. Mają one postać plików YAML bezpośrednio reprezentujących bieżący stan klastra. Generalnie dobrym pomysłem jest wpisanie w tym samym pliku zarówno nazwy usługi, jak i typu obiektu.

Mimo że Kubernetes pozwala na tworzenie plików YAML zawierających wiele obiektów, generalnie takie postępowanie powinno być traktowane jako antywzór. Jest to uzasadnione jedynie wtedy, gdy wszystkie obiekty umieszczone w jednym pliku są koncepcyjnie identyczne. Wyborem elementów do umieszczenia w jednym pliku YAML rządzą takie same zasady jak definiowaniem struktur i klas. Jeśli grupa obiektów nie tworzy spójnej koncepcji, to znaczy, że prawdopodobnie obiekty te nie powinny być zgrupowane w jednym pliku.

Rozszerzmy poprzedni przykład. Teraz nasz system plików może wyglądać tak:

```
frontend/
  frontend-deployment.yaml
  frontend-service.yaml
  frontend-ingress.yaml
usługa-1/
  usługa-1-deployment.yaml
  usługa-1-service.yaml
  usługa-1-configmap.yaml
...
```

Wersje okresowe

A co z zarządzaniem wersjami? Możliwość cofnięcia się w historii aplikacji i przejrzenia jej wcześniejszych wdrożeń bywa bardzo przydatna. Analogicznie, dobrze jest mieć możliwość wdrożenia nowej konfiguracji, zachowując jednocześnie możliwość wdrożenia stabilnej konfiguracji.

W konsekwencji dobrze jest móc przechowywać i utrzymywać wiele różnych wersji konfiguracji jednocześnie. Istnieją dwa odmienne podejścia, które można stosować z systemami kontroli plików i wersji, które tu opisujemy. Pierwsze polega na używaniu znaczników, gałęzi i funkcji systemu kontroli źródła. Wygoda tego rozwiązania polega na tym, że jest podobne do sposobu zarządzania wersjami w systemie kontroli wersji oraz prowadzi do powstania prostszej struktury katalogów.

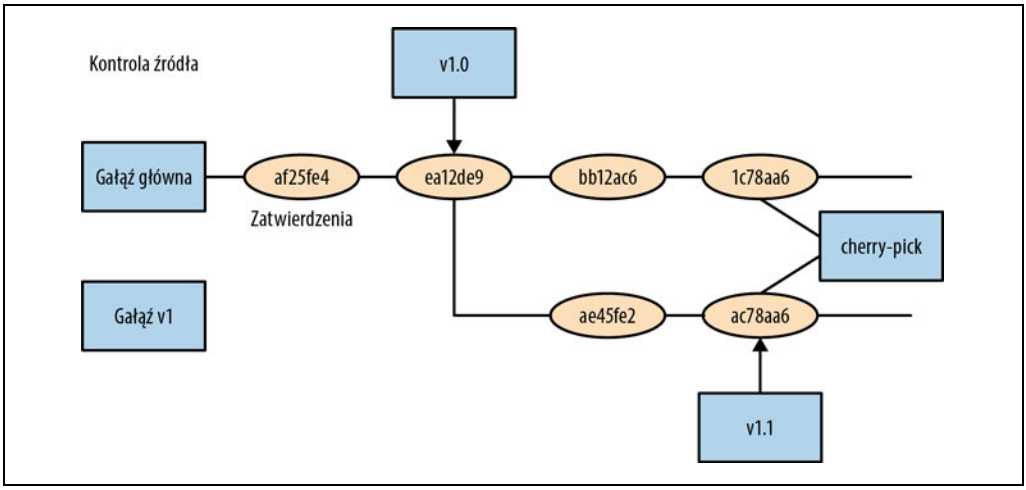
Druga opcja to sklonowanie konfiguracji w systemie plików i przechowywanie różnych wersji w katalogach. Zaletą tej metody jest to, że pozwala na bardzo łatwe przeglądanie kilku konfiguracji jednocześnie.

W rzeczywistości te podejścia są bardzo podobne i wybór między nimi sprowadza się do preferencji estetycznych. Dlatego opisujemy oba, a wybór zostawiamy Tobie lub Twojemu zespołowi.

Kontrola wersji przy użyciu gałęzi i znaczników

Struktura katalogów w przypadku metody zarządzania opartej na katalogach i znacznikach jest taka sama jak opisana w poprzednim przykładzie. Kiedy wersja jest gotowa, w odpowiedniej wersji konfiguracji umieszcza się znacznik kontroli źródła (na przykład `git tag v1.0`) i gałąź `HEAD` kontynuuje iterację do przodu.

Sprawy stają się odrobinę bardziej skomplikowane, gdy trzeba zaktualizować konfigurację wersji, choć całą procedurę określa system kontroli źródła. Najpierw należy wysłać zmianę do gałęzi `HEAD` repozytorium. Następnie trzeba utworzyć nową gałąź o nazwie `v1` w znaczniku `v1.0`. Potem wybiera się żądaną zmianę do gałęzi wydania (`git cherry-pick <wydanie>`) i na koniec oznacza się tę gałąź znacznikiem `v1.1`, aby oznaczyć nowe wydanie. To podejście przedstawiono na rysunku 22.1.



Rysunek 22.1. Podejście cherry-pick

Typowym błędem popełnianym przy wybieraniu poprawek do gałęzi wydania jest wybór zmiany tylko do ostatniego wydania. Dobrym pomysłem jest jej wprowadzenie do wszystkich aktywnych wydań, na wypadek gdyby trzeba było cofnąć wersję, a poprawka byłaby nadal potrzebna.

Kontrola wersji przy użyciu katalogów

Alternatywą dla systemu kontroli wersji jest system plików. W tym podejściu poszczególne wersje są przechowywane w osobnych katalogach. Na przykład system plików naszej aplikacji może wyglądać tak:

```
frontend/
  v1/
    frontend-deployment.yaml
    frontend-service.yaml
  current/
    frontend-deployment.yaml
    frontend-service.yaml
service-1/
  v1/
    service-1-deployment.yaml
    service-1-service.yaml
  v2/
    service-1-deployment.yaml
    service-1-service.yaml
  current/
    service-1-deployment.yaml
    service-1-service.yaml
...
```

Każda wersja znajduje się w równoległej strukturze katalogów w katalogu związanym z tą wersją. Wszystkie wdrożenia wywodzą się z gałęzi `HEAD`, a nie z określonych wersji lub znaczników. Nową konfigurację dodaje się do plików w *bieżącym* katalogu.

Tworzenie nowej wersji polega na skopiowaniu *bieżącego* katalogu w celu stworzenia nowego powiązanego z nową wersją.

Gdy do wydania wprowadzana jest poprawka błędu, żądanie ściągnięcia musi spowodować zmodyfikowanie pliku YAML we wszystkich odpowiednich katalogach wersji. To podejście jest trochę lepsze od poprzedniego, ponieważ wystarczy jedno żądanie zmiany, aby było jasne, że wszystkie odpowiednie wersje zostaną zaktualizowane w taki sam sposób. Nie trzeba osobno wybierać każdej wersji.

Konstruowanie aplikacji w sposób umożliwiający jej rozwój, testowanie i wdrażanie

Struktura aplikacji powinna nie tylko umożliwiać okresowe wprowadzanie kolejnych wersji, ale również powinna pozwalać na zwinne programowanie, testowanie jakości i bezpieczne wdrażanie. Daje to programistom możliwość szybkiego opracowywania i testowania zmian w aplikacjach rozproszonych oraz bezpiecznego przesyłania ich do klientów.

Cele

W zakresie rozwoju i testowania aplikacji można wymienić dwa cele. Pierwszy jest taki, że każdy programista powinien mieć możliwość łatwego tworzenia nowych funkcji. W większości przypadków

programista pracuje tylko nad jednym komponentem, ale ten jest połączony pod wieloma względami ze wszystkimi innymi mikrousługami w klastrze. Dlatego aby praca przebiegała sprawnie, programista powinien pracować we własnym środowisku, ale mieć dostęp do wszystkich usług.

Drugim celem budowy aplikacji jest przygotowanie jej do łatwego i dokładnego przetestowania przed wdrożeniem. Jest to niezbędne do szybkiego wprowadzania nowych funkcji przy zachowaniu wysokiej niezawodności.

Progresja wydania

Aby osiągnąć oba wymienione w poprzednim punkcie cele, należy powiązać te etapy rozwoju z wersjami opisanymi wcześniej. Wyróżnia się następujące etapy rozwoju wersji:

`HEAD`
: Najnowsza wersja konfiguracji, ostatnie zmiany.

Development
: Dość stabilna, ale jeszcze niegotowa do wdrożenia wersja. Odpowiednia dla programistów do budowy funkcji.

Staging
: Początek testowania, małe prawdopodobieństwo zmian, chyba że zostaną znalezione problemy.

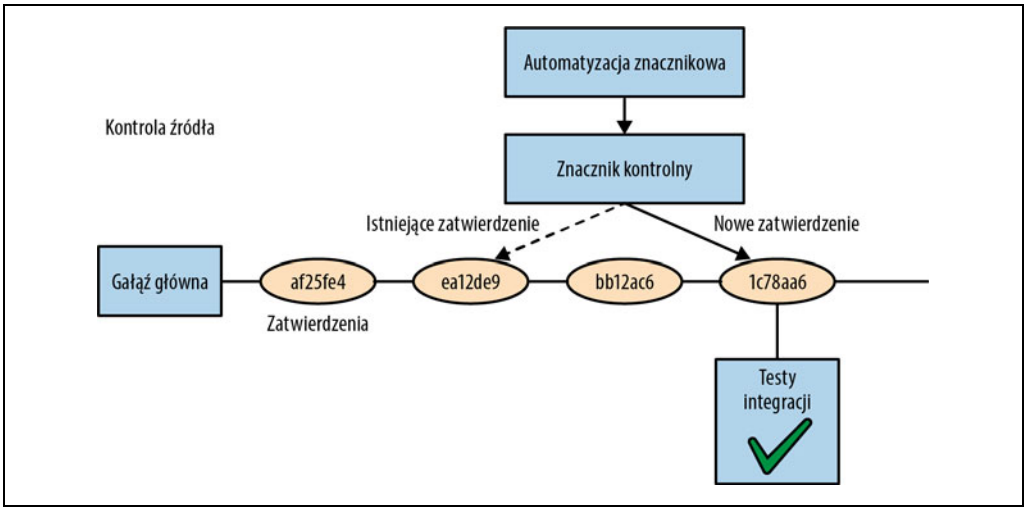
Canary
: Pierwsze prawdziwe wydanie dla użytkowników, przeznaczone do przetestowania pod prawdziwym obciążeniem i dające użytkownikom możliwość wglądu w to, co nadchodzi.

Release
: Aktualna wersja produkcyjna.

Znacznik kontrolny

Bez względu na wybór sposobu zarządzania wersjami (system plików lub system kontroli wersji) na etapie rozwojowym należy korzystać ze znacznika kontrolnego. Ma to związek z tym, że wersja rozwojowa aplikacji jest dynamiczna i tylko nieznacznie mniej stabilna niż `HEAD`.

Etap rozwojowy aplikacji wprowadza się przez dodanie do systemu kontroli źródła znacznika, który jest później obsługiwany automatycznie. W ramach okresowej kadencji gałąź `HEAD` jest poddawana zautomatyzowanym testom integracyjnym. Jeśli je przejdzie, znacznik rozwojowy zostaje przesunięty do `HEAD`. Dzięki temu programiści wdrażający swoje środowiska dysponują całkiem świeżymi wersjami zmian, a jednocześnie mają pewność, że wdrażane konfiguracje przeszły już przynajmniej podstawowe testy. To podejście jest zilustrowane na rysunku 22.2.



Rysunek 22.2. Podejście ze znacznikiem kontrolnym

Etapy i wersje

Niektórych może kusić, aby wprowadzać nowy zestaw konfiguracji dla każdego z etapów, ale w prawdziwym świecie w wyniku takiego postępowania powstałaby taka liczba kombinacji wersji i etapów, że bardzo trudno byłoby się w nich zorientować. Prawidłowe podejście polega na zastosowaniu systemu wiązania wersji z etapami.

Zarówno w podejściu do reprezentowania różnych wersji konfiguracji opartym na systemie plików, jak i na kontroli źródła łatwo można zaimplementować system wiązania etapów z wersjami. W przypadku systemu plików nazwy etapów można łączyć z wersjami za pomocą dowiązań symbolicznych:

```
frontend/
  canary/ -> v2/
  release/ -> v1/
  v1/
    frontend-deployment.yaml
...
```

W przypadku kontroli wersji wystarczy dodać znacznik odpowiadający wersji.

W każdym razie kontrola wersji odbywa się w ramach procesów opisanych wcześniej, a etapy są niezależnie w odpowiedni sposób posuwane do przodu do nowych wersji. To oznacza, że dwa procesy toczą się jednocześnie. Jeden dotyczy nowych wersji wydań, a drugi — kwalifikacji wersji wydania do określonego etapu w cyklu życia aplikacji.

Parametryzacja aplikacji za pomocą szablonów

Gdy powstanie duża liczba kombinacji środowisk i etapów, staje się jasne, że zapewnienie ich identyczności jest niepraktyczne lub nawet niemożliwe. A jednak należy starać się, aby środowiska były do siebie tak podobne, jak to możliwe. Różnice między środowiskami są przyczyną zakłóceń i prowadzą do powstawania bardzo trudnych do zrozumienia systemów. Jeśli środowisko etapowe będzie różnić się od środowiska wydań, to czy można ufać wynikom testów obciążeniowych przeprowadzonych w tym pierwszym, aby zakwalifikować wydanie? By zapewnić maksymalny poziom podobieństwa między środowiskami, można zastosować parametryzację. Środowiska parametryzowane dużą część swoich konfiguracji opierają na *szablonach*, ale ostateczna konfiguracja zależy od pewnej liczby *parametrów*. Dzięki temu większa część konfiguracji znajduje się we wspólnym szablonie, a parametry mają ograniczony zasięg i są przechowywane w małym pliku pozwalającym na łatwą wizualizację różnic.

Parametryzacja przy użyciu narzędzia Helm i szablonów

Istnieje wiele języków do tworzenia konfiguracji parametryzowanych. Generalnie wszystkie one zakładają utworzenie pliku *szablonu*, który zawiera większą część konfiguracji, i pliku *parametrów*, który można połączyć z szablonem w celu utworzenia kompletnej konfiguracji. Dodatkowo większość języków szablonowych umożliwia definiowanie wartości domyślnych, które są wykorzystywane w przypadku, gdy nie zostaną podane inne.

Poniżej znajduje się przykład utworzenia konfiguracji parametryzowanej przy użyciu narzędzia Helm (*https://helm.sh*), czyli menedżera pakietów Kubernetes. Niezależnie od tego, co mówią miłośnicy poszczególnych języków, tak naprawdę wszystkie języki parametryzacyjne są do siebie bardzo podobne i, tak jak w przypadku języków programowania, wybór jednego z nich jest w dużej mierze kwestią gustu. Dlatego też opisane tu wzorce w równym stopniu odnoszą się do Helm i do innych języków szablonowych.

Język szablonowy Helm wykorzystuje składnię szablonów „mustache”, na przykład:

```
metadata:
  name: {{ .Release.Name }}-deployment
```

Ten kod oznacza, że w miejsce `Release.Name` ma zostać wstawiona nazwa wdrożenia.

Aby przekazać parametr dla tej wartości, należy posłużyć się plikiem *values.yaml* o następującej treści:

```
Release:
  Name: moje-wydanie
```

W tej sytuacji po zamianie parametrów otrzymamy następujący wynik:

```
metadata:
  name: moje-wydanie-deployment
```

Parametryzacja systemu plików

Wiesz już, jak definiować parametry w konfiguracjach, więc teraz pokażemy Ci, jak zastosować takie konfiguracje do układów systemu plików. W tym przypadku nie będziemy traktować każdego etapu cyklu wdrażania jako wskaźnika do wersji, tylko zastosujemy rozwiązanie, w którym każdy taki etap będzie kombinacją pliku parametrów i wskaźnika do określonej wersji. W układzie opartym na katalogach może to wyglądać tak:

```
frontend/
  staging/
    templates -> ../v2
    staging-parameters.yaml
  production/
    templates -> ../v1
    production-parameters.yaml
  v1/
    frontend-deployment.yaml
    frontend-service.yaml
  v2/
    frontend-deployment.yaml
    frontend-service.yaml
...
```

W metodzie z wykorzystaniem systemu kontroli wersji struktura wygląda podobnie, tylko parametry każdego etapu cyklu życia są przechowywane w korzeniu drzewa katalogów konfiguracji:

```
frontend/
  staging-parameters.yaml
  templates/
    frontend-deployment.YAML
....
```

Wdrażanie aplikacji na całym świecie

Kiedy różne wersje aplikacji są już wdrożone na różnych etapach rozwoju, pozostaje tylko jedno —przygotowanie konfiguracji do wdrożeń na całym świecie. Myli się jednak ten, kto myśli, że to techniki zarezerwowane wyłącznie dla dużych aplikacji, ponieważ można je stosować zarówno w przypadku dwóch, jak i dziesiątek, a nawet setek regionów. W świecie chmury, w którym awaria może wyłączyć cały region, wdrożenie aplikacji w wielu regionach (i zarządzanie tym wdrożeniem) jest jedynym sposobem na osiągnięcie poziomu niezawodności satysfakcjonującego bardzo wymagających użytkowników.

Architektura umożliwiająca wdrażanie aplikacji na całym świecie

Zasadniczo klaster Kubernetes powinien obejmować jeden region i zawierać jedno kompletne wdrożenie aplikacji. W efekcie wdrożenie aplikacji na całym świecie dotyczy wielu klastrów, z których każdy ma jej własną konfigurację. Opis metod tworzenia aplikacji o zasięgu światowym, zwłaszcza z uwzględnieniem takich skomplikowanych zagadnień jak replikacja danych, nie mieści się w zakresie tego rozdziału, ale pokażemy Ci, jak przygotować pod tym kątem konfiguracje aplikacji w systemie plików.

Pod względem koncepcyjnym konfiguracja dla określonego regionu jest tym samym co etap w cyklu wdrażania. Dlatego regiony do konfiguracji dodaje się dokładnie tak samo jak etapy cyklu życia. Na przykład zamiast takiej struktury:

- *Development*
- *Staging*
- *Canary*
- *Production*

można utworzyć taką:

- *Development*
- *Staging*
- *Canary*
- *EastUS*
- *WestUS*
- *Europe*
- *Asia*

W przypadku konfiguracji system plików dla tej struktury wygląda tak:

```
frontend/
  staging/
    templates -> ../v3/
    parameters.yaml
  eastus/
    templates -> ../v1/
    parameters.yaml
westus/
    templates -> ../v2/
    parameters.yaml
...
```

Jeśli natomiast używasz systemu kontroli wersji i znaczników, Twój system plików będzie wyglądał następująco:

```
frontend/
  staging-parameters.yaml
  eastus-parameters.yaml
  westus-parameters.yaml
  templates/
    frontend-deployment.yaml
...
```

Po wybraniu tej struktury dla każdego regionu należy utworzyć nowy znacznik i wówczas we wdrożeniu w danym regionie trzeba użyć zawartości pliku odpowiadającego temu znacznikowi.

Implementacja wdrożenia światowego

Kiedy mamy już konfiguracje dla wszystkich regionów na świecie, powstaje pytanie, jak aktualizować te konfiguracje. Korzystanie z wielu z nich jest podyktowane głównie chęcią zapewnienia wysokiej niezawodności usług. Choć może się wydawać, że główną przyczyną przestojów są przerwy w dostępie do chmury i centrów danych, prawda jest taka, że najczęściej przestoje są powodowane przez operacje wdrażania nowych wersji oprogramowania. Z tego względu kluczem do utrzymania wysokiej dostępności systemu jest ograniczenie zasięgu skutków wprowadzania jakichkolwiek zmian. Jeśli więc zamierzasz wdrożyć nową wersję w wielu regionach, najlepszym sposobem jest wykonywanie operacji pojedynczo i przechodzenie do kolejnego regionu dopiero po dokładnym sprawdzeniu, czy wszystko jest w porządku w poprzednim.

Wdrażanie oprogramowania na całym świecie to operacja o charakterze procesu, a nie pojedynczego polecenia: najpierw przeprowadza się aktualizację do najnowszej wersji, a następnie przechodzi się przez wszystkie regiony po kolei. Ale jak powinna wyglądać struktura regionów i ile czasu powinno się odczekać przed przejściem do kolejnego regionu?

Do automatyzacji procesu wdrażania można używać narzędzi takich jak GitHub Actions (*https://oreil.ly/BhWxi*). Pozwalają one na deklaratywne definiowanie procesu i również są przechowywane w systemie kontroli wersji.

Ilość czasu, jaką należy odczekać z przejściem do wdrażania w kolejnym regionie, można określić na podstawie średniego czasu wykrycia problemu (jeśli istnieje) po wdrożeniu nowej wersji oprogramowania w regionie. Oczywiście każdy problem jest inny i objawia się po innym czasie, dlatego posługujemy się wartością średnią. Zarządzanie oprogramowaniem na dużą skalę polega na operowaniu prawdopodobieństwem, nie opiera się na pewności. Dlatego z przejściem do następnego regionu należy poczekać, aż prawdopodobieństwo ujawnienia się problemu stanie się wystarczająco małe. Na dobry początek można przyjąć dwu- lub trzykrotność średniego czasu ujawnienia się błędu, ale tak naprawdę wszystko zależy od konkretnej aplikacji.

Przy określaniu kolejności regionów należy uwzględnić ich cechy. Na przykład niektóre regiony są znacznie bardziej obciążone ruchem niż inne. Dana aplikacja może mieć funkcje, które cieszą się dużo większą popularnością w jednym regionie geograficznym niż w innym. Wszystkie te czynniki należy wziąć pod uwagę podczas planowania procesu aktualizacji. Najlepiej jest zaczynać od regionów o małym natężeniu ruchu, aby ewentualne problemy spowodowały jak najmniejsze szkody. Błędy, które ujawniają się szybko, często są też najbardziej dotkliwe, choć oczywiście nie jest to ścisłą regułą. Zwykle udaje się je jednak wychwycić już w pierwszym regionie. Warto więc postarać się, aby ochronić przed ich skutkami jak najwięcej klientów. Następnie dobrze jest przejść do regionu o dużym natężeniu ruchu — gdy wiadomo już, że nowa wersja działa prawidłowo przy małym obciążeniu, należy sprawdzić, jak będzie się zachowywać pod dużym obciążeniem. Jedynym sposobem, aby się o tym przekonać, jest wdrożenie aplikacji w regionie, w którym występuje duże natężenie ruchu. Jeśli operacja powiedzie się w regionie o małym i dużym natężeniu ruchu, można mieć pewność, że pójdzie bez problemu we wszystkich pozostałych. Jeżeli jednak występują jakieś różnice regionalne, dobrym pomysłem może być przedłużenie testów w różnych regionach geograficznych, zanim wypuści się aktualizację na szeroką skalę.

Kiedy opracujesz harmonogram aktualizacji, trzymaj się go zawsze bez względu na wielkość zmian. Niejeden przestój był spowodowany tym, że ktoś chciał przyspieszyć procedurę, aby rozwiązać jakiś dodatkowy problem, albo był pewny, że dana zmiana jest „bezpieczna".

Pulpity i monitorowanie wdrożeń światowych

W przypadku wdrożeń na małą skalę może się to wydawać nieistotne, ale przy średnich i dużych projektach częstym problemem jest występowanie różnych wersji aplikacji w różnych regionach. Przyczyn takiego stanu rzeczy może być wiele (na przykład błąd aktualizacji, anulowanie operacji lub problemy w określonym regionie) i jeśli nie będziesz skrupulatnie wszystkiego kontrolować, wkrótce zostaniesz z zagmatwaną plątaniną różnych wersji wdrożonych w różnych regionach na całym świecie. Ponadto kiedy klient pyta o poprawki błędów, które przeszkadzają mu w pracy, chcemy wiedzieć, czy zostały już wdrożone.

Dlatego należy opracować pulpit, na którym wygodnie można sprawdzić, jaka wersja oprogramowania jest wdrożona w każdym regionie. Powinien on także wysyłać ostrzeżenie, gdy pojawi się zbyt duża liczba różnych wersji wdrożeń aplikacji. Najlepszą praktyką jest zawężenie limitu aktywnych wersji do maksymalnie trzech: testowej, wdrożeniowej i zmienianej przez wdrażaną. Każda dodatkowa wersja jest jak proszenie się o kłopoty.

Podsumowanie

W tym rozdziale opisaliśmy metody zarządzania wersjami aplikacji Kubernetes, etapami wdrażania oraz regionami na całym świecie. Przedstawiliśmy zasady stanowiące podstawę dobrej organizacji oprogramowania: posługiwanie się systemem plików, recenzowanie kodu w celu zapewnienia wysokiej jakości zmian oraz wykorzystanie flag funkcji przy stopniowym dodawaniu lub usuwaniu funkcji.

Zaprezentowane w tym rozdziale rozwiązania należy traktować jako wskazówki, a nie gotowe recepty. Zapoznaj się z nimi, po czym opracuj własne rozwiązanie, które najlepiej sprawdzi się w przypadku Twojej konkretnej aplikacji. Pamiętaj też, że opracowując strukturę aplikacji pod kątem jej wdrażania, planujesz proces, z którym będziesz mieć do czynienia przez wiele lat.

DODATEK A

Budowanie własnego klastra Kubernetes

System Kubernetes jest często postrzegany przez pryzmat wirtualnego świata obliczeń w chmurze publicznej, gdzie najbardziej możesz zbliżyć się do klastra poprzez przeglądarkę internetową lub terminal, ale zbudowanie fizycznego klastra Kubernetes na gołym metalu może być bardzo satysfakcjonującym doświadczeniem. Nic też nie może równać się z uczuciem, kiedy naprawdę podpinasz zasilanie lub sieć do węzła i obserwujesz na żywo działanie systemu Kubernetes, gdy próbuje wyleczyć Twoją aplikację, aby przekonać Cię o swojej użyteczności.

Budowa własnego klastra może wydawać się trudna i kosztowna, ale na szczęście tak nie jest. Możliwość zakupu tanich płyt komputerowych typu **Soc** (ang. *System-on-a-chip*) oraz szeroki wkład społeczności, aby uczynić system Kubernetes łatwiejszym w instalacji, sprawiają, że niewielki klaster Kubernetes da się zbudować w kilka godzin.

W tym rozdziale zamieściliśmy instrukcje budowania klastra maszyn Raspberry Pi, ale przy niewielkich modyfikacjach te same instrukcje można zastosować w pracy z wieloma różnymi maszynami jednopłytowymi lub innymi komputerami, które ma się do dyspozycji.

Lista części

Pierwszą rzeczą, którą musisz zrobić, jest zebranie kilku elementów do Twojego klastra. We wszystkich przedstawionych tu przykładach zakładamy budowę klastra z czterema węzłami. Jeśli chcesz, możesz zbudować klaster trzech węzłów, a nawet stu węzłów, ale cztery to całkiem dobra liczba.

Na początek musisz zakupić (lub „wyżebrać") różne elementy potrzebne do zbudowania klastra. Oto lista zakupów z przybliżonymi cenami (aktualnymi w momencie powstawania tego zestawienia):

1. Cztery płyty Raspberry Pi 4 z przynajmniej 2 GB pamięci: 700 zł;
2. Cztery karty pamięci SDHC, co najmniej o pojemności 8 GB (kupuj produkty wysokiej jakości!): 120 – 200 zł;
3. Cztery 30-centymetrowe kable ethernetowe Cat. 6: 40 zł;
4. Cztery 30-centymetrowe kable USB-A do USB-C: 40 zł;
5. Jeden 5-portowy przełącznik Fast Ethernet 10/100: 40 zł;
6. Jedna 5-portowa ładowarka USB: 100 zł;

7. Jedna stakowalna obudowa na 4 urządzenia Raspberry Pi: 150 zł (możesz zbudować własną);
8. Jeden kabel zasilający USB do przełącznika Ethernet (opcjonalnie): 20 zł.

Łącznie dla całego klastra koszty sprzętu wyniosą około 1200 zł, ale możesz zejść do 800 zł, budując klaster z trzema węzłami i pomijając obudowę oraz kabel zasilający dla przełącznika (chociaż obudowa i kabel sprawiają, że konstrukcja jest schludniejsza).

Jeszcze jedna uwaga w kwestii kart pamięci: nie oszczędzaj na nich. Niskobudżetowe karty pamięci zachowują się nieprzewidywalnie i sprawiają, że klastry są naprawdę niestabilne. Jeśli chcesz zaoszczędzić trochę pieniędzy, kup wysokiej jakości kartę o mniejszej pojemności. W internecie można znaleźć bardzo dobrej jakości karty o pojemności 8 GB już za około 30 zł.

W każdym razie kiedy zaopatrzysz się już w części, będziesz gotowy, aby przejść do budowania klastra.

Podając te instrukcje, zakładamy, że masz urządzenie zdolne do flashowania kart SDHC. Jeśli nie, musisz kupić czytnik USB kart pamięci.

Flashowanie obrazów

Domyślny obraz systemu Ubuntu 20.04 obsługuje Raspberry Pi 4 oraz jest powszechnie używany przez wiele klastrów Kubernetes. Najprostszym sposobem na jego instalację jest wykorzystanie narzędzia Raspberry Pi Imager dostępnego na stronach projektu Raspberry Pi (*https://oreil.ly/4s8Wa*):

- macOS (*https://oreil.ly/g7Lzw*)
- Windows (*https://oreil.ly/Y7CD3*)
- Linux (*https://oreil.ly/u4YvC*)

Za pomocą tego Imagera zapisz obraz systemu Ubuntu 20.04 na każdej swojej karcie pamięci. Ubuntu może nie być domyślnym wyborem w programie, ale można go wskazać jako opcję.

Pierwsze uruchomienie: węzeł główny

Pierwszą czynnością jest uruchomienie tylko serwera API. Złóż swój klaster i zdecyduj, który komputer będzie węzłem serwera API. Włóż kartę pamięci, podłącz płytę do wyjścia HDMI, a klawiaturę do portu USB.

Następnie podłącz zasilanie, aby uruchomić płytę.

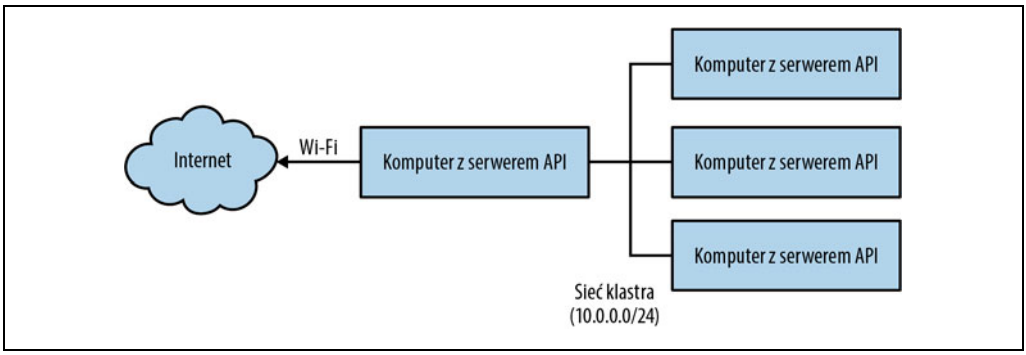
Zaloguj się przy użyciu nazwy użytkownika `ubuntu` i hasła `ubuntu`.

Pierwszą rzeczą, którą powinieneś zrobić ze swoim Raspberry Pi (lub dowolnym nowym urządzeniem), jest zmiana domyślnego hasła. Domyślne hasła dla każdego rodzaju instalacji są dobrze znane ludziom, którzy mogą podejmować złośliwe działania po zalogowaniu się do systemu. To sprawia, że internet jest mniej bezpieczny dla wszystkich. Zmień swoje domyślne hasła!

Powtórz te czynności dla każdego węzła w swoim klastrze.

Konfigurowanie sieci

Następnym krokiem jest skonfigurowanie sieci na serwerze API. Konfiguracja sieci dla klastra Kubernetes może być skomplikowana. W poniższym przykładzie konfigurujemy sieć, w której jeden komputer jest połączony z internetem bezprzewodowo. Komputer ten jest także połączony z siecią klastra przez połączenie przewodowe Ethernet oraz zapewnia serwer DHCP (ang. *Dynamic Host Configuration Protocol*), aby udostępnić adres sieciowy pozostałym węzłom w klastrze. Ilustruje to poniższy rysunek:



Wybierz, która z Twoich płyt będzie hostować serwer API i `etcd`. Najprostszym sposobem na zapamiętanie jej jest uczynienie jej pierwszym lub ostatnim węzłem, ale można też zastosować jakieś oznaczenie.

Aby to zrobić, otwórz plik */etc/netplan/50-cloud-init.yaml*. Jeśli nie istnieje, możesz go utworzyć. Jego zawartość powinna wyglądać tak:

```
network:
        version: 2
        ethernets:
                eth0:
                        dhcp4: false
                        dhcp6: false
                        addresses:
                        - '10.0.0.1/24'
                        optional: true
        wifis:
                wlan0:
                        access-points:
                                <twój-ssid>:
```

```
                    password: '<twoje-hasło>'
        dhcp4: true
        optional: true
```

Powoduje to ustawienie głównego interfejsu Ethernet na statyczny adres 10.0.0.1. i skonfigurowanie interfejsu Wi-Fi do łączenia się z lokalną siecią Wi-Fi. Następnie należy wykonać polecenie `sudo netplan apply`, by wprowadzić te nowe zmiany.

Uruchom ponownie komputer, aby zająć adres 10.0.0.1. Poprawność ustawień można skontrolować przez wykonanie polecenia `ip addr` i sprawdzenie adresu interfejsu `eth0`. Ponadto upewnij się, że połączenie z internetem działa poprawnie.

Następnie zainstaluj na tym serwerze API serwer DHCP, aby przydzielił adresy węzłom roboczym. Uruchom poniższe polecenie:

```
$ apt-get install isc-dhcp-server
```

Teraz skonfiguruj serwer DHCP w następujący sposób (*/etc/dhcp/dhcpd.conf*):

```
# Ustaw nazwę domenową, która zasadniczo może być dowolna
option domain-name "cluster.home";

# Domyślnie używa serwera DNS Google; możesz tutaj wstawić wartości dostarczone przez ISP
option domain-name-servers 8.8.8.8, 8.8.4.4;

# Dla naszej podsieci użyjemy zakresu 10.0.0.X
subnet 10.0.0.0 netmask 255.255.255.0 {
    range 10.0.0.1 10.0.0.10;
    option subnet-mask 255.255.255.0;
    option broadcast-address 10.0.0.255;
    option routers 10.0.0.1;
}
default-lease-time 600;
max-lease-time 7200;
authoritative;
```

Być może trzeba będzie też otworzyć plik */etc/defaults/isc-dhcp-server*, aby ustawić zmienną środowiskową `INTERFACES` na `eth0`. Zrestartuj serwer DHCP za pomocą polecenia `sudo systemctl restart isc-dhcp-server`. Teraz Twój komputer powinien przydzielać adresy IP. Możesz to sprawdzić, podłączając drugą maszynę do przełącznika za pomocą kabla ethernetowego. Ta druga maszyna powinna dostać z serwera DHCP adres 10.0.0.2.

Pamiętaj, aby edytować plik */etc/hostname* w celu zmiany nazwy tego komputera na `node-1`. Żeby pomóc Kubernetes w obsłudze sieci, musisz skonfigurować `iptables`, tak by „widział" mostkowany ruch sieciowy. Utwórz plik */etc/modules-load.d/k8s.conf* zawierający tylko `br_netfilter`. Spowoduje to załadowanie modułu `br_netfilter` do jądra.

Następnie musisz włączyć ustawienia `systemctl` dla mostkowania sieciowego i translacji adresów (NAT), aby działały mechanizmy sieciowe Kubernetes oraz by Twoje węzły stały się obecne w internecie publicznym. Utwórz plik o nazwie */etc/sysctl.d/k8s.conf* i następującej treści:

```
net.ipv4.ip_forward=1
net.bridge.bridge-nf-call-ip6tables=1
net.bridge.bridge-nf-call-iptables=1
```

Następnie edytuj plik */etc/rc.local* (lub jego odpowiednik) i dodaj reguły `iptables` dla przekierowania z `eth0` do `wlan0` (i odwrotnie):

```
$ iptables -t nat -A POSTROUTING -o wlan0 -j MASQUERADE
$ iptables -A FORWARD -i wlan0 -o eth0 -m state \
  --state RELATED,ESTABLISHED -j ACCEPT
$ iptables -A FORWARD -i eth0 -o wlan0 -j ACCEPT
```

W tym momencie podstawowa konfiguracja sieci powinna być kompletna. Podłącz i uruchom pozostałe dwie płyty (powinieneś zobaczyć, że otrzymały adresy 10.0.0.3 i 10.0.0.4). Edytuj plik */etc/hostname* na obu komputerach, aby nazwać je odpowiednio `node-2` i `node-3`.

Zweryfikuj rezultat, zaglądając do pliku */var/lib/dhcp/dhcpd.leases*, a następnie uruchom połączenie SSH z węzłami (pamiętaj, aby najpierw zmienić domyślne hasła). Sprawdź, czy węzły łączą się z internetem.

Dodatkowe zadania

Jest kilka dodatkowych elementów konfiguracji sieciowej, które ułatwiają zarządzanie klastrem. Pierwszym jest edycja pliku */etc/hosts* na każdym komputerze, aby zmapować nazwy na właściwe adresy. Na każdej maszynie dodaj:

```
...
10.0.0.1 kubernetes
10.0.0.2 node-1
10.0.0.3 node-2
10.0.0.4 node-3
...
```

Teraz możesz używać tych nazw podczas łączenia się z tymi maszynami.

Druga rzecz polega na ustawieniu dostępu SSH bez haseł. W tym celu uruchom polecenie `ssh-keygen`, a następnie skopiuj plik *$HOME/.ssh/id_rsa.pub* do `/home/pirate/.ssh/authorized_keys` na komputerach `node-1`, `node-2` i `node-3`.

Instalowanie środowiska wykonawczego kontenerów

Zanim będzie można zainstalować Kubernetes, należy zainstalować środowisko wykonawcze kontenerów. Do wyboru masz kilka opcji, ale najczęściej jest używane środowisko `containerd` z Dockera. Środowisko `containerd` jest dostępne w postaci standardowego pakietu Ubuntu, ale zazwyczaj jest on trochę opóźniony pod względem wersji. Dlatego zalecamy instalację z projektu Docker, choć wymaga to odrobinę więcej pracy.

Pierwszy krokiem jest ustawienie Dockera jako repozytorium do instalacji pakietów w systemie:

```
# Kilka potrzebnych dodatków
sudo apt-get install ca-certificates curl gnupg lsb-release

# Instalacja klucza podpisu Dockera
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor \
-o /usr/share/keyrings/docker-archive-keyring.gpg
```

Na koniec utwórz plik */etc/apt/sources.list.d/docker.list* o następującej zawartości:

```
deb [arch=arm64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] \
https://download.docker.com/linux/ubuntu focal stable
```

Po zainstalowaniu repozytorium pakietów Dockera możesz zainstalować `containerd.io`, wykonując poniższe polecenie. Należy zainstalować `containerd.io`, nie `containerd`, aby otrzymać pakiet Dockera, a nie domyślny pakiet Ubuntu:

```
sudo apt-get update; sudo apt-get install containerd.io
```

W tym momencie `containerd` jest zainstalowany, ale wymaga konfiguracji, ponieważ konfiguracja dostarczona przez pakiet nie pozwala na współpracę z Kubernetes:

```
containerd config default > config.toml
sudo mv config.toml /etc/containerd/config.toml

# Ponowne uruchomienie, aby wprowadzić konfigurację
sudo systemctl restart containerd
```

Po zainstalowaniu środowiska wykonawczego kontenerów możesz przejść do instalacji samego Kubernetes.

Instalacja Kubernetes

W tym momencie powinieneś mieć działające wszystkie węzły, z adresami IP i dostępem do internetu. Teraz czas zainstalować Kubernetes na wszystkich węzłach. Używając SSH, uruchom poniższe polecenia na wszystkich węzłach, by zainstalować narzędzia `kubelet` i `kubeadm`.

Najpierw dodaj klucz szyfrowania dla pakietów:

```
# curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg \
  | sudo apt-key add -
```

Potem dodaj to repozytorium do listy repozytoriów:

```
# echo "deb http://apt.kubernetes.io/ kubernetes-xenial main" \
  | sude tee /etc/apt/sources.list.d/kubernetes.list
```

Na koniec zaktualizuj i zainstaluj narzędzia Kubernetes. Spowoduje to również aktualizację na wszelki wypadek wszystkich pakietów w Twoim systemie:

```
# sudo apt-get update
$ sudo apt-get upgrade
$ sudo apt-get install -y kubelet kubeadm kubectl kubernetes-cni
```

Konfigurowanie klastra

Na serwerze API (tym z uruchomionym serwerem DHCP i podłączonym do internetu) uruchom następujące polecenie:

```
$ sudo kubeadm init --pod-network-cidr 10.244.0.0/16 \
            --apiserver-advertise-address 10.0.0.1 \
            --apiserver-cert-extra-sans kubernetes.cluster.home
```

Zwróć uwagę, że rozgłaszasz swój wewnętrzny adres IP, a nie zewnętrzny adres.

W efekcie wydrukowane zostanie polecenie dodawania węzłów do klastra. Będzie to wyglądać mniej więcej tak:

```
$ kubeadm join --token=<token> 10.0.0.1
```

Używając SSH, połącz się z każdym z węzłów roboczych w klastrze i uruchom to polecenie.

Kiedy to zrobisz, powinieneś być w stanie uruchomić poniższe polecenie i zobaczyć swój działający klaster:

```
$ kubectl get nodes
```

Konfigurowanie ustawień sieciowych klastra

Skonfigurowałeś sieć na poziomie węzłów, ale musisz skonfigurować ustawienia sieciowe do komunikacji między kapsułami. Ponieważ wszystkie węzły w klastrze działają w tej samej fizycznej sieci Ethernet, wystarczy po prostu ustawić poprawne reguły routingu w jądrach hostów.

Najprostszym sposobem jest użycie narzędzia Flannel (*https://oreil.ly/ltaOv*) stworzonego przez CoreOS, a teraz obsługiwanego w ramach projektu Flannel (*https://oreil.ly/RHJMH*). Flannel umożliwia korzystanie z wielu różnych trybów routingu; my użyjemy trybu `host-gw`. Możesz pobrać przykładową konfigurację ze strony projektu Flannel (*https://github.com/coreos/flannel*):

```
$ curl https://oreil.ly/kube-flannelyml \
  > kube-flannel.yaml
```

Domyślna konfiguracja, którą dostarcza Flannel, korzysta z trybu `vxlan`. Aby to poprawić, otwórz ten plik konfiguracyjny w ulubionym edytorze i zamień `vxlan` na `host-gw`.

Możesz to również zrobić za pomocą narzędzia `sed`:

```
$ curl https://oreil.ly/kube-flannelyml \
  | sed "s/vxlan/host-gw/g" \
  > kube-flannel.yaml
```

Po zaktualizowaniu pliku *kube-flannel.yaml* możesz utworzyć konfigurację sieciową Flannel w następujący sposób:

```
$ kubectl apply -f kube-flannel.yaml
```

Spowoduje to utworzenie dwóch obiektów, `ConfigMap`, służącego do konfiguracji Flannel, i `DaemonSet`, który uruchamia rzeczywistego demona Flannel. Możesz to sprawdzić za pomocą następujących poleceń:

```
$ kubectl describe --namespace=kube-system configmaps/kube-flannel-cfg
$ kubectl describe --namespace=kube-system daemonsets/kube-flannel-ds
```

Podsumowanie

W tym momencie powinieneś mieć działający klaster Kubernetes skonfigurowany na Twoich maszynach Raspberry Pi. Nadaje się on doskonale do odkrywania Kubernetes. Zaplanuj pewne prace, otwórz interfejs użytkownika i spróbuj popsuć klaster poprzez restartowanie komputerów lub odłączenie sieci.

Notatki

Notatki

O autorach

Brendan Burns rozpoczął swoją karierę od krótkiego stażu w branży oprogramowania, po czym skupił się na doktoracie z robotyki, dotyczącym planowania ruchu dla ramion robotycznych. Potem przez krótki okres wykładał jako profesor informatyki. W końcu wrócił do Seattle i dołączył do Google, gdzie pracował nad infrastrukturą wyszukiwania internetowego ze szczególnym uwzględnieniem indeksowania o niskim opóźnieniu. Podczas pracy w Google wraz z Joem Bedą i Craigiem McLuckiem stworzył projekt Kubernetes. Brendan jest obecnie dyrektorem ds. technicznych w Microsoft Azure.

Joe Beda rozpoczął karierę w firmie Microsoft, pracując nad przeglądarką Internet Explorer (był młody i naiwny). Przez 7 lat w Microsofcie i 10 lat w Google Joe pracował nad frameworkami GUI, komunikatorami w czasie rzeczywistym, telefonią, uczeniem maszynowym dla celów związanych z reklamami oraz chmurą obliczeniową. Przede wszystkim podczas pracy w Google rozpoczął projekt Google Compute Engine i razem z Brendanem Burnsem oraz Craigiem McLuckiem stworzył Kubernetes. Wraz z Craigiem Joe założył i sprzedał start-up (Heptio) firmie VMware, w której aktualnie jest głównym inżynierem. Joe z dumą nazywa Seattle swoim domem.

Kelsey Hightower pełni funkcję Principal Developer Advocate w Google, gdzie pracuje w dziale Google Cloud Platform. Opracował i udoskonalił wiele produktów Google Cloud, między innymi Google Kubernetes Engine, Cloud Functions i bramę interfejsu API Apigee. Większość czasu spędza w towarzystwie dyrektorów i programistów z firm z listy „Fortune" 1000, którym pomaga zrozumieć i wykorzystywać technologie i platformy Google wspierające rozwój biznesu. Kelsey jest bardzo aktywny w środowisku open source, w którym zajmuje się projektami pomagającymi programistom i specjalistom operacyjnym w budowie i dostarczaniu aplikacji chmurowych. Dużo pisze oraz często występuje na konferencjach, jako pierwszy otrzymał wyróżnienie CNCF Top Ambassador za pomoc w początkowej fazie rozwoju Kubernetes. Jest mentorem i doradcą technicznym, który pomaga założycielom firm zrealizować swoje wizje.

Lachlan Evenson jest głównym menedżerem programu w zespole open source w Azure. Jest aktywnym członkiem społeczności Kubernetes oraz zasiadał w komitecie sterującym i pełnił funkcję koordynatora wydania. Lachlan posiada wiedzę praktyczną na temat wielu projektów chmurowych, a także spędza czas na budowie i wspieraniu projektów open source z ekosystemu chmurowego.

Kolofon

Zwierzę przedstawione na okładce tej książki to delfinowiec (delfin) białoboki (łac. *Lagenorhynchus acutus*). Zgodnie z nazwą boki tego zwierzęcia zdobią jasne łaty, a od oka do płetwy grzbietowej biegnie jasnoszary pas. Jest to przedstawiciel jednego z największych gatunków delfinów oceanicznych, który występuje na obszarze całego północnego Oceanu Atlantyckiego. Woli przebywać na otwartych wodach, przez co jest rzadko widywany w pobliżu wybrzeża, ale chętnie podpływa do łodzi i popisuje się sztuczkami.

Delfiny białobokie to towarzyskie zwierzęta, które zwykle żyją w dużych grupach złożonych z około 60 osobników, choć wiele zależy od miejsca i dostępności pożywienia. Delfiny często grupowo polują na ławice ryb, choć zdarza się im też polować w pojedynkę. Ich podstawowym narzędziem do znajdowania ofiar jest echolokacja, czyli delfini rodzaj sonaru. Większość diety tych oceanicznych ssaków stanowią śledzie, makrele i ośmiornice.

Średnia długość życia delfina białobokiego wynosi od 22 do 27 lat. Samice przystępują do godów tylko raz na dwa – trzy lata, a ciąża trwa u nich 11 miesięcy. Młode zwykle rodzą się w czerwcu lub w lipcu i są karmione przez matkę przez 18 miesięcy. Dzięki wysokiemu stosunkowi masy mózgu do masy ciała (najwyższemu w świecie zwierząt morskich) delfiny są bardzo inteligentne i wykazują złożone zachowania społeczne, takie jak okazywanie żałoby, współpraca czy rozwiązywanie problemów.

Ilustracja na okładce, autorstwa Karen Montgomery, została wykonana na podstawie czarno-białej ryciny zamieszczonej w książce *British Quadrupeds*.

PROGRAM PARTNERSKI

GRUPY HELION

1. ZAREJESTRUJ SIĘ
2. PREZENTUJ KSIĄŻKI
3. ZBIERAJ PROWIZJĘ

Zmień swoją stronę WWW w działający bankomat!

Dowiedz się więcej i dołącz już dzisiaj!
http://program-partnerski.helion.pl

GRUPA Helion